WEB PROGRAMLAMA

(Web Programming)

Platformdan Üretime

TEMELLER BASKISI

(FOUNDATIONS EDITION)

Dr. Mahyar Teymournezhad

Bilgisayar Mühendisliği Programı

Teknik inceleme: 21 October 2026

Bu baskı hakkında

Bu temeller baskısı, kapsamlı bir üniversite ders kitabının tamamlanmış ilk üç oturumunu içerir: Web'in nasıl çalıştığı, anlamsal HTML (semantic HTML) ve erişilebilirlik (accessibility) ile profesyonel duyarlı CSS (responsive CSS). Bu baskı, sonraki kısımlarda da izlenen son bölüm biçimini, laboratuvar modelini, ayrıntılı çözüm politikasını ve görsel sistemi kullanır.

Öğrenci baskısı, biçimlendirici alıştırmaların (formative exercises) ve kısa sınavların (quizzes) eksiksiz yanıtlarını içerir. Sonuç değerlendirmesinde kullanılan güvenli sürümler ise erişim denetimli ayrı bir öğretim elemanı ekinde tutulur.

Kitap nasıl kullanılmalı?

Örnekleri çalıştırmadan önce sonucu tahmin edin, her laboratuvar kontrol noktasını tamamlayın, çalışmanızı sürüm kontrolüne işleyin (commit) ve çözümünü okumadan önce her alıştırmayı yapmayı deneyin. Tarayıcı araçlarını (browser tools), testleri, sürüm geçmişini (version history) ve bunların sunduğu mühendislik kanıtını (engineering evidence) değerlendirin.

Yayın bilgileri

Gözden geçirilmiş temeller baskısı, 28 August 2026. Yazar: Dr. Mahyar Teymournezhad. Telif hakkı (c) 2026 Dr. Mahyar Teymournezhad. Apache License, Version 2.0 kapsamında dağıtılmaktadır; deponun LICENSE dosyasına bakın. Güncel teknik temel, Node.js 24 LTS sürümünü hedeflemektedir.

Önsöz

Web çoğu zaman bir komutlar bütünü olarak tanıtılır: bu etiketi yazın, şu stil kuralını kopyalayın, bir çatı (framework) kurun ve bir sunucu (server) çalıştırın. Bu komutlar bir sayfa üretebilir; ancak öğrenciye henüz güvenilir bir zihinsel model (mental model) kazandırmaz. Bir şey başarısız olduğunda, kopyalanmış adımlar pek az yardım sunar. Bu nedenle bu kitap daha küçük ve daha insani sorularla başlar. Tarayıcı ne yapmaya çalışıyor? Hangi program iletişim kuruyor? Ağdan hangi bilgiler geçiyor? Hangi kısmı gözlemleyebiliriz? Ancak bundan sonra sorunu çözen protokole (protocol), dil özelliğine (language feature) veya araca ad veririz.

Önceden web geliştirme (web development) bilgisine sahip olduğunuz varsayılmaz. Teknik bir terim, sade bir dille tanıtılır, tanıdık bir durumla ilişkilendirilir ve ardından mühendislik çalışmaları için yeterince kesin olana kadar ayrıntılandırılır. Kısaltmalar, ilk kez önem kazandıklarında açık biçimleriyle verilir. Diyagramlar ilişkileri gösterir; örnekler bu ilişkileri somutlaştırır; laboratuvarlar açıklamayı kanıta dönüştürür. İleri düzey içerik, temel bilgilerden ani bir sıçramayla ayrılmaz. Aynı fikirlerin daha büyük ölçekte geliştirilmesiyle oluşur.

Ders boyunca sürdürülen Campus Hub projesi, derse tek bir ana hat kazandırır. Öğrenciler önce küçük bir belge yayımlar; ardından onu anlamsal, duyarlı, etkileşimli, veri odaklı (data-driven), güvenli, test edilmiş ve dağıtıma hazır duruma getirir. Her eklemenin bir nedeni vardır. Amaç yalnızca bir uygulamayı tamamlamak değil, uygulamanın tarayıcıdan, ağdan, sunucudan ve veri tabanından ne yapmasını istediğini anlamaktır.

Öğrenciler için

Bir örneği çalıştırmadan önce sonucu tahmin edin. Gözlemlediklerinizi ve bunlardan çıkardığınız sonuçları kaydedin. Yanıtlarını okumadan önce alıştırmaları yapmayı deneyin ve kodun son satırını ezberlemek yerine akıl yürütme biçimlerini karşılaştırın. Bir paragraf yoğun gelirse somut örneğine dönün ve onu kendi sözcüklerinizle yüksek sesle açıklayın.

Öğretim elemanları için

Her oturum öğretime uygun bir sıra hâlinde düzenlenmiştir: güdüleyici problem, kavram modeli, tahmin, gösterim, yönlendirmeli laboratuvar, bağımsız aktarım, hata ayıklama kliniği (debugging clinic), bilgiyi geri çağırma sınavı (retrieval quiz) ve yansıtıcı değerlendirme (reflection). Açıklamalar hazırlık ve bireysel çalışma için yeterince ayrıntılıdır; kontrol noktaları ise sorular, canlı gösterimler ve biçimlendirici değerlendirme (formative assessment) için doğal duraklar sağlar.

Yazar hakkında

Dr. Mahyar Teymournezhad

Dr. Mahyar Teymournezhad, *Web Programlama: Platformdan Üretime* kitabının yazarı ve öğretim programı tasarımcısıdır (curriculum designer). Kitabı, öğrencinin Web ile ilk karşılaşmasından üretim odaklı tam yığın mühendisliğine (production-oriented full-stack engineering) uzanan tutarlı tek bir yol sunacak biçimde, üniversitede bilgisayar mühendisliği dersi için temel başvuru kaynağı olarak geliştirmiştir.

Yazarlık yaklaşımı, kalıcı kavramlara güncel eğilimlere bağlı araçlardan önce yer verir. Erişilebilirliği, güvenliği (security), test etmeyi (testing), sürüm kontrolünü (version control) ve profesyonel açıklamayı; isteğe bağlı son konular olarak değil, gündelik mühendislik sorumlulukları olarak ele alır. Her laboratuvar gözlemlenebilir kanıt bırakacak biçimde tasarlanır; her ayrıntılı çözüm de bir çözümün neden uygun olduğunu, hangi varsayımlara dayandığını ve farklı bir seçimin hangi durumda gerekçelendirilebileceğini açıklar.

İçindekiler

Yönlendirme - Web Programlama Çalışma İstasyonunuzu (workstation) Hazırlayın 17

Yönlendirme haritası 17

Öğrenme çıktıları 17

1. Araçları kurmadan önce çalışma istasyonunu anlayın 18

1.1 Gerekli donanım ve erişim 18

1.2 Hazırladığınız bilgisayarı kaydedin 19

2. İşletim sistemini (operating system) ve dosya görünümünü hazırlayın 19

3. Güncel bir web tarayıcısı kurun 20

4. Visual Studio Code'u kurun ve ortamı tanıyın 20

4.1 Windows 20

4.2 macOS 20

4.3 Linux 21

4.4 Düzenleyiciyi doğrulayın 21

5. Asgari terminal modelini öğrenin 21

5.1 Proje kökü (project root) soyut bir fikir değil, bir konumdur 22

6. Git'i kurun ve yapılandırın 23

6.1 Git'i kurun 23

6.2 Kurulumu doğrulayın ve yazarı tanımlayın 23

7. Node.js 24 LTS ve npm'yi kurun 24

7.1 Windows ve macOS yükleyici yolu 24

7.2 Linux yolu 24

7.3 İsteğe bağlı sürüm yöneticisi (version manager) yolu 24

7.4 Node.js ve npm'yi doğrulayın 24

7.5 JavaScript dersin birincil programlama dilidir 25

7.6 JavaScript'i tarayıcıda doğrulayın 25

7.7 JavaScript'i Node.js içinde doğrulayın 26

7.8 JavaScript'in ders boyunca nasıl geliştiği 26

8. Geliştirmeye başlamadan önce uzak deponuzu (remote repository) oluşturun veya edinin 26

8.1 Tercih edilen ders yöntemi: öğretim elemanının GitHub şablonundan oluşturun 27

8.2 Öğretim elemanı şablon yerine bir klonlama URL'si (clone URL) sağlarsa 30

8.3 Depo bu bilgisayarda zaten varsa 30

8.4 Öğretim elemanı bir ZIP arşivi sağlarsa 30

8.5 Deponun tamamını VS Code'da açın 31

8.6 Geliştirilen çalışmalarda aynı kaydet-denetle-işle-gönder döngüsünü kullanın 31

9. Yeniden üretilebilir hazırlık testini (readiness test) çalıştırın 31

10. Yerel sunucuyu (local server) başlatın ve inceleyin 32

10.1 Kitabın iki terminal ile neyi kastettiği 32

10.2 Sunucuyu başlatın ve çıktısını okuyun 33

10.3 Adresi doğru biçimde açın 33

10.4 Geliştirici araçlarında (developer tools) Network panelini bulun 34

10.5 Sunucuyu bilinçli olarak durdurun 35

10.6 İsteğe bağlı sorun giderme (troubleshooting): npm package.json dosyasını bulamıyor . . . 35
11. Erişilebilirlik inceleme araçlarını hazırlayın . . . 36
12. Kişisel bir çalışma kanıtı kaydı (proof-of-work commit) oluşturun ve gönderin . . . 36
13. Kanıtlardan hareketle sorun giderin . . . 37
14. Hazırlık kaydı . . . 37

Yönlendirme alıştırmaları . . . 38
Alıştırma 1 - Sorumlu aracı adlandırın . . . 38
Alıştırma 2 - Dizini teşhis edin . . . 38
Alıştırma 3 - Kanıtı açıklayın . . . 38
Alıştırma 4 - Gizli bilgileri koruyun . . . 38
Alıştırma 5 - Terminal rollerini atayın . . . 39
Alıştırma 6 - Boş bir Network panelini bulun . . . 39
Alıştırma 7 - Bağlantı ve yol hatalarını ayırın . . . 39
Alıştırma 8 - Tek dil, iki ortam . . . 39

Yönlendirme ayrıntılı çözümleri . . . 39
Yanıt 1 . . . 39
Yanıt 2 . . . 39
Yanıt 3 . . . 40
Yanıt 4 . . . 40
Yanıt 5 . . . 40
Yanıt 6 . . . 40
Yanıt 7 . . . 40
Yanıt 8 . . . 40

Yönlendirme - bilgiyi geri çağırma sınavı . . . 41
Sınav yanıtları . . . 41
Yönlendirme özeti . . . 42
Yönlendirme kaynakları . . . 42
Neler sık değişir? . . . 43

Oturum 1 - Web Nasıl Çalışır? . . . 44
Oturum haritası . . . 44
Öğrenme çıktıları . . . 44
1. Kanıtla başlayın . . . 44
2. İnternet, Web değildir . . . 45
2.1 Ağ nedir? . . . 46
2.2 İstemciler (clients) ve sunucular rollerdir . . . 47
2.3 HTML, CSS ve JavaScript'in farklı görevleri vardır . . . 48
2.4 Web sitesi (website), web uygulaması (web application) ve API . . . 50
Kontrol noktası 1 - rolleri sınıflandırın . . . 51
3. Bir URL'yi yapılandırılmış veri (structured data) olarak okuyun . . . 51

İki nokta karakteri (colon character) hakkında bir not . . . 52
3.1 Yol (path) bir addır; her zaman diskteki bir konum değildir . . . 52
3.2 Sorgu verileri (query data) ve parçalar (fragments) . . . 52
3.3 Köken (origin) ve aynı köken (same-origin) üzerine akıl yürütme . . . 53
3.4 Kodlama (encoding) ve normalleştirme (normalization) . . . 55
Çalıştırmadan önce tahmin edin . . . 55
4. Gezinmeden piksellere . . . 55
4.0 Protokol yığını (protocol stack): karışıklık olmadan iş birliği . . . 57
4.1 Ad çözümleme (name resolution) . . . 59
4.2 Bağlantı ve TLS . . . 65
4.3 HTTP alışverişi . . . 65
4.4 Tarayıcı işlemesi . . . 67
5. Yerel dosya, yerel sunucu, herkese açık sunucu . . . 68
6. Geliştirici araçları ölçüm araçlarıdır . . . 70
Disiplinli bir hata ayıklama döngüsü (debugging loop) . . . 70
7. Kullanışlı en küçük HTML belgesi . . . 71
7.1 Öğeler (elements), etiketler (tags), içerik ve öznitelikler (attributes) . . . 71
7.2 HTML anlamsaldır . . . 72
7.3 Belge iskeleti (document skeleton) . . . 72
7.4 Ayrıştırma (parsing) hataya dayanıklıdır; hatasız değildir . . . 73
7.5 HTML, DOM ve View Source ilişkili ancak birbirinden farklıdır . . . 73
8. Git kararları kaydeder . . . 74
Depo, commit, dal (branch) ve uzak depo . . . 74
Commit kalitesi . . . 74

Yönlendirmeli laboratuvar - Yerel olarak gözlemleyin ve yayımlayın . . . 75
Amaç ve kabul ölçütleri . . . 75
Güvenlik ve gizlilik (privacy) . . . 75
Adım 1 - Başlangıç durumunu (baseline) oluşturun . . . 75
Adım 2 - Gezilebilir içerik ekleyin . . . 76
Adım 3 - HTTP üzerinden sunun . . . 76
Adım 3A - Ad çözümlemeyi gözlemleyin . . . 76
Adım 4 - Bir arıza oluşturun ve teşhis edin . . . 76
Adım 5 - Doğrulayın . . . 77
Gözlem kaydı . . . 77
Temizlik . . . 77

Alıştırmalar . . . 78
Temel düzey . . . 78
Alıştırma 1 - URL yapısı . . . 78
Alıştırma 2 - Olayları sıralayın . . . 78
Alıştırma 3 - Yanıtı okuyun . . . 78
Uygulama . . . 78
Alıştırma 4 - Üç gözlemi teşhis edin . . . 78

Alıştırma 5 - Sınırı çizin . . . 78
İleri düzey . . . 78
Alıştırma 6 - Bir sayfa, birden çok istek . . . 78
Alıştırma 7 - Git kanıtı . . . 79
Alıştırma 8 - Bir DNS yanıtını izleyin . . . 79
Alıştırma 9 - Kaynak HTML ve DOM . . . 79
Alıştırma 10 - Yöntem özellikleri . . . 79
Alıştırma 11 - Kökenleri karşılaştırın . . . 79
Alıştırma 12 - Katmana göre teşhis edin . . . 79
Alıştırma 13 - IPv6'yı genişletin, sıkıştırın ve yorumlayın . . . 79

Ayrıntılı çözümler . . . 80
Yanıt 1 . . . 80
Yanıt 2 . . . 80
Yanıt 3 . . . 80
Yanıt 4 . . . 81
Yanıt 5 . . . 81
Yanıt 6 . . . 81
Yanıt 7 . . . 81
Yanıt 8 . . . 81
Yanıt 9 . . . 82
Yanıt 10 . . . 82
Yanıt 11 . . . 82
Yanıt 12 . . . 82
Yanıt 13 . . . 83

Hata ayıklama kliniği . . . 83
Teşhis . . . 83

Bilgiyi geri çağırma sınavı . . . 84
Sınav yanıtları ve kavram yanılgısı notları . . . 84
Öz değerlendirme ölçeği . . . 85

Oturum özeti . . . 85
Sözlük . . . 86
Temel kaynaklar ve ileri okumalar . . . 88
Neler sık değişir? . . . 88
Oturum değerlendirmesi . . . 88
Öğrenme günlüğü notları . . . 89

Oturum 2 - Anlamsal HTML ve Erişilebilir Formlar . . . 90
Oturum haritası . . . 90
Öğrenme çıktıları . . . 90
Kavramlardan önce: Campus Hub'ı bulun, düzenleyin ve görüntüleyin . . . 91

Doğru Campus Hub dosyasını bulun . . . 91
Düz metin kod düzenleyicisi kullanın . . . 91
Görünür tek bir değişiklik yapın ve kaydedin . . . 92
Yerel sunucuyu başlatın ve tarayıcı çıktısını açın . . . 92
Sonuç farklı olduğunda kanıtı okuyun . . . 93
1. Kullanıcının göreviyle başlayın . . . 94
2. HTML bir anlam dilidir . . . 94
2.1 Öğeyi seçmeden önce söz dizimini okuyun . . . 95
2.2 Belge genelindeki anlam html ve head içinde başlar . . . 96
2.3 Yapı, yazı tipi boyutunu değil belgeyi izler . . . 97
2.4 Yer işaretleri sayfayı gezilebilir bölgelere ayırır . . . 97
3. Metin içeriği, bağlantılar ve ortam . . . 98
3.1 Metin öğeleri retorik yapıyı iletir . . . 98
3.2 Listeler gruplamayı ifade eder . . . 99
3.3 Bağlantı metni bağlam içinde anlamlı olmalıdır . . . 99
3.4 Görseller amaçla ilgili bir karar gerektirir . . . 99
3.5 Tarihler, kısaltmalar ve veriler makinece okunabilir değerlere sahip olabilir . . . 100
3.6 Ses ve video eşdeğer erişim gerektirir . . . 100
3.7 Tablolar veriler arasındaki ilişkiler içindir . . . 100
4. Formlar, kısıtları olan konuşmalardır . . . 100
4.1 Gönderim yöntemini işleme göre seçin . . . 102
4.2 Etiketler ve açıklamalar erişilebilir kalır . . . 102
4.3 Uygun en dar yerleşik denetimi seçin . . . 102
4.4 Hangi denetimlerin gönderileceğini tahmin edin . . . 103
4.5 Özellikle dosyalar için kodlamayı seçin . . . 104
4.6 Otomatik tamamlama, girdi amacı ve veri minimizasyonu . . . 104
4.7 Kısıt doğrulaması yararlıdır ancak eksiksiz değildir . . . 105
4.8 JavaScript eklemeden önce hata kurtarmayı tasarlayın . . . 105
Çalıştırmadan önce tahmin edin . . . 105
5. Birbiriyle ilişkili üç gösterim . . . 106
5.1 Erişilebilir ad, açıklama, rol ve durum . . . 106
5.2 Klavye kullanımı bir mühendislik testidir . . . 107
6. Erişilebilirlik uygunluktan daha geniştir . . . 107
Yönlendirmeli laboratuvar - Erişilebilir bir etkinlik formu oluşturun . . . 109
Amaç ve kabul ölçütleri . . . 109
Adım 1 - Düzenlemeden önce okuyun . . . 109
Adım 2 - Yer işaretlerini ve gezinmeyi oluşturun . . . 109
Adım 3 - Formu yerleşik denetimlerden oluşturun . . . 110
Adım 4 - Gönderim sözleşmesini tanımlayın . . . 110
Adım 5 - Kısıtları ifade edin . . . 110
Adım 6 - Fare kullanmadan test edin . . . 110
Adım 7 - Kaynağı doğrulayın ve ağacı inceleyin . . . 110
Adım 8 - Doğrulayın, açıklayın ve sürüm kontrolüne işleyin . . . 110

Laboratuvar gözlem kaydı . . . 110

Alıştırmalar . . . 111

Temel düzey . . . 111

Alıştırma 1 - Öğe seçimi . . . 111

Alıştırma 2 - Etiket ve ad . . . 111

Alıştırma 3 - Alternatif metin . . . 111

Uygulama . . . 111

Alıştırma 4 - Ana hat yapısını düzeltin . . . 111

Alıştırma 5 - Erişilemeyen denetimi teşhis edin . . . 111

Alıştırma 6 - Tablo ilişkileri . . . 112

İleri düzey . . . 112

Alıştırma 7 - Doğrulama sınırı . . . 112

Alıştırma 8 - Erişilebilirlik ağacı üzerine akıl yürütme . . . 112

Alıştırma 9 - Gönderilen veriyi tahmin edin . . . 112

Alıştırma 10 - Bir test stratejisi seçin . . . 112

Alıştırma 11 - Söz dizimini ve anlamı düzeltin . . . 112

Alıştırma 12 - Denetimleri ve kodlamayı seçin . . . 112

Alıştırma 13 - Hata kurtarmayı tasarlayın . . . 112

Alıştırma 14 - Karmaşık bir tabloyu modelleyin . . . 113

Ayrıntılı çözümler . . . 113

Yanıt 1 . . . 113

Yanıt 2 . . . 113

Yanıt 3 . . . 113

Yanıt 4 . . . 113

Yanıt 5 . . . 113

Yanıt 6 . . . 114

Yanıt 7 . . . 114

Yanıt 8 . . . 114

Yanıt 9 . . . 114

Yanıt 10 . . . 114

Yanıt 11 . . . 115

Yanıt 12 . . . 115

Yanıt 13 . . . 115

Yanıt 14 . . . 116

Hata ayıklama kliniği . . . 116

Teşhis . . . 116

Bilgiyi geri çağırma sınavı . . . 116

Sınav yanıtları ve kavram yanılgısı notları . . . 117

Öz değerlendirme ölçeği . . . 118

Oturum özeti 118
Sözlük 119
Temel kaynaklar ve ileri okumalar 120
Neler sık değişir? 121
Oturum değerlendirmesi 121
Oturum 3 - Basamaklı Stil Sayfaları, Yerleşim ve Duyarlı Tasarım 122
Oturum haritası 122
Öğrenme çıktıları 122
1. CSS sunum ve kısıtlar için bir dildir 123
1.1 Bir CSS kuralını eksiksiz bir cümle gibi okuyun 123
1.2 Kurallar, @ kuralları ve kısaltmalar farklı yapılardır 124
1.3 CSS'yi HTML'ye bilinçli biçimde bağlayın 124
1.4 Tarayıcının zaten bir stil sayfası vardır 125
2. Seçiciler eşleşir; basamaklama karar verir 125
2.1 Uzun seçiciler oluşturmadan önce seçici okuryazarlığı geliştirin 125
2.2 Basamaklama birbiriyle yarışan bildirimleri çözümler 126
2.3 Özgüllük, en yükseğe çıkarılacak bir puan değil, karşılaştırmadır 128
2.4 Kalıtım, seçici eşleşmesi değildir 128
2.5 Özel özellikler kararları basamaklama boyunca taşır 128
3. Kutular bildirilmiş değerleri geometriye dönüştürür 129
3.1 display dış ve iç davranışı belirler 129
3.2 İçerik, dolgu, kenarlık ve dış boşluk kutu modelini oluşturur 129
3.3 Normal akış dayanıklı bir varsayılandır 130
3.4 Boyutlandırma içeriğin doğal boyutuyla başlar 130
3.5 Mantıksal özellikler yazı sistemini izler 131
3.6 Dış boşlukların çökmesi ve taşma kanıt sunar 131
3.7 Konumlandırılmış yerleşim başvuru noktasını ve akışı değiştirir 132
4. Değerler türü, ölçeği, rengi ve görsel hiyerarşiyi ifade eder 132
4.1 Birimler ilişkileri iletir 132
4.2 Renk gösterimi, rengin kullanılabilirliğinden ayrıdır 136
4.3 Tipografi yalnızca süslemeyi değil, okumayı denetler 136
4.4 Arka planlar, kenarlıklar ve gölgeler içeriği desteklemelidir 137
4.5 Form denetimleri platform davranışını korur 137
5. İlişkiye göre bir yerleşim modeli seçin 138
5.1 Flexbox öğeleri ana eksen boyunca dağıtır 138
5.2 Grid izleri iki boyutta koordine eder 139
5.3 Hizalama, sıralama değildir 139
6. Duyarlı tasarım bağlamı bir girdi olarak ele alır 139
6.1 Görüntü alanı davranışı HTML'de başlar 140
6.2 Ortam sorguları çevreye yanıt verir 140
6.3 Kapsayıcı sorguları bileşenleri bağlama duyarlı kılar 142
6.4 Görseller ve gömülü ortam 143
6.5 Kullanıcı tercihleri duyarlı girdilerdir 143

7. Erişilebilir sunum gerçek kullanıcı koşullarında işlevini korur . . . 144
7.1 Etkileşim durumları algılanabilir kalmalıdır . . . 144
7.2 Hedef boyutu yerleşimden sonra ölçülür . . . 144
7.3 Yakınlaştırma, metin aralığı ve yeniden akış yerleşim testleridir . . . 144
7.4 Gizleme ve görsel sıra erişimi değiştirebilir . . . 145
7.5 Hareket, karşıtlık ve zorunlu renkler birleşik kanıt gerektirir . . . 145
Çalıştırmadan önce tahmin edin . . . 145
8. CSS mimarisi değişikliği düşük maliyetli kılmalıdır . . . 145
9. CSS'de değerleri tahmin ederek değil, kanıtı izleyerek hata ayıklayın . . . 146
9.1 Doğru tarayıcı panellerini açın . . . 146
9.2 Yatay taşmayı sistemli biçimde izleyin . . . 147

Yönlendirmeli laboratuvar - Duyarlı Campus Hub kabuğunu oluşturun . . . 147
Başlamadan önce . . . 147
Amaç ve kabul ölçütleri . . . 148
Adım 1 - Stil uygulanmamış belgeyi inceleyin . . . 148
Adım 2 - Tasarım değişkenlerini ve genel kısıtları oluşturun . . . 149
Adım 3 - Bileşen yerleşimlerini oluşturun . . . 149
Adım 4 - İçerik odaklı bir geliştirme ekleyin . . . 149
Adım 5 - Taşma denetimi yapın . . . 149
Adım 6 - Tercihleri ve etkileşimi doğrulayın . . . 149

Alıştırmalar . . . 150
Temel düzey . . . 150
Alıştırma 1 - Basamaklama kararı . . . 150
Alıştırma 2 - Kutu hesabı . . . 150
Alıştırma 3 - Yazı tipine göreli birimleri hesaplayın ve seçin . . . 151
Uygulama . . . 151
Alıştırma 4 - Taşmayı düzeltin . . . 151
Alıştırma 5 - Flexbox mı Grid mi? . . . 151
Alıştırma 6 - Odak gerilemesi . . . 151
İleri düzey . . . 151
Alıştırma 7 - Kaynak sırası . . . 151
Alıştırma 8 - Duyarlı tasarım üzerine akıl yürütme . . . 151
Alıştırma 9 - Stil uygulanmamış bir sayfayı teşhis edin . . . 151
Alıştırma 10 - Seçicileri bilinçli biçimde eşleştirin . . . 151
Alıştırma 11 - Kalıtım ve CSS genel anahtar sözcükleri . . . 151
Alıştırma 12 - Konumlandırma ve katmanlama . . . 152
Alıştırma 13 - Ortam sorgusu mu kapsayıcı sorgusu mu? . . . 152
Alıştırma 14 - Erişilebilir denetim biçimlendirmesi . . . 152
Alıştırma 15 - Geliştirici araçları kanıt planı . . . 152
Alıştırma 16 - Mimari inceleme . . . 152

Ayrıntılı çözümler . . . 152

Yanıt 1 . . . 152
Yanıt 2 . . . 152
Yanıt 3 . . . 153
Yanıt 4 . . . 153
Yanıt 5 . . . 153
Yanıt 6 . . . 153
Yanıt 7 . . . 153
Yanıt 8 . . . 153
Yanıt 9 . . . 154
Yanıt 10 . . . 154
Yanıt 11 . . . 154
Yanıt 12 . . . 154
Yanıt 13 . . . 155
Yanıt 14 . . . 155
Yanıt 15 . . . 155
Yanıt 16 . . . 155

Hata ayıklama kliniği . . . 156

Teşhis . . . 156

Bilgiyi geri çağırma sınavı . . . 156

Sınav yanıtları ve kavram yanılgısı notları . . . 157

Öz değerlendirme ölçeği . . . 158

Oturum özeti . . . 158

Sözlük . . . 159
Temel kaynaklar ve ileri okumalar . . . 160
Neler sık değişir? . . . 161
Oturum değerlendirmesi . . . 161
Öğretim elemanı kılavuzu . . . 161

Önerilen beş saatlik sıra . . . 161
Kavram yanılgılarını görünür kılan gösterimler . . . 162
Dikkat edilmesi gereken yaygın kavram yanılgıları . . . 162
Uyarlamalar ve alternatifler . . . 163
Biçimlendirici değerlendirme ölçütü . . . 163

Oturum 4 - İlk İlkelerden JavaScript . . . 164

Oturum haritası . . . 164
Ön koşullar . . . 164
Öğrenme çıktıları . . . 164

0. Bilgisayarı hazırlayın ve ilk JavaScript kodunuzu çalıştırın . . . 165

0.1 Beş aracı kurmadan önce anlayın . . . 165

0.2 Node.js'i Windows'a adım adım kurun 166
0.3 Visual Studio Code'u kurun veya doğrulayın 167
0.4 Doğru terminali açın ve geçerli dizinini kanıtlayın 167
0.5 İlk Node.js programını oluşturun ve çalıştırın 168
0.6 JavaScript'i bir tarayıcıda çalıştırın 169
0.7 Baştan başlamak yerine ilk hataları okuyun 170

1. Program, makine için sıralı bir açıklamadır 170
1.1 ECMAScript ve JavaScript 171
1.2 İlk dosyanız, tam olarak belirtilen bir dizinden 171
1.3 Çalıştırma sırası, açıklamalar ve büyük-küçük harf duyarlılığı 171

2. Değerlerin türleri, bağların adları vardır 172
2.1 İfadeler değer üretir 173
2.2 const ve let 173
2.3 Dizgeler ve şablon değişmezleri 174

3. İşleçler ilişkiler oluşturur 174
3.1 Dönüştürme görünür olmalıdır 174
3.2 Doğruluk değeri ve kısa devre değerlendirmesi 175

4. Denetim akışı işi seçer ve yineler 175
4.1 Döngüler değişen bir durumla yineler 176

5. İşlevler bir hesaba ad ve sınır verir 176
5.1 Bildirimler, ifadeler ve ok işlevleri 177
5.2 Kapsam, bir adın nerede kullanılabildiğini belirler 177

6. Diziler sıralı koleksiyonları modeller 178
6.1 Dönüşümler amacı ifade eder 178

7. Nesneler adlandırılmış olguları modeller 178
7.1 İlkel değerlerin kopyalanması ve nesne kimliği 179

8. Hatalar hüküm değil, kanıttır 179

9. Küçük testler davranışı yinelenebilir kılar 180

Yönlendirmeli laboratuvar - Campus Hub etkinlik uygunluğunu modelleyin 181
Amaç ve kanıt 181
Adım 1 - Başlangıç durumunu oluşturun 181
Adım 2 - Etkinlik yapısını okuyun 181
Adım 3 - Her seferinde tek bir işlev yazın 181
Adım 4 - Koleksiyonu dönüştürün 181

Adım 5 - Sınırı doğrulayın . . . 181
Adım 6 - Çıktıyı kodla karşılaştırın . . . 181
Adım 7 - Kanıtı sürüm kontrolüne işleyin . . . 182

Alıştırmalar . . . 183

Temel düzey . . . 183
Alıştırma 1 - Değerler ve türler . . . 183
Alıştırma 2 - Bağ değişikliği mi nesne değişikliği mi? . . . 183
Alıştırma 3 - Katı karşılaştırma . . . 183
Alıştırma 4 - Bir dalı izleyin . . . 183
Uygulama . . . 183
Alıştırma 5 - Tüm girdiler için tanımlı bir işlev yazın . . . 183
Alıştırma 6 - Dizi işlem hattı . . . 183
Alıştırma 7 - Kapsam teşhisi . . . 183
Alıştırma 8 - Kimlik teşhisi . . . 183
İleri düzey . . . 183
Alıştırma 9 - Sınır tablosu . . . 183
Alıştırma 10 - Hatalı bir hesabın hatasını ayıklayın . . . 183
Alıştırma 11 - Döngü mü dönüşüm mü seçin . . . 184
Alıştırma 12 - Test kalitesi . . . 184
Alıştırma 13 - Kod hatasını ayıklamadan önce ortamı kanıtlayın . . . 184
Alıştırma 14 - Kaynağı, komutu ve çıktıyı ayırın . . . 184
Alıştırma 15 - Tarayıcı betiğinin yüklenmesini izleyin . . . 184

Ayrıntılı çözümler . . . 184

Yanıt 1 . . . 184
Yanıt 2 . . . 184
Yanıt 3 . . . 184
Yanıt 4 . . . 184
Yanıt 5 . . . 185
Yanıt 6 . . . 185
Yanıt 7 . . . 185
Yanıt 8 . . . 185
Yanıt 9 . . . 185
Yanıt 10 . . . 185
Yanıt 11 . . . 186
Yanıt 12 . . . 186
Yanıt 13 . . . 186
Yanıt 14 . . . 186
Yanıt 15 . . . 186

Hata ayıklama kliniği . . . 186

Belirti 1 - ReferenceError: remaining is not defined . . . 186
Belirti 2 - TypeError, undefined değerinin bir özelliğini okumaktan söz ediyor . . . 187

Belirti 3 - Hesap rakamları art arda birleştiriyor . . . 187
Belirti 4 - Bir test başka bir testin verisini değiştirdi . . . 187

Bilgiyi geri çağırma sınavı . . . 187

Sınav yanıtları ve kavram yanılgısı notları . . . 187

Oturum özeti . . . 188

Sözlük . . . 189
Temel kaynaklar ve ileri okumalar . . . 189
Neler sık değişir? . . . 190
Öğretim elemanı kılavuzu . . . 190
Önerilen sekiz saatlik sıra . . . 190
Kavram yanılgılarını görünür kılan gösterimler . . . 190
Değerlendirme ve kapsayıcılık . . . 190

Ek A - Test edilmiş laboratuvar başvuru çözümleri . . . 191

Hafta 01 - index.html . . . 191
Hafta 02 - index.html . . . 192
Hafta 03 - index.html . . . 194
Hafta 03 - styles.css . . . 195
Hafta 04 - events.js . . . 197
Hafta 04 - events.test.js . . . 198
Hafta 04 - hello-browser.html . . . 198
Hafta 04 - hello-browser.js . . . 198
Hafta 04 - hello.js . . . 199
Hafta 04 - index.html . . . 199
Hafta 04 - styles.css . . . 199

Ek B - Hızlı komutlar . . . 200

Yönlendirme - Web Programlama Çalışma İstasyonunuzu Hazırlayın

Temel soru: Bir öğrenci, ilk programlama laboratuvarından önce bilgisayarının web programlarını düzenlemeye, çalıştırmaya, incelemeye, test etmeye ve korumaya hazır olduğunu nasıl kanıtlayabilir?

Bu yönlendirme, geliştirici araçlarıyla ilgili daha önce deneyiminiz olmadığını varsayar. Laboratuvarlara getireceğiniz bilgisayarda baştan sona uygulayın. Bir yükleyici *Finished* dediğinde durmayın. Kurulum ancak aracı çalıştırabildiğinizde, sürümünü gözlemleyebildiğinizde, ders sayfasını sunabildiğinizde, isteği inceleyebildiğinizde, testleri çalıştırabildiğinizde ve bir Git commit'i kaydedebildiğinizde tamamlanmış sayılır.

Yönlendirme haritası

Tablo 0.1, hazırlık görevini ve hazır oluşu gösteren kanıtları tanımlar.

Tablo 0.1 - Çalışma istasyonu hazırlığının kapsamı ve kanıtları

Öğe	Plan
Hedef kitle	Web geliştirme araçları konusunda önceden deneyim sahibi olduğu varsayılmayan öğrenciler
Süre	İndirme hızına ve işletim sistemine bağlı olarak yaklaşık 90-150 dakika
Gerekli sonuç	Düzenleyicide klasör olarak açılmış, çalışan bir yerel ders deposu
Birincil programlama dili	Önce tarayıcıda, daha sonra sunucudaki Node.js ortamında modern JavaScript
Çalışma zamanı	npm'yi içeren Node.js 24 Uzun Süreli Destek (Long-Term Support, LTS)
Temel kanıt	Araç sürümleri, başarılı ders testleri, yerel HTTP yanıtı, Network paneli gözlemi ve bir Git commit'i
Yardım isteme eşiği	Tam komutu, hata metninin tamamını, işletim sistemini ve denenen çözümü kaydettikten sonra laboratuvar desteği isteyin

Öğrenme çıktıları

Bu yönlendirmeyi tamamladıktan sonra:

1 düzenleyiciyi, terminali, çalışma zamanını, paket yöneticisini, sunucuyu, tarayıcıyı, geliştirici araçlarını, test çalıştırıcısını ve sürüm kontrol sistemini birbirinden ayırt edebilecek;

2 Windows, macOS veya Linux üzerinde desteklenen bir tarayıcıyı, Visual Studio Code'u, Git'i, Node.js 24 LTS'yi ve npm'yi kurup doğrulayabilecek;

3 geçerli dizini belirleyebilecek, klasörler arasında güvenle hareket edebilecek ve görüntülenen komut istemi simgesini yazmadan bir komut çalıştırabilecek;

4 klasör yapısını kaybetmeden ders deposunu edinebilecek ve açabilecek;

5 ders doğrulama paketini ve yerel geliştirme sunucusunu çalıştırabilecek;

6 tarayıcı geliştirici araçlarında bir localhost isteğini inceleyebilecek;

7 Git kimliğini yapılandırabilecek, küçük ve anlamlı bir commit oluşturabilecek ve bunu onaylanmış uzak depoya gönderebilecek;

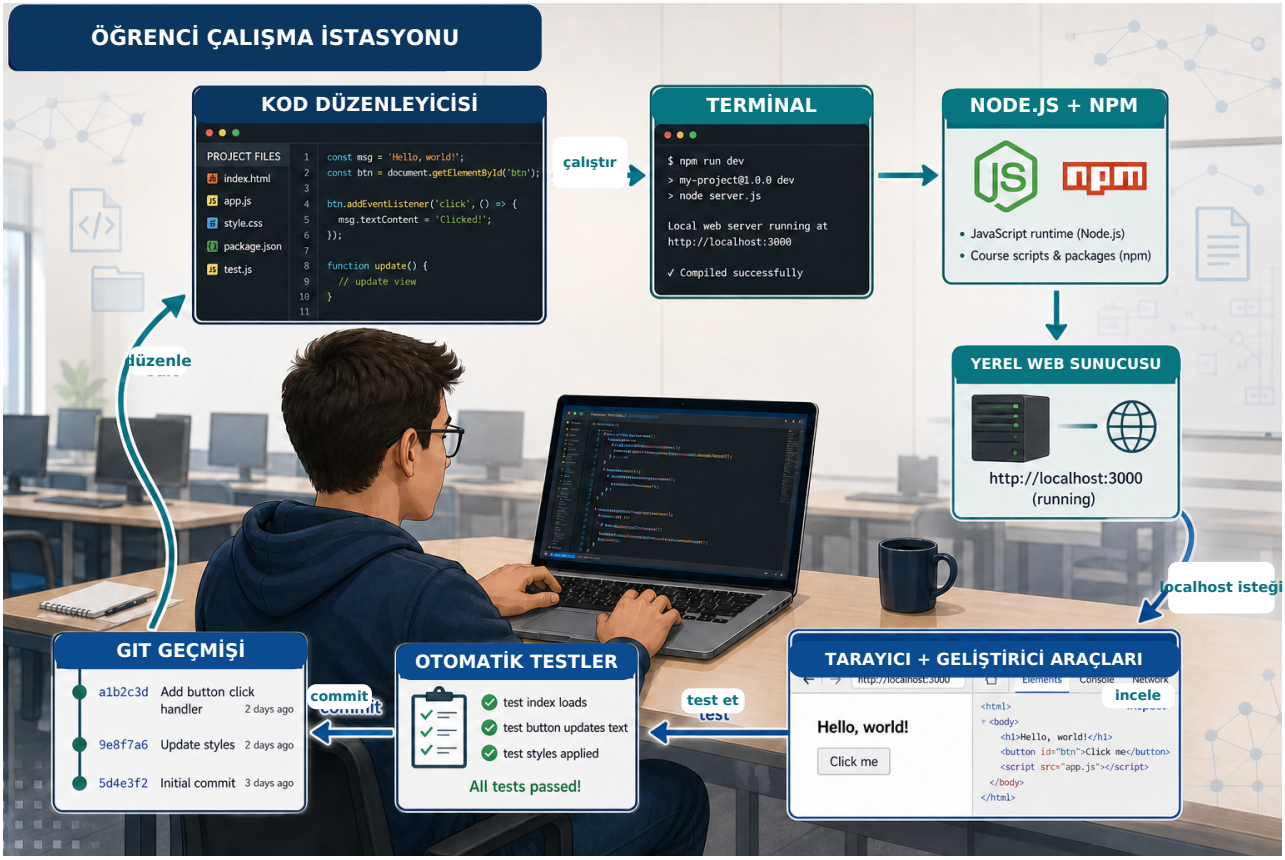
8 JavaScript dilini, onu çalıştıran tarayıcı ve Node.js ortamlarından ayırt edebilecek ve

9 parolaları, erişim belirteçlerini veya öğrencilere ait özel bilgileri açığa çıkarmadan yararlı tanılama kanıtları toplayabileceksiniz.

1. Araçları kurmadan önce çalışma istasyonunu anlayın

Bir web programlama çalışma istasyonu tek bir büyük uygulama değil, iş birliği yapan programlar kümesidir. Kod düzenleyicisi metin dosyalarını değiştirir. Terminal, komutları çalıştırmak için metin tabanlı bir arayüz sağlar. Node.js, JavaScript'i tarayıcının dışında çalıştırır. Node.js ile birlikte kurulan npm, bir projenin tanımladığı betikleri çalıştırır. Yerel web sunucusu HTTP isteklerini dinler. Tarayıcı kaynakları ister ve görüntüler; geliştirici araçları ise kanıtları görünür kılar. Otomatik testler belirtilmiş özellikleri denetler. Git, projenin seçilmiş sürümlerini kaydeder.

Şekil 0.2, bu kitap boyunca kullanılacak çalışma döngüsünü gösterir. Şekli saat yönünde okuyun. Düzenleme işlemi proje dosyalarını değiştirir; terminal komutu, Node.js destekli yerel sunucuyu çalıştırır; tarayıcı `localhost` adresinden istekte bulunur; geliştirici araçları incelemeyi destekler; testler projeyi denetler ve bilinçli bir Git commit'i anlamlı bir kontrol noktasını korur. Oklar bir çalışma sürecini anlatır; araçların tek bir program olduğu anlamına gelmez.



Şekil 0.2. Ders çalışma istasyonu; düzenleme, komut çalıştırma, Node.js çalışma zamanı, yerel sunum, tarayıcı incelemesi, otomatik testler ve Git geçmişini ayrı fakat iş birliği yapan sorumluluklar olarak korur.

1.1 Gerekli donanım ve erişim

Tablo 0.2, gereksinimleri yararlı eklerden ayırır. Ders; pahalı bir oyun bilgisayarı, ücretli bir düzenleyici, herkese açık bir sunucu veya ikinci bir monitör gerektirmez.

Tablo 0.2 - Ders için donanım ve erişim

Öğe	Gerekli başlangıç düzeyi	Varsa önerilen
Bilgisayar	Fiziksel klavyeli, desteklenen 64 bit Windows, macOS veya Linux dizüstü ya da masaüstü bilgisayar	Derse getirilebilen ve harici ekrana bağlanabilen bir dizüstü bilgisayar
Bellek	Temel laboratuvarlar için 8 GB RAM	Daha sonraki veri tabanları, tarayıcı araçları ve eş zamanlı hizmetler için 16 GB veya daha fazlası
Depolama	İşletim sistemi güncellemelerinden sonra en az 10 GB boş alan	Depoların, paket önbelleklerinin, veri tabanı dosyalarının ve ekran görüntülerinin alan için yarışmaması amacıyla 20 GB boş alan
İşlemci	Güncel olarak desteklenen x64 veya ARM64 işlemci	Eş zamanlı araçların daha akıcı çalışması için dört veya daha fazla mantıksal çekirdek
Ekran	Tarayıcı yakınlaştırmasına elverişli, okunabilir ekran	İkinci ekran yararlıdır ancak hiçbir zaman zorunlu değildir
Ağ	Kurulum, belgeler ve depo eşitlemesi için güvenilir İnternet erişimi	Kablosuz hizmet kararsızsa büyük indirmeler için kablolu bağlantı
Girdi ve ses	Fiziksel klavye; yerleşik işaretleme aygıtı yeterlidir	Ortam ve ekran okuyucu alıştırmaları için kulaklık; harici fare isteğe bağlıdır
İzinler	Onaylı yazılımları kurma izni veya yardımcı olabilecek bir yöneticiye erişim	Günlük kullanım için standart hesap ve kurulum gerektirdiğinde ayrı yetki yükseltme
Hesaplar	Üniversite hesabı ve ders deposuna ya da öğrenme sistemine erişim	Uzağa gönderim gerekiyorsa çok faktörlü kimlik doğrulamayla korunan Git barındırma hesabı

Tablet veya telefon duyarlı tasarım testleri için değerlidir; ancak birincil çalışma istasyonunun yerini tutmaz. Ders; dosya sistemi, terminal, yerel sunucu, geliştirici araçları ve Git gerektirir. Yönetilen bir laboratuvar bilgisayarı kullanan öğrenciler, oturum kapatıldıktan sonra çalışmaların korunup korunmadığını doğrulamalı; ayrılmadan önce onaylanmış çalışmaları kalıcı depolamaya göndermeli veya kopyalamalıdır.

1.2 Hazırladığınız bilgisayarı kaydedin

Yazılım indirmeden önce şunları kaydedin:

- işletim sisteminin adı ve sürümü;

- genellikle x64 veya ARM64 olan işlemci mimarisi;

- kullanılabilir bellek ve depolama;

- hesabın uygulama kurup kuramadığı ve

- ders çalışmalarının saklanacağı klasör.

Yeniden bulabileceğiniz kısa bir yerel yol kullanın. Windows için `C:\Users\Ada\Documents\web-course`, macOS için `/Users/ada/Documents/web-course` örnek verilebilir. Etkin depoyu, eşitleme yazılımının üretilen dosyaları sürekli kilitlediği bir klasöre yerleştirmeyin. Doğrudan bir e-posta ekinde, sıkıştırılmış ZIP arşivinde veya tarayıcının geçici indirme önizlemesinde çalışmayın.

2. İşletim sistemini ve dosya görünümünü hazırlayın

1 Bekleyen güvenlik güncellemelerini kurun ve bilgisayarı yeniden başlatın.

2 Tarihin, saatin ve saat diliminin doğru olduğunu doğrulayın. Yanlış saatler sertifika denetimlerini, Git zaman damgalarını ve kimlik doğrulamayı bozabilir.

3 Kendi kullanıcı hesabınızda ders çalışmaları klasörünü oluşturun.

4 Dosya yöneticisini tam dosya adlarını ve uzantıları gösterecek biçimde yapılandırın. `index.html.txt` adlı dosya bir HTML belgesi değil, metin dosyasıdır.

5 UTF-8 metin dosyaları oluşturulabildiğinden emin olun. Deponun `.editorconfig` dosyası daha sonra uyumlu düzenleyiciden UTF-8, Satır Besleme (Line Feed, LF) satır sonları, iki boşluklu girinti, dosya sonunda yeni satır ve satır sonunda gereksiz boşluk bulunmamasını ister.

6 Kurulum kanıtının nerede saklanacağına karar verin. Kişisel ders notlarınızın içinde `setup-evidence.txt` adlı düz metin dosyası yeterlidir. Parolaları, kurtarma kodlarını veya erişim belirteçlerini kaydetmeyin.

Windows 11'de Dosya Gezgini içinde Görünüm > Göster > Dosya adı uzantıları seçeneği bulunur. macOS'ta Finder içinde Ayarlar > İleri Düzey > Tüm dosya adı uzantılarını göster seçeneği bulunur. Linux dosya yöneticileri farklılık gösterir; önemli kanıt, `index.html` ile `index.html.txt` dosyalarını birbirinden ayırt edebilmenizdir.

3. Güncel bir web tarayıcısı kurun

Chrome, Edge veya Firefox'un güncel kararlı sürümünü tarayıcının resmî sitesinden kurun. İlk tarayıcı, dersin birincil inceleme ortamıdır. İkinci bir tarayıcı motoru daha sonra uyumluluk denetimleri için yararlıdır; ancak ilk laboratuvar için gerekli değildir.

1 Yükleyiciyi resmî yayıncıdan indirin.

2 Sitenin HTTPS kullandığını ve yayıncı adının doğru olduğunu denetleyin.

3 Yükleyiciyi çalıştırın ve tarayıcının otomatik güncellenmesine izin verin.

4 Tarayıcıyı açıp `https://example.com/` adresini ziyaret edin.

5 Geliştirici araçlarını açın. Windows ve Linux'ta F12 veya Ctrl+Shift+I; macOS'ta Command+Option+I kullanın.

6 Network panelini seçin, sayfayı yeniden yükleyin ve bir belge isteğinin göründüğünü doğrulayın.

Panelin tam yerleşimi sürümler arasında değişir. Rengi veya ekrandaki konumu değil, kanıtı öğrenin: istek URL'si, yöntem, durum, yanıt başlıkları, zamanlama ve başlatan öğe. Çok sayıda tarayıcı uzantısı kurmayın. Bir uzantı istekleri değiştirebilir, içerik ekleyebilir, bellek tüketebilir veya laboratuvar gözlemini karıştırabilir. Sonuçlar şaşırtıcıysa temiz bir tarayıcı profili kullanın.

4. Visual Studio Code'u kurun ve ortamı tanıyın

Bu kitap, desteklenen üç masaüstü platformunda da kullanılabildiği için Visual Studio Code'u (VS Code) kullanır. VS Code bir metin düzenleyicisidir; Visual Studio ile aynı ürün değildir. Öğrenciler başka bir yetenekli düzenleyici kullanabilir; ancak öğretim elemanları yayımlanan adımların yalnızca VS Code ile eşleşmesini garanti edebilir.

4.1 Windows

1 Resmî VS Code indirme sayfasını açın.

2 Doğru x64 veya ARM64 mimarisi için User Installer paketini indirin.

3 Yükleyiciyi çalıştırın. Sunulduğunda VS Code'u PATH'e ekleyen ve klasörün Code ile açılmasına izin veren seçenekleri koruyun.

4 Güncellenmiş PATH'in okunması için kurulumdan sonra PowerShell'i kapatıp yeniden açın.

4.2 macOS

1 Resmî VS Code sitesinden macOS sürümünü indirin. Universal sürüm hem Intel hem de Apple silicon işlemcileri destekler.

2 İndirilen disk görüntüsünü açın ve `Visual Studio Code.app` uygulamasını Applications klasörüne sürükleyin.

3 VS Code'u Applications klasöründen başlatın.

4 Command+Shift+P ile Command Palette'i açın, `Shell Command: Install 'code' command in PATH` komutunu çalıştırın ve Terminal'i yeniden başlatın.

4.3 Linux

1 Debian veya Ubuntu'da resmî `.deb` paketini, Fedora veya Red Hat uyumlu sistemlerde resmî `.rpm` paketini ya da dağıtımınız için belgelenmiş paketi kullanın.

2 Güncellemelerin olağan biçimde ulaştırılabilmesi için dağıtımın paket aracıyla kurun.

3 Kurulumdan sonra yeni bir terminal başlatın.

4.4 Düzenleyiciyi doğrulayın

Yeni bir terminalde şu komutu çalıştırın:

```
code --version
```

İlk satır bir sürüm numarası olmalıdır. Grafik düzenleyici açılıyor ancak komut tanınmıyorsa VS Code kurulmuştur; fakat komut satırı başlatıcısı PATH üzerinde değildir. İlgili platform adımını yineleyin. Geçici olarak File > Open Folder seçeneğini kullanarak devam edebilirsiniz; ancak sonraki otomasyon alıştırmalarından önce komutu düzeltin.

Yalnızca tek bir HTML dosyasını değil, bir klasörü açın. Explorer alanı proje ağacını göstermelidir. VS Code; temel HTML, CSS, JavaScript, JSON, Markdown, terminal ve Git desteğini içerir; ilk üç oturum için uzantı gerekmez. Uzantılar çalıştırılabilir yazılımlardır. Yalnızca öğretim elemanının onayladığı uzantıyı kurun, yayıncısını ve izinlerini inceleyin, ders sonuçlarını değiştirirse kaldırın. Yapay zekâ ile tamamlama isteğe bağlıdır ve üniversitenin değerlendirme ve gizlilik politikasına uymalıdır; bu kitap için hiçbir zaman ön koşul değildir.

5. Asgari terminal modelini öğrenin

Terminal penceresi çıktıyı gösterir ve bir kabuk aracılığıyla komutları kabul eder. Bu kitapta Windows öğrencileri PowerShell kullanır. macOS genellikle Z shell (zsh), Linux ise Bash kullanır. Kabukların söz dizimi farklıdır; bu nedenle platformunuz için etiketlenmiş kod bloğunu kullanın.

Komut istemi, ardından > veya $ bulunan bir yol gösterebilir. Öğreticide gösterilen komut istemi simgesini yazmayın. Yalnızca komutu yazın. Çalıştırmak için Enter'a basın. Komutlar geçerli çalışma dizinine göre işlem yapar; bu nedenle silme, taşıma, kurma veya sunucu başlatma işleminden önce dizini doğrulayın.

Tablo 0.3 - Temel gezinme komutları

Amaç	Windows Komut İstemi	Windows PowerShell	macOS veya Linux kabuğu
Geçerli dizini göster	cd	Get-Location	pwd
Öğeleri listele	dir	Get-ChildItem	ls
Klasöre gir	cd /d "full-path"	Set-Location "full-path"	cd "full-path"
Üst klasöre geç	cd ..	Set-Location ..	cd ..
Klasör oluştur	mkdir web-course	New-Item -ItemType Directory web-course	mkdir web-course
Ekranı temizle	cls	Clear-Host	clear
Ön plandaki sunucuyu durdur	Ctrl+C	Ctrl+C	Ctrl+C

Boşluk içeren yollar tırnak içine alınmalıdır; örneğin `Set-Location "C:\Users\Ada\Course Work"` veya `cd "/Users/ada/Course Work"`. Sekme ile tamamlama yazım hatalarını azaltır: dosya adının ilk birkaç karakterini yazıp Tab tuşuna basın. Yukarı Ok önceki komutu geri getirir. Bilmediğiniz bir komutta Enter'a basmadan önce komutun ne okuyacağını, değiştireceğini, oluşturacağını veya kaldıracağını açıklayın.

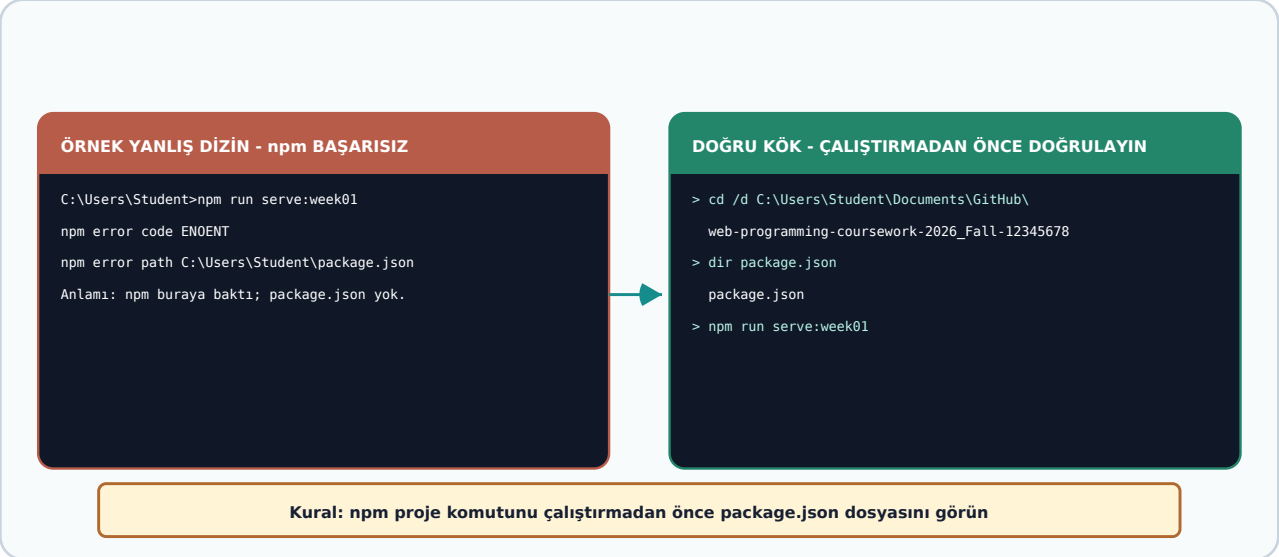
5.1 Proje kökü soyut bir fikir değil, bir konumdur

Her terminal o anda bir klasörün içindedir. `npm run ...` gibi bir komut, aklınızdaki projeyi bulmak için tüm bilgisayarı aramaz. npm, terminalin geçerli klasöründen başlar ve projenin `package.json` dosyasını orada bulmayı bekler. Bu dersteki öğrenci çalışmalarında depo kökü; `package.json`, `labs` ve `scripts` öğelerini içeren dıştaki `web-programming-coursework-2026_Fall-STUDENT_NUMBER` klasörüdür. Klasör adının son kısmı, `STUDENT_NUMBER` değişmez metni değil, öğrencinin gerçek üniversite numarasıdır.

Varsayımsal bir örnekte Komut İstemi'nin şöyle başladığını düşünün:

```
C:\Users\Student>
```

`>` işaretinden önceki yol, geçerli klasörün `C:\Users\Student` olduğunu belirtir. Bu, ders deposu değildir. npm burada çalıştırılırsa `package.json` dosyasını bu dizinde arar ve `C:\Users\Student\package.json` gibi bir yol bildirebilir. Bu, başarılı yönlendirme işleminde görünmesi gereken bir çıktı değil, sorun giderme örneğidir. Şekil 0.3 varsayımsal yanlış konumu doğru depo köküyle karşılaştırır.



Şekil 0.3. npm, package.json dosyasını terminalin geçerli proje dizininde arar.

Her proje komutundan önce bu dört parçalı denetimi kullanın:

Adım 1 - Proje klasörünü bulun. Dosya Gezgini'nde `package.json` dosyasını görünür biçimde içeren klasöre gidin. Student adlı bir öğrenci depoyu olağan GitHub klasörüne klonladıysa örnek konum `C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678` olur. Buradaki `12345678` kurgusal bir öğrenci numarasıdır. Windows hesap adınız, öğrenci numaranız ve seçtiğiniz klonlama konumu farklı olacaktır.

Adım 2 - Bu klasöre girin. Komut İstemi'nde aşağıdaki komutu tam olarak yazın. `/d`, `cd` komutunun klasörle birlikte sürücüyü de değiştirmesini sağlar:

```
cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"
```

Adım 3 - Konumu kanıtlayın. Önce `cd`, ardından `dir package.json` komutunu çalıştırın. İlk komut depo yolunu yazdırmalı, ikincisi `package.json` dosyasını listelemelidir. İkinci komut `File Not Found` diyorsa durun. Henüz npm'yi çalıştırmayın.

Adım 4 - Proje komutunu yalnızca kanıt başarılı olduktan sonra çalıştırın. Örneğin `npm test` veya `npm run serve:week01` komutunu çalıştırın.

Kullanışlı bir görsel yol da vardır. Dosya Gezgini'nde doğru proje klasörünü açın, adres çubuğuna tıklayın, `cmd` yazın ve Enter'a basın. Windows, Komut İstemi'ni doğrudan bu klasörde açar. Alternatif olarak Windows bu komutu sunuyorsa klasör içinde boş bir alana sağ tıklayıp Open in Terminal seçeneğini kullanın. Yine de `dir package.json` komutunu çalıştırın; görsel gezinme de yanlış iç klasörde sona erebilir.

6. Git'i kurun ve yapılandırın

Git; kontrol noktaları, inceleme, iş birliği ve kurtarma için kullanılan sürüm kontrol sistemidir. Git ile GitHub gibi bir barındırma hizmeti ilişkili ancak farklıdır: Git yerel olarak çalışır; barındırma hizmeti uzak depoları saklar ve hesap, inceleme ile iş birliği özellikleri ekler.

6.1 Git'i kurun

- **Windows:** Git for Windows'u resmî Git kurulum sayfasından indirin. Standart seçenekler bu ders için uygundur. PowerShell ve VS Code'un Git'i bulmasına izin verilmelidir.

- **macOS:** `git --version` komutunu çalıştırın. macOS, Command Line Tools paketini kurmayı önerebilir. Alternatif olarak güncel resmî Git yükleyicisini veya kurum tarafından onaylanmış paket yöneticisini kullanın.

- **Debian veya Ubuntu:** `sudo apt update`, ardından `sudo apt install git` komutunu çalıştırın.

- **Fedora:** `sudo dnf install git` komutunu çalıştırın.

Dağıtımınız listelenmemişse dağıtımın karşılık gelen destekli komutunu kullanın. Bir reklamdan veya incelenmemiş forumdan kurulum komutu kopyalamayın.

6.2 Kurulumu doğrulayın ve yazarı tanımlayın

Şu komutu çalıştırın:

```
git --version
```

Ardından yeni commit'lerde saklanması gereken kimliği yapılandırın:

```
git config --global user.name "Your Name"
git config --global user.email "your-address@example.edu"
git config --global init.defaultBranch main
```

Örnekleri kendi onaylanmış kimliğinizle değiştirin. Herkese açık bir depo kişisel e-posta adresini göstermemeliyse dersin barındırma hizmetinin sunduğu gizlilik adresini kullanın. Sonucu doğrulayın:

```
git config --global --list
```

Depoda metin dosyalarını standartlaştıran `.editorconfig` kuralları bulunur. Depoyu edindikten sonra deponun içinde `git config core.autocrlf input` komutunu çalıştırın. Bu, ilgisiz depoları değiştirmeden LF satır sonlarının korunmasına yardımcı olan, depoya özgü bir ayardır.

Git kimlik bilgileri commit kimliğiyle aynı değildir. Bir parolayı, kişisel erişim belirtecini, özel anahtarı veya çok faktörlü kimlik doğrulama kurtarma kodunu hiçbir zaman kaynak dosyasına koymayın. Uzak erişim tanıtıldığında barındırma hizmetinin desteklediği kimlik bilgisi yöneticisini veya Güvenli Kabuk (Secure Shell, SSH) iş akışını kullanın ve çok faktörlü kimlik doğrulamayı etkinleştirin.

7. Node.js 24 LTS ve npm'yi kurun

Node.js, tarayıcının dışındaki bir JavaScript çalışma zamanıdır. npm, Node.js ile kurulan paket yöneticisi ve proje betiği çalıştırıcısıdır. Depo, `package.json` içinde `>=24 <25` aralığını bildirir; `.nvmrc` ise test edilen v24 sürümünü kaydeder. Daha önce kurulmuş olması nedeniyle kullanım ömrü sona ermiş bir Node.js sürümünü bunun yerine kullanmayın.

Bu kitabın teknik inceleme tarihinde Node.js 24 bir LTS sürümüdür. Yeni bir akademik yıldan önce bakım durumunu her zaman resmî Node.js sürümleri sayfasından doğrulayın.

7.1 Windows ve macOS yükleyici yolu

1 Resmî Node.js indirme sayfasını açın.

2 Bilgisayarın x64 veya ARM64 mimarisi için mevcut en güncel 24.x LTS yükleyicisini seçin.

3 Yükleyiciyi çalıştırın; npm ve PATH bütünleştirmesini etkin bırakın.

4 Tüm eski terminal pencerelerini kapatın ve yeni bir tane açın.

7.2 Linux yolu

Dağıtımınız için resmî Node.js indirme sayfasından bağlantı verilen kurulum yöntemini veya kurumca onaylanmış bir Node sürüm yöneticisini kullanın. Varsayılan dağıtım depoları daha eski bir ana sürüm içerebilir; bu nedenle sürümü doğrulamak zorunludur. Bir forumdan kopyalanmış ve incelemediğiniz `curl ... | sudo bash` komutunu kullanmayın.

7.3 İsteğe bağlı sürüm yöneticisi yolu

Farklı projeler farklı Node.js ana sürümleri gerektirdiğinde sürüm yöneticisi yararlıdır. Yönetici ve komutlar Windows ile POSIX sistemlerinde farklı olduğundan, ortak sınıf başlangıç düzeyi yerine isteğe bağlıdır. Kullanıyorsanız `.nvmrc` dosyasında kayıtlı sürümü kurun, geçerli kabukta etkinleştirin ve aşağıdaki doğrulama komutlarının tümünü yine de çalıştırın. Sürüm yöneticisi Node.js'in yerini almaz; kabuğun kurulu Node.js sürümlerinden hangisini kullandığını seçer.

7.4 Node.js ve npm'yi doğrulayın

Şu komutları çalıştırın:

```
node --version
npm --version
```

Node.js çıktısı `v24.` ile başlamalıdır. npm kendi sürümünü yazdırmalıdır. Node.js ile npm ayrı paketler olduğu için iki sayının eşleşmesi beklenmez. `node` bir terminalde çalışıp diğerinde çalışmıyorsa yazılımı yeniden kurmadan önce kabukları karşılaştırın, VS Code'u yeniden başlatın ve PATH'i inceleyin.

Sıradan ders paketleri için `sudo npm install -g ...` komutunu veya yükseltilmiş PowerShell'i kullanmayın. Bu depo şu anda temel testler için üçüncü taraf paket kurulumu gerektirmez. Sonraki bölümler, `package.json` ve bir kilit dosyasında kayıtlı yerel proje bağımlılıklarını kullanacaktır.

7.5 JavaScript dersin birincil programlama dilidir

Bu ders web programlamayı öncelikle JavaScript ile öğretir. JavaScript programlama dilidir; tarayıcı ve Node.js ise bu dilde yazılmış programları çalıştırabilen iki farklı ortamdır. Modern JavaScript'in standartlaştırılmış tanımına ECMAScript denir. Geliştiriciler gündelik konuşmada, bu standarda uyan programlarla çalışma ortamının sağladığı olanakların bütününden söz ederken JavaScript adını kullanır.

Bu ayrım önemlidir; çünkü dil ile ortam farklı yetenekler sağlar:

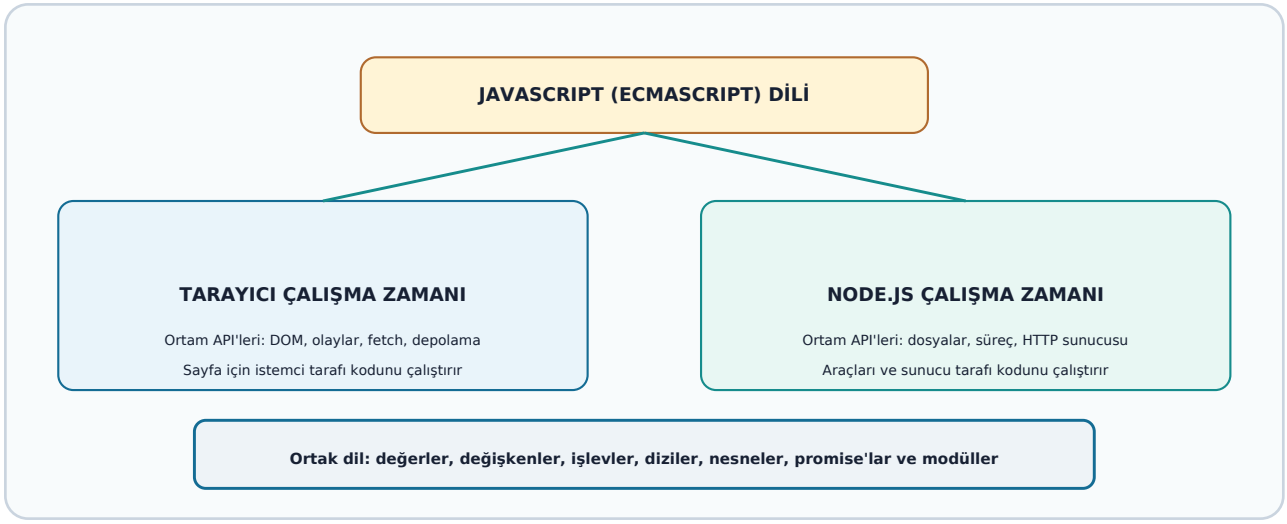
- JavaScript'in kendisi değerleri, değişkenleri, işleçleri, kararları, döngüleri, işlevleri, dizileri, nesneleri, sınıfları, promise'ları ve modülleri sağlar.

- Tarayıcı; Belge Nesne Modeli (Document Object Model, DOM), olaylar, `fetch` ve web depolaması dâhil tarayıcıya özgü Uygulama Programlama Arayüzleri (Application Programming Interfaces, APIs) ekler. Tarayıcı JavaScript'i bir tıklamaya yanıt verebilir ve sayfayı değiştirebilir.

- Node.js; dosyalar, süreçler, akışlar ve HTTP sunucuları dâhil sunucu ve işletim sistemi API'leri ekler. Node.js varsayılan olarak bir tarayıcı sayfası veya DOM sağlamaz.

- npm başka bir programlama dili değildir. Proje bağımlılıklarını kurar ve adlandırılmış betikleri çalıştırır; Node.js ise bu araçların ve sunucuların kullandığı JavaScript'i yürütür.

Şekil 0.4, tek bir dilin iki çalışma zamanına girdiğini gösterir. Ortak orta bölüm, öğrencilerin dil bilgisini istemci tarafı kodundan sunucu tarafı koduna aktarabilmesini açıklar. Farklı ortam API'leri ise kodun iki ortam arasında her zaman değiştirilmeden kopyalanamamasının nedenidir.



Şekil 0.4. JavaScript tarayıcıda ve Node.js'te çalışır; çalışma zamanları farklı ortam API'leri sağlar.

HTML ve Basamaklı Stil Sayfaları (Cascading Style Sheets, CSS) temel önemini korur; ancak farklı görevleri vardır. HTML içeriği ve ilişkileri tanımlar. CSS sunumu ve yerleşimi tanımlar. JavaScript algoritmaları, davranışı, durum değişikliklerini ve uygulama mantığını ifade eder. Üçüne birden “programlama dili” demek, hata ayıklama sırasında önem kazanan ayrımları gizler.

7.6 JavaScript'i tarayıcıda doğrulayın

Desteklenen her tarayıcı zaten bir JavaScript motoru içerir; ayrıca tarayıcı JavaScript'i yükleyicisi yoktur. Dosya oluşturmadan doğrulamak için:

1 Ders tarayıcısında `https://example.com/` adresini açın.

2 Geliştirici araçlarını açın ve Console panelini seçin. Console gizliyse `>>` taşma menüsünü seçip Console'u açın.

3 `>` giriş isteminin yanına tıklayın, `1 + 2` ifadesini kendiniz yazın ve Enter'a basın. Sonraki satırda `3` görünmelidir.

4 `typeof document` yazıp Enter'a basın. Tarayıcı geçerli sayfa için bir `document` sağladığından sonuç `"object"` olmalıdır.

Console, kodu açık sayfanın bağlamında hemen çalıştırır. Bilmediğiniz bir komutu hiçbir zaman buraya yapıştırmayın. Bazı tarayıcılar kullanıcıların kötü amaçlı kod yapıştırmaya kandırılmasını önlemek için self-XSS uyarısı gösterir. Bu alıştırmada iki kısa ifadeyi elle yazın ve uyarıyı etkisizleştirmeyin.

7.7 JavaScript'i Node.js içinde doğrulayın

`node --version` komutunun başarılı olduğu herhangi bir terminalde şunları çalıştırın:

```
node -e "console.log(1 + 2)"
node -e "console.log(typeof document)"
```

İlk komut `3` yazdırmalıdır. İkincisi `undefined` yazdırmalıdır. İkinci sonuç beklenir: Node.js JavaScript'i doğru biçimde çalıştırmıştır; ancak tarayıcı belgesi oluşturmaz. `-e` seçeneği Node.js'ten ardından gelen kısa programı değerlendirmesini ister. Sonraki bölümlerde, komut satırında uzun programlar yazmak yerine programları `.js` ve `.mjs` dosyalarına yerleştirecek, modülleri içe aktaracak ve test edeceksiniz.

7.8 JavaScript'in ders boyunca nasıl geliştiği

Dizilim, birleştirmeden önce sorumlulukları bilinçli olarak ayırır:

1 Bölüm 1, JavaScript'in web isteği ve görüntülenen sayfa içindeki yerini açıklar.

2 Bölüm 2 ve 3, betiklerin sağlam bir belgenin yerini almak yerine onu geliştirebilmesi için anlamsal HTML'yi ve profesyonel CSS'yi kurar.

3 Bölüm 4 JavaScript'i ilk ilkelerden öğretir: değerler, değişkenler, türler, denetim akışı, işlevler, kapsam, diziler, nesneler, hatalar, hata ayıklama ve testler.

4 Bölüm 5 dili tarayıcıdaki DOM'a, olaylara ve formlara uygular.

5 Bölüm 6 eşzamansız JavaScript'i, promise'ları, `async/await`, `fetch`, JavaScript Nesne Gösterimi (JavaScript Object Notation, JSON) ve Web API'lerini açıklar.

6 Bölüm 7 ECMAScript modüllerini, npm araçlarını ve ardından TypeScript'i ekler. Öğrencilerin bir dili, tür sistemini ve DOM'u aynı anda öğrenmemesi için JavaScript önce gelir.

7 Bölüm 8, sunucu tarafı JavaScript'i ve HTTP hizmetlerini öğretmek için Node.js'i kullanır. Sonraki projeler, ayrı sorumluluklarını koruyarak tarayıcı kodunu, sunucu kodunu ve kalıcı verileri birbirine bağlar.

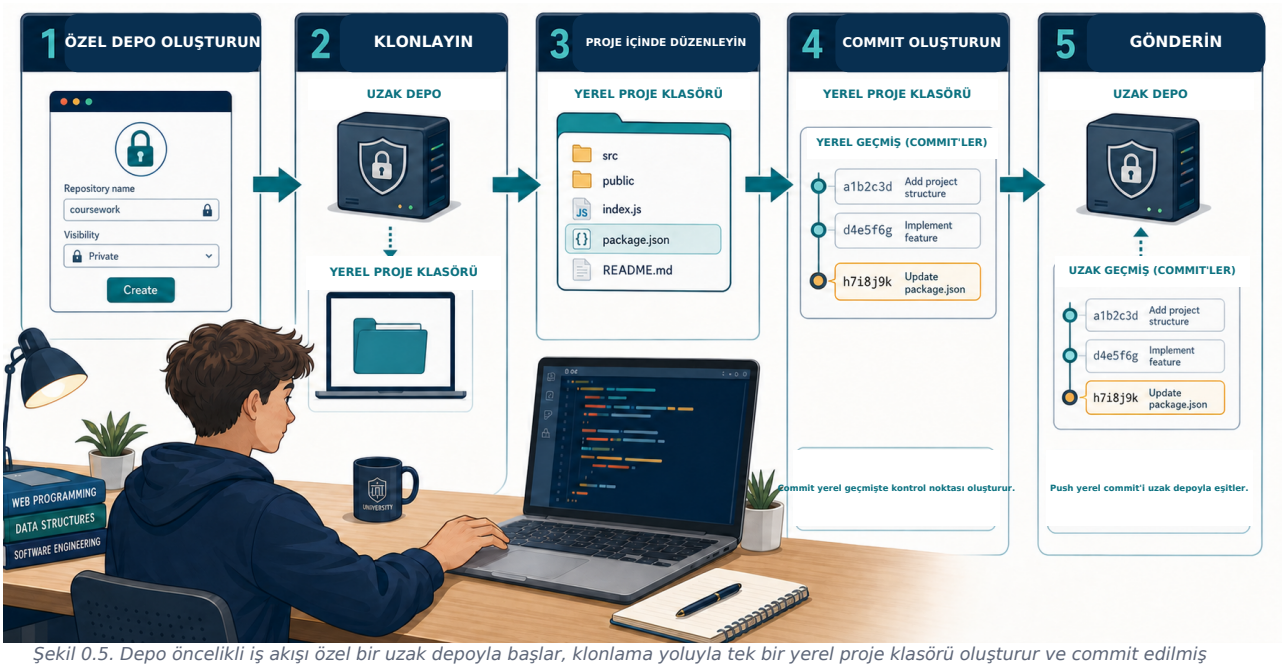
Bu nedenle istemci tarafı ve sunucu tarafı programlama önce ayrı ayrı öğretilir, ardından tam yığın uygulamalarda bütünleştirilir. Dil ortaktır; güvenlik sınırları, kullanılabilir API'ler, yaşam süresi ve sorumluluklar ortak değildir.

8. Geliştirmeye başlamadan önce uzak deponuzu oluşturun veya edinin

En güvenli ders iş akışı, öğrenciye atanmış özel bir uzak depoyla başlar. Depo, Git'in kaydettiği değişiklik geçmişiyle birlikte bir proje klasörüdür. Git barındırma hizmeti, Git depolarını İnternet'e bağlı bir sunucuda saklayan ve bunları kimin okuyabileceğini veya değiştirebileceğini denetleyen web sitesidir. GitHub, GitLab ve Bitbucket buna örnektir. Bu ders `https://github.com/` adresindeki GitHub'ı kullanır. Git bilgisayarınızdaki sürüm kontrol programıdır; GitHub çevrim içi hizmettir. Birlikte çalışırlar, ancak aynı ürün değildir.

Yerel sözcüğü “bu bilgisayarda” anlamına gelir. *Uzak* sözcüğü “başka bir konumda saklanan ve ağ üzerinden erişilen” anlamına gelir. Bu nedenle klonlamadan sonra aynı proje iki yerde bulunur: düzenlediğiniz yerel proje klasörü ve gönderim yaptığınızda commit'leri alan GitHub'daki uzak depo. Dosyayı kaydetmek Git geçmişini güncellemez, commit oluşturmak onu karşıya yüklemez ve gönderim yapmak commit edilmemiş değişiklikleri otomatik olarak içermez.

Şekil 0.5 ilişkinin tamamını gösterir. Uzak depo ile yerel proje klasörü birbiriyle ilişkili iki konumdur; sihirli tek bir paylaşılan klasör değildir.



Şekil 0.5. Depo öncelikli iş akışı özel bir uzak depoyla başlar, klonlama yoluyla tek bir yerel proje klasörü oluşturur ve commit edilmiş çalışmayı bilinçli bir gönderimle geri taşır.

8.1 Tercih edilen ders yöntemi: öğretim elemanının GitHub şablonundan oluşturun

Bu alt bölüm tek bir yolu açıklar: öğretim elemanı bir GitHub şablon deposu hazırlar ve her öğrenci bundan ayrı bir özel depo oluşturur. Şablon, klasörleri ve dosyaları GitHub üzerindeki bir düğmeyle kopyalanabilen başlangıç projesidir. Microsoft Word'deki belge şablonu değildir ve öğrenciler öğretim elemanının özgün deposunu düzenlemez.

Öğretim elemanından gelmesi gereken bilgiler

Fall 2026 sınıfı aşağıdaki özel GitHub şablon deposunu kullanır. Yetkili GitHub hesabınızla oturum açmışken Fall 2026 ders çalışmaları şablonunu açın.

```
https://github.com/Mahyar2020/web-programming-coursework-2026_Fall
```

Öğretim elemanı aynı tıklanabilir adresi Moodle, Canvas veya Blackboard gibi dersin Öğrenme Yönetim Sistemi'nde (Learning Management System, LMS) de yayımlamalıdır. Öğrenciler doğrulanmamış bir hesaptan gelen adres yerine LMS kopyasını kullanmalıdır. Bu adreste `Mahyar2020` sahip, `web-programming-coursework-2026_Fall` ise depo adıdır.

Bunu, kitabı içeren depoyla karıştırmayın. Ders kitabının kaynak deposu kitabı ve öğretim elemanı kaynaklarını üretmek için kullanılır; öğrencilerin laboratuvar çalışmalarını tamamladığı depo değildir.

LMS'de ödev bağlantısı yoksa veya bağlantı 404 sayfası açıyorsa durun ve dönem GitHub şablonunu ya da davetini öğretim elemanından isteyin. Adresi tahmin etmeyin, ilgisiz boş bir depo oluşturmayın, web aramasında bulunan bir depoyu kullanmayın ve öğretim elemanı bu iş akışlarından birini seçmedikçe Fork veya Download ZIP seçeneğini bunun yerine kullanmayın.

Oturum açılan hesaba erişim verilmemişse özel bir depo da 404 olarak görünebilir.

Adım 1 - Doğru GitHub hesabını oluşturun veya bu hesapta oturum açın

Tarayıcıda `https://github.com/` adresini açın. Hesabınız yoksa Sign up, varsa Sign in seçeneğini kullanın. Üniversitenin onayladığı hesabı kullanın. Üniversite yönetilen bir hesap oluşturduysa ikinci bir hesap açmak yerine onun oturum açma talimatlarını izleyin. Sağ üst köşedeki profil fotoğrafını seçerek oturum açmış kullanıcı adını doğrulayın; GitHub bu menüde kullanıcı adını gösterir.

Tarayıcı burada çevrim içi hesabı yönetmek için kullanılır. Terminal ve `git` komutu daha sonra yerel kopyayı oluşturmak için kullanılacaktır.

Adım 2 - Hesabı çok faktörlü kimlik doğrulamayla koruyun

Çok Faktörlü Kimlik Doğrulama (Multi-Factor Authentication, MFA), oturum açarken birden fazla kanıt türü gerektirir. GitHub kendi uygulamasına İki Faktörlü Kimlik Doğrulama (Two-Factor Authentication, 2FA) adını verir. Parola bir faktördür; kimlik doğrulayıcı uygulamanın değişen kodu, geçiş anahtarı veya fiziksel güvenlik anahtarı başka bir faktör sağlar. Bu, birisi parolayı öğrendiğinde oluşabilecek zararı sınırlar.

GitHub şu anda kısa mesaj yerine Zamana Dayalı Tek Kullanımlık Parola (Time-Based One-Time Password, TOTP) üreten kimlik doğrulayıcı uygulamayı önerir. Telefonda veya bilgisayarda bir kimlik doğrulayıcı uygulama hazırlayın, ardından GitHub'da şu adımları izleyin:

1 Sağ üst köşedeki profil fotoğrafını seçin ve Settings'i açın.

2 Sol kenar çubuğunda Access altında Password and authentication seçeneğini açın.

3 Two-factor authentication bölümünü bulun ve Enable two-factor authentication seçeneğini kullanın.

4 Kimlik doğrulayıcı uygulamayla görüntülenen Hızlı Yanıt (Quick Response, QR) kodunu tarayın. Aygıt tarayamıyorsa GitHub elle giriş için kurulum anahtarı gösterebilir.

5 Kimlik doğrulayıcı uygulamanın ürettiği altı basamaklı kodu girin. Kod belirli aralıklarla değiştiğinden güncel olanı kullanın.

6 GitHub gösterdiğinde kurtarma kodlarını indirin. Bunları güvenli bir parola yöneticisinde veya dizüstü bilgisayardan ayrı, korunan başka bir yerde saklayın.

7 Kurtarma kodlarının kaydedildiğini doğrulayın. Varsa yedek yöntem olarak geçiş anahtarı veya güvenlik anahtarı ekleyin.

Kurtarma kodları, kimlik doğrulayıcı aygıt kaybolduğunda erişimi geri getirebilir. QR görselini, kurulum anahtarını, parolayı, kimlik doğrulayıcı kodunu veya kurtarma kodlarını hiçbir zaman ders deposuna, değerlendirme için gönderilen ekran görüntüsüne, e-postaya ya da paylaşılan sohbete koymayın. Ayrıntılı ve güncellemeye duyarlı süreç GitHub'ın resmî iki faktörlü kimlik doğrulama talimatlarında korunur.

Adım 3 - Öğretim elemanının tam bağlantısını açın ve doğrulayın

Önce GitHub'ın gönderdiği özel depo davetini kabul edin. Bu ders için kaydedilmiş GitHub hesabıyla oturum açmışken bildirimi açın, davetin `Mahyar2020` ve `web-programming-coursework-2026_Fall` adlarını içerdiğini doğrulayın ve Accept invitation seçeneğini kullanın. Davet görünmüyorsa GitHub bildirim gelen kutusunu ve GitHub'a kayıtlı e-posta adresini denetleyip öğretim elemanıyla iletişime geçin. Depo davetleri hesaba özgü olduğundan başka bir öğrenciden daveti iletmesini istemeyin.

LMS'ye dönün ve bu dönem için yayımlanan şablon bağlantısını seçin. Tıklanabilir bağlantı varken adresi yeniden yazmayın. Tarayıcı adresi şu olmalıdır:

```
https://github.com/Mahyar2020/web-programming-coursework-2026_Fall
```

Sayfa, ders dosyalarını ve dosya listesinin üzerinde Use this template düğmesini göstermelidir. Düğme yoksa üç olasılıktan biri geçerlidir: bağlantı şablon değildir, yanlış hesapla oturum açılmıştır veya hesabın erişimi yoktur. Sayfa adresini kaydedin ve hangi durumun geçerli olduğunu öğretim elemanına sorun. Yalnızca devam edebilmek için başka bir düğme seçmeyin.

Adım 4 - Şablondan kendi uzak deponuzu oluşturun

Use this template, ardından Create a new repository seçeneğini kullanın. Oluşturma sayfasında:

1 Owner alanını öğretim elemanının belirttiği kişisel hesaba veya üniversite kuruluşuna ayarlayın.

2 Depoya `web-programming-coursework-2026_Fall-STUDENT_NUMBER` adını verin ve `STUDENT_NUMBER` bölümünü üniversite öğrenci numaranızla değiştirin. Boşluk kullanmayın.

3 İsteğe bağlı olarak `Coursework for Web Programming` gibi bir açıklama girin.

4 Üniversite açıkça başka bir görünürlük istemedikçe Private seçeneğini kullanın. Private, yalnızca yetkili hesapların depoyu okuyabileceği anlamına gelir; güvenlik uygulamalarının gereksiz olduğu anlamına gelmez.

5 Include all branches onay kutusunu işaretsiz bırakın. İşaretsiz, kapalı anlamına gelir: GitHub yalnızca varsayılan `main` dalını kopyalar. Bunu yalnızca öğretim elemanı ek şablon dallarının gerekli olduğunu açıkça duyurursa seçin.

6 Başka bir README, `.gitignore` veya lisans eklemeyin. Bu dosyalar zaten şablondan gelir.

7 Create repository from template seçeneğini kullanın ve yeni sayfanın yüklenmesini bekleyin.

Sayfanın üst kısmında seçtiğiniz sahibin ve depo adının göründüğünü doğrulayın. Bu yeni sayfa uzak deponuzdur. Depoyu oluşturmak henüz bilgisayara bir proje klasörü yerleştirmedi.

Adım 5 - Özel deponuza öğretim elemanı erişimi verin

Yeni depo öğrenciye aittir; bu nedenle şablona erişim, öğretim elemanına öğrencinin kopyasına otomatik erişim vermez. Yeni depo sayfasında Settings, ardından Collaborators veya Collaborators and teams seçeneğini açın. Add people seçeneğini kullanın, `Mahyar2020` yazın, hesap adını doğrulayın ve daveti gönderin. Özel ders çalışması incelenmeden önce öğretim elemanı daveti kabul etmelidir.

Daveti gönderdikten sonra deponun Code sayfasına dönün. Görünen adı öğretim elemanının adına benzediği için tanımadığınız bir hesabı hiçbir zaman eklemeyin.

Adım 6 - HTTPS klonlama adresini kopyalayın

Yeni depo sayfanızda yeşil Code düğmesini seçin. Local, ardından HTTPS seçeneğini açın ve görüntülenen adresin yanındaki kopyalama düğmesini kullanın. Adres, gerçek sahibiniz ve depo adınızla aşağıdaki örneğe benzemelidir:

```
https://github.com/YOUR_GITHUB_USERNAME/web-programming-coursework-2026_Fall-
YOUR_STUDENT_NUMBER.git
```

Güvenli Kabuk (SSH) başka bir geçerli bağlantı yöntemidir; ancak yalnızca derste SSH anahtarları öğretildikten sonra kullanın. Adresi kopyalamak hiçbir şeyi klonlamaz; sonraki komut için metni panoya yerleştirir.

Adım 7 - Bilgisayarda üst klasörü seçin

Windows Dosya Gezgini'nde `C:\Users\YourName\Documents\GitHub` gibi bir üst klasör oluşturun veya açın. Üst klasör projeyi içerecektir; ancak kendisi proje değildir. Dosya Gezgini adres çubuğuna tıklayın, `cmd` yazın ve Komut İstemi'ni bu üst klasörde açmak için Enter'a basın. İstem `Documents\GitHub>` ile bitmelidir.

macOS veya Linux'ta terminal açın ve `cd` ile seçilen üst klasöre girin. Konumunu yazdırıp doğrulamak için `pwd` kullanın.

Adım 8 - Yerel projeyi oluşturmak için uzak depoyu klonlayın

Terminal isteminde `git clone` yazın, bir boşluk ekleyin, Adım 6'da kopyalanan HTTPS adresini yapıştırın ve Enter'a basın. Temsilî bir komut şöyledir:

```
git clone https://github.com/YOUR_GITHUB_USERNAME/web-programming-coursework-2026_Fall-
YOUR_STUDENT_NUMBER.git
```

Örnek URL'nin tamamını deponuzdan kopyaladığınız adresle değiştirin. `YOUR_GITHUB_USERNAME` veya `YOUR_STUDENT_NUMBER` yer tutucu sözcüklerini yazmayın. Git yeni bir alt klasör oluşturur, proje dosyalarını ve Git geçmişini indirir ve uzak bağlantıyı normalde `origin` olarak adlandırır. İlk özel depo işleminde Git Credential Manager tarayıcı açıp Git'i yetkilendirmenizi isteyebilir. Tarayıcıdaki oturum açma işlemini tamamlayın; GitHub parolasını komutun içine koymayın.

Adım 9 - Projeye girin ve iki konumun bağlı olduğunu kanıtlayın

Windows Komut İstemi'nde `git clone` tarafından oluşturulan gerçek klasör adını kullanın:

```
cd /d "C:\Users\YourName\Documents\GitHub\web-programming-coursework-2026_Fall-
YOUR_STUDENT_NUMBER"
dir package.json
git status
git remote -v
```

macOS veya Linux'ta karşılık gelen yolla `cd` kullanın ve `dir package.json` yerine `ls -l package.json` yazın. Başarıyı varsaymak yerine kanıtı okuyun:

- `dir package.json` veya `ls -l package.json` proje dosyasını göstermelidir;

- `git status` geçerli dalın adını ve çalışma ağacının durumunu bildirmelidir;

- `git remote -v`, fetch ve push için uzak deponuzun HTTPS adresini `origin` adının yanında göstermelidir ve

- bu adresteki sahip ve depo, yalnızca öğretim elemanının şablonuyla değil, Adım 4'te oluşturulan yeni GitHub sayfasıyla eşleşmelidir.

`git clone`, `Repository not found` bildirirse önce kopyalanan adresi ve oturum açılmış GitHub hesabını doğrulayın. Özel depoda bu ileti çoğu zaman kimlik doğrulamanın başarısız olduğu veya hesabın izninin bulunmadığı anlamına gelir; deponun silindiğini göstermek zorunda değildir.

8.2 Öğretim elemanı şablon yerine bir klonlama URL'si sağlarsa

Sağlanan URL'yi ders çalışmalarının üst klasörüne klonlayın, yeni klasöre girin ve aynı dört doğrulama komutunu uygulayın. Yalnızca uzak depo hesabınıza izin veriyorsa push yapabilirsiniz. Korunan ortak kitap dalında deneysel commit'ler oluşturmayın. Öğretim elemanı bunun yerine her öğrenci için bir uzak depo oluşturabilir veya kişisel fork isteyebilir.

8.3 Depo bu bilgisayarda zaten varsa

Terminal başka yerde açıldı diye ikinci bir kopya klonlamayın. Dosya Gezgini'nde mevcut dış proje klasörünü bulun. Bu yönlendirmedeki varsayımsal Windows konumu için Komut İstemi sırası şöyledir:

```
cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"
dir package.json
git status
```

Üçü de başarılıysa bu klasörde devam edin. Benzer adlara sahip iki klon, bir kopyayı düzenleyip diğerini çalıştırmayı kolaylaştırır; bu nedenle adı açık tek bir çalışma kopyası kullanın.

8.4 Öğretim elemanı ZIP arşivi sağlarsa

1 ZIP dosyasını ders çalışmalarının üst klasörüne kaydedin ve tümüyle ayıklayın.

2 Ayıklanan dış klasörün `package.json` içerdiğini doğrulayın; sıkıştırılmış önizleme içinde hiçbir zaman çalışmayın.

3 Ayıklanan klasör içinde `git status` çalıştırın. Git bunun bir depo olmadığını söylüyorsa ZIP dosyaları içerir, fakat Git geçmişini içermez.

4 Tarayıcıda onaylanmış boş özel uzak depoyu oluşturun. Ayıklanan projede zaten README varsa yeni bir tane eklemeyin.

5 Ayıklanan proje kökünde Git'i başlatmak ve bu belirli uzak depoyu bağlamak için öğretim elemanının komutlarını izleyin. Yanlış bir uzak depo çalışmayı yanlış yerde yayımlayabileceğinden, ilk push işleminden önce öğretim elemanı `git remote -v` çıktısını doğrulamalıdır.

8.5 Deponun tamamını VS Code'da açın

Doğrulanmış depo kökünde şu komutu çalıştırın:

```
code .
```

Nokta, bu geçerli klasör anlamına gelir. VS Code'da Explorer; `labs`, `scripts`, `package.json` ve `.nvmrc` öğelerini göstermelidir. Alttaki terminal istemi proje klasörünün adıyla bitmelidir. Explorer yalnızca tek bir HTML dosyası gösteriyorsa File > Open Folder seçeneğini kullanın, dış depo klasörünü seçin ve Select Folder düğmesine basın. VS Code yazarlara güvenip güvenmediğinizi sorduğunda önce deponun yetkili ders kaynağından geldiğini doğrulayın.

8.6 Geliştirilen çalışmalarda aynı kaydet-denetle-commit-push döngüsünü kullanın

Her küçük ve çalışan değişikliğin sonunda:

```
git status
git diff
git add path-to-the-file-you-changed
git diff --staged
git commit -m "Describe the completed change"
git push
```

`git status` ve her iki diff çıktısını okuyun. Örnek yolu ve iletiyi değiştirin; her dosyayı düşünmeden eklemeyin. İlk push işlemi tarayıcıdan veya kimlik bilgisi yöneticisinden kimlik doğrulaması isteyebilir. Komuta gömülmüş hesap parolası yerine barındırma hizmetinin onaylanmış oturum açma akışını kullanın. Ardından uzak depo sayfasını yenileyin ve en son commit'in görünür olduğunu doğrulayın. Görünen uzak commit, push işleminin başarılı olduğuna ilişkin kanıttır.

9. Yeniden üretilebilir hazırlık testini çalıştırın

`npm test` ile başlamayın. Önce terminalin nerede olduğunu kanıtlayın. Windows Komut İstemi'nde başarılı bir ön denetim şöyledir:

```
cd
dir package.json
```

İlk çıktı depo kökü olmalıdır. İkincisi `package.json` dosyasını göstermelidir. PowerShell'de `Get-Location` ve `Get-Item package.json`; macOS veya Linux'ta `pwd` ve `ls -l package.json` kullanın. Ancak bundan sonra şu komutları tek tek çalıştırın:

```
node --version
npm --version
git --version
npm test
```

`npm test`, “npm'yi test et” anlamına gelmez. npm `package.json` dosyasını okur ve bu projenin adlandırılmış test betiğini çalıştırır. Öğrenci ders çalışmaları deposundaki betik; gerekli proje dosyalarını, başlangıç HTML belgelerini, Node.js ana sürümünü ve Hafta 1-3 sunucularının her birinden gelen HTTP yanıtını denetler.

Başarılı sonuç, çalışma zamanının projeyi okuyabildiğini ve sağlanan hazırlık denetimlerini çalıştırabildiğini gösterir. Gelecekteki her programın doğru olduğunu kanıtlamaz.

Yalnızca son satırı değil, ilk hatanın tamamını kaydedin. Yaygın ayrımlar şunlardır:

- `command not found`: araç eksiktir veya PATH üzerinden kullanılamıyordur;

- `wrong major version`: başka bir kurulum veya sürüm yöneticisi etkindir;

- `file not found`: terminal büyük olasılıkla yanlış dizindedir;

- `permission denied`: geçerli hesap hedefi okuyamıyor veya çalıştıramıyordur;

- `test assertion failed`: araçlar çalışmıştır; ancak proje içeriği denetlenen bir gereksinimi karşılamıyordur.

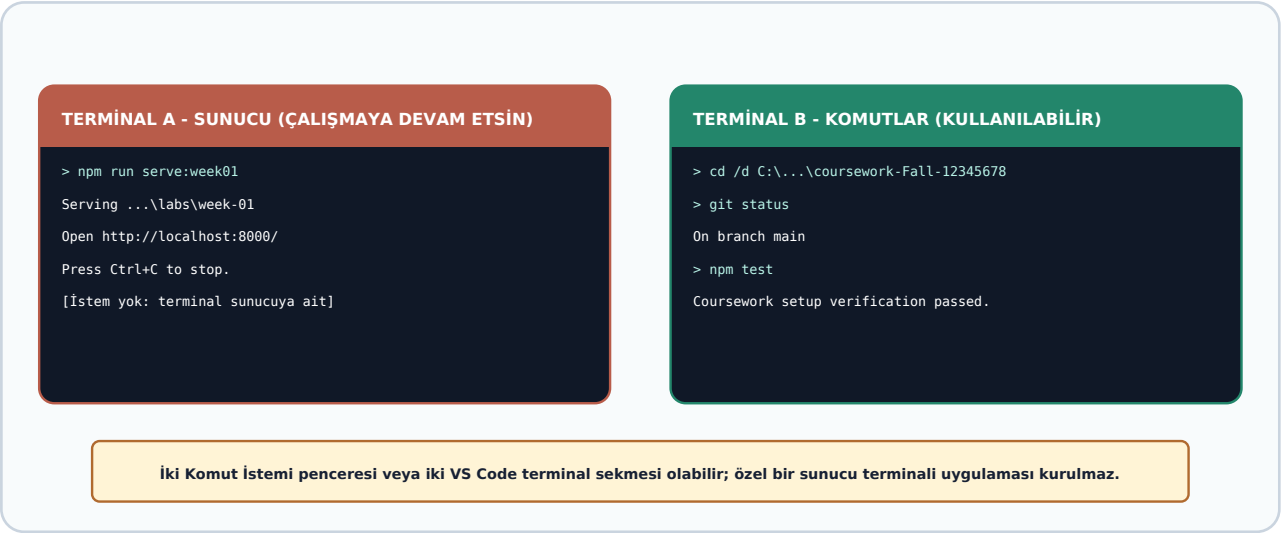
Denetim başarıyla bittiğinde son satırlar `Local Markdown link verification passed`, `Week 1 verification passed` ve `Foundation verification passed` gibi iletiler içerir. Sürüm numaralarının alt ve yama bölümleri değişebilir; ancak bu baskıda Node.js satırı `v24.` ile başlamalıdır.

10. Yerel sunucuyu başlatın ve inceleyin

10.1 Kitabın iki terminal ile neyi kastettiği

Kurulacak iki farklı terminal ürünü yoktur. *Sunucu terminali*, yalnızca o anda sunucunun çalıştığı sıradan bir Komut İstemi, PowerShell, macOS Terminal veya VS Code terminal sekmesidir. *Komut terminali* ise `git status`, `npm test` ve gezinme komutları için boşta kalan ikinci sıradan pencere veya sekmedir.

İki terminal gerekir; çünkü ön plandaki sunucu çalışmayı sürdürür ve ilk terminalin denetimini elinde tutar. Etkinken normal istem geri dönmez. Bu bir donma değildir. Sürecin tarayıcı isteklerini beklediğinin kanıtıdır. Şekil 0.6 iki rolü nasıl tanıyacağınızı gösterir.



Şekil 0.6. Sunucu ve komut terminalleri, o andaki görevleri farklı olan sıradan terminallerdir.

VS Code'da iki terminal oluşturmak için:

1 Depoyu `code .` ile açın.

2 Terminal > New Terminal seçeneğini kullanın. Altta bir panel açılır. İstem yolunun depo klasörünün adıyla bittiğini denetleyin.

3 İkinci sekmeyi oluşturmak için terminal panelinde artı işaretini (+) seçin. VS Code sürümünüz sekmenin bağlam menüsünde Rename seçeneğini sunuyorsa sekmelerin adını SERVER ve COMMANDS olarak değiştirin.

4 Sunucu komutunu çalıştırmadan önce SERVER sekmesini seçin. Başka bir komuta gereksinim duyana kadar COMMANDS sekmesini kullanmayın.

Windows Komut İstemi'nde bunun yerine iki ayrı pencere açabilirsiniz. Her terminalin kendi geçerli dizini olduğundan aynı `cd /d "..."` komutunu iki pencerede de çalıştırın.

10.2 Sunucuyu başlatın ve çıktısını okuyun

SERVER olarak belirlenen terminalde `package.json` dosyasının bulunduğunu kanıtlayın, ardından öğrenci ders çalışmaları deposundaki Hafta 1 sayfasıyla sunucuyu başlatın:

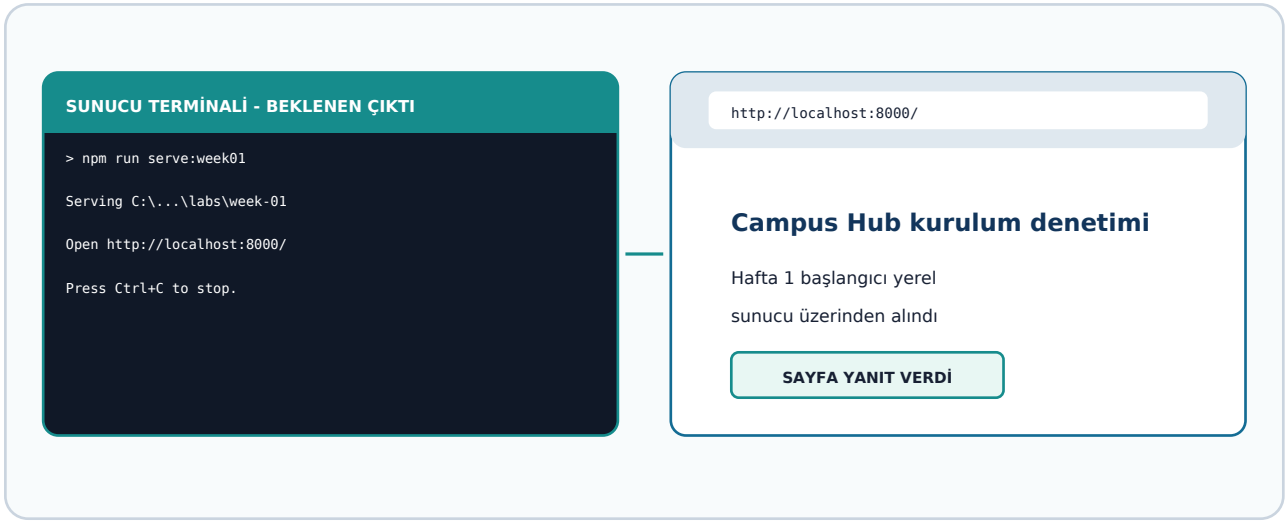
```
dir package.json
npm run serve:week01
```

Bu noktada Node.js, yerel sunucu ve tarayıcının birlikte çalışıp çalışmadığını denetliyorsunuz; Campus Hub belgesini henüz tamamlamanız beklenmez. Başlangıç dosyası görünür bir kurulum iletisi ve Oturum 1 çalışmasını işaretleyen açıklamalar içerir; dolayısıyla başarılı sunucu açıklamasız beyaz bir sayfa üretmez.

macOS veya Linux'ta `dir package.json` yerine `ls -l package.json` kullanın. Sunucu çıktısının beklenen son üç satırı şöyledir:

```
Serving ...\labs\week-01
Open http://localhost:8000/
Press Ctrl+C to stop.
```

İlk yolun başlangıcı bilgisayarınıza bağlıdır. Önemli olan sonunun Windows'ta `labs\week-01`, macOS ve Linux'ta `labs/week-01` olmasıdır. Bu terminali çalışır durumda bırakın. Şekil 0.7 beklenen terminal durumunu beklenen tarayıcı sonucuyla eşleştirir.



Şekil 0.7. Terminal çıktısı ve tarayıcı sayfası birlikte localhost'un yanıt verdiğini kanıtlar.

10.3 Adresi doğru biçimde açın

Tarayıcının adres çubuğuna tıklayın, tam olarak `http://localhost:8000/` yazın ve Enter'a basın. Markdown köşeli parantezlerini, parantezleri, tırnak işaretlerini veya sona nokta eklemeyin. Özellikle şu, yazılacak bir değer değil kitap bağlantısı gösterimidir: `http://localhost:8000/`. Tarayıcı yalnızca düz adresi alır.

`localhost`, geçerli bilgisayarı ifade eden özel bir ana makine adıdır. `8000`, bu ders sunucusunun dinlediği bağlantı noktasıdır. Bu adım hiçbir şeyi İnternet'te yayımlamaz. Sayfa şunu göstermelidir:

`Campus Hub setup check` başlığı ile tarayıcının Hafta 1 başlangıcını yerel sunucu üzerinden aldığını belirten ileti. Oturum 1'de bu kurulum içeriğini eksiksiz Campus Hub yapısıyla değiştireceksiniz.

Adres yükleniyor ancak sayfa beyaz görünüyorsa önce terminaldeki `Serving ...` satırının `labs\week-01` ile bittiğini doğrulayın; ardından VS Code'da `labs/week-01/index.html` dosyasını açıp kurulum denetimi başlığını içerdiğini doğrulayın. Dosyayı kaydedin ve tarayıcıyı yeniden yükleyin. Bu durum, tarayıcının dinleyen sunucuya bağlanamadığı anlamına gelen `This site can't be reached` ile aynı değildir.

10.4 Geliştirici araçlarında Network panelini bulun

Network paneli VS Code'a veya Windows ağ ayarlarına değil, tarayıcının geliştirici araçlarına aittir.

1 Campus Hub sekmesini seçili tutun.

2 Windows/Linux'ta F12 veya Ctrl+Shift+I; macOS'ta Command+Option+I ile geliştirici araçlarını açın.

3 Panel normalde sayfanın yanında veya altında açılır. Üst kısmında Elements, Console, Sources ve Network gibi sekmeler bulunur. Network'ü seçin.

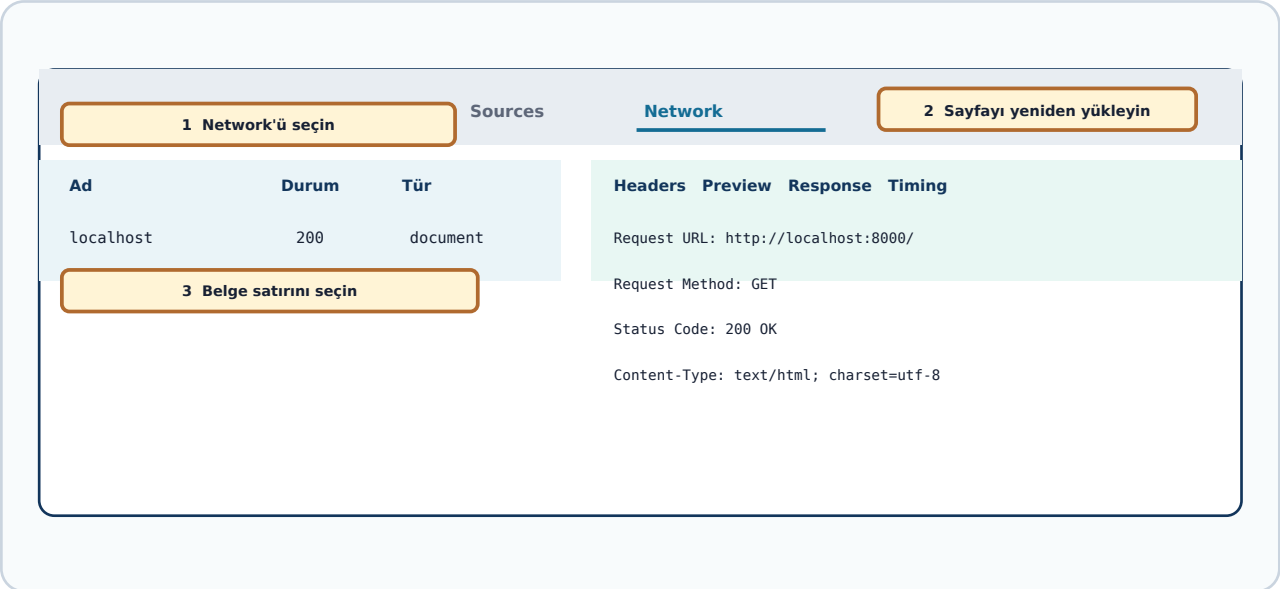
4 Panel dar olduğu için Network gizliyse çift şeritli `>>` taşma düğmesini seçin ve Network'ü açın. Geliştirici araçları komut menüsünü açıp `Show Network` araması da yapabilirsiniz.

5 Network bu andan başlayarak etkinliği kaydeder. Tablo boşsa geliştirici araçları açıkken Ctrl+R, F5 veya tarayıcı yenileme düğmesiyle sayfayı yeniden yükleyin.

6 Type değeri `document` olan `localhost` veya `/` satırını seçin. Görsel veya simge satırını seçmeyin.

7 Ayrıntılar bölmesinde Headers'ı seçin. General altında Request URL, Request Method ve Status Code alanlarını bulun. Response Headers altında Content-Type değerini bulun.

Şekil 0.8 kararlı ve temsilî bir ekran haritasıdır. Tarayıcı renkleri ve tam sekme konumları değişebilir; ancak kanıt adları tanınabilir kalır.



Şekil 0.8. Network istekleri kaydeder; HTTP kanıtı için belge satırını seçin.

Beklenen şu değerleri kaydedin:

Tablo 0.4 - Hafta 1 localhost belge isteği için beklenen kanıt

Kanıt	Beklenen değer
Request URL	http://localhost:8000/
Request method	GET
Status code	200 OK
Resource type	document
Response Content-Type	text/html; charset=utf-8

Durum `404` ise sunucuya erişilebiliyor ancak istenen yol bir dosyayla eşleşmiyordur. Bu, tarayıcının bağlantı kuramadığı anlamına gelen `This site can't be reached` durumundan farklıdır.

10.5 Sunucuyu bilinçli olarak durdurun

SERVER terminaline dönün, içine tıklayın ve Ctrl+C tuşlarına bir kez basın. Windows `Terminate batch job (Y/N)?` diye sorarsa `Y` yazıp Enter'a basın. Olağan istem geri döner. Tarayıcıyı yeniden yükleyin: siteye erişilemediğini bildirmelidir. Bu denetimli hata, sayfanın daha önce açılmış yerel dosyaya değil, sunucu sürecine bağlı olduğunu kanıtlar.

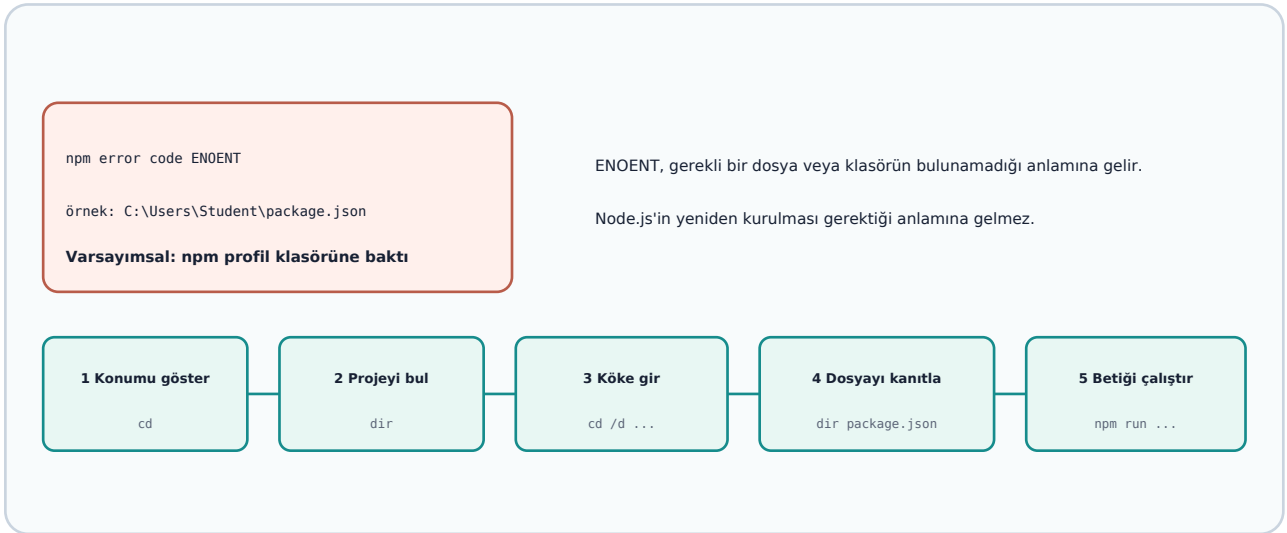
10.6 İsteğe bağlı sorun giderme: npm package.json dosyasını bulamıyor

Sunucu başlıyor ve terminal `Serving`, `Open` ve `Press Ctrl+C` satırlarını yazdırıyorsa bu alt bölümü atlayın. Bu bölüm, öğrencinin daha sonra karşılaşabileceği farklı bir hatayı açıklar; beyaz sayfayı açıklamaz.

ENOENT, gerekli bir dosya veya klasörün bulunamadığını belirten işletim sistemi hata adıdır. Örneğin öğrenci npm'yi depo kökü yerine Windows profil klasöründen çalıştırırsa npm şöyle bir satır yazdırabilir:

```
npm error path C:\Users\Student\package.json
```

Bu satır yönlendirmeden beklenen çıktı değil, açıklayıcı örnektir. Kendi hesap yolunuzla görünürse npm çalışıyordur ancak terminal deponun dışındadır. npm'yi yeniden kurmak terminali taşımaz. Şekil 0.9 bu varsayımsal durumun tanılama sırasını gösterir.



Şekil 0.9. ENOENT yolu, npm'nin package.json dosyasını nerede aradığını gösterebilir.

Bu bölümde kullanılan örnek yol için düzeltme şöyle olur:

```
cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"
```

```
dir package.json
npm run serve:week01
```

Terminal `Open http://localhost:8000/` yazdırmadan tarayıcıyı açmayın. `dir package.json` başarısız olursa başka bir npm komutu tahmin etmek yerine klasörü Dosya Gezgini'nde doğrulayın.

`8000` bağlantı noktası zaten kullanılıyorsa önce önceki ders sunucusunu arayın. Windows PowerShell'de `Get-NetTCPConnection -LocalPort 8000`; macOS'ta `lsof -i :8000`; Linux'ta `ss -ltnp | grep 8000` kullanın. Tanımadığınız kurumsal süreci sonlandırmayın. Kaydettiğiniz süreç bilgisiyle yardım isteyin.

11. Erişilebilirlik inceleme araçlarını hazırlayın

Ücretli erişilebilirlik ürünü gerekmez. Şunları hazırlayın:

1 Tarayıcının Elements, Accessibility ve Network inceleme alanlarını öğrenin.

2 Tab ve Shift+Tab tuşlarının odağı taşıdığını ve işletim sisteminin bu kısayolları engellemediğini doğrulayın.

3 Tarayıcıda yüzde `200` yakınlaştırmayı deneyin ve ardından eski duruma getirin.

4 Platformunuzdaki yerleşik ekran okuyucuyu belirleyin: Windows'ta Narrator, macOS'ta VoiceOver veya Linux'ta Orca gibi kurumca desteklenen ekran okuyucu. Ekran okuyucu kullanımı kendi eğitimini gerektirir; onu bir dakika açmak eksiksiz erişilebilirlik testi değildir.

5 W3C Nu HTML Checker ve W3C erişilebilirlik değerlendirmesi genel bakış sayfalarını yer imlerine ekleyin.

Ders ve veri politikası izin vermedikçe özel ders çalışmasını veya kişisel verileri çevrim içi denetleyiciye yüklemeyin. Tarayıcı incelemesi, klavye testi, doğrulama, otomatik erişilebilirlik denetimleri, yardımcı teknoloji ve temsili kullanıcı geri bildirimi farklı kanıtlar sağlar. Tek bir kurulum erişilebilirliği kanıtlamaz.

12. Kişisel bir çalışma kanıtı commit'i oluşturun ve gönderin

Öğretim elemanının belirlediği öğrenci deposunu veya dalını kullanın; kurulum notlarını doğrudan korunan ortak kitap dalına commit etmeyin. Yalnızca araçların ana sürümleri, test sonucu, işletim sistemi ve tarih gibi gizli olmayan kanıtları içeren `setup-evidence.txt` adlı düz metin dosyası oluşturun. Ardından şunları çalıştırın:

```
git status
git add setup-evidence.txt
git diff --staged
git commit -m "Record development environment verification"
git log -1 --oneline
git push
```

Commit oluşturmadan önce hazırlanmış diff'i okuyun. `git add` sonraki commit'in içeriğini seçer; dosyayı karşıya yüklemez. `git commit` yerel anlık görüntü kaydeder; uzak hizmete otomatik push yapmaz. `git push`, yapılandırılmış uzak dalda henüz bulunmayan commit'leri gönderir. Push işleminden sonra tarayıcıdaki depo sayfasını yenileyin. En yeni commit iletisinin ve zamanının görünür olduğunu doğrulayın. Git bir upstream dalı olmadığını bildirirse hedefi tahmin etmeyin; `git remote -v` çıktısını doğrulayın ve öğretim elemanının sağladığı ilk push komutunu tam olarak kullanın.

Bu küçük döngüyü ders boyunca yineleyin: tutarlı bir değişikliği düzenleyin, kaydedin, ilgili denetimi çalıştırın, `git diff` çıktısını inceleyin, yalnızca amaçlanan dosyaları hazırlayın, anlamlı iletiyle commit oluşturun ve push yapın. “Her şeyi push et” ifadesi “önbellekleri, parolaları, üretilmiş bağımlılıkları veya ilgisiz dosyaları yükle” değil, “proje için gereken incelenmiş her commit'i gönder” anlamına gelmelidir. `.gitignore`, sürüm geçmişinin dışında kalması gereken dosyaları tanımlar.

13. Kanıttan hareketle sorun giderin

Tablo 0.5 yaygın belirtileri verimli ilk denetimlerle eşleştirir. Her aracı yeniden kurmak nadiren en iyi ilk tepkidir.

Tablo 0.5 - Kurulum belirtileri ve verimli ilk denetimler

Belirti	İlk denetimler
code tanınmıyor	Terminali yeniden başlatın; VS Code komut satırı başlatıcısının PATH üzerinde olduğunu doğrulayın; geçici olarak Open Folder kullanın
node --version, v24'ten eski	Kabuğun hangi çalıştırılabilir dosyayı bulduğunu denetleyin; amaçlanan sürüm yöneticisini etkinleştirin veya desteklenen ana sürümü yeniden kurun
npm, PowerShell ilkesi tarafından engelleniyor	Resmî Node kurulumunu doğrulayın; uygun npm komut shim'ini kullanın veya yürütme ilkesini değiştirmeden önce kurumsal destek isteyin
npm test, package.json dosyasını bulamıyor	Geçerli dizin komutunu çalıştırın ve depo köküne geçin
Tarayıcı localhost'a erişemiyor	Sunucu terminalinin hâlâ çalıştığını, URL ile bağlantı noktasının eşleştiğini ve hiçbir güvenlik duvarı isteminin reddedilmediğini doğrulayın
Kaydettikten sonra sayfa değişmiyor	Dosyayı kaydedin, sunucu kökünü doğrulayın, istenen URL'yi inceleyin ve Network paneli açıkken yeniden yükleyin
Git commit'i reddediyor	İletiyi okuyun; yazar adını/e-postasını yapılandırın, amaçlanan dosyayı hazırlayın ve depo ile dalı doğrulayın
Git beklenmedik çok sayıda satır değişikliği gösteriyor	Commit oluşturmadan önce .editorconfig dosyasını, LF satır sonlarını ve depoya özgü core.autocrlf ayarını doğrulayın
Yönetilen bilgisayar kurulumu reddediyor	Ürün, sürüm, yükleyici kaynağı ve hatayı kaydedin; üniversitenin yazılım merkezini kullanın veya yönetici desteği isteyin

Yardım isterken komutu metin olarak, eksiksiz hatayı metin olarak, geçerli dizini, işletim sistemini, ilgili sürüm çıktısını ve hatadan hemen önce neyin değiştiğini verin. Ekran görüntüsü bu kanıtı tamamlayabilir; ancak aranabilir hata metninin yerini almamalıdır. Gerekli olmadıklarında kullanıcı adlarını, özel depo adreslerini, belirteçleri ve kişisel bilgileri kaldırın.

14. Hazırlık kaydı

Tablo 0.6'yı tamamlayın. “Kuruldu” ifadesi yeterli kanıt değildir.

Tablo 0.6 - Öğrenci çalışma istasyonu hazırlık kaydı

Denetim	Kaydedilecek kanıt
İşletim sistemi ve mimari	
Tarayıcı ve DevTools'un başarıyla açılması	
VS Code sürümü ve açılmış depo klasörü	
Git sürümü ve yapılandırılmış yazar kimliği	
Node.js v24 çıktısı	
npm sürüm çıktısı	
Tarayıcı Console'u: 1 + 2 ve typeof document sonuçları	
Node.js: aritmetik ve typeof document sonuçları	
npm test sonucu	
Localhost URL'si, yöntem, durum ve Content-Type	
Sunucunun Ctrl+C ile durdurulması ve hatanın gözlenmesi	
En son kişisel Git commit'i	
Push sonrasında uzak depoda görünen en son commit	
Belirlenen erişilebilirlik araçları	

Her satırda gözlenmiş kanıt veya onaylanmış bir uyarlama bulunduğunda çalışma istasyonu hazırdır. Ders süresini yeniden kuruluma ayırmamak için bu kaydı ilk laboratuvarda yanınızda bulundurun.

Yönlendirme alıştırmaları

Alıştırma 1 - Sorumlu aracı adlandırın

Her görev için birincil aracı adlandırın: `index.html` dosyasını değiştirmek, bir npm betiğini çalıştırmak, HTML'yi piksellere dönüştürmek, HTTP durumunu incelemek, yerel anlık görüntüyü korumak ve sunucu tarafı JavaScript'i çalıştırmak.

Alıştırma 2 - Dizini tanılayın

`node --version` başarılı oluyor; ancak `npm test`, `package.json` dosyasını bulamadığını bildiriyor. En olası nedeni belirtin ve Windows ile macOS/Linux için tahribatsız bir tanılama sırası verin.

Alıştırma 3 - Kanıtı açıklayın

Bir öğrenci `index.html` dosyasını `file:` URL'siyle açıyor ve yerel web sunucusunun çalıştığını söylüyor. Bu gözlemin neden yetersiz olduğunu açıklayın ve ders sunucusuyla HTTP alışverişini gösteren en küçük testi tasarlayın.

Alıştırma 4 - Gizli bilgileri koruyun

Bir terminal hatası, kişisel erişim belirteci içeren uzak URL barındırıyor. Öğrencinin belirteci yayımlamadan nasıl yardım alması gerektiğini ve hangi acil güvenlik işleminin gerekebileceğini açıklayın.

Alıştırma 5 - Terminal rollerini atayın

Bir öğrenci sunucuyu başlatıyor, ardından aynı terminale `git status` yazmayı deniyor; ancak olağan istem görünmüyor. Ne olduğunu, sunucunun nasıl çalışır durumda tutulacağını ve `git status` komutunun nerede çalıştırılacağını açıklayın.

Alıştırma 6 - Boş Network panelini çözün

Geliştirici araçları açık; ancak Network tablosu boş. Belge isteğini oluşturup inceleyecek kesin sırayı verin. Beklenen yöntemi, durumu, kaynak türünü ve Content-Type değerini belirtin.

Alıştırma 7 - Bağlantı ve yol hatalarını ayırın

Şu iki tarayıcı sonucunu karşılaştırın: `This site can't be reached` ve HTTP `404 Not Found` yanıtı. Her gözlem neyi kanıtlar ve ardından hangi kanıtı incelersiniz?

Alıştırma 8 - Tek dil, iki ortam

`typeof document` ifadesi tarayıcı Console'unda `"object"`, Node.js'te ise `undefined` döndürüyor. Her iki sonucun da nasıl doğru JavaScript olabileceğini açıklayın. Ardından her olanağı dil düzeyi, tarayıcının sağladığı veya Node.js'in sağladığı olarak sınıflandırın: bir dizi, `document.querySelector`, `fetch`, `process.argv` ve bir işlev.

Yönlendirme alıştırmalarının ayrıntılı yanıtları

Yanıt 1

Kod düzenleyicisi `index.html` dosyasını değiştirir. npm, `package.json` dosyasını okur ve adı verilen proje betiğini çalıştırır. Tarayıcı HTML'yi ayrıştırır ve ekrana işler. Tarayıcı geliştirici araçları HTTP durumunu görünür kılar. Git yerel anlık görüntüyü kaydeder. Node.js JavaScript'i tarayıcının dışında çalıştırır. Tek bir görevde birkaç araç yer alabilir; ancak bunlar araçların birincil sorumluluklarıdır.

Yanıt 2

Kabuk büyük olasılıkla depo kökünün dışındadır. Önce PowerShell'de `Get-Location`, macOS/Linux'ta `pwd` ile geçerli dizini gösterin. `Get-ChildItem` veya `ls` ile öğeleri listeleyip `package.json` dosyasını arayın. `Set-Location` veya `cd` ile depo klasörüne geçin, yeniden listeleyin ve `npm test` komutunu tekrar çalıştırın. Bu denetimler dosyaları değiştirmez. Node.js'i yeniden kurmak geçerli dizini düzeltmez.

Bölümde kullanılan varsayımsal Windows yolu için eksiksiz tahribatsız kurtarma `cd`, ardından `dir`, sonra `cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"` ve son olarak `dir package.json` ile başlar. Öğrenci gerçek depo yolunu yazmalıdır. Yalnızca son komut dosyayı gösterdikten sonra npm yeniden çalıştırılmalıdır. Bir ENOENT iletisi varsa içindeki yol, npm'nin aradığı klasörün kanıtıdır; böyle bir ileti yoksa bu tanı geçerli değildir.

Yanıt 3

Bir `file:` URL'si, tarayıcının HTTP sunucusu olmadan yerel dosyayı okumasını sağlar. `npm run serve:week01` komutunu başlatın, `http://localhost:8000/` adresini ziyaret edin, Network panelini açın, yeniden yükleyin ve belge isteğini seçin. Çalışan sunucu süreciyle birlikte kaydedilmiş HTTP yöntemi, durumu ve yanıt Content-Type değeri alışverişi kanıtlar. Süreç sınırını doğrulamak için sunucuyu durdurun ve URL'nin artık yanıt vermediğini gözlemleyin.

Yanıt 4

Öğrenci, açığa çıkmış belirteci barındırma hizmeti üzerinden hemen iptal etmeli veya döndürmelidir; çünkü daha sonra ekran görüntüsünü karartmak, daha önce ifşa edilmiş kimlik bilgisini güvenli hâle getirmez. Yardım isteği hata türünü, komut biçimini, ana makineyi ve gizli olmayan bağlamı korurken gizli bilgiyi `[REDACTED TOKEN]` gibi bir işaretle değiştirmelidir. Belirteç commit edilmemeli, sohbete yapıştırılmamalı, issue'ya yüklenmemeli veya kurulum kanıtına konulmamalıdır.

Yanıt 5

Sunucu ön plandaki bir süreçtir ve şu anda o terminali kullanmaktadır. İstekleri dinlerken istemin görünmemesi beklenen durumdur. İlk terminali açık bırakın ve ikinci bir olağan terminal penceresi veya VS Code terminal sekmesi oluşturun. İkinci terminalde depo köküne girin, `package.json` dosyasının bulunduğunu kanıtlayın ve `git status` komutunu orada çalıştırın. “Sunucu terminali” ve “komut terminali” ifadeleri o anki görevlerini anlatır; farklı uygulamalar değildir. Sunucu terminalinde Ctrl+C tuşlarına basmak sunucuyu durdurur; dolayısıyla bunu yalnızca yerel oturumu bitirmek istediğinizde yapın.

Yanıt 6

Geliştirici araçlarını açık tutun, Network'ü seçin (gerekirse `>>` taşma menüsünü kullanın) ve sayfayı yeniden yükleyin. Type değeri `document` olan `localhost` veya `/` satırını seçin, Headers'ı açın ve General ile Response Headers bölümlerini okuyun. Hafta 1 başlangıcı için beklenen kanıt; yöntem `GET`, durum `200 OK`, tür `document` ve Content-Type değeri `text/html; charset=utf-8` biçimindedir. Network'ü sayfa yüklendikten sonra açmak boş tablo gösterebilir; çünkü kayıt çok geç başlamıştır. Yeniden yükleme, gözlenebilir yeni bir istek oluşturur.

Yanıt 7

`This site can't be reached`, tarayıcının verilen ana makine ve bağlantı noktasında yanıt veren bir süreçle bağlantıyı tamamlayamadığı anlamına gelir. SERVER terminalinin hâlâ çalıştığını, aynı bağlantı noktasını yazdırdığını ve URL'nin düz adres olarak girildiğini denetleyin. `404 Not Found` yanıtı ise bir HTTP sunucusunun yanıt verdiğini; ancak istenen yolun kullanılabilir bir dosyayı tanımlamadığını kanıtlar. Bu durumda Network istek URL'sini ve sunucunun yapılandırılmış kökü ile yolunu inceleyin. Bu ayrım, öğrencinin her tarayıcı hatasını aynı sorun gibi ele almasını önler.

Yanıt 8

JavaScript `typeof` işlecini tanımlar; ancak hangi global nesnelerin var olduğuna yürütme ortamı karar verir. Tarayıcı, yüklenen sayfayı temsil etmek için `document` oluşturur; bu nedenle `typeof document` sonucu `"object"` olur. Node.js varsayılan olarak tarayıcı `document` nesnesine sahip değildir; dolayısıyla türünü güvenli biçimde okumak `undefined` üretir. Sonuçların hiçbiri dilin farklı veya bozuk olduğu anlamına gelmez.

Diziler ve işlevler dil düzeyindeki olanaklardır. `document.querySelector`, tarayıcının Belge Nesne Modeli (Document Object Model) tarafından sağlanır. `process.argv`, Node.js tarafından sağlanır. `fetch` bir Web API'si olarak başlamış, artık güncel Node.js tarafından da sağlanmaktadır; bu nedenle kullanılabilirliği çekirdek dilden çok ana makine

ortamına aittir, çekirdek dile değil. Bu son örnek, geliştiricilerin bildikleri her adın JavaScript'in kendi içinde yerleşik olduğunu varsaymak yerine neden hem dil standardını hem de çalışma zamanı belgelerini denetlediğini gösterir.

Yönlendirme bilgiyi geri çağırma kısa sınavı

1 Bir yükleyiciyi açmak, aracın hazır olduğunun neden kanıtı değildir?

2 Node.js ile npm arasındaki fark nedir?

3 `code .` ne anlama gelir?

4 Geçerli çalışma dizini neden önemlidir?

5 Bu baskı hangi Node.js ana sürümünü gerektirir?

6 `git commit` çalışmayı otomatik olarak karşıya yükler mi?

7 `file:` yüklemesini HTTP yüklemesinden hangi kanıt ayırır?

8 Laboratuvarlar sırasında tarayıcı uzantıları neden en az düzeyde tutulmalıdır?

9 `setup-evidence.txt` içinde hiçbir zaman ne bulunmamalıdır?

10 Bilinmeyen bir hataya verimli ilk tepki nedir?

11 Olağan bir terminali “sunucu terminali” yapan nedir?

12 Bir sayfa görünür durumdayken Network neden boş olabilir?

13 Bir npm ENOENT iletisindeki yol size neyi söyler?

14 Kaydetme, commit ve push arasındaki fark nedir?

15 Bu derste birincil programlama dili hangisidir?

16 Tarayıcı JavaScript'i `document` kullanabilirken Node.js normalde neden kullanamaz?

17 İstemci tarafı ile sunucu tarafı programlama derste ne zaman birleştirilir?

Kısa sınav yanıtları

1 Hazırlık, çalıştırılabilir kanıt gerektirir: sürüm çıktısı ve aracın ders iş akışında başarıyla kullanılması.

2 Node.js, JavaScript'i tarayıcının dışında çalıştıran çalışma zamanıdır; npm ise Node.js ile dağıtılan paket yöneticisi ve proje betiği çalıştırıcısıdır.

3 VS Code'dan geçerli dizini proje klasörü olarak açmasını ister.

4 Göreli yollar ve proje keşfi oradan başlar; yanlış dizinde çalıştırılan doğru bir komut yanlış hedefi okuyabilir veya değiştirebilir.

5 `package.json` tarafından sınırlandırıldığı ve `.nvmrc` içinde kaydedildiği üzere sürüm `24.x`.

6 Hayır. Yerel bir anlık görüntü oluşturur. Uzak depoyla iletişim kurmak için ayrı bir push işlemi gerekir.

7 Bir HTTP URL'siyle birlikte yöntem, durum ve HTTP yanıt alanlarını gösteren Network paneli belge alışverişi.

8 Uzantılar içerik ekleyebilir, istekleri değiştirebilir, paneller ekleyebilir veya incelenen davranışı belirsizleştiren değişkenler oluşturabilir.

9 Parolalar, erişim belirteçleri, özel anahtarlar, kurtarma kodları veya gereksiz özel öğrenci ve depo bilgileri.

10 Tam hatayı, komutu, geçerli dizini, sürümleri ve son değişikliği koruyun; tek bir etkeni değiştirmeden önce hangi katmanın başarısız olduğunu belirleyin.

11 İçinde şu anda ön planda çalışan bir sunucu süreci vardır ve olağan kabuk istemi henüz geri dönmemiştir.

12 Geliştirici araçları kayda istek tamamlandıktan sonra başlamış olabilir; Network açıkken yeniden yükleyin.

13 npm'nin eksik dosyayı, örneğin `package.json` dosyasını, bulmaya çalıştığı konumu gösterir.

14 Kaydetme çalışma dosyasını yazar, commit seçilmiş içeriği yerel Git geçmişine kaydeder ve push yerel commit'leri yapılandırılmış uzak depoya aktarır.

15 ECMAScript olarak standartlaştırılmış modern JavaScript; TypeScript yalnızca öğrenciler JavaScript'i ve modülleri anladıktan sonra tanıtılır.

16 `document`, çekirdek JavaScript dili tarafından değil, tarayıcının DOM ortamı tarafından sağlanır. Node.js varsayılan olarak tarayıcı DOM'u oluşturmaz.

17 API'leri ve sorumlulukları açıkça anlaşılabilsin diye önce ayrı ayrı öğretilir; daha sonraki tam yığın uygulama projelerinde birleştirilir.

Yönlendirme özeti

- Çalışma istasyonu, ayrı sorumluluklara sahip ve birlikte çalışan araçlar sistemidir.

- Kurulum ancak hem sürüm hem de iş akışı kanıtı başarılı olduğunda tamamlanır.

- Dersin başlangıç düzeyi; güncel tarayıcı, VS Code, Git, Node.js `24 LTS`, npm, tarayıcı geliştirici araçları ve yetkilendirilmiş depodan oluşur.

- Modern JavaScript birincil programlama dilidir; tarayıcı ile Node.js aynı çekirdek dili farklı ortam API'leriyle çalıştırır.

- Depo kökü, ders komutları için başvuru dizinidir.

- `npm test`, localhost isteği, Network kanıtı, Git commit'i ve doğrulanmış uzak push, yeniden üretilebilir en düşük hazırlık kanıtını oluşturur.

- Geliştirici araçları ve otomatik denetimler kanıt sağlar; akıl yürütmenin veya kullanıcı değerlendirmesinin yerini almaz.

- Kimlik bilgileri ve özel veriler; kaynakta, herkese açık paylaşılan ekran görüntülerinde ve günlüklerde ya da kurulum kayıtlarında asla yer almamalıdır.

Yönlendirme kaynakları

1 Node.js, *Node.js releases*.

2 Node.js, *Download Node.js*.

3 Microsoft, *Get started with Visual Studio Code*.

4 Git, *Install Git*.

5 Chacon ve Straub, *Pro Git - First-Time Git Setup*.

6 Chrome for Developers, *Open Chrome DevTools*.

7 Chrome for Developers, *Inspect network activity*.

8 GitHub Docs, *Creating a repository from a template*.

9 GitHub Docs, *Cloning a repository*.

10 GitHub Docs, *Pushing commits to a remote repository*.

11 GitHub Docs, *About repositories*.

12 GitHub Docs, *Configuring two-factor authentication*.

13 W3C, *Nu HTML Checker*.

14 W3C Web Accessibility Initiative, *Evaluating Web Accessibility*.

Neler sık değişir?

Yükleyici ekranları, paket sürümleri, işletim sistemi izin istemleri ve tarayıcı paneli düzenleri sık değişir. Her aracın sorumluluğu ve kanıta dayalı iş akışı daha kararlıdır. Teknik inceleme tarihi: `22 September 2026`.

Oturum 1 - Web Nasıl Çalışır?

Temel soru: Tekdüzen Kaynak Bulucu (Uniform Resource Locator - URL) yazmakla bir sayfayı görmek arasında gerçekte ne olur ve geliştirici her adımı nasıl gözlemleyebilir?

Oturum haritası

Tablo 1.1, teknik tartışma başlamadan önce öğretim elemanlarına ve öğrencilere oturumun ortak görünümünü sunar.

Tablo 1.1 - Oturum kapsamı, hazırlık ve kanıt

Öğe	Ayrıntılar
Hedef kitle	Programlama deneyimi olan; ancak web bilgisi olduğu varsayılmayan öğrenciler
Hazırlık	Çalışma istasyonu yönlendirmesini tamamlayın ve hazırlık kaydını getirin
Ders süresi	100 dakikalık kavram atölyesi ve 110 dakikalık laboratuvar
Bağımsız çalışma	Yaklaşık 2-3 saat
Ana ürün	Yerel olarak sunulan ve Git'e kaydedilen küçük Campus Hub sayfası
Değerlendirme	Network gözlemi, depo geçmişi, alıştırmalar ve kısa sınav

Öğrenme çıktıları

Bu oturumdan sonra şunları yapabilmelisiniz:

1 İnternet'i, Dünya Çapında Ağ'ı (World Wide Web), tarayıcıyı, sunucuyu, web sitesini, web sayfasını ve kaynağı birbirinden ayırmak.

2 Güvenli Hiper Metin Aktarım Protokolü (Hypertext Transfer Protocol Secure - HTTPS) URL'sini şema, ana makine, bağlantı noktası, yol, sorgu ve parçaya ayırmak.

3 Önek uzunlukları dâhil İnternet Protokolü sürüm 4 (IPv4) ve İnternet Protokolü sürüm 6 (IPv6) adreslerinin yapısını ve gösterimini okumak.

4 Ad çözümlemesi, bağlantı kurulumu, Hiper Metin Aktarım Protokolü (Hypertext Transfer Protocol - HTTP) isteği, HTTP yanıtı ve tarayıcı işlemesi üzerinden basitleştirilmiş gezinmeyi izlemek.

5 HTTP alışverişinin yöntemini, hedefini, durumunu ve seçilmiş üstbilgilerini okumak.

6 `localhost`, yerel dosya ve herkese açık dağıtımın neden farklı olduğunu açıklamak.

7 En küçük geçerli Hiper Metin İşaretleme Dili (HyperText Markup Language - HTML) belgesini oluşturmak ve HTTP üzerinden sunmak.

8 Tahmin etmek yerine kanıt toplamak için tarayıcı geliştirici araçlarını kullanmak.

9 Git deposu oluşturmak ve küçük, anlamlı commit'ler yapmak.

1. Kanıtla başlayın

Bu oturum, bu bölümün hemen öncesindeki *Web Programlama Çalışma İstasyonunuzu Hazırlayın* yönlendirmesini tamamladığınızı varsayar. Yönlendirme yalnızca yazılımları sıralamaz: her aracın ne işe yaradığını açıklar, işletim sistemine özgü kurulum adımları verir, sürümleri doğrular, yerel sunucuyu başlatır, tarayıcı geliştirici araçlarını açar, otomatik denetimleri çalıştırır ve ilk Git commit'ini kaydeder. Bu adımlardan biri yabancıysa veya başarısız olursa devam etmeden önce yönlendirmeye dönün. Bölümler, konuyu öğrenmeyle sürekli araç değiştirmeyi birbirine karıştırmamak için aynı düzenleyici-terminal-tarayıcı-test-Git döngüsünü bilinçli olarak yeniden kullanır.

Tanıdığınız bir web sitesini açın. Devam etmeden önce kısa bir tahmin yazın:

- Sayfa bilgisayarınızda önceden depolanmış mıdır?

- Tarayıcı doğru makineyi nasıl bulur?

- Tek bir sayfa tek bir istek mi gerektirir?

- HTML'yi hangi bilgisayar çalıştırır?

- Sunucu URL'nin `#fragment` bölümünü görebilir mi?

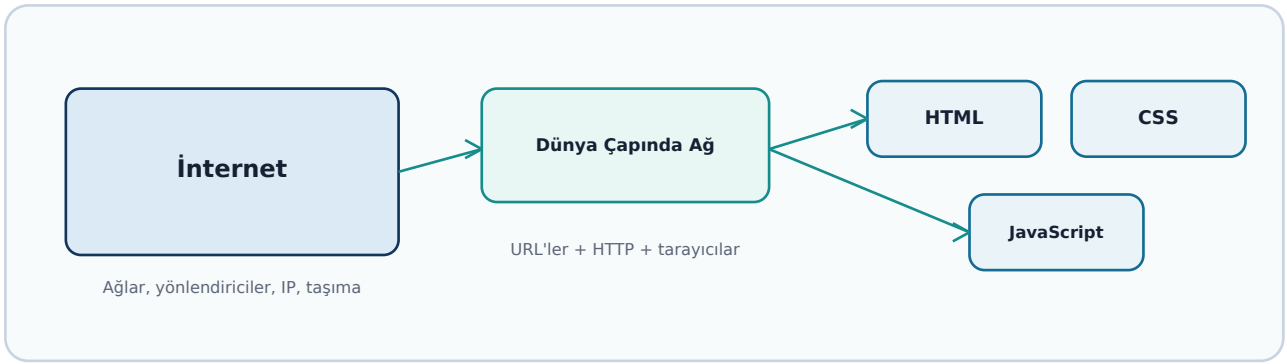
Tahmini henüz düzeltmeyin. Oturumun sonunda, neyin değiştiğini üzerine not edin. Öğrenme, zihinsel model sınanabilir hâle geldiğinde başlar.

Notlarınızda iki sütun bulundurun: gözlem ve yorum. “Network paneli `404` durumunu gösteriyor” bir gözlemdir. “Sunucuda bu yola uyan rota yok” ise kanıtın destekleyebileceği bir yorumdur. Bu alışkanlık, makul görünen açıklamanın ölçülmüş gerçekle karıştırılmasını önler.

2. İnternet, Web değildir

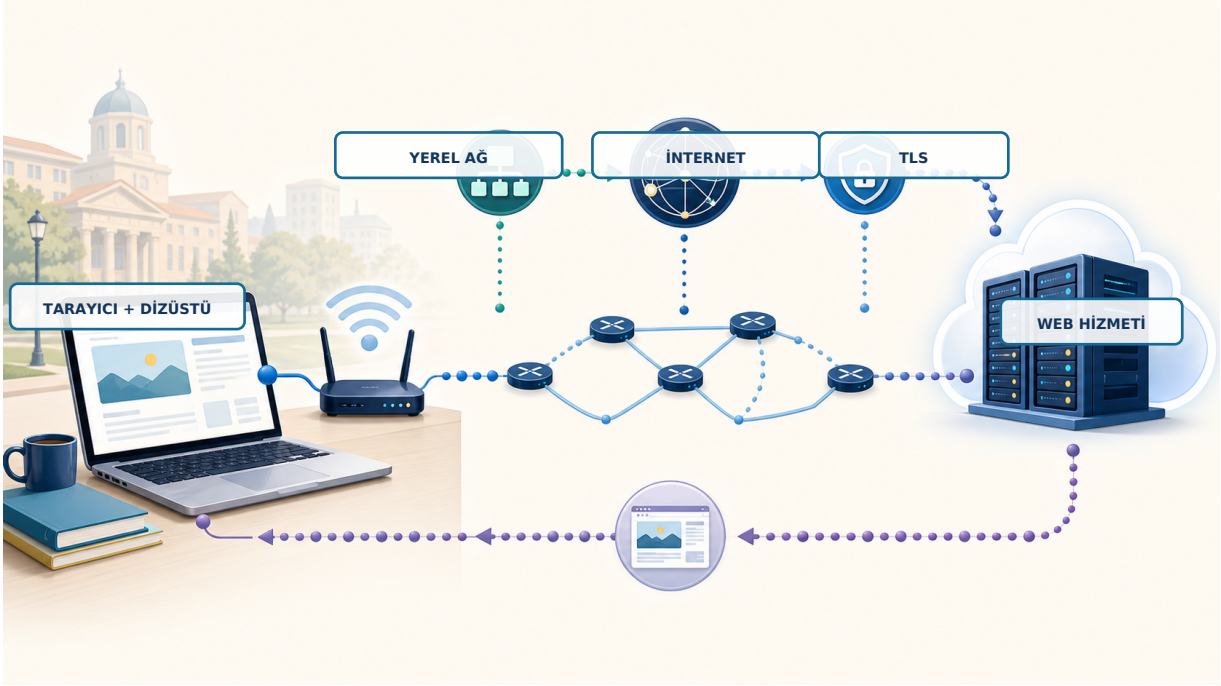
İnternet, verileri makineler arasında taşımak için bir protokol ailesi kullanan ağlar ağıdır. Dünya Çapında Ağ, bu ağ üzerinde kurulan uygulamalardan biridir. E-posta, sesli aramalar, çevrim içi oyunlar ve Web, İnternet'i kullanabilir; ancak bunların tümü Web değildir.

Bu ayrım önemlidir; çünkü web geliştiricisi aynı anda birkaç katmanla çalışır. Ağ katmanındaki hata HTTP hatasından farklıdır; ikisi de HTML belgesindeki hatadan farklıdır. Her başarısızlığa “web sitesi çöktü” denildiğinde tanılama tahmine dönüşür. Profesyonel çalışma, kanıtın işaret ettiği katmanı adlandırmakla başlar. Şekil 1.1, İnternet altyapısını onun üzerinde kurulan Web uygulamasından ayırır.



Şekil 1.1. Web, İnternet altyapısı üzerinde kurulan uygulamalardan biridir.

Şekil 1.2 aynı düşünceyi daha somut gösterir. Dizüstü bilgisayar, Wi-Fi yönlendiricisi, aradaki ağ aygıtları, adlandırma hizmeti, güvenlik adımı ve veri merkezi sunucusu tek bir dev makine değildir. Bunlar birlikte çalışan ayrı parçalardır. Noktalı dönüş yolu, yanıt verisinin tarayıcıya geri gidişini temsil eder; sayfanın görünmesi için tarayıcının bu veriyi yine de yorumlaması gerekir.



Şekil 1.2. İstek bağlantılı ağlardan web hizmetine gider; yanıt tarayıcıya döner ve işlenir.

2.1 Ağ nedir?

Ağ, veri alışverişi yapabilen bilişim aygıtları grubudur. Evinizdeki Wi-Fi ağı bunun bir örneğidir. Üniversite ağı başka bir örnektir. İnternet, bağımsız işletilen çok büyük sayıdaki ağı ortak protokoller aracılığıyla birbirine bağlar. Her kabloya, yönlendiriciye, adrese veya hizmete sahip tek bir kuruluş yoktur.

Bir aygıt, Wi-Fi radyosu veya Ethernet bağdaştırıcısı gibi ağ arayüzü üzerinden ağa katılır. Arayüzler arasındaki doğrudan bağlantıya bağ denir. Yerel ağda anahtar, kablolu aygıtlar arasında genellikle çerçeveleri iletir; kablosuz erişim noktası radyo kullanan aygıtları bu yerel altyapıya bağlar. Yönlendirici farklı ağları bağlar ve İnternet Protokolü paketinin sıradaki hangi noktaya iletileceğine karar verir. Bu aygıtlar birlikte çalışır; ancak aynı görevi yapmaz.

Veri normalde paket adı verilen yönetilebilir birimlere ayrılır. Yönlendiriciler paketleri hedeflerine doğru iletir. Aynı alışverişe ait farklı paketler farklı gecikmelerle karşılaşabilir; ağlar paketleri kaybedebilir, çoğaltabilir veya sırasını değiştirebilir. Üst katmanlar, vaat ettikleri hizmete göre bunları telafi eder.

Gündelik iki ölçüm kolayca karıştırılır. Bant genişliği, yolun zaman birimi başına ne kadar veri taşıyabildiğini; gecikme ise verinin gidip yanıtın alınmasının ne kadar sürdüğünü açıklar. Yüksek bant genişlikli bağlantı, gecikme yüksek olduğunda yine yavaş hissedilebilir. Düşük bant genişlikli bağ küçük bir iletiyi hızla aktarırken büyük bir videoda zorlanabilir.

İnternet Protokolü (Internet Protocol - IP) adresi, İnternet Protokolüyle teslim için bir ağ arayüzünü tanımlar. İnternet Protokolü sürüm 4 (IPv4) adresleri genellikle `192.0.2.10` gibi dört ondalık sayıyla yazılır; İnternet Protokolü sürüm 6 (IPv6) adresleri `2001:db8::10` gibi daha geniş onaltılık gösterim kullanır. Bu örnekler belgeleme için ayrılmış aralıklardandır. IP adresi alan adı, URL, kişi veya zorunlu olarak tek bir fiziksel bilgisayarla aynı şey değildir. Bulut hizmetleri, vekil sunucular, yük dengeleyiciler ve paylaşımlı barındırma bire bir varsayımları güvenilmez kılar.

Birçok ev ve kampüs ağı içeride özel IPv4 adresleri kullanır ve yönlendiricide Ağ Adresi Çevirisi (Network Address Translation - NAT) yoluyla herkese açık bir adresi paylaşır. NAT yaygındır; ancak yönlendirmenin tanımı değildir ve her ağ tasarımında zorunlu değildir. IPv6 çok daha geniş adres alanı sağlar; dağıtımı ve gizlilik mekanizmaları, “yönlendiricileri ortadan kaldırdığı” iddiasından daha inceliklidir.

2.2 İstemci ve sunucu rollerdir

İstemci bir hizmet isteğini başlatır. Sunucu istekleri dinler ve yanıt üretir. Bunlar kalıcı donanım kategorileri değil, rollerdir. Dizüstü bilgisayar aynı anda istemci olarak tarayıcıyı ve geliştirme sunucusunu çalıştırabilir.

Ana makine, ağa katılan fiziksel makine veya sanal ortamdır; süreç ise bu ana makinede çalışan programdır. Tek ana makine aynı anda tarayıcı sürecini, geliştirme sunucusu sürecini, veri tabanı sürecini ve ilgisiz birçok programı çalıştırabilir. Bağlantı noktaları, işletim sisteminin gelen taşıma verisini amaçlanan sürece teslim etmesine yardımcı olur.

Sunucu sözcüğü birbiriyle ilişkili iki anlamda kullanılır:

- Hiper Metin Aktarım Protokolü (HTTP) isteklerini dinleyen süreç gibi bir sunucu programı; ve

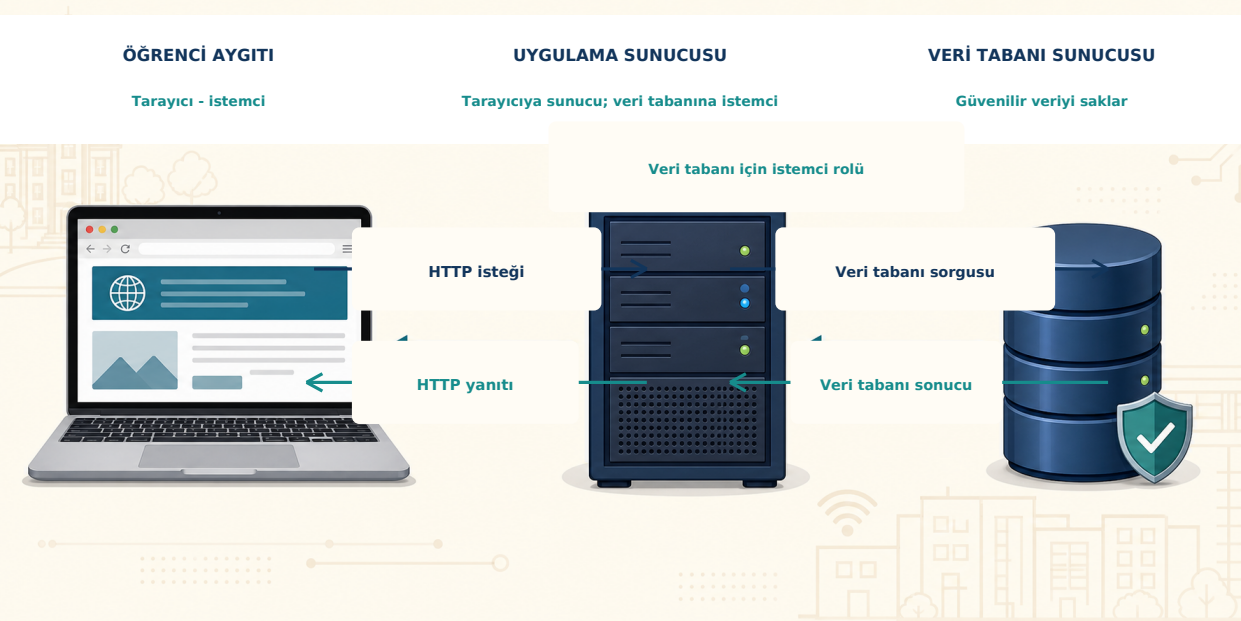
- bir veya daha fazla sunucu programını çalıştıran bilgisayar anlamındaki sunucu makinesi.

İki anlamı ayrı tutun. “Sunucu `404` döndürdü” program davranışını anlatır. “Sunucunun elektriği kesildi” makine veya altyapı hatasını anlatır.

Tarayıcı, web standartlarını anlayan istemci uygulamasıdır. Web sayfası normalde Hiper Metin İşaretleme Dili (HTML) belgesidir. Web sitesi, birbiriyle ilişkili sayfalar ve diğer kaynaklar bütünüdür. Kaynak, Web üzerinde tanımlanan herhangi bir şeydir: HTML, Basamaklı Stil Sayfaları (Cascading Style Sheets - CSS), görsel, yazı tipi, JavaScript Nesne Gösterimi (JSON) verisi, üretilmiş rapor veya başka bir sonuç.

İstemci ve sunucu “zayıf bilgisayar” ve “güçlü bilgisayar” anlamına gelmez; kalıcı kimlikler de değildir. Tarayıcı isteğini işleyen sunucu, başka hizmetten veri istediğinde istemciye dönüşebilir. Aynı biçimde geliştirme sırasında aynı dizüstü bilgisayar iki tarafı da çalıştırabilir. Rol, anlatılan alışveriş tarafından belirlenir.

Şekil 1.3 rol değişimini görünür kılar. Uygulama sunucusu, tarayıcının HTTP isteğini alıp HTTP yanıtı gönderdiği için tarayıcıyla ilişkisinde sunucudur. Aynı uygulama süreci veri tabanı hizmetinden veri istediğinde yeni alışveriş başlatır ve bu nedenle istemci olarak davranır. Veri tabanı sunucusu ikinci alışverişi yanıtlar. Etiketler her ilişkiyi açıklar; makinelere tek ve kalıcı kimlik atamaz.



Şekil 1.3. Uygulama sunucusu tarayıcıya sunucu, veri tabanına istemci olarak davranır.

2.3 HTML, CSS ve JavaScript'in görevleri farklıdır

Web'e tamamen yeni başlayan biri çoğu zaman bu üç adı birlikte duyar ve bunların birbirinin yerine geçebilen programlama dilleri olduğunu varsayar. Öyle değildir.

HTML, yani Hiper Metin İşaretleme Dili, belgenin yapısını ve anlamını açıklar. “Hiper metin”, belgelerin başka kaynaklara bağlantı verebilmesi demektir. “İşaretleme”, metnin başlıklar, paragraflar, bağlantılar, görseller, listeler ve formlar gibi ögelerle açıklanmasıdır. HTML öncelikle algoritma yazma dili değildir; belgenin parçalarının ne olduğunu açıklar.

CSS, yani Basamaklı Stil Sayfaları, sunumu açıklar: tipografi, renk, boşluk, yerleşim, duyarlı davranış ve işlemenin diğer yönleri. “Basamaklı” sözcüğü, farklı kaynaklardan gelen bildirimleri birleştiren ve çakışmaları çözen kural sistemini ifade eder. CSS süslemeden çok daha fazlasıdır; ekranlar, baskı, farklı yazı sistemleri, kullanıcı tercihleri ve erişilebilirlik gereksinimleri arasında yerleşimi denetler.

JavaScript bir programlama dilidir. Tarayıcıda olaylara yanıt verebilir, değer hesaplayabilir, veri isteyebilir, belgeyi güncelleyebilir ve tarayıcının sağladığı arayüzleri kullanabilir. JavaScript, Node.js gibi ortamlar aracılığıyla sunucular dâhil tarayıcı dışında da çalışabilir.

Benzer adlarına karşın JavaScript ile Java; tür sistemleri, yürütme ortamları ve yaygın kullanım alanları farklı programlama dilleridir. Modern JavaScript, ECMAScript adlı dil belirtimini izler. JavaScript motoru bu dili uygular; çevresindeki çalışma zamanı ise dilin parçası olmayan olanaklar ekler. Tarayıcı DOM'u, olayları ve Web API'lerini sağlar. Node.js dosyalar, süreçler, ağ sunucuları ve komut satırı programları için olanaklar sağlar. Bu nedenle işlev veya dizi iki ortamda da çalışırken `document` normalde yalnızca tarayıcıda bulunur.

Bu ders, HTTP sınırının iki tarafında da modern JavaScript'i birincil programlama dili olarak kullanır. Tarafları birleştirmeden önce ayrı ayrı öğretir. Öğrenciler önce dilin temellerini; ardından DOM ve olaylarla tarayıcı programlamayı, eşzamansız istekleri ve modülleri öğrenir. Daha sonra aynı dili Node.js üzerinden sunucu rotalarına ve veri erişimine uygular. Ortak dil kullanmak sınırı ortadan kaldırmaz: tarayıcı kodu kullanıcının aygıtında çalışır ve sunucu kurallarını uygulayacağına güvenilemez; sunucu kodu uygulamanın denetiminde çalışır ve

her isteği doğrulamalıdır. TypeScript yalnızca JavaScript ve modüller anlaşıldıktan sonra tanıtılır.

Tek bir etkinlik sayfasını düşünün. HTML, “Bu ana başlıktır; bu etkinlik tarihidir; bu da kayıt düğmesidir” der. CSS, “Bu punto ölçeğini kullan, metni okunaklı tut ve etkinlik kartlarını kullanılabilir genişliğe göre düzenle” der. JavaScript ise “Öğrenci filtreyi etkinleştirdiğinde eşleşen etkinlikleri iste ve bütün sayfayı değiştirmeden sonuçları güncelle” diyebilir. Üç teknoloji aynı arayüz hakkında farklı soruları yanıtlar.

Tarayıcı HTML'yi algoritma gibi çalıştırmaz. HTML'yi, canlı belgeyi temsil eden bellek içi ağaç olan Belge Nesne Modeli'ne (Document Object Model - DOM) ayrıştırır. CSS kuralları bu ağacın parçalarıyla eşleşerek sunuma katkıda bulunur. Tarayıcının JavaScript motoru JavaScript'i çalıştırır; JavaScript de tarayıcı arayüzleri aracılığıyla DOM'u okuyabilir veya değiştirebilir. Sonraki bölümlerde bu mekanizmalar ayrı ayrı incelenecektir. Şimdilik önemli sınır anlam, sunum ve davranış arasındadır.

Tablo 1.2, üç teknolojiyi öğrencilerin karşılaştığı dosya uzantılarına göre değil, sorumluluklarına göre ayırır.

Tablo 1.2 - Temel tarayıcı teknolojilerinin sorumlulukları

Teknoloji	Birincil sorumluluk	Basit örnek
HTML	Yapı ve anlam	“Bu metin sayfanın ana başlığıdır.”
CSS	Sunum ve yerleşim	“Bu kartları duyarlı bir ızgaraya yerleştir.”
JavaScript	Davranış ve durum	“Kullanıcı gönderdiğinde doğrula ve sayfayı güncelle.”

Şekil 1.4 aynı sayfayı üç aşamada gösterir. Soldan sağa okuyun: sade fakat anlamlı belge; profesyonel sunum eklenmiş aynı belge; etkileşim sonucunda durumu değişmiş, sunumu tamamlanmış belge.

Şekil 1.4. HTML yapıyı, CSS sunumu, JavaScript ise etkileşimli durumu sağlar.

Bu ayrım bir tasarım ilkesidir; mutlak fiziksel sınır değildir. JavaScript programı HTML ögeleri oluşturabilir ve modern CSS birçok koşula tepki verebilir. Yine de bir meselenin yapısal mı, sunumsal mı yoksa davranışsal mı olduğunu sormak genellikle daha açık kod üretir.

Bu ayrım dayanıklılığı da artırır. Stil sayfası yüklenemezse sağlam HTML belgesi sade görünse bile başlıklarını, bağlantılarını ve form denetimlerini sunmayı sürdürebilir. İsteğe bağlı JavaScript başarısız olduğunda sayfa aşamalı geliştirme (progressive enhancement) yaklaşımıyla tasarlanmışsa olağan gezinme ve sunucu destekli form çoğu zaman kullanılabilir kalır. Bu, her uygulamanın JavaScript olmadan çalışabileceği anlamına gelmez; temel görevlerin gerçekten gerektirdiklerinden daha fazla teknolojiye bağımlı olmaması gerektiğini anlatır.

2.4 Web sitesi, web uygulaması ve API

Web sitesi genellikle belgeleri ve gezinmeyi öne çıkarır. Web uygulaması çoğunlukla kullanıcı etkileşimini ve değişen durumu vurgular. Aralarındaki sınır standartlaştırılmış teknik çizgi değildir: üniversite portalı tek sistem içinde makaleler, arama, formlar, gösterge panoları ve gerçek zamanlı bildirimler barındırabilir.

Bu terimler bir vurguyu açıklar; birbiriyle uyuşmayan üç teknolojiyi değil. Haber sitesi etkileşimli arama içerebilir. Web uygulaması uzun biçimli belgelere sahip olabilir. Her ikisi de aynı URL'leri, HTTP anlamlarını, HTML'yi, CSS'yi, JavaScript'i, kimlik doğrulama sistemini ve sunucu altyapısını kullanabilir.

API, yani Uygulama Programlama Arayüzü (Application Programming Interface), yazılım bileşenlerinin iletişim kurması için tanımlanmış yoldur. Web API'si çoğu zaman HTTP isteklerini kabul eder ve eksiksiz HTML belgesi yerine JSON döndürür. Tarayıcı arayüzü ile API, aynı temel uygulamanın iki görünümü olabilir.

API bir sözleşmedir; herkese açılmış veri tabanı değildir ve kişinin ziyaret ettiği ekran olmak zorunda da değildir. Sözleşme işlemleri, girdileri, çıktıları, hataları ve kuralları tanımlar. Campus Hub'da herkese açık etkinlik sayfaları okumayı ve gezinmeyi; düzenleyici gösterge panosu düzenlemeyi ve durum değiştirmeyi öne çıkarabilir. API ise bu gösterge panosuna, mobil istemciye veya başka üniversite hizmetine etkinlik verisi sağlayabilir. Sunucu bilgiyi veri tabanından alabilir; ancak API istemcilerle iç depolama arasında durur.

Şekil 1.5 bu vurguları üç panelle karşılaştırır. Sol panel bağlantılı belgeleri, orta panel etkileşimli durum taşıyan arayüzü, sağ panel ise yazılım istemcileriyle hizmet arasındaki yapılandırılmış alışverişleri temsil eder. Altlarındaki ortak altyapı, tek üretim sisteminin üçünü birden sağlayabileceğini vurgular.

Şekil 1.5. Web sitesi, web uygulaması ve API farklı arayüzleri vurgular; ortak Web altyapısını paylaşır.

Yanıtın HTML veya JSON olması, yanıta kimlerin erişebileceğini belirlemez. API'ler de doğrulama, yetkilendirme, istek hızı denetimi, hata yönetimi ve dikkatle çizilmiş gizlilik sınırları gerektirir. İlerleyen bölümler, öğrenciler temeldeki HTTP alışverişlerini gözlemlemeyi öğrendikten sonra bu sorumlulukları ayrıntılandırır.

Kontrol noktası 1 - rolleri sınıflandırın

Tarayıcınız http://localhost:8000/index.html adresini istediğinde aşağıdaki ifadelerden hangileri doğrudur?

1 Bilgisayarınız hem istemci ana makinesi hem sunucu ana makinesidir.

2 Tarayıcı ile sunucu aynı programdır.

3 Bilgisayarınızdan hiçbir paket dışarı çıkmasa da alışveriş HTTP kullanabilir.

4 `localhost`, "öğretim elemanının sunucusu" anlamına gelir.

Yanıt: 1 ve 3. İstemci ve sunucu, yazılım rollerini açıklar. `localhost` normalde tarayıcının çalıştığı makinenin geridöngü (loopback) arayüzüne çözümlenir. Bu nedenle iki program aynı bilgisayarda çalışırken de istemci-sunucu alışverişi gerçekleşebilir.

3. URL'yi yapılandırılmış veri olarak okuyun

Tekdüzen Kaynak Konum Belirleyicisi (Uniform Resource Locator - URL), tarayıcının bir kaynağa başvurmak ve o kaynağa nasıl erişeceğini belirlemek için kullandığı adres benzeri tanımlayıcıdır. URL, gündelik bir etiket veya cümle değildir. Tanımlı bir dil bilgisine göre serileştirilmiş veridir; bu nedenle `:`, `/`, `?`, `&` ve `#` gibi karakterler belirli konumlarda kesin görevler üstlenir. Bir URL'yi bileşenlerine ayırarak okumak; gezinmeyi, güvenlik sınırlarını, yönlendirmeyi, önbelleğe almayı ve form gönderimini tahmin yürütmeden açıklamanın ilk adımıdır.

Aşağıdaki URL'yi inceleyin.

```
https://events.campus.example:8443/clubs/robotics?semester=fall&limit=20#schedule
\___/   \____________________/ \__/\_____________/ \___________________/ \_____/
 şema             ana makine      port       yol                 sorgu        parça
```

Şekil 1.6 aynı URL'yi etiketli bileşenlere dönüştürür; böylece her noktalama işareti ve alt dize bağımsız olarak incelenebilir.

Şekil 1.6. URL, bileşenleri farklı görevler üstlenen yapılandırılmış bir tanımlayıcıdır.

Tablo 1.3, etiketlenmiş her URL bileşeninin ne sağladığını açıklar.

Tablo 1.3 - URL bileşenleri ve amaçları

Bileşen	Amaç
Şema	URL'nin nasıl işleneceğini belirtir; `https`, taşıma güvenliğiyle birlikte HTTP kullanır.
Ana makine	Ağ ana makinesini belirtir. Alan adı bir IP adresine çözümlenebilir.
Bağlantı noktası	Ana makinedeki hizmet uç noktasını seçer. Varsayılan değer geçerliyse çoğu zaman yazılmaz.
Yol	Sunucunun adlandırma alanındaki bir kaynağı belirtir; kaynak fiziksel dosya olmak zorunda değildir.
Sorgu	Ek parametreler sağlar. Bunların anlamını uygulamanın sözleşmesi belirler.
Parça	Alınan kaynak içindeki ikincil konumu veya durumu belirtir. İstemci tarafından işlenir ve HTTP isteğinde gönderilmez.

HTTPS için geleneksel varsayılan bağlantı noktası `443`, HTTP için ise `80`'dir. URL'de bağlantı noktasının yazılmaması kavramın olmadığı anlamına gelmez; varsayılan değeri şema sağlar.

İki nokta karakteri hakkında bir not

İki noktanın tek ve evrensel bir anlamı yoktur; anlamı, yer aldığı gösterimden doğar. `https://` içinde iki nokta URL şemasını bitirir. `campus.example:8443` içinde ana makineyi açıkça yazılmış bağlantı noktasından ayırır. IPv6 adresi `2001:db8::10` içinde iki noktalar onaltılık sayı gruplarını ayırır; çift iki nokta ise sıfır gruplarından oluşan tek bir diziyi kısaltır. Olağan düzyazıda iki nokta yalnızca bir açıklamayı veya listeyi başlatabilir. Bu kitap, teknik değere ait noktalama işaretlerini çevreleyen cümlenin noktalamasından ayırmayı kolaylaştırmak için URL ve adres örneklerini kod biçiminde gösterir.

3.1 Yol bir addır; her zaman disk konumu değildir

İlk web siteleri `/notes/week1.html` yolunu çoğu zaman doğrudan `week1.html` adlı dosyayla eşlerdi. Bu eşleme hâlâ mümkündür; ancak modern sunucular sıklıkla bir yönlendirici kullanır. `/students/42` gibi bir yol, uygulamaya veri tabanından `42` numaralı öğrenciyi almasını ve bir yanıt üretmesini söyleyebilir. İstemci, gösterimin dosya, veri tabanı sorgusu, önbellek girdisi, işlev veya başka bir hizmet tarafından üretildiğini yalnızca URL'ye bakarak belirleyemez.

3.2 Sorgu verisi ve parçalar

Sorgu `?` ile başlar. Birden çok ad-değer çifti yaygın olarak `&` ile ayrılır; ancak uygulamanın bunları nasıl yorumlayacağı kendi sözleşmesine bağlıdır. Bu seçimlerin paylaşılabilir URL'de yer alması gerektiğinde filtreleme, sıralama, arama ve sayfalama gibi konular için sorgu parametreleri uygundur.

Sorgular özel değildir. Tarayıcı geçmişinde, sunucu günlüklerinde, analiz sistemlerinde, kopyalanmış bağlantılarda ve aracı sistem kayıtlarında görünebilir. Parolaları, oturum tanımlayıcılarını veya başka gizli bilgileri asla URL'ye koymayın.

Parça # ile başlar. Tarayıcı parçayı HTTP istek hedefine katmak yerine kendisinde tutar. Geleneksel belgede parça, `id` değeri kendisiyle eşleşen bir öğeyi belirleyebilir. İstemci tarafı uygulamalar parçalara başka yerel anlamlar verebilir. Bununla birlikte parça güvenli depolama alanı değildir; tarayıcı geçmişi, ekran görüntüleri, eklentiler ve istemci tarafı betikleri onu açığa çıkarabilir.

3.3 Köken ve aynı-köken mantığı

Tarayıcının Campus Hub etkinlik sayfasını aşağıdaki URL'den yüklediğini varsayın.

> Bu, açılacak bir web sitesi değil, kurgusal bir adrestir. `.example` son eki eğitim ve belgelemede kullanılan adlar için ayrılmıştır; bu nedenle `hub.campus.example` arkasında bilerek herkese açık bir Campus Hub sunucusu yoktur. Bazı PDF görüntüleyicileri URL biçimindeki basılı metni otomatik olarak bağlantı gibi gösterebilir. Bu örneği açarsanız `DNS_PROBE_FINISHED_NXDOMAIN` iletisi beklenen sonuçtur: Alan Adı Sistemi (Domain Name System - DNS) bu kurgusal adı bulamamıştır. Daha sonraki bir etkinlik çalışan sayfa gerektirdiğinde kitap açıkça bir `localhost` adresi veya doğrulanmış herkese açık bağlantı verir.

```
https://hub.campus.example/events
```

Tam URL bir kaynağı tanımlar; fakat tarayıcı şu güvenlik sorusuna kararlı biçimde yanıt vermelidir: Başka hangi belgeler ve hizmetler bu sayfayla aynı kaynağa aittir? Tarayıcı bu soruyu sayfanın kökenini hesaplayarak yanıtlar.

Olağan bir HTTP veya HTTPS URL'sinde köken tam olarak üç değerden oluşur:

1 `http` veya `https` gibi şema;

2 `hub.campus.example` gibi ana makine adı;

3 HTTPS için `443` gibi etkin bağlantı noktası.

Burada *etkin* sözcüğü önemlidir. URL bağlantı noktasını açıkça yazmak zorunda değildir. HTTPS varsayılan olarak `443`, HTTP ise `80` kullanır. Bu nedenle `https://hub.campus.example` ile `https://hub.campus.example:443` aynı kökene sahiptir. İlk URL bağlantı noktasını yazmaz; ancak etkin bağlantı noktası yine `443`'tür.

Campus Hub sayfasının kökeni şöyle yazılabilir:

```
https://hub.campus.example
```

Nelerin kaybolduğuna dikkat edin. `/events` yolu kökenin parçası değildir; sorgu ve parça da değildir. Şekil 1.6 bütün URL bileşenlerini gösterir. Köken karşılaştırmasında yalnızca şemayı, ana makine adını ve etkin bağlantı noktasını kullanın. Yollar kaynakları ayırır, fakat farklı köken oluşturmaz.

İki kökeni mekanik olarak karşılaştırma

İki URL'nin “ilişkili görünüp görünmediğine” bakarak karar vermeyin. Her seferinde aynı dört adımı uygulayın:

1 Gerekirse göreli başvuruyu çözümleyerek iki değeri de tam URL yapın.

2 Şemaları karşılaştırın; farklılarsa kökenler farklıdır.

3 Ana makine adlarını karşılaştırın; farklılarsa kökenler farklıdır.

4 Yazılmamış bağlantı noktası için şemanın varsayılanını ekleyerek etkin bağlantı noktalarını karşılaştırın. Üç karşılaştırma da eşleşirse URL'ler aynı kökenlidir.

Bu adımları sayfa URL'si `https://hub.campus.example/events` için uygulayın.

- `https://hub.campus.example/clubs?active=true` aynı kökenlidir. Yalnızca yol ve sorgu farklıdır.
- `https://hub.campus.example:443/help` aynı kökenlidir. Yazılan `443`, HTTPS'in varsayılan bağlantı noktasıdır; bu nedenle iki etkin bağlantı noktası da `443`'tür.
- `http://hub.campus.example/events` farklı kökenlidir. Şema `https` yerine `http`'dir; varsayılan bağlantı noktası da `443` yerine `80`'dir.
- `https://api.campus.example/events` farklı kökenlidir. `api.campus.example` ile `hub.campus.example` farklı ana makine adlarıdır. İkisinin de `campus.example` ile bitmesi onları aynı kökenli yapmaz.
- `https://hub.campus.example:8443/events` farklı kökenlidir. Ana makine adı eşleşir; fakat `8443`, etkin bağlantı noktası `443` ile eşleşmez.

Bu kural yerel geliştirmede öğrencileri sıkça şaşırtır. `http://localhost:8000` sayfası ile `http://localhost:3000` adresindeki Uygulama Programlama Arayüzü (Application Programming Interface - API) aynı bilgisayarda çalışır; fakat bağlantı noktaları farklı olduğu için farklı kökenlidir. Benzer biçimde `http://localhost:8000` ile `http://127.0.0.1:8000`, `localhost` ve `127.0.0.1` farklı ana makine değerleri olduğundan farklı kökenlidir. “Aynı makine” ile “aynı köken” farklı soruları yanıtlar.

Tarayıcı bu sınıra neden ihtiyaç duyar?

Bir öğrencinin özel Campus Hub sayfasında oturum açtığını, ardından başka sekmede ilişkisiz ve kötü niyetli bir sayfa açtığını düşünün. Her betik tarayıcının ulaşabildiği her kökendeki veriyi serbestçe okuyabilseydi kötü niyetli sayfa öğrencinin özel ders programını, notlarını veya kurum içi sayfalarını okumaya çalışabilirdi. Aynı-köken ilkesi, bir kökendeki belge veya betiğin başka kökendeki kaynaklarla nasıl etkileşebileceğini sınırlar. Bu, tarayıcının temel yalıtım mekanizmalarından biridir.

İlke, “kökenler arası trafik hiçbir zaman izinli değildir” demez. Tarayıcılar kullanıcının başka kökene bağlantı izlemesine, formun başka kökene gönderilmesine ve sayfanın görsel gibi bazı kökenler arası kaynakları göstermesine genellikle izin verir. Önemli kısıtlamalar etkileşim türüne bağlıdır. Özellikle JavaScript, alıcı sunucu bu erişime bilerek izin vermedikçe kökenler arası `fetch()` yanıtını normalde okuyamaz.

Kökenler Arası Kaynak Paylaşımı (Cross-Origin Resource Sharing - CORS), sunucunun seçilmiş kökenler arası erişime izin verebildiği HTTP mekanizmasıdır. CORS iki kökeni tek kökene dönüştürmez ve tarayıcı kodu kendisine izin veremez. Sunucu politikasını yanıt üstbilgilerinde bildirir; tarayıcı sonucu uygular. 6. Bölüm bu istek-yanıt sürecini ayrıntılandırır.

Kökeni gündelik ve daha gevşek “web sitesi” sözcüğünden de ayırın. Tek web sitesi sayfalar için `www.campus.example`, veriler için `api.campus.example` gibi birkaç köken kullanabilir. Adlar aynı kuruluşa ait olsa da tarayıcı onları farklı kökenler olarak görür. İleride aynı-köken ile aynı-site kavramları da ayrılacaktır; bunlar birbirinin yerine kullanılamaz.

Devam etmeden önce kuralı sınayın

Bir sayfa `http://localhost:8000/week-01/` adresinden yüklenmiştir. Her hedefin sayfayla aynı kökenli olup olmadığına karar verin.

- `/images/campus.svg` aynı kökenlidir; göreli URL sayfanın şemasını, ana makine adını ve bağlantı noktasını devralır.
- `http://localhost:8000/api/events` aynı kökenlidir; üç köken değerinin tümü eşleşir.

- `http://localhost:3000/api/events` farklı kökenlidir; bağlantı noktası farklıdır.
- `https://localhost:8000/api/events` farklı kökenlidir; ana makine adı ile yazılan bağlantı noktası eşleşse de şema farklıdır.
- `http://127.0.0.1:8000/api/events` farklı kökenlidir; iki ad aynı bilgisayara ulaşabilse bile ana makine değeri farklıdır.

Korunacak alışkanlık kesindir: şemayı, ana makine adını ve etkin bağlantı noktasını karşılaştırın. Yolu karşılaştırmayın, kurumsal sahipliğe bakarak karar vermeyin ve “aynı bilgisayar” ifadesini “aynı köken” yerine koymayın. İlerleyen bölümler bu modeli çerezlere, tarayıcı depolamasına, `iframe` öğelerine, CORS'a ve içerik yalıtımına bağlayacaktır.

3.4 Kodlama ve normalleştirme

URL'ler tanımlı ayrıştırma ve yüzde kodlama kurallarına göre serileştirilir. Boşluk `%20` biçiminde görünebilir; form kodlu sorgu verisi ise boşluğu çoğu zaman `+` ile temsil eder. Görünüşte benzer veya farklı kodlanmış URL'ler gelişigüzel bakılarak güvenle karşılaştırılamaz. Uygulamalar URL dizelerini elle bölmek yerine standartlara uygun URL ayrıştırıcısı kullanmalıdır.

Ana makine adının soldan sağa görünümü yeni başlayanları yanıltabilir. `accounts.university.example` içinde kayıtlı alan adı sınırı adlandırma politikasına bağlıdır; DNS hiyerarşisi sağa doğru daha genel hâle gelir. Ana makine adının herhangi bir yerinde tanıdık sözcük görülmesini sitenin güvenilir olduğuna kanıt saymayın.

Çalıştırmadan önce tahmin edin

Aşağıdaki temel URL verildiğinde başvuruların hedeflerini tahmin edin:

```
https://hub.campus.example/events/today
../clubs
/help
?format=json
#registration
```

Yanıtlar:

- `../clubs` -> `https://hub.campus.example/clubs`
- `/help` -> `https://hub.campus.example/help`
- `?format=json` -> `https://hub.campus.example/events/today?format=json`
- `#registration` -> `https://hub.campus.example/events/today#registration`

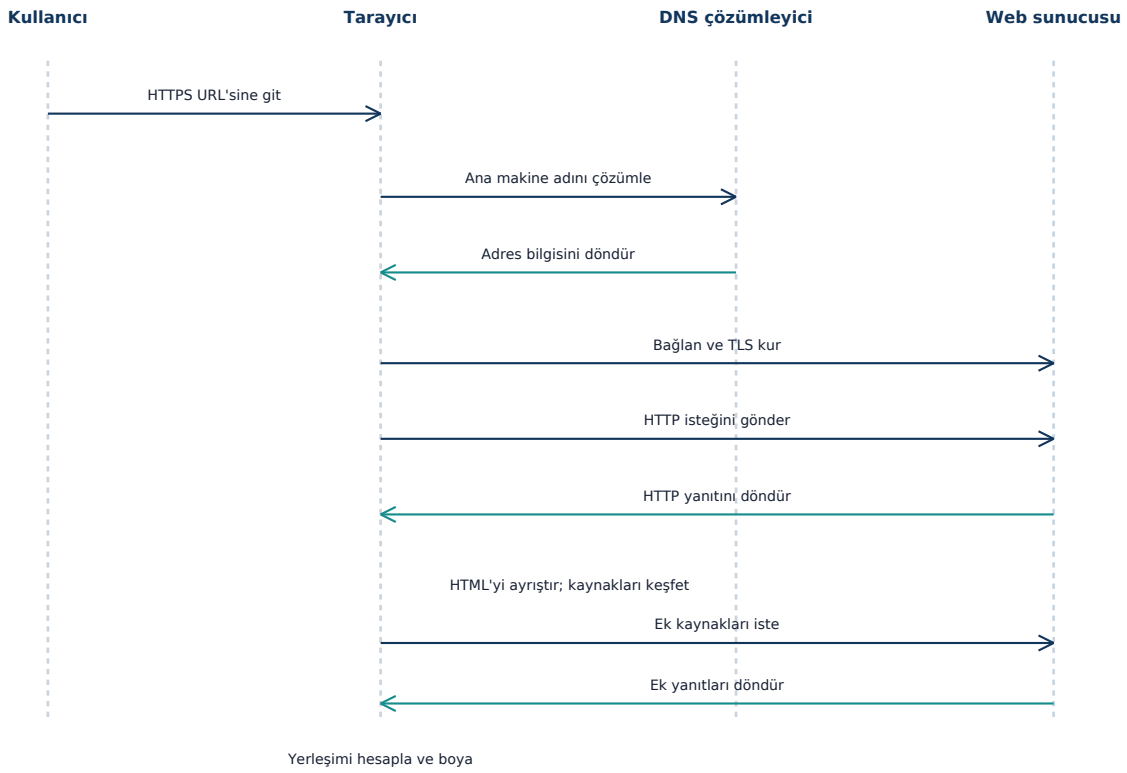
Göreli URL çözümleme tanımlı kurallara uyar; basit dize birleştirme değildir. Göreli başvuru bir temel URL'ye bağlıdır. HTML belgesinde bir `base` öğesi açıkça değiştirmedikçe temel normalde belge URL'sidir. Bu nedenle `../clubs` gibi aynı metin, farklı konumlardaki belgelerde farklı hedefleri belirleyebilir.

4. Gezinmeden piksellere

Aşağıdaki model bilinçli olarak sadeleştirilmiştir; ancak her adım gözlemlenebilir veya kitabın ilerleyen bölümlerinde daha ayrıntılı hâle getirilebilir.

Dizide iki kısaltma vardır. Alan Adı Sistemi (Domain Name System - DNS) adlandırma bilgisi sağlar; Taşıma Katmanı Güvenliği (Transport Layer Security - TLS) güvenli bağlantıyı korur. Şekilden sonraki bölümler bu protokollerin nereye oturduğunu ve neleri yapmadığını açıklar.

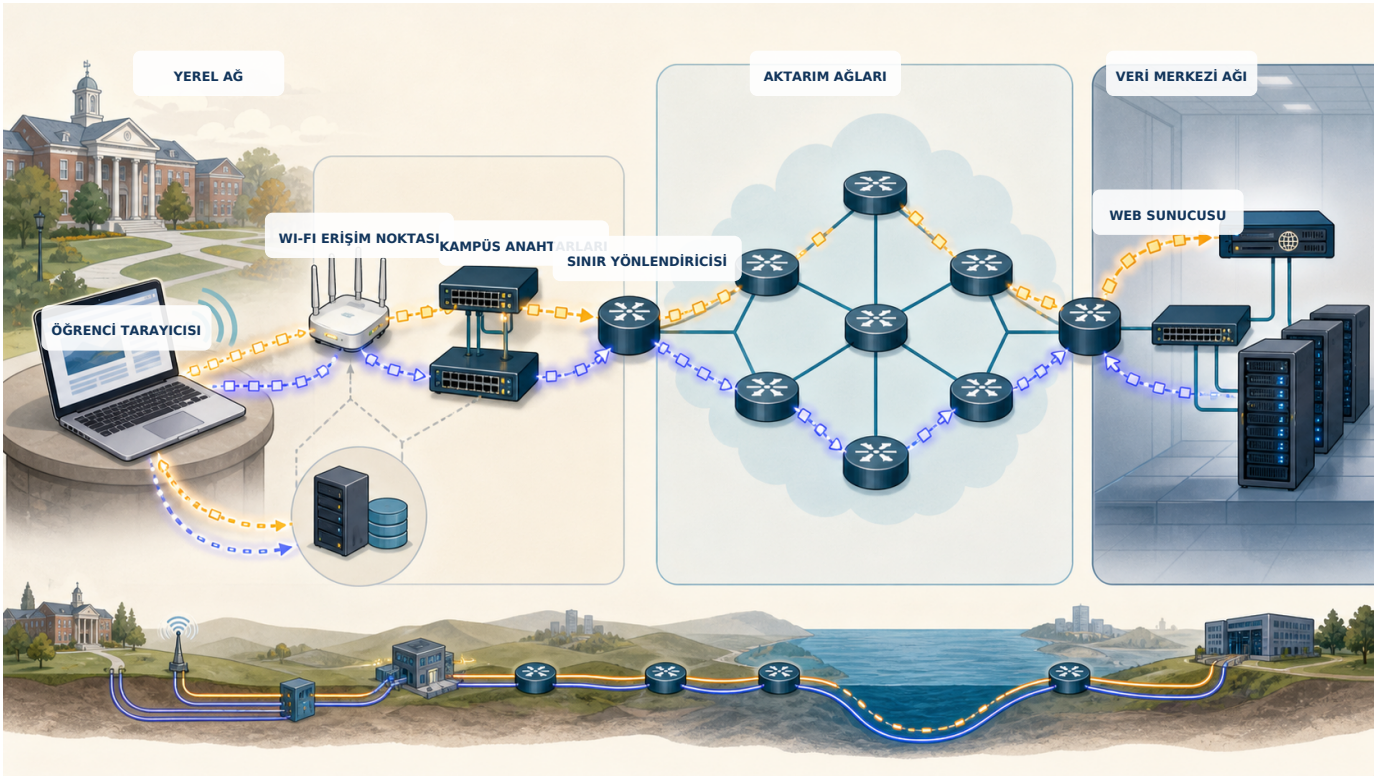
Şekil 1.7, bu sorumlulukları ayrı ayrı incelemeden önce bütün sayfa gezinme dizisini izler.



Şekil 1.7. Sayfa gezinmesi adlandırma, bağlantı güvenliği, HTTP, ayrıştırma ve görüntülemeyi koordine eder.

Şekil 1.7 bir dizi diyagramıdır: her katılımcının ne zaman davrandığını ve her iletiyi kimin gönderdiğini vurgular. Nedensel sıranın okunabilir kalması için fiziksel ağın büyük bölümünü bilerek gizler. Tarayıcı normalde sunucuya kesintisiz tek kabloyla bağlanmaz; verileri yerel ve uzak ağ bağlantılarından oluşan bir diziyi geçer.

Şekil 1.8 aynı gezinmeyi mekânsal açıdan gösterir. Dizüstü bilgisayar Wi-Fi çerçevesini erişim noktasına gönderebilir; kampüs ağı veriyi anahtarlar ve yönlendirici üzerinden taşır; bağımsız işletilen aktarım ağları IP paketlerini iletir; veri merkezi ağı sonunda paketi sunucuya ulaştırır. Yanıt verisi birbirine bağlı ağlardan geri döner, ancak ters yönde tam olarak aynı yolu izlemesi garanti değildir. Yönlendiriciler paketleri adreslere doğru iletir; sayfanın başlıklarını, CSS kurallarını veya JavaScript davranışını yorumlamaz.



Şekil 1.8. İstek yerel, aktarım ve veri merkezi ağlarını geçer; yanıt bağlantılı yollar üzerinden geri döner.

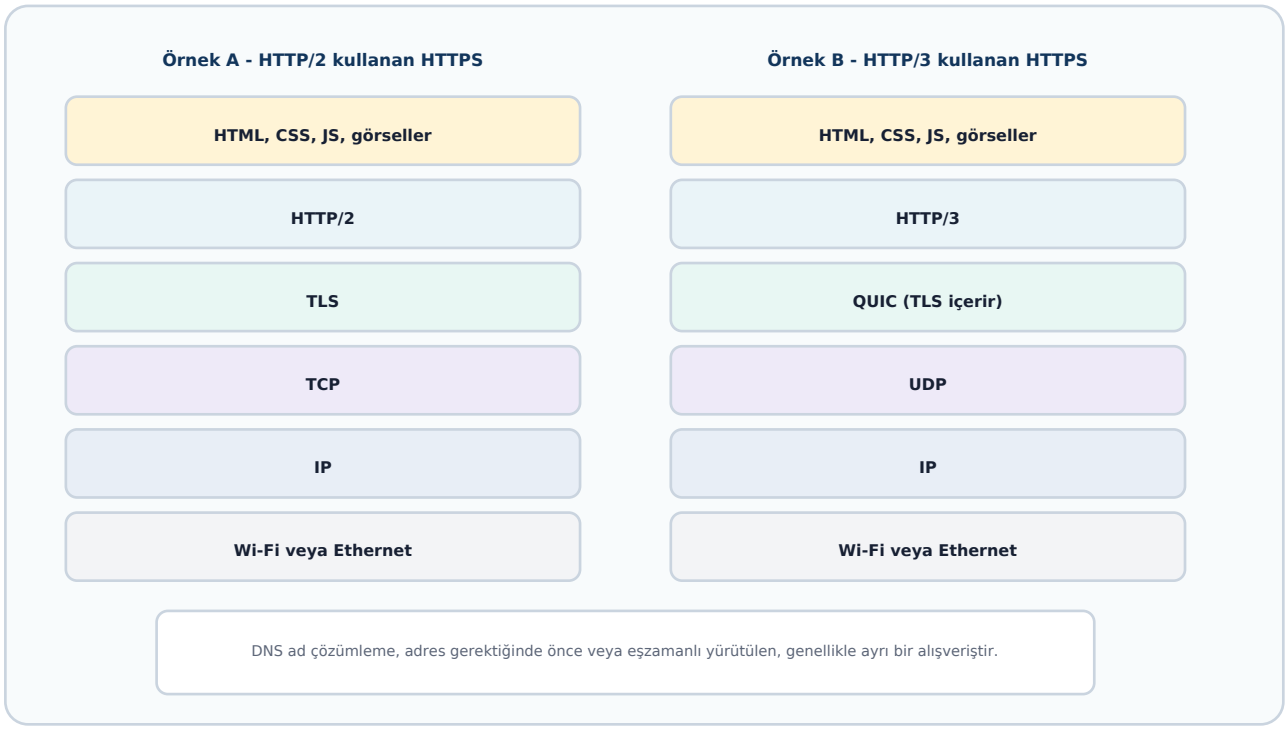
Başlangıca yakın gösterilen DNS sorgusu, sayfa isteğinin içindeki başka bir kablo değil, ilişkili bir alışveriştir. İstemcinin hedef bilgilerini edinmesine yardım eder. Tarayıcı önbellekte bulunan veya yeni çözümlenen uygun adres bilgisine sahip olduğunda HTTPS için kullanılacak bağlantıyı kurabilir. Şekil 1.7 ile Şekil 1.8 iki tamamlayıcı görünüm sunar: biri işlerin sırasını, diğeri verinin nereden geçtiğini açıklar.

4.0 Protokol yığını: karışıklık olmadan iş birliği

Tarayıcı “web sitesi paketi” adlı tek ve özel bir nesne göndermez. Sayfa isteği, birkaç protokolün birlikte çalışmasıyla tamamlanır. *Yığın* sözcüğü yararlıdır; çünkü her katman üstündeki katmana hizmet sağlar ve altındaki hizmete dayanır. Katmanlar, tarayıcının yalnızca birini seçtiği menü seçenekleri değildir.

Tek güvenli sayfa isteğinde aynı genel alışveriş boyunca birkaç katman etkindir. Tarayıcı HTTP isteği oluşturabilir. Taşıma Katmanı Güvenliği (TLS) bu HTTP verisini korur. İletim Denetim Protokolü (Transmission Control Protocol - TCP) korunan baytları güvenilir biçimde taşıyabilir. İnternet Protokolü (IP) paketleri adresler ve yönlendirir. Son olarak Wi-Fi veya Ethernet gibi bağlantı teknolojisi, her yerel ağ bağlantısında çerçeveleri taşır. Yönlendirici bir bağlantı katmanı çerçevesini çıkarıp sıradaki bağlantı için yenisini kurarken IP paketi hedefe doğru ilerlemeyi sürdürür.

Bazı yığın girdileri alternatiftir; çünkü farklı protokol sürümleri aynı katmanın işini farklı biçimde çözer. Hiper Metin Aktarım Protokolü sürüm 2 (HTTP/2) yaygın olarak TCP üzerinde TLS kullanır. Hiper Metin Aktarım Protokolü sürüm 3 (HTTP/3), Kullanıcı Datagram Protokolü (User Datagram Protocol - UDP) üzerinde güvenli taşıma protokolü QUIC'i kullanır. QUIC, HTTP/3'ün gerektirdiği güvenlik el sıkışmasını içerdiğinden HTTP/3 yığını, HTTP/2 ve TCP örneğindeki gibi QUIC'in altına ayrı TLS kutusu yerleştirmez. İki örnek de IP ve bağlantı teknolojisi kullanır.



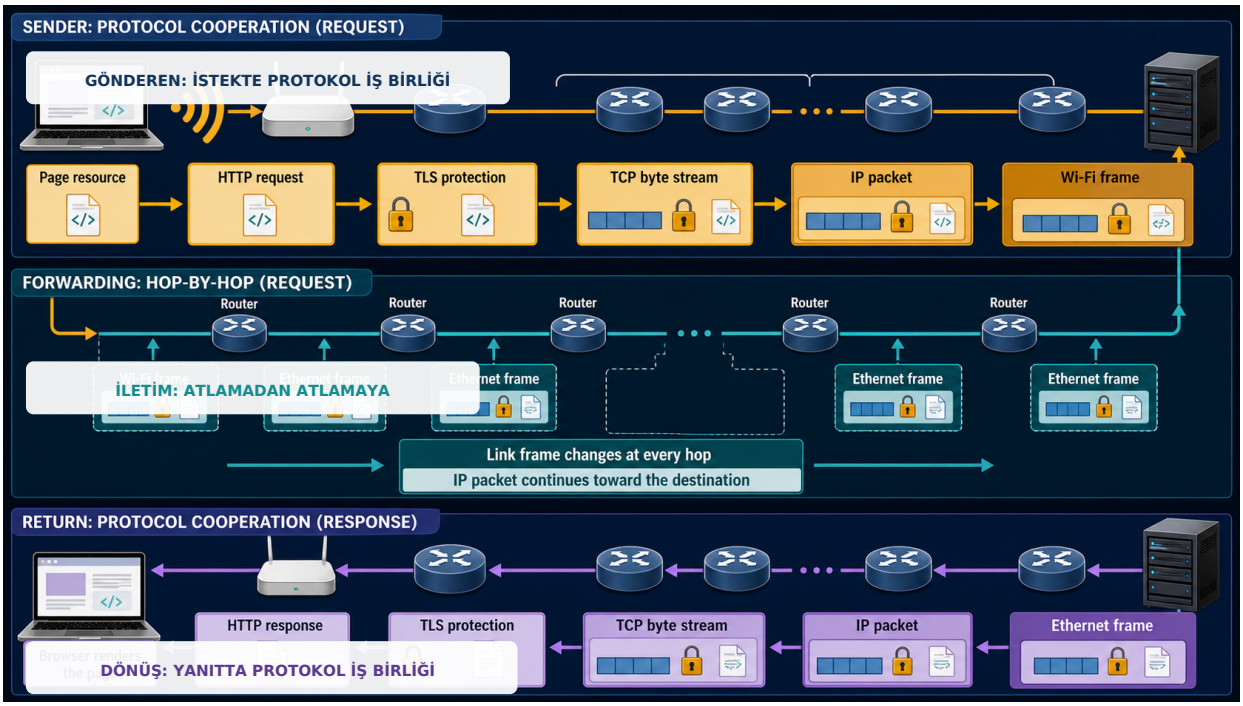
Şekil 1.9. Yaygın iki HTTPS yığını. Her sütunu web içeriğinden ağ bağlantısına doğru okuyun.

Şekil 1.9'daki her sütunu yukarıdan aşağıya okuyun. Sütun, sayfa alışverişinde kullanılabilecek bir birleşimi temsil eder; yalnızca bir kutunun çalıştığı anlamına gelmez. Aynı sütundaki kutular iş birliği yapar. İki sütun, protokol desteği ve uzlaşma sonucunda seçilen alternatiflerdir.

Başlangıç düzeyinde yararlı sorumluluklar şunlardır:

- Alan Adı Sistemi (DNS) sorgusu, çoğu zaman adres olmak üzere türü belirtilmiş adlandırma bilgisini edinir. Genellikle tarayıcı sayfa bağlantısını açmadan önce yürütülen ayrı alışveriştir; DNS sayfanın HTTP isteğinin içinde taşınmaz.
- İnternet Protokolü (IP), paketleri birbirine bağlı ağlardaki ağ arayüzleri arasında taşır.
- İletim Denetim Protokolü (TCP), HTTP/1.1 ve HTTP/2 gibi protokoller için güvenilir ve sıralı bayt akışı sağlar.
- QUIC, Kullanıcı Datagram Protokolü (UDP) üzerinde güvenli ve çoklanmış taşıma sağlar; HTTP/3 tarafından kullanılır.
- Taşıma Katmanı Güvenliği (TLS), veriyi kimlik doğrulama, gizlilik ve bütünlükle korur. HTTP/2 örneğinde HTTP ile TCP arasındadır; HTTP/3 karşılık gelen korumayı QUIC üzerinden edinir.
- Hiper Metin Aktarım Protokolü (HTTP), kaynaklarla ilgili isteklerin ve yanıtların anlamını tanımlar.
- Sayfa içeriği HTML, CSS, JavaScript, görseller ve JavaScript Nesne Gösterimi (JavaScript Object Notation - JSON) içerir. Bu kaynakları uç noktalardaki uygulamalar yorumlar.

Bu model bilinçli olarak sadeleştirilmiştir. Gerçek sistemler bağlantıları yeniden kullanabilir, DNS trafiğini şifreleyebilir, vekil sunuculardan geçebilir veya yolun farklı noktalarında farklı bağlantı teknolojileri seçebilir. Temel düşünce değişmez: birkaç protokol farklı işler yaptığı için birlikte etkin olabilir; TCP ile QUIC ise taşıma işi için alternatif örneklerdir. Alt taşıma ayrıntıları değişse bile `GET`, `404` ve `Content-Type` uygulama anlamını korur.



Şekil 1.10. HTTPS alışverişi uygulama, güvenlik, taşıma, ağ ve bağlantı sorumluluklarını birleştirir.

Şekil 1.10, soyut katmanları ağ üzerindeki bir yolculuğa dönüştürür. Göndericide sayfa kaynağı bir HTTP isteğinin içeriğine dönüşür. TCP tabanlı örnekte TLS, HTTP verisini korur; TCP sıralı bayt akışı sağlar; IP, yönlendiricilerin kullandığı paket adreslemesini sağlar; ilk yerel bağlantı da paketi Wi-Fi çerçevesi içinde taşır. Bu sarmalama işleminde her katman kendi işi için gereken denetim bilgisini ekler.

Yönlendiricide gelen bağlantı çerçevesi yerel yolculuğunun sonuna ulaşır. Yönlendirici bu çerçeveyi çıkarır, iletme için gereken IP bilgisini inceler ve paketi Ethernet gibi sonraki bağlantıya uygun yeni bir çerçeveye yerleştirir. Gerçek baytlar ve üstbilgiler Şekil 1.10'un gösterebileceğinden daha ayrıntılıdır; ancak sınır önemlidir: yönlendiriciler IP paketini atlama atlama iletir, tarayıcı ve sunucu uç noktaları HTTP'yi ve sayfa içeriğini yorumlar. Dönüş yanıtı aynı iş birliği yapan sorumlulukları uygulama yönünde tersine kullanır.

4.1 Ad çözümleme

İnsanlar alan adlarını tercih eder; ağ yönlendirmesi ise sonunda adres bilgisine ihtiyaç duyar. Tarayıcı ve işletim sistemi önbelleklere ve bir DNS çözümleyiciye danışır. Çözümleme zorunlu olarak tek bir merkezî dizine yapılan tek yolculuk değildir. DNS dağıtık ve hiyerarşiktir; yanıtlar belirli bir süre için önbelleğe alınabilir.

Önemli düzeltme: DNS sayfayı indirmez. İstemcinin isteği nereye göndereceğini bulmasına yardım eder.

DNS aslında nedir?

Alan Adı Sistemi (DNS), dağıtık ve hiyerarşik bir veri tabanı ile sorgu protokolüdür. Her adın yanında türü belirtilmemiş tek bir yanıt saklamaz. Bunun yerine kaynak kayıtları adı verilen türü belirli girdiler saklar. İstemci bir ad ve kayıt türü sorar; tür, DNS'in istediği bilgi türünü belirtir. Adı adrese dönüştürmek en bilinen kullanımdır; aynı sistem e-posta teslimi, yetkili sunucular, takma adlar ve başka hizmetlerle ilgili bilgi de saklar.

Bir kaynak kaydının alanları görünür olduğunda anlaşılması kolaylaşır. Aşağıdaki öğretim örneğini inceleyin:

```
hub.campus.example.    300    IN   A   192.0.2.40
```

Kaydı soldan sağa okuyun.

- `hub.campus.example.` kaydın ait olduğu DNS adıdır. Son nokta DNS ağacının kökünü açıkça gösterir; adın yerel bir son eke göreli değil, tam olduğunu belirtir.

- `300`, saniye cinsinden yaşam süresidir (Time to Live - TTL). Önbellek bu kaydı yeni bilgi istemeden önce normalde 300 saniye yeniden kullanabilir.

- `IN`, İnternet sınıfıdır. DNS birden çok veri sınıfını destekleyecek şekilde tasarlanmıştır; öğrencilerin olağan İnternet kullanımında karşılaşacağı sınıf `IN`'dir.

- `A`, kayıt türüdür ve son alanın nasıl yorumlanacağını söyler.

- `192.0.2.40`, kaydın verisidir. A kaydında bu veri tek bir IPv4 adresidir; burada kullanılan belgeleme adresi herkese açık bir Campus Hub sunucusu değildir.

192.0.2.40'ı IPv4 adresi olarak okumak

İnternet Protokolü sürüm 4 (IPv4) adresleri tam olarak 32 ikili rakamdan, yani bitten oluşur. İnsanların okuyabileceği gösterimde bu 32 bit dört adet sekiz bitlik gruba ayrılır. Sekiz bitlik gruba oktet denir. Her oktet olağan ondalık sayıya dönüştürülür ve yazılan dört sayı noktalarla ayrılır:

```
192 . 0 . 2 . 40
 |    |   |    |
oktet oktet oktet oktet
```

Sekiz bit, 0'dan 255'e kadar 256 farklı değer gösterebilir. Bu nedenle noktalı ondalık IPv4 oktetlerinin her biri 0-255 kapsayıcı aralığında olmalıdır. `192.0.2.40` ve `10.24.0.255` sözdizimi bakımından geçerli örneklerdir; `192.0.2.256`, `192.0.-1.40` ve `192.0.2` ise geçerli tam IPv4 adresleri değildir.

```
192        0          2            40
11000000   00000000   00000010     00101000
```

Noktalar yazılı gösterime aittir; paketi dört ayrı iletime bölmez. Dört sayı da `hub.campus.example` içindeki dört ad düzeyine karşılık gelmez. Alan adı ile IP adresi, farklı amaçlar için farklı yapılar kullanır.

IPv4 adresi yönlendirme için bir IP uç noktasını veya ağ arayüzünü tanımlar. Tek başına bir uygulama sürecini tanımlamaz; ana makinedeki hizmeti seçmeye TCP'nin geleneksel HTTPS için `443` bağlantı noktası gibi taşıma katmanı bağlantı noktası yardım eder. Adres tek başına ağ önekiyle ana makine bölümünün nerede bittiğini de göstermez. Bu sınırı önek uzunluğu sağlar. Örneğin `192.0.2.0/24`, ilk 24 bitin ağ öneki olduğunu ve kalan sekiz bitin bu önek içinde değiştiğini belirtir.

`192.0.2.0/24` bloğu belgeleme ve örnekler için ayrılmıştır. Bu nedenle `192.0.2.40` kitap için uygundur; gerçek Campus Hub herkese açık adresi olarak sunulmamalıdır. Gerçek DNS sorgusunda A kaydı, alan işletmecisinin seçtiği bir veya birden çok çalışan IPv4 adresi döndürebilir.

IPv6 adresini okuma

İnternet Protokolü sürüm 6 (IPv6), çok daha büyük adres alanı ve İnternet katmanı işletiminde iyileştirmeler sağlamak üzere tasarlanmıştır. IPv6 adresi tam 128 bit içerir; bu, IPv4 adresinin dört katı bittir. Bu durum IPv6 adresinin dört IPv4 adresi olarak yazıldığı anlamına gelmez. IPv6'nın kendine özgü gösterimi ve işletim kuralları vardır.

Boyut farkı çok büyüktür. IPv4'te 2^32, yani yaklaşık 4,3 milyar olası bit deseni vardır. IPv6'da 2^128, yani yaklaşık 3,4 x 10^38 olası bit deseni vardır. Adresleme politikaları bu alanların bazı bölümlerini ayırır veya yapılandırır; dolayısıyla iki sayı da bağlanabilecek aygıtların basit sayısı değildir. Karşılaştırma, IPv6'nın adresleri korumak için yaygın adres paylaşımına dayanmak zorunda kalmadan cömert ağ önekleri atayabilmesini açıklar.

128 ayrı sıfır ve biri yazmak hata üretmeye açık olduğundan IPv6, taban-16 sayı sistemi olan onaltılık gösterimi kullanır. Onaltılıkta 0-9 ve ardından ondalık 10-15 değerleri için a-f rakamları bulunur. Her onaltılık rakam dört biti temsil eder; dört onaltılık rakam 16 bit eder.

Tam yazım biçimi 128 biti sekiz adet 16 bitlik gruba ayırır. Gruplar iki noktayla ayrılır:

```
2001 : 0db8 : 0000 : 0000 : 0000 : 0000 : 0000 : 0040
 |       |      |      |      |      |      |      |
16 bit  16 bit 16 bit 16 bit 16 bit 16 bit 16 bit 16 bit
```

Her grup 0000-ffff kapsayıcı onaltılık aralığındadır. Harf büyüklüğünün sayısal anlamı yoktur; `0DB8` ile `0db8` aynı değerdir. Bu kitapta kullanılan geleneksel biçim küçük harflidir. İki noktalar yazılı adresin ayırıcılarıdır; IPv6 paketinde karakter olarak iletilmez.

Örneğin onaltılık `2001` şu on altı biti temsil eder:

```
onaltılık      2      0      0      1
ikili         0010   0000   0000   0001
```

IPv6 gösterimi iki kısaltma kuralı sağlar:

1 Bir grup içindeki baştaki sıfırları atlayın. `0db8` -> `db8`, `0040` -> `40` ve `0000` -> `0` olur. Sıfır olmayan grubun sağındaki sıfırı atmayın: `0040`, `4` değil `40` olur.

2 Tamamı sıfır olan ardışık grup dizisini `::` ile değiştirin. Çift iki nokta bir adreste yalnızca bir kez görünebilir; iki sıkıştırılmış dizi olursa her konuma kaç sıfır grubunun ait olduğu bilinemeyecektir.

İki kural birlikte uygulanınca tam adres `2001:db8::40` olur.

Bu kurallar sözdizimi denetimi için de yararlıdır. `2001:db8::40`, `::1` ve `fe80::a:b:c:d` geçerli IPv6 yapısına sahiptir. `2001::db8::40`, iki çift iki nokta sıkıştırması içerdiği için geçersizdir. `2001:db8:12345::40` içinde beş onaltılık rakamlı grup vardır ve geçersizdir. `2001:db8::g` içinde onaltılık rakam olmayan `g` vardır. Sözdizimi geçerli bir adres ayrılmış, atanmamış, erişilemez veya belirli bir arayüz için uygunsuz olabilir.

Sıkıştırılmış adresi genişletmek için görünür 16 bitlik grupları sayın ve `::` konumuna toplam sekiz grup olacak kadar `0000` ekleyin. `2001:db8::40` içinde üç görünür grup bulunduğundan çift iki nokta beş sıfır grubunu temsil eder. Böylece aşağıdaki biçimler aynı 128 bitlik adresi açıklar:

```
2001:0db8:0000:0000:0000:0000:0000:0040
2001:db8:0:0:0:0:0:40
```

```
2001:db8::40
```

Bir adres, `::` kullanmadan da baştaki sıfırları atlayabilir. Tersine `::`, birkaç sıfır grubunu veya yalnızca tek bir sıfır grubunu temsil edebilir. Örneğin `::1`, yedi sıfır grubunun ardından `0001` gelir ve geçerli bilgisayarı gösteren IPv6 geridöngü arayüzünü belirtir. `::` belirsiz adrestir; sıradan bir kaynak adresi henüz seçilmediğini ifade eder ve bir bağlantıya yazılacak herkese açık hedef değildir.

Tek bir sıfır grubunu sıkıştırmak sözdizimi bakımından mümkün olsa da önerilen standart metin biçimi yalnızca bir grup için `::` kullanmaz. Birden çok sıfır dizisi eşit uzunluktaysa önerilen biçim ilk diziyi sıkıştırır. Bu kurallar günlüklerin, yapılandırma dosyalarının ve karşılaştırma araçlarının aynı adres için öngörülebilir tek bir yazım kullanmasına yardım eder.

IPv4'te olduğu gibi önek uzunluğu ağ bölümünü tanımlar. `2001:db8:1234:5678::/64`, ilk 64 bitin ağ öneki, kalan 64 bitin alt ağ içindeki arayüz tanımlayıcısı olduğunu belirtir. Çoğu sıradan IPv6 yerel ağ alt ağında `/64` kullanılır; başka önek uzunluklarının da tanımlı görevleri vardır. Eğik çizgi adresin kendisine değil, adres-önek gösterimine aittir.

Tanımak için yararlı birkaç adres kapsamı vardır:

- Genel tekil adresler, yönlendirme ve güvenlik politikasına bağlı olarak IPv6 ağları arasında yönlendirilebilir. Geniş biçimde ayrılmış genel tekil alan `2000::/3` içinde başlar.
- Yerel bağlantı adresleri `fe80::/10` kullanır ve yalnızca yerel ağ bağlantısında çalışır. Yönlendiriciler bunları başka bağlantılara iletmez. İşletim sistemi hangi yerel arayüzün kastedildiğini göstermek için `fe80::12ab%7` gibi bölge tanımlayıcısı gösterebilir; `%7` arayüz gösterimidir, 128 bitlik adresin parçası değildir.
- Geridöngü `::1`'dir ve geçerli ana makineyi belirtir.
- Çoklu gönderim adresleri `ff00::/8` ile başlar ve alıcı grubunu belirtir. IPv6'da yayın adresi yoktur; IPv4'ün yayın için kullandığı durumlarda çoklu gönderim ve başka mekanizmalar görev alır.

`2001:db8::/32` öneki belgeleme için ayrılmıştır. Bu nedenle `2001:db8::40` bu bölüm için uygundur; gerçek Campus Hub sunucusu olarak tanımlanmamalıdır. Üretim ana makinesinin sabit ve bir veya daha çok geçici gizlilik adresi dâhil olmak üzere birden çok IPv6 adresi olabilir. Adres ataması yönlendirici duyuruları, Durumsuz Adres Otomatik Yapılandırması (Stateless Address Autoconfiguration - SLAAC), IPv6 için Dinamik Ana Bilgisayar Yapılandırma Protokolü (DHCPv6), elle yapılandırma veya ağ işletmecisinin seçtiği bunların birleşimiyle yapılabilir.

IPv6 adreslemeyi değiştirir; uygulama güvenliğini otomatik olarak sağlamaz. Genel yönlendirilebilir adres yine güvenlik duvarı politikası, güvenli hizmet yapılandırması, HTTPS için Taşıma Katmanı Güvenliği (TLS), kimlik doğrulama ve yazılım bakımı gerektirir. IPv6 yönlendiricileri de ortadan kaldırmaz; yönlendiriciler IPv6 paketlerini ağlar arasında iletmeyi sürdürür. IPv6 adreslerini korumak için Ağ Adresi Çevirisi normal bir gereklilik değildir; ancak çevirinin kaldırılması güvenlik denetimlerinin kaldırılması demek değildir.

DNS, IPv6 adres bilgisini AAAA kaydında saklar. Bir adın hem A hem AAAA kaydı olabilir; böylece IPv4/IPv6 çift yığınlı istemci çalışan bir yolu seçebilir. Modern bağlantı algoritmaları çoğu zaman kullanıcıdan protokol sürümünü seçmesini istemeden kullanılabilir IPv6 ve IPv4 olasılıklarını dener. DNS'den IPv6 adresi almak sunucunun erişilebilir olduğunu veya HTTPS'in başarılı olacağını kanıtlamaz.

IPv6 değişmez değeri URL'de ana makine olarak kullanıldığında köşeli parantez gerekir. Köşeli parantez, adresin içindeki iki noktaları isteğe bağlı bağlantı noktasını başlatan iki noktadan ayırır:

```
https://[2001:db8::40]/status
https://[2001:db8::40]:8443/status
```

İkinci URL'de `2001:db8::40` ana makine, `8443` bağlantı noktasıdır. Köşeli parantezler URL sözdizimidir; 128 bitlik IPv6 adresinin parçası değildir. Üretimde herkese açık HTTPS URL'lerinde alan adları genellikle tercih edilir; çünkü sertifikalar, hizmet taşınması ve insanların tanıması kararlı adlar etrafında düzenlenir.

Tablo 1.4, öğrencilerin başlangıç düzeyinde Web geliştirme ve dağıtım sırasında karşılaşma ihtimali yüksek DNS kayıt türlerini karşılaştırır.

Tablo 1.4 - Yaygın DNS kaynak kayıt türleri

Tür ve ad	Saklanan değer	Yaygın kullanım
A - Address	Dört ondalık oktet olarak yazılan tek 32 bitlik IPv4 adresi; ör. `192.0.2.40`	Bir ad için IPv4 adresi bilgisi verir. Bir adın birkaç A kaydı olabilir.
AAAA - IPv6 Address	Onaltılık gruplarla yazılan tek 128 bitlik IPv6 adresi; ör. `2001:db8::40`	Bir ad için IPv6 adresi bilgisi verir. Dört A harfi daha yeni adres kaydı türünün resmî adıdır.
CNAME - Canonical Name	IP adresi değil, başka bir alan adı	Sahip adın takma ad olduğunu belirtir. Çözümleyici hedef adı izler ve gereken kayıt türünü orada sorar.
MX - Mail Exchange	Tercih numarası ve e-posta sunucusunun alan adı	Alan adına e-posta teslimini yönlendirir. Daha düşük tercih numaraları önce denenir.
NS - Name Server	Yetkili ad sunucusunun alan adı	Bir bölge için DNS yetkisini devreder veya tanımlar. Sunucuya ulaşmadan önce kendisi de adrese çözülmelidir.
TXT - Text	Bir veya daha çok karakter dizisi	Belirli sözleşmelerin yorumladığı metni taşır; e-posta güvenliği ve alan adı doğrulaması buna örnektir.

Aşağıdaki örneklerde öğretim için ayrılmış ad ve adresler kullanılır:

```
hub.campus.example.        300   IN   AAAA    2001:db8::40
www.campus.example.        300   IN   CNAME   hub.campus.example.
campus.example.            3600  IN   MX      10 mail.campus.example.
campus.example.            86400 IN   NS      ns1.campus.example.
campus.example.            3600  IN   TXT     "course-verification=sample-value"
```

AAAA örneği, Tablo 1.4'ten hemen önce açıklanan IPv6 adresini sağlar. Kayıt verisi 128 bitlik tam değeri taşır; sıkıştırılmış yazım adresin kendisini küçültmez. DNS adres verisini, tarayıcının gösterilen noktalamayı kopyalamasından farklı olarak protokol gösteriminde taşır.

CNAME örneği `www.campus.example` adının gerçek adresinin `hub.campus.example` olduğunu söylemez. İlk adın takma ad olduğunu ve çözümleyicinin kanonik hedefi izleyerek gereken kayıt türünü orada sorması gerektiğini söyler. MX örneği `mail.campus.example` için 10 tercihli posta teslimi atar; bu sunucunun adresini vermez. NS örneği yetkili sunucuyu adlandırır; onun da adres kayıtlarına ihtiyacı vardır. TXT örneği, uygulama düzeyindeki bir sözleşme nasıl yorumlayacağını belirleyene kadar yalnızca bir dizedir.

DNS yanıtı birkaç kayıt içerebilir ve farklı istemciler aynı sahip adı için farklı türler isteyebilir. Bu nedenle A yanıtı almak, aynı ad için AAAA, MX, TXT veya başka kayıtların bulunmadığı anlamına gelmez.

Yalın çözümleyici, yinelemeli çözümleyici ve yetkili sunucu

Uygulamaların çoğu DNS hiyerarşisini kendisi dolaşmaz. İşletim sisteminin genellikle sağladığı yerel yalın çözümleyiciye sorar. Bu çözümleyici, bir ağ sağlayıcısının, kurumun, herkese açık DNS hizmetinin veya yerel kuruluşun işlettiği yinelemeli çözümleyiciyle iletişim kurar.

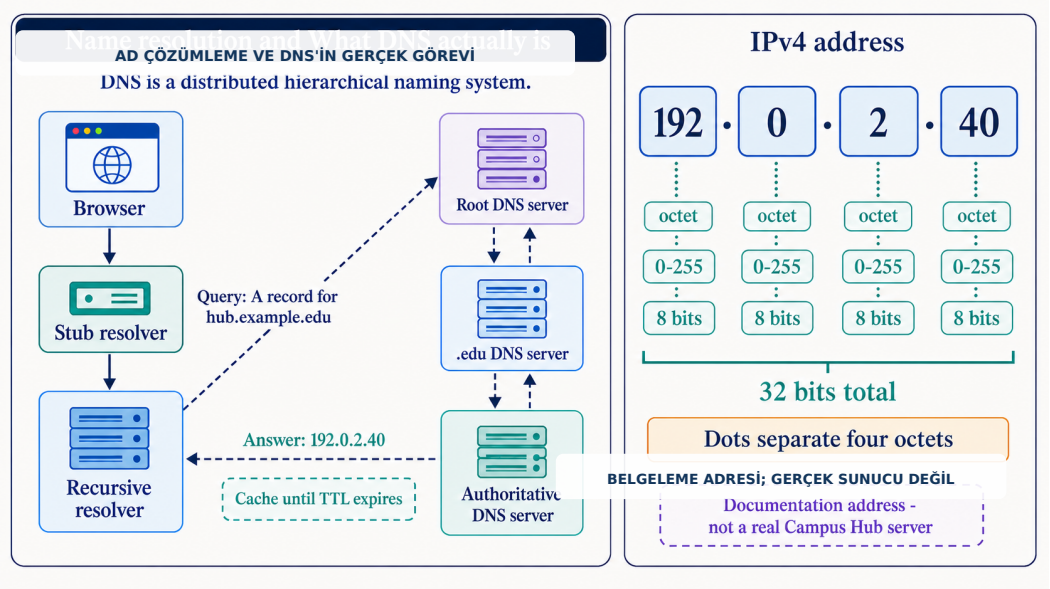
Yinelemeli çözümleyicide kullanılabilir önbellek yanıtı yoksa yönlendirmeleri izler. Sadeleştirilmiş düzeyde şunları sorabilir:

1 Üst düzey alan için bilginin nerede olduğunu öğrenmek üzere kök ad sunucusuna;

2 Alanın yetkili sunucularını öğrenmek üzere üst düzey alan sunucusuna;

3 İstenen kayıt için yetkili sunucuya.

Çözümleyici sonucu istemciye döndürür ve önbelleğe alabilir. Gerçek çözümleme takma adları, birden çok adresi, DNSSEC doğrulamasını, arama son eklerini, yerel politikayı, bölünmüş görünüm DNS'ini veya şifreli çözümleyici taşımasını içerebilir. Sadeleştirilmiş sıra eksiksiz uygulama izi değil, bir temeldir. Şekil 1.11 arama yolunu dönen IPv4 adresinin noktalı ondalık yapısıyla birleştirir. Sol paneli konuşma olarak okuyun: tarayıcı yerel yalın çözümleyiciye sorar, yalın çözümleyici yinelemeli çözümleyiciye sorar ve yinelemeli çözümleyici kullanılabilir önbellek yanıtı yoksa DNS hiyerarşisine danışır. Sağ paneli A kaydı verisinin sayısal açıklaması olarak okuyun; ek DNS hiyerarşisi değildir.



Şekil 1.11. DNS, türü belirtilmiş yanıt için hiyerarşiyi izler; IPv4 dört sekiz bitlik oktetin noktalı gösterimidir.

Önbelleğe alma ve TTL

DNS yanıtı, kaydın ne kadar süre yeniden kullanılabileceğini önbelleklere bildiren yaşam süresi (TTL) içerebilir. Önbelleğe alma gecikmeyi azaltır ve yükü düşürür; ancak kayıt değişikliğinin her yerde aynı anda görünmemesine de yol açar. “DNS yayılımı” çoğu zaman eski önbellek yanıtlarının farklı zamanlarda sona ermesini anlatan gevşek bir ifadedir.

Olumsuz yanıtlar da önbelleğe alınabilir. Bir ad sorgulandığında mevcut değilse çözümleyici bu sonucu bir süre saklayabilir. Tarayıcıyı tekrar tekrar yenilemek tarayıcı, işletim sistemi veya çözümleyici önbelleklerini zorunlu olarak atlamaz.

Başarılı DNS yanıtı uygulamanın kullanılabilir olduğunu garanti etmez. Yalnızca çözümleyicinin ilgili adlandırma bilgisini aldığını kanıtlar. Dönen hizmet hâlâ erişilemez olabilir, bağlantıyı reddedebilir, TLS el sıkışmasını tamamlayamayabilir, `503` döndürebilir veya yanlış içerik üretebilir. Tersine DNS hatası, sunucu süreci sağlıklı olsa bile normal ad tabanlı erişimi engeller.

4.2 Bağlantı ve TLS

İstemcinin sunucuya bir taşıma yolu kurması gerekir. Farklı HTTP sürümleri farklı taşıma ayrıntıları kullanır; ilerleyen bölümler bunları ele alacaktır. HTTPS URL'sinde istemci ve sunucu ayrıca korumalı bir kanal kurar ve istemci, istenen kimlik için sunucunun sertifikasını doğrular.

TCP tabanlı HTTP'de bağlantı kurulması, uç nokta adresleri ve bağlantı noktaları arasında sıralı ve güvenilir bir bayt akışı oluşturur. “Güvenilir” sözcüğünün burada özel bir anlamı vardır: taşıma katmanı kaybı algılar ve veriyi yeniden gönderir. Sunucunun hızlı yanıt vereceğini, çevrim içi kalacağını veya uygulama isteğini doğru işleme alacağını garanti etmez.

TLS, kriptografik parametreleri oluşturur ve normalde güven deposuna dayanan bir sertifika zinciri üzerinden sunucunun kimliğini doğrular. El sıkışma başarılı olduktan sonra korunan uygulama verisinin üç temel özelliği vardır:

- gizlilik: gözlemciler korunan içeriği okuyamamalıdır;
- bütünlük: fark edilmeden değişiklik yapılması mümkün olmamalıdır;
- kimlik doğrulama: istemci, sertifika ve doğrulama kurallarına bağlı olarak iletişim kurmak istediği sunucunun kimliğini doğrular.

HTTPS, aktarım sırasında veriyi korur ve sertifikada temsil edilen sunucu kimliğini doğrular. Uygulamanın iddialarının doğru olduğunu, kodunda güvenlik açığı bulunmadığını veya kuruluşun güvenilmeyi hak ettiğini kanıtlamaz.

Şifreleme uç noktaları görünmez de yapmaz. Ağ gözlemcileri adresler, zamanlama ve trafik hacmi gibi bazı üstverileri hâlâ öğrenebilir. Gizlilik bir sistem özelliğidir; yalnızca kilit simgesi değildir.

4.3 HTTP alışverişi

HTTP, uygulama düzeyinde ve durumsuz bir protokoldür.

“Durumsuz”, protokolün mevcut isteği yorumlamak için sunucunun önceki istekleri hatırlamasını zorunlu kılmadığı anlamına gelir. Uygulamalar veri tabanları, çerezler, oturumlar, önbellekler ve başka mekanizmalarla durum oluşturabilir. Uygulama durumu HTTP tarafından otomatik sağlanmaz; HTTP'nin üzerinde kurulur. Şekil 1.12 istemciden sunucuya giden isteği sunucuya dönen yanıttan ayırır.

Şekil 1.12. HTTP, istemci isteğinin niyetini sunucu yanıtının sonucundan ayırır.

Basitleştirilmiş istek şöyle olabilir:

```
GET /events?semester=fall HTTP/1.1
Host: hub.campus.example
Accept: text/html
User-Agent: ExampleBrowser/1.0
```

Basitleştirilmiş yanıt şöyle olabilir:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 4182
Cache-Control: max-age=60

<!doctype html>
<html lang="en">
...
```

HTTP/2 ve HTTP/3'te ileti biçimi farklıdır; ancak yöntem, hedef, alanlar, durum ve içerik gibi anlamsal fikirler geçerliliğini korur.

Yöntem, isteğin niyetini ifade eder. GET bir gösterimi almayı ister. Durum kodu sonucu özetler. Tablo 1.5 durum kodu ailelerinin ilk okumasını sağlar.

Tablo 1.5 - HTTP durum kodu aileleri

Aralık	Anlam	Örnekler
1xx	Bilgilendirici	103 Early Hints
2xx	Başarılı	200 OK, 201 Created, 204 No Content
3xx	Yönlendirme	301 Moved Permanently, 304 Not Modified
4xx	İstek gönderildiği biçimiyle yerine getirilemez	400 Bad Request, 403 Forbidden, 404 Not Found
5xx	Sunucu görünüşte geçerli isteği işlerken başarısız oldu	500 Internal Server Error, 503 Service Unavailable

Her 4xx yanıtını “öğrenci hata yaptı”, her 5xx yanıtını da “sunucu bilgisayarı bozuk” diye çevirmeyin. Durum kodları protokol düzeyindeki sonuçları açıklar.

Yöntemler ve anlamsal özellikler

Sık karşılaşılan yöntemler Tablo 1.6'da karşılaştırılır.

Tablo 1.6 - Yaygın HTTP istek yöntemleri ve amaçlanan anlamsal özellikleri

Yöntem	Amaçlanan kullanım	Önemli özellik
GET	Güncel gösterimi alır.	Tanım gereği güvenli ve idempotenttir.
HEAD	GET'in döndüreceği alanları içerik olmadan alır.	Üstveri denetimleri için yararlıdır.
POST	Hedef kaynağın gönderilen içeriği işlemesini ister.	Çoğu zaman bir şey oluşturur veya tetikler; genellikle idempotent değildir.
PUT	Hedef kaynağın durumunu oluşturur veya değiştirir.	Idempotenttir.
PATCH	Kısmi değişiklik uygular.	Idempotent olması garanti edilmez.
DELETE	Hedefin temsil ettiği ilişkiyi kaldırır.	Amaçlanan etkide idempotenttir.

Güvenli yöntem, kullanıcının bakış açısından salt okunur olmayı amaçlar. Idempotent yöntem, aynı istek bir veya birkaç kez yapıldığında aynı amaçlanan etkiye sahiptir. Idempotent olması “aynı yanıtı döndürür” veya “günlükleme yan etkisi yoktur” demek değildir. Bu özellikler yeniden deneme davranışını, önbelleğe almayı, bağlantı izlemeyi ve API tasarımını biçimlendirir.

Alanlar ve içerik uzlaşması

HTTP alanları üstveri taşır. `Content-Type` içeriğin ortam türünü, `Content-Length` varsa boyutunu, `Accept` istemcinin işleyebileceği biçimleri, `Cache-Control` önbellek yönergelerini ve `Location` yönlendirme veya oluşturma sonrasında başka bir kaynağı tanımlayabilir.

Yanıt biçimini yalnızca dosya adına bakarak çıkarmayın. `.html` ile biten URL JSON, uzantısı olmayan URL HTML döndürebilir. Protokol düzeyindeki bildirim yanıtın `Content-Type` alanıdır. Yanlış ortam türleri görüntüleme hatalarına ve güvenlik sorunlarına yol açabilir.

Gösterimler ve kaynaklar

Kaynak, URL'nin tanımladığı kavramsal hedeftir. Gösterim, mevcut veya amaçlanan durumun belirli bir aktarılabilir biçimidir. Aynı kaynak HTML ve JSON gösterimlerine veya uzlaşma yoluyla seçilen yerelleştirilmiş biçimlere sahip olabilir. Bu ayrım, “URL dosyaya eşittir” düşüncesinin modern web mimarisi için neden yetersiz olduğunu açıklar.

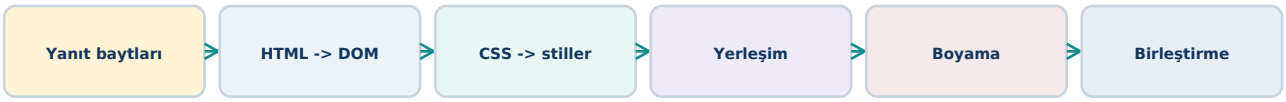
Yönlendirmeler, önbellekleme ve koşullu istekler

Yönlendirme görünmez bir sıçrama değil, bir yanıttır. Tarayıcı `3xx` durumunu ve `Location` alanı gibi bilgileri alır; ardından yeni istek gönderebilir. Bu nedenle Network paneli çoğu zaman hem ilk alışverişi hem izleyen alışverişi gösterir.

Önbelleğe alma, tanımlı kurallar altında saklanan yanıtın sonraki istekleri karşılamasına izin verir. İstemci, ETag gibi bir doğrulayıcıyla koşullu istek yapabilir. `304 Not Modified`, istemcinin sakladığı gösterimi yeniden kullanmasını söyler; sunucunun yanıt vermediği anlamına gelmez.

4.4 Tarayıcı işleme süreci

HTML'yi almak son değildir. Tarayıcı onu iç belge modeline ayrıştırır, başvurulan kaynakları keşfeder, stilleri uygular, geometriyi hesaplar ve pikselleri boyar. Betikler daha sonra olaylara yanıt verip belgeyi değiştirebilir. Görünen tek sayfa çoğu zaman birçok HTTP alışverişinin sonucudur. Şekil 1.13, alınan baytlardan görünür piksellere giden tarayıcı hattını özetler.



Betikler ve kullanıcı eylemleri DOM'u değiştirip yeni görüntüleme işlerini başlatabilir.

Şekil 1.13. Tarayıcı baytları ve kaynakları piksellere ve etkileşime dönüştürür.

Yararlı bir görüntüleme modeli şöyledir:

1. HTML baytları karakterlere kodu çözülür ve düğümlerden oluşan ağaç olan Belge Nesne Modeli'ne (Document Object Model - DOM) ayrıştırılır.
2. CSS ayrıştırılır, seçiciler eşleştirilir ve hesaplanmış stiller belirlenir. Kavramsal CSS nesne modeli bu çalışmaya katılır.
3. Tarayıcı, görüntüleme için hangi kutuların oluşturulması gerektiğini belirler.
4. Yerleşim boyutları ve konumları hesaplar.
5. Boyama, arka planların, kenarlıkların, metnin ve görsellerin nasıl çizileceğini kaydeder.
6. Birleştirme katmanları görüntüleme için birleştirir.

Bu model bir öğretim modelidir; her tarayıcının altı aşamayı kesinlikle ardışık yürüttüğünü vaat etmez. Motorlar işi iyileştirir, paralelleştirir, önbelleğe alır ve yeniden ziyaret eder. Yerleşimle ilgili özellikleri değiştiren betik ek stil, yerleşim, boyama veya birleştirme çalışması başlatabilir.

Aşamalı görüntüleme ve kaynak keşfi

Tarayıcılar yanıtın tamamı gelmeden ayrıştırmaya başlayabilir. HTML ayrıştırıcısı stil sayfalarını, betikleri, görselleri, yazı tiplerini ve başka kaynakları keşfeder. Bazı kaynaklar sonraki çalışmayı engelleyebilir; bazıları eşzamansız yüklenir. Ön yükleme taraması, önbellek durumu, öncelik kararları ve betik öznitelikleri gözlenen sırayı etkiler.

Bu nedenle sayfa performansı “HTML dosyasını küçült” cümlesine indirgenemez. Gecikme, istek sayısı, kritik kaynaklar, sunucu yanıt süresi, önbellek politikası, görsel boyutları, yazı tipi stratejisi, JavaScript çalıştırma süresi ve görüntüleme maliyeti birlikte katkıda bulunur.

5. Yerel dosya, yerel sunucu, herkese açık sunucu

Bu adresler farklı ortamları açıklar:

```
file:///C:/course/week-01/index.html
http://localhost:8000/index.html
https://student.campus.example/index.html
```

- `file:` URL'si tarayıcının yerel dosyayı doğrudan okumasına izin verir. Bir web sunucusuna HTTP isteği yoktur.
- `localhost` geçerli makineyi adlandırır. Seçilen bağlantı noktasında bir sunucu programı dinliyor olmalıdır.
- Herkese açık HTTPS URL'si herkese açık ad çözümleme, erişilebilir barındırma ve TLS gerektirir.

`localhost` çoğu zaman `127.0.0.1` veya `::1` gibi bir geridöngü adresine çözümlenir. Geridöngüye gönderilen trafik geçerli ana makinede kalır. Bağlantı noktası yine önemlidir: `http://localhost:8000/` ile `http://localhost:3000/` farklı sunucu süreçlerine ulaşabilir.

IPv4 adresi `127.0.0.1` ve IPv6 adresi `::1` geridöngüyü tanımlar; ancak farklı İnternet Protokolü sürümlerine aittir. Buna karşılık `0.0.0.0` üzerinde dinlediğini söyleyen sunucu yapılandırması “kullanılabilir bütün yerel IPv4 arayüzleri” anlamına gelir. `0.0.0.0`, öğrencilerin herkese açık bağlantıya yazması gereken bir hedef değil, bağlama talimatıdır. Geridöngünün ötesine bağlanmak geliştirme sunucusunu yerel ağdaki başka aygıtlara açabileceğinden bilinçli seçim olmalıdır.

Yerel sunucudan herkese açık hizmete geçmek ana makine adını değiştirmekten fazlasıdır. Herkese açık dağıtım normalde erişilebilir ana makine, güvenlik duvarı politikası, DNS kayıtları, geçerli TLS sertifikası ve çoğu zaman gelen istekleri kabul edip uygulama sürecine ileten ters vekil sunucu gerektirir. Üretim günlükleri, izleme, güncellemeler, yedekler ve erişim kontrolleri de operasyon sorumluluğuna dönüşür. `localhost` üzerinde çalışan sayfa önemli bir testi geçmiştir; güvenli biçimde dağıtılabileceğini henüz kanıtlamamıştır.

Şekil 1.14 bu ortamları sınırlarını gizlemeden karşılaştırır. Yerel geliştirmede tarayıcı ve geliştirme sunucusu aynı bilgisayarda geridöngü üzerinden iletişim kurabilir; istek herkese açık İnternet'ten geçmez. Herkese açık örnekte DNS araması, adres bilgisini döndüren ayrı bir ön alışveriştir. Daha sonraki HTTPS isteği DNS'in içinden geçmez; birbirine bağlı yönlendiricileri aşarak HTTPS ağ geçidine ulaşır ve ağ geçidi isteği uygulama sunucusuna iletir. Veri tabanı, doğrudan herkese açık tarayıcı bağlantısını kabul etmek yerine uygulama sunucusunun arkasında kalır. Yanıt uygulama yolu üzerinden tarayıcıya döner.

Geliştirme sunucusu düzenleme ve hata ayıklamayı desteklemek üzere tasarlanır. Otomatik yeniden yükleme yapabilir, ayrıntılı hatalar gösterebilir ve üretim sertleştirmesini içermeyebilir. Üretim sunucusu denetimli dağıtım, güvenlik, güvenilirlik, performans ve gözlemlenebilirlik için yapılandırılır. Geliştirme sunucusu yerelde çalışıyor diye onu güvenilmeyen ağlara açmayın.

Kurs çalışmaları için, bir laboratuvar özellikle `file:` davranışını incelemiyorsa her zaman yerel HTTP sunucusu kullanın. Tarayıcı güvenlik kuralları ve URL çözümleme, yerel dosyalarla HTTP kaynakları arasında farklılık gösterebilir.

6. Geliştirici araçları ölçüm araçlarıdır

Tarayıcının geliştirici araçlarını açın ve Network panelini bulun. Gezinme veya yönlendirme listenin temizlenmesine neden olacaksa “Preserve log” seçeneğini etkinleştirin; ardından sayfayı yeniden yükleyin.

Tek bir belge isteği için şunları kaydedin:

- tam istek URL'si;

- istek yöntemi;

- durum kodu;

- gösteriliyorsa uzak adres;

- istek `Accept` değeri;

- yanıt `Content-Type`;

- varsa `Cache-Control`;

- aktarılan boyut ve zamanlama;

- başka isteklerin izleyip izlemediği ve onları neyin başlattığı.

Kanıtın sınırları vardır. Panel geçici bir üstbilgi, önbelleğe alınmış yanıt, eklenti tarafından oluşturulan istek veya tarayıcıya özgü zamanlama ayrıntısı gösterebilir. Aracın bildirdiğini kaydedin ve gözlem ile çıkarımı birbirinden ayırın.

Geliştirici araçlarında birbiriyle ilişkili birkaç araç bulunur. Elements paneli canlı Belge Nesne Modeli'ni (DOM) ve eşleşen stilleri gösterir. Console betik iletilerini raporlar ve geçerli sayfa bağlamında küçük ifadeleri değerlendirmeye izin verir. Network paneli kaynak alışverişlerini kaydeder. Sources veya Debugger paneli yüklenen dosyaları gösterir ve kesme noktalarını destekler. Application veya Storage paneli istemci tarafı depolamayı, çerezleri, önbellekleri ve service worker'ları gösterir. Performance paneli zamanlanmış tarayıcı çalışmasını kaydeder. Adlar tarayıcılar arasında biraz değişir; ancak sorular aynıdır: ne istendi, ne oluşturuldu, hangi kod çalıştı, hangi durum mevcut ve zaman nerede harcandı?

Hiçbir panel her şeyi bilen bir doğruluk makinesi değildir. Network kaydı yalnızca yakalama etkin olduğunda başlar; Elements paneli ilk kaynak kodunu değil canlı ağacı gösterir; Console çıktısı filtrelenebilir veya eklentiler tarafından üretilebilir. İyi hata ayıklama, gözlemin aracını, zamanını, kapsamını ve koşullarını kaydeder.

Disiplinli hata ayıklama döngüsü

Bu döngüyü ders boyunca kullanın:

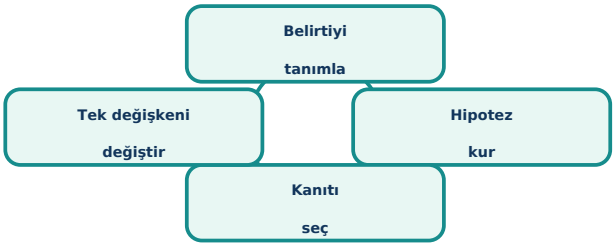
1. Belirtiyi kesin ifade edin. “Sayfa bozuk” kesin değildir. “Belge 200 döndü, ancak logo.svg 404 döndürdü” eyleme dönüktür.

2. Tek bir hipotez kurun. Örneğin göreli görsel URL'si yanlış yola çözümleniyor.

3. Hipotezi çürütebilecek bir gözlem seçin. İstenen URL'yi ve başlatanı inceleyin.

4. İlgili tek bir etkeni değiştirin. İlişkisiz tarayıcı ayarlarını değiştirmek yerine yolu düzeltin.

5. Tekrarlayın ve kaydedin. Açıklaması olmayan düzeltme kırılgan bilgidir.


Belirtiyi tanımla
Hipotez kur
Tek değişkeni değiştir
Kanıtı seç

7. En küçük kullanışlı HTML belgesi

HTML, belge yapısını ve anlamını açıklar. Web sayfalarının çoğunun yazıldığı dildir ve HTTP yanıtı `Content-Type: text/html` bildirdiğinde tarayıcının başlangıçta beklediği biçimdir. Tarayıcı HTML'yi ayrıştırır; belgeyi programlama yönergeleri dizisi gibi "çalıştırmaz". Ayrıştırma sırasında `link`, `img` ve `script` gibi öğeler tarayıcının alıp kullanabileceği başka kaynakları tanımlayabilir.

7.1 Öğeler, etiketler, içerik ve öznitelikler

Şunu düşünün:

```
<a href="/events">Etkinlikleri görüntüle</a>
```

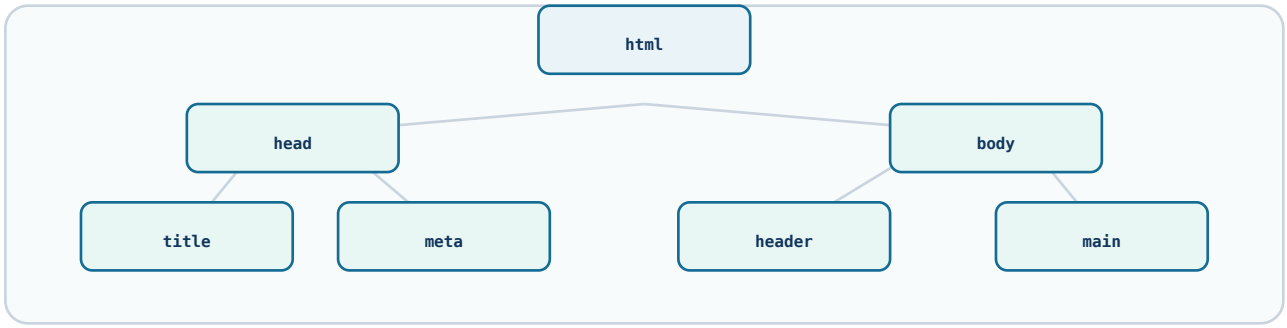
Yapının tamamı bir öğedir. `<a ...>` başlangıç etiketidir, `</a>` bitiş etiketidir ve `Etkinlikleri görüntüle` öğenin metin içeriğidir. `href`, değeri bağlantı hedefini sağlayan bir özniteliktir. Geliştiriciler günlük konuşmada "etiket" derken çoğu zaman "öğe"yi kasteder; ancak belirtimleri ve belge ağacını okurken ayrım yararlıdır.

`meta` ve `img` gibi bazı öğeler boş öğedir; bitiş etiketi veya alt öğe içeriği yoktur. `<img />` yazımı izin verilen bir sözdizimidir; ancak eğik çizgi HTML'yi XML'e dönüştürmez ve boş olmayan HTML öğelerini kapatmaz.

İlişkileri ifade etmek için öğeler iç içe yerleştirilebilir:

```
<article>
  <h2>Robotik laboratuvarı tanıtım akşamı</h2>
  <p>Ekip ile tanışın ve güncel projeleri inceleyin.</p>
</article>
```

`h2` ve `p`, `article` öğesinin çocuklarıdır. Doğru iç içelik önemlidir; çünkü ayrıştırıcı açılış ve kapanış satırlarını görsel yığın olarak değil, ağaç olarak kurar. Şekil 1.16 bu üst öğe-alt öğe ilişkilerini açıkça gösterir.



Şekil 1.16. İç içe HTML öğeleri bir belge ağacına dönüşür.

7.2 HTML anlamsaldır

Anlamsal HTML amacı iletir. Başlık büyük ve kalın göründüğü için değil, bir bölümü tanıttığı için başlıktır. Düğme yalnızca metin içeren dikdörtgen değil, bir eylemi temsil eder. Anlam; klavye kullanıcılarına, yardımcı teknolojiye, arama sistemlerine, okuyucu kiplerine, bakım yapanlara ve otomatik araçlara yarar sağlar.

Önce anlamına göre öğe seçin, sonra biçimlendirin. Yerel bir `button` etkinleştirme davranışı, klavye desteği ve platform anlambilimi sağlarken tıklama işleyicili `div` kullanmayın.

7.3 Belge iskeleti

2. Oturum anlambilim ve erişilebilirliği derinlemesine inceler. Şimdilik aşağıdaki eksiksiz asgari yapıyı anlayın:

```
<!doctype html>
<html lang="tr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Hub</title>
  </head>
  <body>
    <header>
      <h1>Campus Hub</h1>
    </header>
    <main>
      <p>Öğrenci etkinliklerini ve toplulukları bulun.</p>
    </main>
  </body>
</html>
```

Kod bloğu eksiksiz girdidir. Şekil 1.17, aynı yapısal fikirleri kullanan daha büyük 1. Hafta çözümünün tarayıcı çıktısını gösterir. Sade görünüm bilinçlidir: HTML yapı ve anlam sağlar; 3. Bölüm Basamaklı Stil Sayfaları (Cascading Style Sheets - CSS) sunumu ekler.

- `<!doctype html>` modern HTML ayrıştırma kipini seçer.

- `lang`, yardımcı teknoloji dâhil yazılımlar için belge dilini tanımlar.

- UTF-8, öğrencilerin ihtiyaç duyduğu metni destekler ve erken bildirilmelidir.

- `viewport`, mobil aygıtlarda yerleşim görünüm alanını kullanışlı kılar.

- `title`, tarayıcı sekmesini, geçmiş girdisini ve yer imini etiketler.

- `h1`, ana başlığı tanımlar; metni büyütme komutu değildir.

`head`, belge üstverisini ve destek kaynaklarına başvuruları içerir; görünür sayfa başlığı değildir. `body`, kullanıcıya sunulan belge içeriğini taşır. Görünür site başlığı `body` içinde, genellikle bir `header` öğesinde yer alır.

7.4 HTML hatalara toleranslıdır; hatasız değildir

Tarayıcılar birçok HTML hatasından standartlaştırılmış ayrıştırma kurallarına göre kurtulur. Hatalı sayfaların çalışıyor gibi görünmesinin nedeni bu dayanıklılıktır. Bu durum doğruluğu göz ardı etme izni değildir. Kurtarma, öğrencinin tasarladığını sandığı DOM'dan farklı bir DOM oluşturabilir ve tutarsız stil, betik ve erişilebilirliğe yol açabilir.

Bir doğrulayıcı kullanın ve tarayıcının Elements panelini inceleyin. Kaynak dosyası yazdığınızı kaydeder; DOM, tarayıcının oluşturduğunu kaydeder.

7.5 HTML, DOM ve View Source ilişkili ama farklıdır

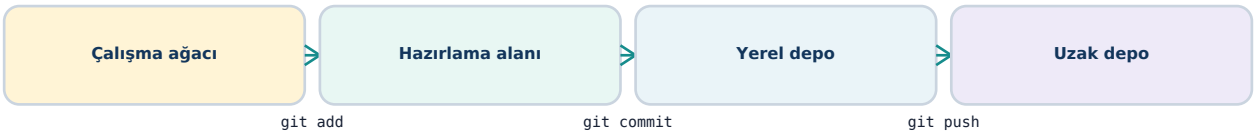
- HTML kaynağı, teslim edilen veya saklanan serileştirilmiş metindir.

- HTML ayrıştırıcısı bu metni HTML ayrıştırma modeline göre yorumlar.

- DOM, tarayıcı özelliklerine ve JavaScript'e açılan bellek içi nesne ağacıdır.

- Geliştirici araçları betiklerin değiştirdiği bir DOM gösterebilir.

Bu ayrım, işlenmiş sayfada görünen bir değerin View Source'da bulunmadığı veya geçersiz iç içeliğin ayrıştırıcı tarafından yeniden düzenlendiği durumlarda temel önem taşır.



Şekil 1.18. Git, seçilen değişiklikleri çalışma ağacından kalıcı geçmişe taşır.

8. Git kararları kaydeder

Git, dağıtık bir sürüm kontrol sistemidir. Bir ekibin çalışmayı inceleyebilmesi, karşılaştırabilmesi, paylaşabilmesi ve kurtarabilmesi için anlık görüntüleri ve ilişkilerini kaydeder. Şekil 1.18 çalışma ağacını, hazırlama alanını, yerel geçmişi ve uzak depoyu ayırır.

İlk laboratuvar için üç fikir yeterlidir:

1. Çalışma ağacı düzenlemekte olduğunuz dosyaları içerir.

2. Hazırlama alanı bir sonraki commit için amaçlanan değişiklikleri seçer.

3. Commit, üst ilişki ve üstveriyle birlikte bir anlık görüntüyü kaydeder.

```
git init
git status
git add index.html
git commit -m "Campus Hub belgesini oluştur"
```

Her işlemden önce ve sonra `git status` çalıştırın. Commit mesajı tuş vuruşlarını değil, anlamlı değişikliği belirtmelidir. “Etkinlik gezinmesini ekle”, “dosyayı güncelle”den daha yararlıdır. Parola, belirteç, özel anahtar veya kişisel öğrenci verisi commit etmeyin.

Depo, commit, dal ve uzak depo

Depo, projenin geçmişini ve üstverisini saklar. Commit; yazar bilgisi, zaman, mesaj ve üst ilişki içeren tanımlı bir anlık görüntüdür. Dal, bu geçmişteki bir konuma işaret eden hareketli addır. Uzak depo, iş birliği için başka bir depo konumuna adlandırılmış başvurudur; GitHub, Git'in kendisi değil, olası bir barındırma hizmetidir.

Git yerel çalışır. Ağ bağlantısı olmadan başlatabilir, hazırlayabilir, commit edebilir, geçmişi inceleyebilir, dal oluşturabilir ve önceki içeriği geri getirebilirsiniz. `push` ve `fetch` gibi komutlar başka bir depoyla nesne ve başvuru alışverişi yapar. Yerel commit kendiliğinden yükleme, dağıtım, yayımlama veya yedekleme değildir. Bunlar ayrı hedef ve erişim kuralları olan ayrı işlemlerdir. Bu ayrım, `git commit` başarılı olduğu için kodun uzak sunucuda güvende olduğunu sanma hatasını önler.

Commit kalitesi

Yararlı commit tutarlıdır: tek ve açıklanabilir bir değişikliği temsil eder ve başka bir geliştiricinin inceleyebileceği durumda bırakır. Akıl yürütmeyi koruyacak kadar erken, her tuş vuruşundan sonra olmayacak kadar anlamlı commit edin. Commit öncesinde:

1. ilgili denetimleri çalıştırın;

2. `git diff` ile `git diff --staged` çıktısını inceleyin;

3. yanlışlıkla eklenen dosyaları ve gizli bilgileri kaldırın;

4. “Bu commit ... yapacak” cümlesini tamamlayan bir mesaj yazın.

Sürüm kontrolü yedekleme, erişim kontrolü veya gizli bilgi yönetiminin yerine geçmez. Bir gizli bilgi commit edilip paylaşıldıktan sonra sonraki commit'te silinmesi onu mevcut geçmişten kaldırmaz. Kimlik bilgisini iptal edin veya döndürün.

Yönlendirmeli laboratuvar - Yerelde gözlemleyin ve yayımlayın

Amaç ve kabul ölçütleri

Fall 2026 coursework şablonundan oluşturduğunuz kişisel depoda `labs/week-01/index.html` dosyasından başlayın, küçük bir Campus Hub belgesi oluşturun ve HTTP üzerinden sunun. Şablon deposu yalnızca okunur ders materyalidir; çalışmayı yalnızca ondan oluşturduğunuz özel depoya commit edip push edin.

Sonucunuz şunları içermelidir:

- gerekli belge üstverisi;

- bir header, etiketlenmiş bir nav, bir main, en az iki section ve bir footer;

- bir açık h1 ve mantıklı başlık sırası;

- bölüm parçalarına çalışan göreli bağlantılar;

- satır içi CSS, JavaScript, çatı veya kopyalanmış şablon kullanılmaması;

- tarayıcı %200 yakınlaştırmada kullanılabilirlik;

- sağlanan Hafta 1 denetiminden geçme;

- en az üç anlamlı commit;

- tamamlanmış gözlem kaydı.

Güvenlik ve gizlilik

Kurgusal etkinlik adları ve iletişim bilgileri kullanın. Network panelleri sorgu parametrelerini, çerezleri, yetkilendirme verilerini ve kurum içi ana makine adlarını gösterebilir. Ekran görüntülerine veya teslimlere gizli bilgi koymayın.

1. adım - başlangıç durumunu oluşturun

Kişisel coursework deponuzun kökünde bir terminal açın. Doğru klasörde olduğunuzu kanıtlayın ve mevcut Git çalışma ağacını inceleyin:

```
Get-Item package.json
git status
```

VS Code'da `labs/week-01/index.html` dosyasını açın. Karşılaştırma için dosyayı doğrudan tarayıcıda açıp `file:` URL'sini not edebilirsiniz. Geliştirici araçlarında HTTP belge isteğini arayın; doğrudan yerel dosya yüklemesinde bunu görmemelisiniz.

İlk TODO'ları tamamlayın, ardından commit edin:

```
git add labs/week-01/index.html
git commit -m "Campus Hub belgesini oluştur"
```

Kontrol noktası A: `git log --oneline` bir commit gösterir ve sekmede kullanışlı bir başlık bulunur.

2. adım - gezinilebilir içerik ekleyin

Bir header ve gezinme ekleyin; ardından `events` ve `communities` gibi kimliklere sahip iki ana bölüm ekleyin. Her gezinme bağlantısı `href="#events"` gibi göreli bir parça başvurusu kullanmalıdır.

Altbilgiye bunun öğrenci öğrenme projesi olduğunu belirten kısa bir ifade ekleyin. Yeniden commit edin:

```
git add labs/week-01/index.html
git commit -m "Campus Hub gezinmesini ve bölümlerini ekle"
```

Kontrol noktası B: her bağlantı seçildiğinde URL parçası değişir ve odak veya görünüm alanı amaçlanan bölüme gider. Yüklenmiş belge içinde yalnızca parça değiştirmek için sunucu isteği gerekmez.

3. adım - HTTP üzerinden sunun

Kişisel coursework deponuzun kökünden, sağlanan bağımlılıksız sunucuyu 8000 bağlantı noktasında çalıştırın:

```
npm run serve:week01
```

npm betiği `labs/week-01` klasörünü sunar. Sunucu yalnızca yerel geridöngü adresine bağlanır ve paket indirmez.

`http://localhost:8000/` adresine gidin ve sunucu terminalini açık tutun.

Kontrol noktası C: Network paneli HTTP kullanan bir belge isteği gösterir. Yöntemi, durumu, istek URL'sini ve yanıt `Content-Type` değerini kaydedin.

3A. adım - ad çözümlemeyi gözlemleyin

`localhost`, işletim sistemi onu geridöngü adı olarak gördüğünden normalde herkese açık DNS hiyerarşisine başvurulmadan çözülür. PowerShell terminalinde sonucu doğrulayın:

```
Resolve-DnsName localhost
```

`Resolve-DnsName` bulunmayan sistemlerde `nslookup localhost` kullanın. Komut `127.0.0.1`, IPv6 geridöngü adresi `::1` veya ikisini gösterebilir. Bu adresler “bu bilgisayar” anlamına gelir; İnternet'teki başka bir makineyi tanımlamaz.

Şimdi öğretim için güvenli bir herkese açık adı inceleyin:

```
Resolve-DnsName example.com
```

Dönen kayıt türünü ve adresi kaydedin. Önbellekten alınan sonuç da çözümleme kanıtıdır; DNS önbelleğe alma sistemin parçasıdır. Görüntülenen adresin kalıcı olduğunu varsaymayın. Alan adı zaman içinde veya farklı ağlardan farklı adreslere çözülebilir.

Kontrol noktası D: başarılı DNS çözümlemesinin dönen adreste HTTP sunucusunun çalıştığını neden kanıtlamadığını kendi sözcüklerinizle açıklayın.

4. adım - hata oluşturun ve tanılayın

`http://localhost:8000/missing.html` adresine gidin.

Ağ girdisine bakmadan önce durumu tahmin edin. Ardından gerçek durumu ve yanıt gövdesini kaydedin. Hata sayfası görmenin HTTP yanıtı gelmediği anlamına neden gelmediğini açıklayın.

Çalışan sayfaya dönün, sorunları düzeltin ve commit edin:

```
git add labs/week-01/index.html
git commit -m "Hafta 1 HTTP gözlemini tamamla"
```

5. adım - doğrulayın

Kişisel coursework deponuzda çalıştırın:

```
npm test
```

Otomatik denetim sağlanan proje yapısını, Node.js sürümünü, başlangıç belgelerini ve yerel HTTP sunucularını doğrular. Tamamlanan Hafta 1 HTML'sini puanlamaz. Kendi çalışmanızı tamamlayıp commit ettikten sonra belgenizi yukarıdaki kabul ölçütleriyle ve Ek A'daki çözümlü örnekle karşılaştırın.

Gözlem kaydı

Laboratuvar sırasında araçların gösterdiği ile bu kanıttan çıkardığınızı ayırmak için Tablo 1.7'yi kullanın.

Tablo 1.7 - Laboratuvar gözlem kaydı

Gözlem	Kanıtınız
Doğrudan dosya URL'si	
Yerel sunucu URL'si	
İstek yöntemi	
Başarılı durum	
Eksik kaynak durumu	
Yanıt Content-Type	
Bir istek üstbilgisi	
Bir yanıt üstbilgisi	
localhost adres bilgisi	
Herkese açık DNS kayıt türü ve değeri	
İlk yüklemedeki istek sayısı	
Gözlem ve çıkarım	“Gözlemledim ...; ... nedeniyle çıkarıyorum.”

Temizlik

Yerel sunucuyu Ctrl+C ile durdurun. Tüm amaçlanan çalışmanın commit edildiğini ve hiçbir gizli veya ilgisiz dosyanın hazırlama alanında olmadığını `git status` ile doğrulayın.

Alıştırmalar

Cevapları açmadan önce bütün alıştırmaları yapmayı deneyin.

Temel

Alıştırma 1 - URL anatomisi

Aşağıdaki URL'de her bileşeni belirleyin ve hangi bileşenin HTTP isteğinde gönderilmediğini belirtin:

```
https://library.campus.example:9443/search/books?q=distributed+systems#results
```

Alıştırma 2 - Olayları sıralayın

Aşağıdaki sadeleştirilmiş olayları geçerli bir nedensellik sırasına koyun:

- tarayıcı HTML'yi ayrıştırır;

- DNS çözümleyici adres bilgisini döndürür;

- kullanıcı URL girer;

- sunucu HTTP yanıtı gönderir;

- tarayıcı HTTP isteği gönderir;

- tarayıcı bir görsel URL'si keşfeder.

Alıştırma 3 - Yanıtı okuyun

Bu durum satırını ve iki alanı yorumlayın. Ne çıkarılabilir, ne çıkarılamaz?

```
HTTP/1.1 304 Not Modified
Cache-Control: max-age=120
ETag: "events-v7"
```

Uygulama

Alıştırma 4 - Üç gözlemi tanılayın

Her durum için en yararlı sonraki incelemeyi verin:

1. `file:///.../index.html` görüntüleniyor; ancak Network'te belge isteği yok.

2. `http://localhost:8000/` bağlantının reddedildiğini bildiriyor.

3. Belge 200 döndürüyor; ancak bir görsel 404 döndürüyor.

Alıştırma 5 - Sınırı çizin

Bir öğrenci “Parça özeldir; çünkü HTTPS onu sunucuya göndermeden önce şifreler” diyor. Bu ifadeyi kesin biçimde düzeltin.

Zorluk

Alıştırma 6 - Bir sayfa, birden çok istek

`index.html` bir stil sayfasına, iki görsele ve bir betiğe başvuruyor. Önbellekler boş ve satır içine gömme yokken normal yüklemede beklenen en az HTTP kaynak isteği sayısı nedir? Gerçek sayı neden farklı olabilir?

Alıştırma 7 - Git kanıtı

Aşağıdaki geçmişin neden zayıf bir mühendislik süreci kanıtı olduğunu açıklayın ve daha iyi üç commit'lik geçmiş önerin:

```
a13f99c final
```

Alıştırma 8 - DNS yanıtını izleyin

Yinelemeli çözümleyicinin önbelleğinde şu kayıtlar vardır:

```
www.department.example.   CNAME   web-host.example.net.   TTL 240
web-host.example.net.      A       192.0.2.40             TTL 45
```

IPv4 istemcisinin bağlantı denemesi yapabilmesi için çözümleyici ne döndürmelidir? Hangi önbellek bilgisi önce eskiyebilir ve bunun gerçekleşebileceği belirtilen en erken süre nedir? Yanıtın web sitesinin sağlıklı olduğunu neden kanıtlamadığını açıklayın.

Alıştırma 9 - Kaynak HTML ve DOM

Şu geçersiz kaynağı düşünün:

```
<p>First paragraph
<p>Second paragraph
```

Tarayıcının Elements paneli, kaynakta kapanış etiketi görünmediği hâlde iki ayrı `p` öğesi gösteriyor. Tarayıcı kaynak dosyasını değiştirdi mi? Ne oldu ve her gösterimi incelemek için hangi görünüm kullanılmalıdır?

Alıştırma 10 - Yöntem özellikleri

Her işlemi güvenli veya güvensiz ve idempotent veya idempotent olmayan olarak sınıflandırın. Sınıflandırmayı belirli bir uygulamaya değil amaçlanan anlambilime dayandırın:

1. `GET /courses/42`
2. Eksiksiz değiştirme belgesiyle `PUT /students/7/preferences`
3. Yeni kayıt oluşturmak için `POST /registrations`
4. `DELETE /registrations/91`

Alıştırma 11 - Kökenleri karşılaştırın

`https://portal.campus.example/index.html` belgesi için aşağıdaki URL'lerin aynı kökenli olup olmadığını belirtin. Her yanıtı şema-ana makine-bağlantı noktası üçlüsüyle açıklayın:

1. `https://portal.campus.example/help`
2. `http://portal.campus.example/help`
3. `https://api.campus.example/help`
4. `https://portal.campus.example:443/help`
5. `https://portal.campus.example:8443/help`

Alıştırma 12 - Katmana göre tanılayın

Tarayıcı bir URL için `DNS_PROBE_FINISHED_NXDOMAIN` bildiriyor. İkinci URL çözülüyor; ancak `ERR_CONNECTION_REFUSED` bildiriyor. Üçüncüsü HTTPS kurup HTTP `503 Service Unavailable` alıyor. Her durumda başarısız olan veya yanıt veren katmanı belirleyin ve üretken bir sonraki denetim adımı adlandırın. Üçünü birden “İnternet kapalı” saymanın tanılamayı neden engelleyeceğini açıklayın.

Alıştırma 13 - IPv6'yı genişletin, sıkıştırın ve yorumlayın

Bütün bölümleri yanıtlayın ve her dönüşümü açıklayın.

1. `2001:db8:12::40` adresini dört hexadecimal rakamdan oluşan sekiz gruba genişletin.
2. `2001:0db8:0000:0000:0000:00ab:0000:0001` adresini geçerli daha kısa biçimde yazın.
3. `2001::db8::1` adresinin neden geçersiz olduğunu açıklayın.
4. `https://[2001:db8::40]:8443/status` içinde ana makineyi, bağlantı noktasını ve yolu belirleyin.
5. `2001:db8:12:34::/64` içindeki `/64` ne demektir ve adresin TCP 64 bağlantı noktası kullandığı anlamına gelir mi?

Çözümlü cevaplar

Cevap 1

- Şema: `https`; ana makine: `library.campus.example`; bağlantı noktası: `9443`; yol: `/search/books`; sorgu: `q=distributed+systems`; parça: `results`.

Parça HTTP istek hedefinin parçası olarak gönderilmez; tarayıcı kaynağı aldıktan sonra onu yerelde kullanır. Sorgu verisi gönderilir ve gizli kabul edilmemelidir.

Cevap 2

Geçerli bir sıra şöyledir: 1) kullanıcı URL girer; 2) DNS çözümleyici adres bilgisini döndürür; 3) tarayıcı HTTP isteği gönderir; 4) sunucu HTTP yanıtı gönderir; 5) tarayıcı HTML'yi ayrıştırır; 6) tarayıcı görsel URL'si keşfeder. Tarayıcı daha sonra görsel için başka istek gönderebilir. Gerçek uygulamalar eşzamanlı çalışabilir, bağlantıları ve önbelleğe alınmış DNS yanıtlarını yeniden kullanabilir veya akışın tamamı gelmeden işlemeye başlayabilir.

Cevap 3

`304 Not Modified`, istemcinin koşullu istek yaptığını ve sunucunun saklanan gösterimin yeniden kullanılabileceğini bildirir; hedef için yeni gösterim gövdesi taşımaz. `Cache-Control: max-age=120` önbellek tazeliğini, ETag ise gösterimle ilişkili doğrulayıcıyı ifade eder. “Hiçbir şey olmadı” veya kaynağın sonsuza kadar değişmeden kaldığı sonucu çıkarılamaz; sayfanın doğru olduğu da bu durumdan anlaşılamaz.

Cevap 4

1. Zorunlu olarak yanlış bir şey yoktur. `file:` yüklemesi HTTP belge isteği oluşturmaz. Adres çubuğunu inceleyin, ardından dizini yerel sunucuyla sunup karşılaştırın. 2. Bir sunucu sürecinin çalıştığını ve 8000 bağlantı noktasını dinlediğini kontrol edin; terminaldeki başlangıç hatalarını, URL'yi ve bağlantı noktasını doğrulayın. 3. Başarısız görsel isteğini seçip istenen URL'yi ve başlatanı inceleyin; HTML başvurusunu, yol çözümlemesini, dosya adının büyük-küçük harfini ve sunucu dosyalarını karşılaştırın.

Cevap 5

Parça HTTP isteğinde hiç gönderilmez; sunucuya ulaşmamasının nedeni HTTPS şifrelemesi değildir. Tarayıcı parçayı kendinde tutar ve yorumlar. Bununla birlikte parçalar tarayıcı geçmişinde, ekran görüntülerinde, kopyalanmış bağlantılarda, istemci günlüklerinde veya istemci kodunda görünebilir. “İstek hedefi olarak gönderilmemek”, gizli bilgiler için güvenli olmak anlamına gelmez.

Cevap 6

En az beş istek gerekir: bir HTML belgesi, bir stil sayfası, iki görsel ve bir betik. Gerçek sayı yönlendirmeler, CSS'nin başvurduğu kaynaklar, yazı tipleri, kaynak haritaları, favicon'lar, betik istekleri, service worker'lar, tarayıcı eklentileri, ön yüklemeler, hatalar, önbellekleme veya doğrudan veri olarak gömülmüş kaynaklar nedeniyle farklı olabilir.

Cevap 7

Tek bir “final” commit'i ara kararları gizler, incelemeyi zorlaştırır ve yararlı kurtarma noktaları bırakmaz. Daha iyi bir geçmiş örneği şöyle olabilir:

```
8ac501e HTTP gözlem notlarını tamamla
340de18 Campus Hub gezinmesini ve bölümlerini ekle
bd102aa Campus Hub belgesini oluştur
```

Hash'ler değişebilir. İyileştirme, amacı anlatan mesajlara sahip, tutarlı ve incelenebilir değişiklikler dizisidir.

Cevap 8

Çözümleyici takma adı izlemeli ve kanonik hedefe bağlı adresi sağlamalıdır: `www.department.example`, `web-host.example.net` için takma addır ve IPv4 adresi `192.0.2.40`'tır. İstemcinin hem ilişkiyi hem kullanılabilir adresi anlaması için yanıt CNAME ile A kaydını birlikte içerebilir. A kaydı 45 saniyelik TTL ile önce eskiyebilir; CNAME 240 saniye kalabilir. TTL, sunucunun bu süre boyunca erişilebilir kalacağı sözü değil, önbellek yaşam süresidir. DNS yalnızca adlandırma bilgisi sağlar; ana makine çevrim dışı olabilir, güvenlik duvarı bağlantıyı reddedebilir, TLS başarısız olabilir veya HTTP uygulaması hata döndürebilir.

Cevap 9

Tarayıcı kaynak dosyasını değiştirmemiştir. HTML ayrıştırıcısı, ikinci `p` başlangıç etiketiyle karşılaşınca ilk paragrafı HTML kurallarına göre örtük olarak kapatmıştır. Sonuçtaki bellek içi DOM iki paragraf düğümü içerir. View Source veya özgün düzenleyici teslim edilen ya da kaydedilen baytları; Elements paneli ise ayrıştırma ve JavaScript değişikliklerinden sonraki canlı DOM'u gösterir. Görünen sonuç kabul edilebilir olsa bile geçersiz kaynağı doğrulayıp düzeltin.

Cevap 10

1. `GET /courses/42` güvenli ve idempotenttir; amacı veri almak ve tekrarlanan istek aynı amaçlanan etkiye sahiptir. Günlükleme veya erişim sayacı gibi iç yan etkiler yöntemin anlamını değiştirmez. 2. Eksiksiz değiştirme yapan PUT güvensiz fakat idempotenttir; aynı değiştirme tekrarlansa da amaçlanan son durum aynıdır. 3. Kayıt oluşturan POST güvensiz ve genellikle idempotent değildir; tekrarı birden fazla kayıt oluşturabilir. 4. DELETE güvensiz fakat idempotenttir; ilk istek hedefi kaldırır, tekrarlar farklı durum döndürebilse de amaçlanan son durum kaydın bulunmamasıdır. Idempotent, her yanıtın bayt bayt aynı olması anlamına gelmez.

Cevap 11

Başvuru kökeni `(https, portal.campus.example, 443)`'tür; HTTPS bağlantı noktası yazılmadığında 443 kullanır. 1 aynı kökenlidir. 2 şema farklı olduğu için farklı kökenlidir. 3 ana makine farklı olduğu için farklı kökenlidir; aynı kuruluşa ait olabilir. 4 varsayılan HTTPS bağlantı noktasını açıkça yazdığı için aynı kökenlidir. 5 bağlantı noktası 8443 olduğu için farklı kökenlidir. Yollar köken üçlüsüne katılmaz.

Cevap 12

`DNS_PROBE_FINISHED_NXDOMAIN`, ad çözümlemesinin alan adının mevcut olmadığını bildirdiğini gösterir; yazımı, yapılandırmayı ve çözümleyici yanıtını denetleyin. IP bağlantısı henüz denenmemiştir. `ERR_CONNECTION_REFUSED`, adres bilgisinin bulunduğunu ve istemcinin bağlantı aşamasına ulaştığını ancak hiçbir hizmetin adres ve bağlantı noktasını kabul etmediğini gösterir; sunucunun beklenen arayüz ve bağlantı noktasında dinleyip dinlemediğini kontrol edin. HTTPS kurulup HTTP 503 alınması ise ağ ve HTTPS aşamalarının geçildiğini, fakat uygulama hizmetinin isteği kullanılamaz olduğunu bildirir; uygulama günlüklerini ve sağlık durumunu inceleyin. Üçünü “İnternet kapalı” saymak farklı katmanları ve doğru sonraki denetimleri gizler.

Cevap 13

1. `2001:db8:12::40` -> `2001:0db8:0012:0000:0000:0000:0000:0040`. 2. `2001:0db8:0000:0000:0000:00ab:0000:0001` -> `2001:db8::ab:0:1`. 3. `2001::db8::1` iki ayrı çift iki nokta sıkıştırması içerdiği için kaç sıfır grubunun nereye ait olduğu belirlenemez. 4. URL'de ana makine `2001:db8::40`, bağlantı noktası `8443`, yol `/status`'tur; köşeli parantez URL sözdizimidir. 5. `/64`, ilk 64 bitin ağ öneği olduğunu belirtir; TCP 64 bağlantı noktası anlamına gelmez ve IPv6 adresinin parçası değildir.

HTTP 503 Service Unavailable bir HTTP yanıtıdır. DNS, bağlantı kurulumu ve HTTPS, bir HTTP sunucusunun veya aracının yanıt verebileceği kadar başarıyla ilerlemiştir. Yanıt alanlarını, sunucu günlüklerini, sağlık denetimlerini, kapasiteyi ve üst hizmet bağımlılıklarını inceleyin.

Her koşulu “İnternet kapalı” diye adlandırmak en güçlü kanıtı yok sayar: her ileti farklı bir katmanda ilerleme konumunu gösterir. Katman farkındalığı aramayı daraltır ve ilgisiz sistemleri değiştirmeyi önler.

Cevap 13

1. Genişletilmiş adres `2001:0db8:0012:0000:0000:0000:0000:0040` olur; görünür dört grup olduğu için `::` dört sıfır grubu sağlar. 2. `2001:db8::ab:0:1` geçerli kısa biçimlerden biridir. 3. Adreste iki `::` bulunduğundan sıfır gruplarının sayısı belirsizdir. 4. Köşeli parantezler IPv6 ana makinesini sınırlar; bağlantı noktası kapanış parantezinden sonra `8443`, yol `/status`'tur. 5. `/64`, ilk 64 adres bitinin ağ öneği olduğunu belirtir; taşıma katmanı bağlantı noktasıyla ilişkisi yoktur.

Hata ayıklama kliniği

Bir öğrenci sunucuyu `C:\course` içinde başlatıyor; ancak dosyayı `C:\course\week-01\index.html` içine koyuyor. `http://localhost:8000/` adresine gidince Campus Hub yerine dizin listesi görüyor.

Tanı

Sunucu kökü `C:\course` olduğu için `/` bu dizini adlandırır. HTML belgesi `/week-01/index.html` yolundadır. Bu yola gidin veya sunucuyu durdurup `C:\course\week-01` içinden yeniden başlatın. Ağ adını yeniden adlandırmak, tarayıcı verilerini temizlemek veya DNS'i değiştirmek gözlenen eşlemeyi düzeltmez.

Bilgiyi geri çağırma sınavı

Geriye bakmadan yanıtlarınızı yazın. Ardından zihinsel modelinizi düzeltmek için açıklamaları kullanın.

1. İnternet ile Web'i tek cümlede ayırın. 2. Hangi URL bileşeni normalde yalnızca istemci tarafından yorumlanır: sorgu mu parça mı? 3. DNS normal gezinmeye ne katar? 4. HTTPS bir web sitesinin dürüst ve hatasız olduğunu garanti eder mi? 5. HTTP yöntemi neyi bildirir? 6. Eksik sayfa görünür HTML ve 404 döndürdü; sunucu yanıt verdi mi? 7. Tek görünür sayfa neden birçok HTTP isteğine yol açabilir? 8. Git çalışma ağacı ile hazırlama alanı arasındaki fark nedir?

9. HTTP yüklemesini `file:` yüklemesinden ayıran bir gözlem söyleyin. 10. Geliştirici neden hem gözlemi hem çıkarımı kaydetmelidir? 11. HTML ile DOM arasındaki fark nedir? 12. DNS kaydının TTL'si neden olabilir? 13. Web kökenini hangi üç değer tanımlar? 14. HTTP'nin durumsuz olması ne demektir? 15. 200 OK neden sayfanın doğru olduğunu kanıtlamaz? 16. Tam genişletilmiş IPv6 adresinde kaç grup vardır ve sıkıştırılmış biçimde `::` ne anlama gelir?

Quiz cevapları ve yanılgı notları

1. İnternet, birbirine bağlı ağların temel ağıdır; Web ise bu ağı kullanarak bağlantılı kaynakları tanımlayan ve değiştiren uygulamadır. İlişkilidirler, eş anlamlı değildir. 2. Parça. Sorgu istek hedefinin parçasıdır; parça istemcide tutulur. 3. DNS, hizmete ulaşmak için ana makine adını adres bilgisine çözer; sayfayı indirmez veya görüntülemez. 4. Hayır. HTTPS bağlantıyı korur ve sertifikalı sunucu kimliğini doğrular; uygulamanın doğru veya hatasız olduğunu onaylamaz. 5. İsteğin amaçlanan anlambilimini, örneğin GET ile gösterim alma niyetini bildirir. 6. Evet. Hata belgesi yanıt içeriğidir ve 404 yanıt durumudur. 7. Belgenin ayrıştırılması stil sayfaları, betikler, görseller, yazı tipleri ve veriler gibi ayrı alınması gereken başvuruları keşfeder. 8. Çalışma ağacı güncel dosya içeriğidir; hazırlama alanı bir sonraki commit'e girecek içeriği seçer.

9. HTTP yüklemesinde `http:` veya `https:` URL'si, Network'te yöntem, durum ve HTTP alanları görünür; doğrudan `file:` yüklemesinde HTTP belge alışverişi yoktur. 10. Araçlar kanıt gösterir, açıklamalar varsayım ekler. İkisini ayırmak akıl yürütmeyi incelenebilir kılar. 11. HTML belgeyi anlatan işaretleme dili ve kaynak sözdizimidir; DOM ayrıştırma sonucunda tarayıcının oluşturduğu canlı bellek içi ağaçtır. DOM ayrıştırıcı onarımlarını ve sonraki JavaScript değişikliklerini yansıtabilir.

12. TTL, çözümleyicilere önbellekteki DNS kaydını ne kadar süre kullanabileceklerini bildirir. Önbellekleme gecikmeyi ve DNS trafiğini azaltır; sürenin dolması yetkili verinin değişmesine izin verir. TTL hizmetin erişilebilirlik garantisi değildir. 13. Şema, ana makine ve etkin bağlantı noktası. Yol, sorgu ve parça kökene katılmaz. 14. HTTP bağımsız istekler arasında konuşmayı kendiliğinden hatırlamaz; uygulamalar çerez, yetkilendirme veya belirteçlerle süreklilik kurar. 15. 200 OK yalnızca isteğin HTTP anlamına göre işlendiğini bildirir; gövde yanlış, erişilemez veya hatalı olabilir. 16. Tam IPv6 adresi sekiz grup ve her grupta 16 bit içerir; `::`, toplam sekiz gruba ulaşmak için ardışık sıfır gruplarını bir kez değiştirir.

Kendini değerlendirme

14-16: 2. Oturuma hazırsınız. 10-13: IP adresleme, URL, DNS ve HTTP'yi yeniden inceleyip diziyi yüksek sesle açıklayın. 0-9: ilerlemeden önce Network laboratuvarını bir arkadaş veya öğretim asistanıyla tekrarlayın.

Oturum özeti

Web, İnternet üzerindeki bir uygulamadır. İstemciler istek başlatır; sunucular yanıt üretir. Bir makine iki rolü de üstlenebilir.

URL'ler yapılandırılmış tanımlayıcılardır; düzensiz dizeler değildir. Köken, URL'nin şeması, ana makine adı ve etkin bağlantı noktasını birleştirir; tarayıcılar bunu güvenlik sınırı olarak kullanır.

IPv4, 32 bitlik adres için dört ondalık oktet kullanır. IPv6, 128 bitlik adres için sekiz onaltılık grup ve dikkatli sıkıştırma kuralları kullanır.

DNS çözümlemesi, bağlantı kurulumu, HTTP alışverişi ve tarayıcı işlemesi farklı sorumluluklardır. HTTP, standart anlamlarla istek niyetini ve yanıt sonuçlarını iletir.

HTTPS aktarım sırasında veriyi korur; uygulamayı güvenilir kılmaz. Tek sayfa çoğu zaman birçok kaynağa ve isteğe bağlıdır.

`file:`, `localhost` ve herkese açık HTTPS ortamları birbirinin yerine kullanılamaz. Geliştirici araçları kanıt sağlar; Git anlamlı anlık görüntüleri ve niyeti kaydeder.

Sözlük

Tablo 1.8, bölümün tekrar ve sınıf içi geri çağırma çalışması için temel çalışma söz varlığını bir araya getirir.

Tablo 1.8 - 1. Oturum sözlüğü

API	Uygulama programlama arayüzü; yazılım bileşenlerinin işlem istemesi veya veri değiştirmesi için tanımlı yol. Web API'si kaynakları çoğu zaman HTTP ile sunar.
Tarayıcı	Web kaynaklarını alan, yorumlayan ve sunan web istemcisi.
İstemci	Bir hizmet için istek başlatan yazılım.
CSS	Yapılandırılmış belgelerin sunumunu ve yerleşimini açıklayan Basamaklı Stil Sayfaları dili.
DNS	Alan adları için adres bilgisi edinmek de dâhil dağıtık adlandırma sistemi.
DOM	Belge Nesne Modeli; tarayıcının belge için kurduğu ve betiklere açtığı canlı düğüm ağacı.
Alan adı	Adresleri ve hizmetleri açıklayabilen, `www.campus.example` gibi hiyerarşik okunabilir ad.
Parça	URL'de `#` sonrasındaki istemci taraflı ikincil başvuru.
HTML	Web belgesine yapı ve anlam veren Hiper Metin İşaretleme Dili.
HTTP	Kaynaklarla ilgili iletileri değiştiren durumsuz uygulama protokolü.
HTTPS	Kimliği doğrulanmış, şifreli taşıma kanalı üzerinden iletişim kuran HTTP.
İnternet	İnternet protokol ailesini kullanan birbirine bağlı ağların küresel ağı.
IP adresi	Arayüzü adreslemek ve paketleri yönlendirmek için kullanılan sayısal tanımlayıcı; ör. `192.0.2.40` veya `2001:db8::40`.
IPv4	Yaygın olarak dört ondalık oktet şeklinde yazılan 32 bitlik İnternet Protokolü sürümü.
IPv6	Belirli kısaltma kurallarıyla sekiz onaltılık grup şeklinde yazılan 128 bitlik İnternet Protokolü sürümü.
JavaScript	Tarayıcılarda ve birçok sunucu ortamında davranış ve uygulama mantığı ifade eden programlama dili.
Localhost	Geçerli ana makinenin geridöngü arayüzü için kullanılan ad.
Köken	Birlikte değerlendirilen şema, ana makine ve etkin bağlantı noktası; tarayıcıların önemli güvenlik sınırı.
Paket	Teslim için gereken denetim bilgisiyle paket anahtarlamalı ağda taşınan sınırlı veri birimi.
Bağlantı noktası	Ana makinedeki belirli hizmete trafiği yönlendiren taşıma katmanı numarası.
Yinelemeli çözümleyici	İstemci adına aramaları yürüten veya koordine eden ve yanıtları TTL'lerine göre önbelleğe alan DNS hizmeti.

Gösterim	Kaynağın HTML belgesi veya JSON açıklaması gibi belirli aktarılabilir biçimi.
Kaynak	URL ile tanımlanan ve web protokolleriyle işlem yapılan şey.
Yönlendirici	Yönlendirme bilgisine göre ağlar arasında paket ileten ağ aygıtı veya yazılım işlevi.
Sunucu	İstekleri dinleyen ve yanıt sağlayan yazılım.
TLS	HTTPS bağlantıları için kimlik doğrulama, gizlilik ve bütünlük sağlayan Taşıma Katmanı Güvenliği protokolü.
TTL	Yaşam süresi; DNS'te kaydın yenilenmeden önce önbellekten kullanılabileceği süre.
URL	Bir kaynağı bulmak ve erişmek için kullanılan yapılandırılmış tanımlayıcı.
Web	İnternet teknolojileri üzerinde kurulan bağlantılı bilgi ve uygulama sistemi.

Birincil başvurular ve ileri okuma

1. WHATWG, HTML Living Standard - Web developer edition. 2. WHATWG, URL Living Standard. 3. WHATWG, Fetch Standard; CORS bölümleri HTTP yanıtlarının kökenler arasında nasıl paylaşılacağını tanımlar. 4. MDN, Same-origin policy. 5. IETF, RFC 9110: HTTP Semantics. 6. IETF, RFC 1034: Domain Names - Concepts and Facilities. 7. IETF, RFC 3596: DNS Extensions to Support IP Version 6. 8. IETF, RFC 8200: Internet Protocol, Version 6 (IPv6) Specification. 9. IETF, RFC 4291: IP Version 6 Addressing Architecture. 10. IETF, RFC 5952: IPv6 Address Text Representation. 11. IETF, RFC 9846: The Transport Layer Security (TLS) Protocol Version 1.3. 12. IETF, RFC 9000: QUIC - A UDP-Based Multiplexed and Secure Transport. 13. IETF, RFC 9114: HTTP/3. 14. W3C, CSS Snapshot 2026. 15. MDN, How the web works. 16. Chacon ve Straub, Pro Git, 2. baskı. 17. IANA, Example Domains. 18. IETF, RFC 2606: Reserved Top Level DNS Names.

Neler daha sık değişir?

Bu bölümdeki kavramsal roller ve HTTP anlambilimi kararlıdır. Tarayıcı geliştirici araçlarının etiketleri, kesin zamanlama panelleri, kurulum ekranları ve komut çıktıları daha sık değişir. Bu bölümün teknik inceleme tarihi 29 Eylül 2026'dır.

Oturum yansıtması

- Oturumun başındaki hangi tahmininiz değişti ve hangi kanıt bunu değiştirdi?
- DNS ile HTTP, istemci ile sunucu veya `file:` ile HTTP ayrımlarından hangisini başka bir öğrenciye anlatırken artık daha güvendesiniz?
- Hangi konu hâlâ belirsiz ve hangi gözlem bu belirsizliği azaltabilir?

Öğrenme günlüğü notları

Bu alana 1. oturumdan kalan sorularınızı, değiştirdiğiniz zihinsel modelleri ve sonraki oturumda doğrulamak istediğiniz tahminleri yazın.

Not: İyi bir günlük yalnızca sonuçları değil, gözlemi ve o gözlemden çıkardığınız yorumu da birbirinden ayırır.

Oturum 2 - Anlamsal HTML ve erişilebilir formlar

Temel soru: Tek bir Hiper Metin İşaretleme Dili (HyperText Markup Language - HTML) belgesi, Basamaklı Stil Sayfaları (Cascading Style Sheets - CSS) veya JavaScript eklenmeden önce insanlar, tarayıcılar, arama motorları ve yardımcı teknolojiler için nasıl anlaşılır kalabilir?

Oturum haritası

Tablo 2.1, hazırlığı, ders süresini, beklenen ürünü ve kanıtları özetler.

Tablo 2.1 - Oturum kapsamı, hazırlık ve kanıtlar

Öğe	Plan
Hedef kitle	1. Oturumu tamamlayan öğrenciler; erişilebilirlik konusunda ön bilgi varsayılmaz.
Hazırlık	Çalışma istasyonu yönlendirmesini tamamlayın. HTML kavramlarına geçmeden önce düzenle-kaydet-sun-yenile döngüsü tekrarlanır.
Ders süresi	Yaklaşık 3 saat ders/atölye ve 2 saat laboratuvar.
Bireysel çalışma	Yaklaşık 3 saat.
Ana ürün	Campus Hub için erişilebilir etkinlik gönderme sayfası.
Değerlendirme kanıtı	Doğrulayıcı çıktısı, klavye testi, anlamsal gerekçe, alıştırmalar ve quiz.

Öğrenme çıktıları

Bu oturumun sonunda şunları yapabileceksiniz:

1. Campus Hub HTML kaynağını bulmak, kod düzenleyicisinde düzenleyip kaydetmek, doğru dizini sunmak ve tarayıcıdaki çıktıyı doğrulamak.

2. Geçerli HTML sözdiziminde öğe, etiket, öznitelik, içerik, boş öğe, Boolean öznitelik ve karakter başvurusunu ayırt etmek.

3. HTML'nin yalnızca kutuları yerleştirmediğini, anlam taşıdığını açıklamak.

4. Belge dili, yazım yönü, başlık, karakter kodlaması, açıklama ve görünüm alanı meta verilerini kurmak.

5. Belgenin bilgi yapısına göre başlıkları ve bölgesel öğeleri seçmek.

6. Yararlı metin, bağlantı, görsel, liste, tablo, tarih ve medya alternatifleri yazmak.

7. Adı, etiketi, açıklaması ve gruplaması olan bir form oluşturmak.

Kavramlardan önce: Campus Hub HTML'sini bulun, düzenleyin ve görüntüleyin

HTML için özel bir görsel tasarım programı gerekmez. HTML belgesi, adı normalde `.html` ile biten düz metin dosyasıdır. Kaynağı metin düzenleyicisinde yazar, dosyayı kaydeder ve tarayıcıdan yüklemesini istersiniz. Tarayıcı işaretlemeyi okur ve görünür sayfayı üretir. Bu nedenle düzenleyici ve tarayıcı aynı çalışmanın iki görünümünü gösterir: düzenleyici talimatları, tarayıcı ise bu talimatların yorumlanmış sonucunu gösterir.

Doğru Campus Hub dosyasını bulun

Fall 2026 ders çalışma şablonundan oluşturduğunuz kişisel depoda her laboratuvar için bir öğrenci çalışma dosyası bulunur. 2. Oturumda `labs/week-02/index.html` dosyasını düzenleyin. Dosyada dış belge yapısı ve `Submit a campus event` başlığı hazırdır; yorumlar henüz tamamlanmamış işleri gösterir. Eksiksiz referans cevap kitabın Ek A bölümündedir ve özel öğrenci deposuna bilerek konulmamıştır.

`index.html` dosya adının tekrarlanması bilerek yapılmıştır. Web sunucuları bir dizin için çoğu zaman `index.html` dosyasını varsayılan belge kabul eder. Dosyaları klasörler ayırır. Yalnızca `index.html` yazan bir sekme yeterli kanıt değildir; Explorer panelindeki klasör yolunu veya düzenleyicinin breadcrumb yolunu okuyun. Okuduğunuz PDF web sitesinin kaynağı değildir; PDF'yi düzenlemek Campus Hub sayfasını değiştiremez.

```
web-programming-coursework-2026_Fall-YOUR_STUDENT_NUMBER\labs\week-02\index.html  ← bu dosyayı düzenleyin
```

Düz metin kod düzenleyicisi kullanın

Not Defteri dâhil her düz metin düzenleyicisi HTML yazabilir. Bu derste klasörleri, satır numaralarını, sözdizimi renklendirmesini, tümleşik terminali ve hata göstergelerini sunduğu için Visual Studio Code (VS Code) kullanılır. Microsoft Word gibi kelime işlemcileri kullanmayın; kelime işlemci HTML kaynağı olmayan biçimlendirme verileri ekleyebilir.

Araçların görevleri ayrıdır:

- VS Code Explorer depo klasörlerini ve dosyalarını gösterir. - VS Code düzenleyicisi HTML kaynağını yazıp kaydettiğiniz yerdir. - Terminal yerel sunucuyu ve geliştirme komutlarını çalıştırır. - Tarayıcı kaydedilmiş dosyayı ister ve görüntüler. - Tarayıcı geliştirici araçları isteği, ayrıştırılmış Belge Nesne Modeli'ni (DOM), erişilebilirlik bilgisini ve diğer kanıtları gösterir.

Projeyi ve doğru dosyayı adım adım açın: 1) VS Code'u başlatın. 2) File > Open Folder'ı seçin. 3) `package.json`, `labs` ve `scripts` içeren dış `web-programming-coursework-2026_Fall-...` klasörünü seçin. 4) Explorer panelinde `labs`, ardından `week-02` klasörünü genişletin. 5) `index.html` dosyasını seçin ve breadcrumb yolunda `labs > week-02 > index.html` bulunduğunu doğrulayın. 6) Düzenleyicide önizleme açılırsa dosyaya sağ tıklayın, Open With seçin ve Text Editor'ı açın.

Görünür bir değişiklik yapın ve kaydedin

Başlangıç dosyasında şu başlığı bulun: `<h1>Submit a campus event</h1>`. Bir sonraki satıra şu paragrafı ekleyin: `<p>This page is being edited in Visual Studio Code.</p>`. Açılış etiketi `<p>` paragrafı başlatır, cümle metin içeriğidir ve kapanış etiketi `</p>` paragrafı bitirir. Paragrafı, ikisi de `main` öğesinin çocukları olduğu için başlıkla aynı girinti düzeyine yerleştirin. Girinti insanların kaynağı okumasına yardım eder; gerçek yapıyı etiketler kurar.

Windows veya Linux'ta Ctrl+S, macOS'ta Command+S tuşlarına basın. Düzenleyici sekmesindeki kaydedilmemiş göstergesi kaybolmalıdır. Kaydetmek değişikliği diske yazar; her tarayıcıyı kendiliğinden yenilemez.

Yerel sunucuyu başlatın ve tarayıcı çıktısını açın

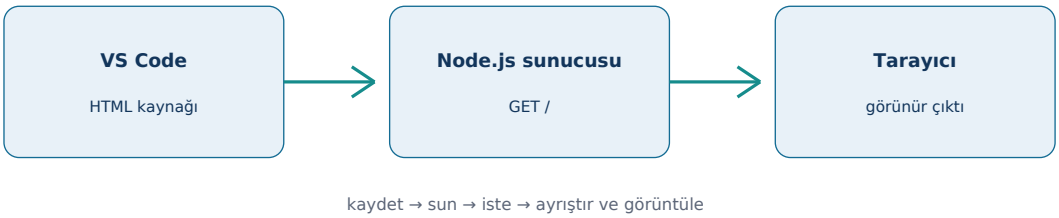
VS Code'da Terminal > New Terminal komutunu seçin. İstem, depo kökünde olmalıdır. Windows Komut İstemi'nde konumu doğrulayın:

```
dir package.json
npm run serve:week02
```

PowerShell'de aynı kanıt için `Get-Item package.json`; macOS veya Linux'ta npm komutundan önce `ls -l package.json` kullanın. Başarılı bir sunucu, anlamı şu olan satırlar yazdırır: `Serving ...\labs\week-02`. Ardından tarayıcıda `http://localhost:8000/` adresini açın. Durdurmak için Ctrl+C'ye basın; o zamana kadar bu terminal sunucu terminalidir ve sunucu durmadan yeni istem göstermez.

Adresi terminale değil, tarayıcının adres çubuğuna yazın. Tarayıcı başlangıç sayfasında `Submit a campus event` başlığını ve eklediğiniz paragrafı göstermelidir. Sayfa sade görünecektir; çünkü 2. Oturum bilerek CSS kullanmaz. Bu başarısızlık değildir. Biçimlendirme eklenmeden önce HTML yapısını incelemenizi sağlar.

Kaynağı her değiştirdiğinizde kısa döngüyü tekrarlayın: 1) VS Code'da `labs/week-02/index.html` dosyasını düzenleyin; 2) kaydedin; 3) tarayıcıya dönün; 4) Ctrl+R, F5 veya yenile düğmesiyle yeniden yükleyin. Şekil 2.1 bu yolun tamamını gösterir.



Şekil 2.1. Campus Hub için düzenleme, kaydetme, sunma, isteme ve görüntüleme döngüsü.

Okların anlamı alternatif programlar değildir: VS Code dosyayı kaydeder; Node.js sunucusu tarayıcı HTTP isteği gönderince dosyayı okur; tarayıcı yanıtı ayrıştırır ve görüntüler. Sağlanan sunucu otomatik canlı yenileme yapmaz. Eski sayfa görünüyorsa önce düzenleyici sekmesinin kaydedildiğini doğrulayın, sonra tarayıcıyı yenileyin. Ayrıca sunucunun `Serving` satırının `labs\week-02` ile bittiğini kontrol edin.

`index.html` dosyasının başka bir kopyasını düzenlediyseniz bu adrese yeniden yükleme o kopyayı gösteremez. Bu oturumun ilerleyen bölümündeki gerçek tarayıcı görüntüsü tamamlanmış Week 2 çözümünü gösterir. Başlangıç dosyanız, denetimleri laboratuvarda siz oluşturana kadar daha az kontrol içerecektir; hemen tamamlanmış sayfaya benzemesini beklemeyin.

Sonuç farklı olduğunda kanıtı okuyun

Tarayıcı sayfası ile hata iletisi farklı kanıt türleridir:

- `Submit a campus event` görünüyorsa tarayıcı doğru başlangıç dosyasına ulaşmıştır. - Sayfa `200 OK` durumuyla yükleniyor ama paragraf yoksa olası nedenler kaydedilmemiş sekme, eksik yenileme veya başka bir `index.html` dosyasını düzenlemektir. - Tarayıcı neredeyse boş sayfa gösteriyorsa geliştirici araçlarını açın, `body` öğesini inceleyin ve sunucunun `Serving` yolunu okuyun. Başka bir haftanın bilerek eksik başlangıç dosyasını sunuyor olabilirsiniz. - Tarayıcı “This site can't be reached” diyorsa o adres ve bağlantı noktasında sunucu yanıt vermiyordur; sunucu terminaline dönün. - Tarayıcı `404 Not Found` alıyorsa sunucu yanıt vermiştir fakat istenen yol mevcut bir dosyayı tanımlamamıştır.

Tarayıcının Elements veya Inspector paneli ayrıştırılmış DOM'u gösterir; kaydedilmiş kaynak dosyanın düzenlenebilir yerine geçmez. Bu paneldeki geçici değişiklikler deney için yararlıdır, ancak yenileme sonrasında kaybolur. Kalıcı düzeltmeleri VS Code dosyasında yapın ve kaydedin.

1. Kullanıcı göreviyle başlayın

Özenli görünen bir etkinlik formu düşünün. Her alan açık renkli bir dikdörtgenle gösteriliyor; yer tutucular “Name”, “Date” ve “Email” diyor; son dikdörtmeye tıklamak formu gönderiyor. Görmesi iyi olan bir fare kullanıcısı formu tamamlayabilir. Klavye kullanıcısı odağın nereye gittiğini göremez. Ekran okuyucu kullanan kişi üç kez “edit text, blank” duyabilir. Bir değer reddedildiğinde kenarlık kırmızıya döner, fakat nedenini açıklayan mesaj bulunmaz.

Sayfanın görsel bir düzeni vardır; ancak henüz güvenilir bir arayüz değildir. Erişilebilirlik, tamamlanmış tasarıma sonradan eklenen süs değildir. Farklı kullanıcılar ve teknolojiler için sistemin sunduğu ilişkilerin, adların, durumların ve etkileşimlerin niteliğidir.

İlerlemeye başlamadan kısa bir tahmin yazın:

- Tüm CSS ortadan kalksaydı hangi bilgiler kalırdı? - Fare olmadan hangi işlemler tamamlanabilir? - Yardımcı teknoloji her denetim için hangi adı duyururdu? - Gönderimin neden başarısız olduğunu kişi nasıl anlardı?

Bu sorular erişilebilirliği belirsiz bir erdem olmaktan çıkarıp gözlenebilir mühendislik davranışına dönüştürür.

2. HTML bir anlam dilidir

HTML öğeleri içeriğin ne olduğunu ve parçaların nasıl ilişkili olduğunu açıklar. Başlık bir bölümü tanıtır; liste ilişkili öğeleri gruplar; düğme bir eylemi temsil eder; bağlantı bir kaynağa gider; form denetimi veri kabul eder. Tarayıcılar bu öğelere davranış ve erişilebilirlik anlamı bağlar.

Bir `div` öğesini düğme gibi göstermek teknik olarak mümkündür. Görünüş, ona klavye davranışı, erişilebilir rol, devre dışı durumu veya etkinleştirme anlamı vermez. Yerel `button` öğesi kavramı zaten ifade ederken bunların hepsini yeniden kurmak gereksizdir:

```
<button type="submit">Publish event</button>
```

İlke basittir: görevin anlamıyla eşleşen öğeyi kullanın. Daha özel bir öğe uygun değilse `div` ve `span` gibi genel kapsayıcıları tercih edin. Şekil 2.2, anlamsal landmark öğelerinin bölgeleri amaçlarına göre nasıl tanımladığını gösterir.

Şekil 2.2. Anlamsal landmark öğeleri sayfayı amaçlarına göre adlandırılmış bölgelere ayırır.

Şekil 2.3. Tek bir anlamsal belge, farklı sunum ve etkileşim yollarını destekler.

Şekil 2.3. Aynı anlamsal belge; görsel sunumu, klavye kullanımını, yardımcı teknolojiyi ve farklı görünüm alanlarını destekleyebilir.

2.1 Öğeyi seçmeden önce sözdizimini okuyun

HTML metin olarak yazılır; fakat noktalama işaretleri dil bilgisel anlam taşır. Eksiksiz bir bağlantı öğesini düşünün:

```
<a class="event-link" href="/events/robotics">Robotics event</a>
```

Açılış etiketi `<a` ile başlar ve `>` ile biter. Kapanış etiketi `</a>`'dır. Aradaki metinle birlikte bunlar öğeyi oluşturur. `class` ve `href`, açılış etiketindeki özniteliklerdir; her özniteliğin adı ve tırnak içindeki değeri vardır. “Robotics event” öğenin metin içeriğidir. Etiket, öğe sınırındaki işaretlemedir; eksiksiz öğenin eş anlamlısı değildir.

Şekil 2.4 bu parçaları ve ayrıştırılmış öğeye katkılarını etiketler. Tarayıcı etiketleri düzyazı olarak göstermez; onları özellikleri, içeriği ve ilişkileri olan bir Belge Nesne Modeli (DOM) düğümüne ayrıştırır.

```
<a> etiket adı   class="event-link" öznitelik   href="/events" öznitelik
Robotics event  metin içeriği   </a> kapanış etiketi
DOM bağlantı düğümü: class + href + text
```

Şekil 2.4. Bir HTML öğesi etiketleri, öznitelikleri, içeriği ve ayrıştırılmış DOM gösterimini birleştirir.

Çoğu öğenin açılış etiketi, içeriği ve kapanış etiketi vardır. Öğeler doğru iç içe geçirilmelidir: üst öğeyi kapatmadan önce en son açılan çocuk öğeyi kapatın.

```
<!-- Doğru iç içelik -->
<p>Read the <strong>important guidance</strong> first.</p>

<!-- Öğeler üst üste bindirilmemelidir -->
<p>Read the <strong>important guidance</p></strong>
```

Tarayıcılar hatalı HTML için hata düzeltme kuralları uygular; ancak bu düzeltme, kaynağın amaçlanan ağacı ifade ettiğinin kanıtı değildir. Farklı bir hata veya bağlam farklı bir DOM üretebilir. Kurallara uygun kaynak yazın, doğrulayın ve sonuç şaşırtıcı olduğunda oluşturulan DOM'u inceleyin.

Boş öğeler çocuk içerik taşıyamaz ve kapanış etiketine sahip değildir. Yaygın örnekler `img`, `input`, `meta`, `link`, `br`, `hr` ve `source` öğeleridir. `<img src="map.png" alt="Campus map">` yazın; `<img>Campus map</img>` yazmayın. `<img ... />` sonundaki eğik çizgi HTML'de tolere edilir, fakat XML anlamında öğeyi kapatmaz ve gerekli değildir.

Bir öznitelik normalde tırnak içine alınmış değere sahiptir. Tutarlı çift tırnaklar boşluk veya noktalama işaretlerinin yeni öznitelik sanılmasını önler. Boolean öznitelik mevcutsa doğru, yoksa yanlıştır. Bu nedenle `required="false"` denetimi yine zorunlu yapar; çünkü öznitelik mevcuttur. Açmak için `required` yazın, kapatmak için özniteliği kaldırın.

Normal düzyazıda boşluklar çoğunlukla birleşir; birkaç boşluk bir yerleşim sistemi oluşturmaz. `pre` öğesi anlamlı önceden biçimlendirilmiş boşlukları korur. `<`, `>` veya `&` karakterlerini işaretleme yerine metin olarak göstermek için `<`, `>` ve `&` gibi karakter başvuruları kullanın. Yorumlar istemciye gönderilir; parola, belirteç, özel not veya yalnızca sunucuya ait bilgi içermemelidir.

Genel özniteliklerin görevleri farklıdır:

- `id`, parçalar, etiketler, betikler ve erişilebilirlik ilişkileri için belge genelinde benzersiz tanımlayıcıdır. - `class`, CSS ve betikler için yeniden kullanılabilir sınıflandırmalar atar; kendi başına kullanıcıya dönük anlam taşımaz. - `data-*`, betikler için uygulamaya özel veri saklar; görünür içeriğin veya standart anlamsal özniteliğin yerine geçmez. - `lang`, `dir`, `hidden`, `title` ve `tabindex` dil, yön, görünürlük, yardımcı bilgi veya odak davranışını etkiler; bilinçli kullanılmalıdır.

HTML ayrıca içerik modelleri tanımlar: hangi çocukların ve alt öğelerin izinli olduğunu belirleyen kurallar. Örneğin `ul`, doğrudan çocukları `li` olan bir listeyi temsil eder. `p` başka bir paragraf içeremez; `form` iç içe form içermemelidir. Her kuralı ezberlemek yerine güncel öğe başvurusuna ve W3C Nu HTML Checker'a başvurmayı öğrenin. Doğrulayıcı yinelenen `id` veya yasak çocuk öğeyi bulabilir; fakat başlığın veya alternatif metnin doğru anlamı taşıyıp taşımadığına karar veremez.

2.2 Belge geneli anlam `html` ve `head` içinde başlar

İlk görünür başlıktan önce belge önemli bilgiler iletir. Doctype çağdaş HTML ayrıştırma kipini seçer. `lang`, belgenin insan dilini belirtir; böylece telaffuz, yazım, çeviri ve dile duyarlı araçlar uygun davranabilir. `title`, tarayıcı sekmelerinde, geçmişte, yer imlerinde ve yardımcı gezinmede sayfayı adlandırır. Karakter kodlaması, baytların metni nasıl temsil ettiğini bildirir.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="description" content="Submit a student event to Campus Hub.">
<title>Submit an event - Campus Hub</title>
</head>
</html>
```

Görünüm alanı bildirimi, mobil tarayıcılardan yerleşim görünüm alanını başlangıç ölçeğinde aygıt genişliğiyle eşleştirmesini ister. Tek başına sayfayı duyarlı yapmaz; HTML ve CSS yine kullanılabilir alana uyum sağlamalıdır. `user-scalable=no` gibi değerlerle yakınlaştırmayı kapatmayın. İnsanlar görme güçlüğü, yorgunluk, küçük ekran veya geçici ortam koşulları nedeniyle yakınlaştırma kullanabilir.

Açıklama meta verisi arama, paylaşım ve dizinleme sistemlerine kullanabilecekleri kısa bir özet verir. Görüntülenmesi garanti edilen metin değildir ve görünür girişin yerine geçmez. `title` tarayıcı ve gezinme bağlamında sayfayı tanımlar; görünür `h1` ise belge içindeki ana konuyu tanımlar. İkisi kısa ve belirgin olmalı, karakter karakter aynı olmak zorunda değildir.

Geçerli bir BCP 47 dil etiketi (`en`, `tr`, `fa` gibi) kullanın. Dil değiştiğinde en yakın kapsayıcı öğede `lang` belirtin. Yazım yönü ayrı bir özelliktir: sağdan sola temel yön için `dir="rtl"`, önceden bilinmeyen kullanıcı metni için `dir="auto"` kullanın. Dil ve yön meta verisi arama motoru süslemesi değil, işlevsel bilgidir.

```
<p>Organizer: <bdi>مریم</bdi></p>
```

2.3 Yapı yazı boyutunu değil belgeyi izler

Başlık düzeyi iç içeliği ifade eder. `h1` belgenin ana konusunu adlandırır; `h2` büyük alt bölümü başlatır; `h3` önceki `h2`'nin alt konusudur. Varsayılan yazı boyutu küçük göründüğü için `h4` seçmek, sunumu yapıyla karıştırır. Görünüşü daha sonra CSS denetler.

```
Campus Hub                 h1
  Upcoming events           h2
    Robotics laboratory evening  h3
    Sustainable design workshop  h3
  Student communities       h2
```

Bu yapı yalnızca metin görünümünde, okuyucu kipinde, arama dizininde ve yardımcı gezinmede anlamını korur. Büyük varsayılan başlıktan kaçınmak için düzey seçmeyin.

2.4 Landmark öğeleri sayfayı gezinilebilir bölgelere ayırır

Yaygın yapısal öğeler Tablo 2.2'de özetlenmiştir.

Tablo 2.2 - Yaygın belge bölgeleri için anlamsal öğeler

Öğe	Uygun kullanım
header	Sayfa veya bölüm için giriş içeriği
nav	Önemli bir gezinme bağlantıları grubu
main	Sayfanın kendine özgü birincil içeriği; genellikle bir görünür örnek
section	Normalde başlığı olan tematik grup
article	Bağımsız durabilecek kendi kendine yeten kompozisyon
aside	Çevredeki içerikle dolaylı ilişkili içerik
footer	Sayfa veya bölüm için kapanış bilgileri

Bir öğenin adı ekrandaki konumunu belirlemez. `nav` sayfanın altında bulunabilir; `aside` görsel olarak yanda olmak zorunda değildir. Anlamsal öğeler amacı açıklar.

Birden çok gezinme bölgesi varsa her birine ayrı ad verin:

```
<nav aria-label="Primary navigation">...</nav>
<nav aria-label="Event categories">...</nav>
```

Atlama bağlantısı, normalde `body` içindeki ilk yararlı odaklanabilir öğe olan birincil içeriğe giden sıradan bağlantıdır. Klavye kullanıcılarının tekrarlanan gezinmeyi atlamasını sağlar:

```
<a href="#main-content">Skip to main content</a>
<main id="main-content">...</main>
```

Geçerli sayfa veya adımı `aria-current="page"` ya da uygun başka bir belirteçle işaretleyin. `href` özniteliğini kaldırmayın; geçerli sayfa bilgisi yerel gezinme anlamını tamamlar.

Erişilebilir Zengin İnternet Uygulamaları (Accessible Rich Internet Applications - ARIA) özelliklerini gerçek anlamsal boşluğu doldurmak veya erişilebilir ad sağlamak için kullanın. Yerel öğelere gereksiz rol eklemeyin. `<button role="button">` öğenin zaten söylediğinden fazlasını söylemez. ARIA rolü davranış hakkında bir sözdür; `div` öğesine `role="button"` eklemek onu odaklanabilir yapmaz, klavyeyle etkinleştirmez, form göndermez veya devre dışı durum sağlamaz. Yerel HTML görevi ifade edebiliyorsa genellikle daha güvenilir başlangıç noktasıdır.

3. Metin içeriği, bağlantılar ve medya

`p` paragrafı; `em` okuma vurgusunu; `strong` önem veya aciliyeti; `blockquote` uzun alıntıyı; `q` kısa satır içi alıntıyı ifade eder. `cite`, alıntıyı söyleyen kişiyi değil yaratıcı eseri adlandırır. `code`, `kbd` ve `samp` kaynak kodu, kullanıcı girdisini ve program çıktısını ayırır. Satır sonları ve boşluklar gerçekten önemliyse, çoğu zaman içindeki `code` öğesiyle birlikte `pre` kullanın.

Daha seyrek öğeler de genel işaretlemenin anlamı silmesini önler:

- `figure`, isteğe bağlı `figcaption` ile kendi kendine yeten içeriği gruplar. - `address`, düzyazıda geçen rastgele posta adresini değil, en yakın makale veya sayfanın iletişim bilgisini içerir. - `mark`, arama eşleşmesi gibi geçerli bağlamda ilgili olduğu için vurgulanan metni gösterir. - `small`, her küçük görünmesi gereken cümleyi değil, hukuki veya kaynak bilgisi gibi yan yorumları belirtir. - `sub` ve `sup`, anlam taşıyan alt ve üst indisleri gösterir; örneğin H₂O veya x². - `ins` ve `del`, eklenen ve silinen içeriği belirtir. - `details` ve `summary`, temel bilgiyi keşfetmeyi engellemediği sürece yerel açılır açıklama sağlar.

`div` ve `span`, belirli bir anlamsal öğe uygun olmadığında gruplama için kullanılan nötr kapsayıcılardır. `class="event-card"` gibi anlamlı bir sınıf adı kodu korumaya yardım eder; fakat erişilebilirlik ağacında makale, düğme, başlık veya landmark oluşturmaz. Bu öğeleri yalnızca varsayılan görünüşleri için seçmeyin. `strong` rastgele sözcükleri kalınlaştırma komutu, `br` de boşluk bırakma aracı değildir. Anlam HTML'ye aittir; sunumu daha sonra CSS denetler.

3.2 Listeler gruplamayı ifade eder

Sıra önemli değilse `ul`, sıra veya sıralama önemliyse `ol`, ad-değer grupları için `dl` kullanın. CSS daha sonra bağlantıları yatay gösterse bile bir gezinme bağlantıları sırası kavramsal olarak listedir.

3.3 Bağlantı metni bağlam içinde anlamlı olmalıdır

“Click here” okuyucuya fareyi nasıl kullanacağını söyler; bağlantının nereye gittiğini söylemez. Hedefi veya eylemi adlandıran metni tercih edin:

```
<a href="/events/robotics-open-evening">Robotics open-evening details</a>
```

Gezinmeden mevcut arayüzü değiştiren eylem için bağlantı kullanmayın; bu bir düğmedir. Yalnızca gezinmek için düğme de kullanmayın. Doğru öğeler öngörülebilir klavye, geçmiş, bağlam menüsü ve yardımcı teknoloji davranışı sağlar. Dosya indiriliyorsa dosya türünü ve yararlıysa yaklaşık boyutunu yazın. Yeni gezinme bağlamı açıyorsa etkinleştirmeden önce okuyucuyu uyarın.

URL'nin kendisini, okuyucunun gerçek adresi bilmesi gerekmiyorsa düzyazıya koymaktan kaçının; açıklayıcı bağlantı metni genellikle daha kolay anlaşılır ve yeniden bulunur.

3.4 Görseller bir amaç kararı gerektirir

`alt` özniteliği görsel için metin alternatifi sağlar. İçeriği yalnızca piksellerin ne gösterdiğine değil, bu bağlamdaki amacına bağlıdır:

- Bilgilendirici görsel kısa ve eşdeğer bilgi ister. - Dekoratif görsel çoğunlukla `alt=""` kullanır; böylece ekran okuyucu onu atlar. - Bağlantı olan görsel, bağlantının hedefini veya eylemini adlandırmalıdır. - Karmaşık grafik kısa alternatif ve yakında ayrıntılı açıklama veya veri tablosu ister.

Her alternatife “image of” diye başlamayın; yazılım öğenin görsel olduğunu zaten bildirir. Aynı cümleyi iki kez dinletiyorsa başlığı `alt` içine kopyalamayın.

```
<figure>
  <img src="capacity.svg" width="640" height="360"
       alt="Registration rises from 20 to 48 students over four weeks">
  <figcaption>Workshop registration by week.</figcaption>
</figure>
```

Bu örnek ancak kısa eğilim önemli bilgiyse uygundur. Kesin değerler önemliyse temel değerleri metin veya tablo olarak da verin. Bilinen görsel boyutlarında `width` ve `height`, dosya gelmeden önce doğru en-boy oranı için alan ayırmaya yardım eder. CSS görseli Bölüm 3'te yine duyarlı biçimde ölçekleyebilir.

Salt simgeli bağlantı veya düğmede görsel alternatifi, simgenin geometrik görünüşünü değil işlemi veya hedefi tanımlamalıdır. Satır içi Ölçeklenebilir Vektör Grafikleri (Scalable Vector Graphics - SVG) ve metin görselleri aynı amaç analizini gerektirir; farklı dosya biçimi erişilebilir ad veya eşdeğer bilgi ihtiyacını ortadan kaldırmaz.

3.5 Tarihler, kısaltmalar ve veriler makinece okunabilir değerler taşıyabilir

`time` öğesi, görünür ifade ile makinenin okuyabileceği tarih veya saatin birlikte bulunmasını sağlar. `datetime` değeri yazılımın ayrıştırabileceği standart biçimdedir:

```
<p>The workshop begins <time datetime="2026-10-14T18:30:00+03:00">14 October at 18:30</time>.</p>
```

Görünür metin okuyucuya göre yazılır; öznitelik anı belirlemek için yılı ve Eşgüdümlü Evrensel Zaman (Coordinated Universal Time - UTC) farkını içerir. Yerel etkinliğin zaman dilimi bağlamda kesin ise yerel tarih veya saat değeri kullanılabilir; içerikte belirtilmeyen bir zaman dilimini yazılım kendiliğinden icat etmemelidir.

Kısaltmayı açmak gerçekten yardımcı oluyorsa `abbr` kullanın; yaygın olmayan kısaltmaları normal düzyazıda ilk kullanımda açın. Temel öğretimi `title` ipucuna koymayın; bu bilgi klavye, dokunma veya yardımcı teknoloji kullanıcılarına tutarlı biçimde ulaşmaz.

3.6 Ses ve video eşdeğer erişim gerektirir

Denetimlerin bulunması kaydedilmiş medyayı erişilebilir yapmaz. Anlamlı konuşma içeren önceden kaydedilmiş videoda eşzamanlı altyazı gerekir. Yalnızca ses içeren içerikte döküm bulunmalıdır. Ses kanalında aktarılmayan önemli görsel bilgi sesli betimleme veya eşdeğer metin açıklaması gerektirebilir. Otomatik altyazıya adları, teknik terimleri, noktalama işaretlerini, konuşmacı değişimlerini ve zamanlamayı incelemeden güvenmeyin.

`track` öğesi altyazı dosyalarını videoyla ilişkilendirebilir; fakat medya öğesini yazmakla doğru alternatifi üretmek ayrı sorumluluklardır. Tarayıcı seçilen biçimi oynatamazsa bir yedek bağlantı da sağlayın.

3.7 Tablolar verilerdeki ilişkiler içindir

Veri tablosu, başlıklarla hücreler arasındaki ilişkileri temsil eder; sayfa yerleşim mekanizması değildir.

```
<table>
  <caption>Remaining places by workshop</caption>
  <thead><tr><th scope="col">Workshop</th><th scope="col">Places</th></tr></thead>
  <tbody><tr><th scope="row">Robotics safety</th><td>12</td></tr></tbody>
</table>
```

`caption` tabloyu adlandırır. Başlık hücreleri ve `scope`, satır-sütun ilişkilerini yazılıma açar; yalnızca görsel hizalama bu ilişkileri iletemez. Basit tabloda `scope="col"` ve `scope="row"` genellikle yeterlidir. Karmaşık tablolar bilişsel ve test maliyeti doğurur; önce veriyi daha basit tablolara bölüp bölemeyeceğinizi sorun.

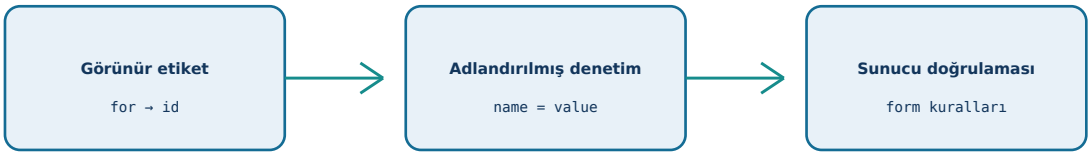
Boş bir başlık hücresini yalnızca görsel boşluk oluşturmak için kullanmayın; toplamı veya kategoriyi yalnızca renge ya da konuma dayandırmayın.

4. Formlar kısıtları olan konuşmalardır

Form, bir işlem için değerleri toplar. Başarılı her denetim, form gönderildiğinde bir ad-değer çifti sağlar. `name` özniteliği gönderimde kullanılacak adı verir; `id` ise belge içindeki öğeyi tanımlar. Değerleri çoğu zaman aynı olsa da görevleri farklıdır.

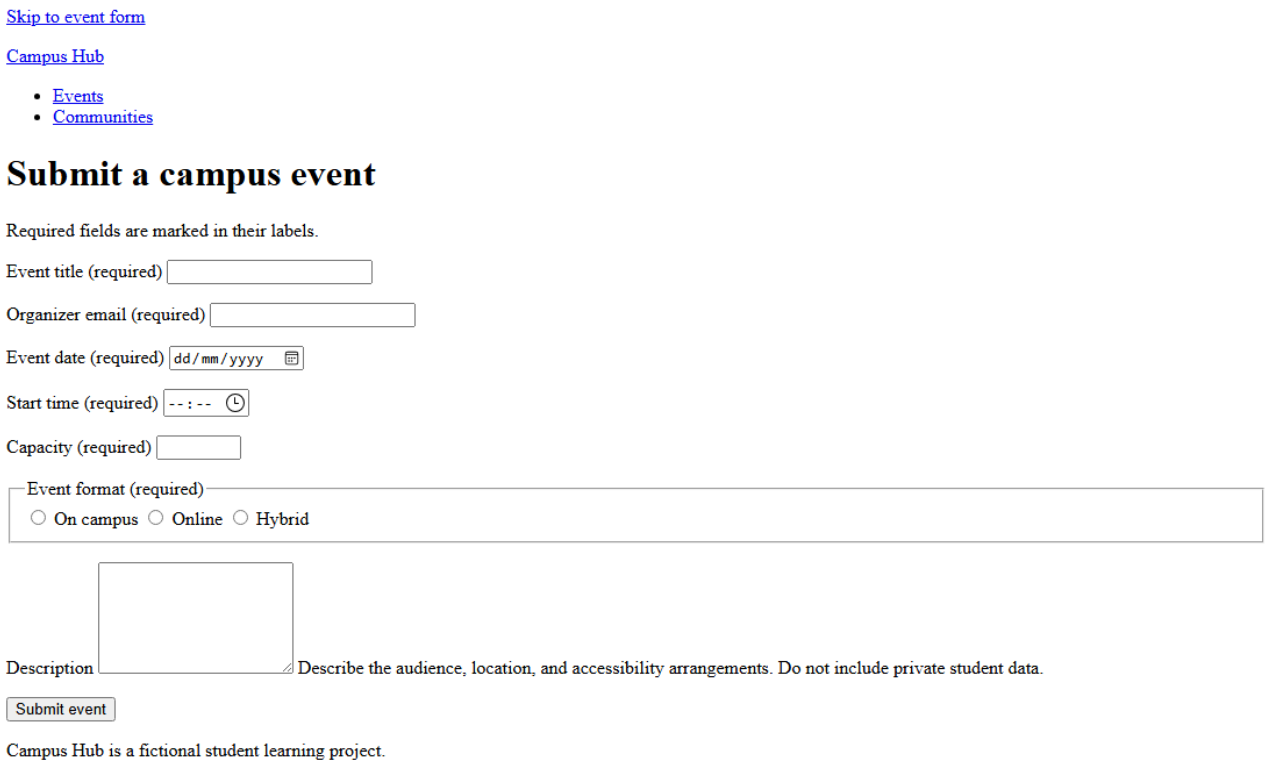
```
<form action="/events" method="post">
  <label for="event-title">Event title</label>
  <input id="event-title" name="title" type="text" required>
  <button type="submit">Submit event</button>
</form>
```

Etiketin `for` değeri denetimin `id` değeriyle eşleşir. Etiketi etkinleştirmek odağı girdiye taşır. Yardımcı teknoloji “Event title, edit text, required” diyebilir. `name="title"`, gönderilen alan adını belirler. `action` gönderim hedefini, `method` ise tarayıcının form gönderme algoritmasında kullanacağı HTTP yöntemini seçer.



Şekil 2.5. Etiket denetimi tanımlar; name değeri istek sınırından geçer; sunucu değeri yeniden doğrular.

Şekil 2.6, tamamlanmış Week 2 formunun tarayıcı çıktısını gösterir. Özel CSS eklenmeden önce doğru HTML'nin sağladıklarını görebilmeniz için yerel tarayıcı görünüşünü bilerek kullanır: görünür etiketler, yerel denetimler, gruplandırılmış seçenekler, yardım metni ve gönder düğmesi.



Şekil 2.6. Tamamlanmış anlamsal formun yerel tarayıcı çıktısı.

4.1 İşleme göre gönderim yöntemini seçin

`get` formu bir alma işlemini temsil eder. Başarılı ad-değer çiftleri normalde sonuç URL'sindeki sorguya dönüşür; bu, aramayı yer imine eklemeyi ve paylaşmayı kolaylaştırır. `post` formu form verisini istek içeriğinde gönderir ve sunucu durumunu oluşturan veya değiştiren işlemler için uygundur.

`post` veriyi şifrelemez ve işlemi güvenli yapmaz: HTTPS kullanın, sunucuda doğrulayın, işlemi yetkilendirin ve hizmetin gerektirmediği bilgileri toplamayın. Yöntem yalnızca sözdizimi tercihi değildir; yenileme, yer imi, önbellek, geçmiş, günlükler ve yeniden denemeler farklı davranır. Arama formu `get`, etkinlik oluşturma formu `post` kullanabilir.

4.2 Etiketler ve açıklamalar kullanılabilir kalmalıdır

Yer tutucu bir örnek veya ipucudur; etiketin yerine geçmez. Kullanıcı yazmaya başlayınca kaybolur, çoğu zaman düşük kontrastla gösterilir ve erişilebilir adı tutarlı biçimde sağlamaz. Görünür etiketi koruyun. Biçim açıklaması önemliyse kalıcı yardım metnini denetimle ilişkilendirin:

```
<label for="event-date">Event date</label>
<input id="event-date" name="date" type="date"
       aria-describedby="event-date-help" required>
<p id="event-date-help">Choose a date within the current semester.</p>
```

Bağlantılı etiket erişilebilir adı, yani denetimin kısa kimliğini sağlar. `aria-describedby` ek yönergeleri veya bağlamı içeren erişilebilir açıklamayı ekler; açıklama adın yerine geçmez. Yardımcı teknoloji kullanmayan kişiler de aynı bilgiyi alabilsin diye önemli talimatları görünür tutun.

4.3 En dar uygun yerel denetimi seçin

Girdi türleri amacı bildirir, mobil klavyeler ve yerleşik denetimler sağlayabilir. `email`, `url`, `date`, `time`, `number`, `search` ve `password` örnekleridir. Çok satırlı girişte `textarea`, sınırlı listede `select`, bağımsız seçimlerde onay kutusu, bir kümeden tek seçimde radyo düğmeleri kullanın.

Türü, değerlerin taşıdığı anlama göre seçin; içerdiği karakterlere göre değil. Posta kodu, telefon, öğrenci numarası, kredi kartı numarası veya tek kullanımlık kod bir hesaplama miktarı değil tanımlayıcıdır. Genellikle `type="number"` yerine uygun `autocomplete` ve `inputmode` ile metin benzeri giriş kullanılır. `inputmode` klavye ipucudur; değeri doğrulamaz.

Tablo 2.3 yaygın seçimleri özetler.

Tablo 2.3 - Yerel form denetimleri ve destekleyici öznitelikleri seçme

Görev	Yerel başlangıç	Önemli nokta
Kısa genel metin	type="text"	Görünür etiket, yararlı autocomplete ve görev tanımlıyorsa uzunluk kısıtı ekleyin.
E-posta	type="email"	Tarayıcı biçime bakar; posta kutusunun varlığını veya kullanıcıya ait olduğunu doğrulamaz.
Parola	type="password"	Doğru autocomplete belirteci; parola yöneticisi ve yapıştırmaya izin verin. Maskeleme şifreleme değildir.
Arama	type="search"	Bilgi alma veya filtreleme yaptığı için normalde GET formundadır.
Miktar	type="number"	Aritmetik miktarlarında anlamlı min, max ve gerekirse step kullanın; rakam dizisi tanımlayıcısında kullanmayın.
Telefon / sayısal kimlik	type="tel" veya inputmode="numeric"	Baştaki sıfırları ve biçimi koruyun; gerçek alan kuralına göre doğrulayın.
Tarih / saat	type="date" veya type="time"	Yerel görünüş değişir; önemliyse zaman dilimini ve aralığı belirtin.
Çok satırlı metin	textarea	rows başlangıç boyutudur, içerik sınırı değildir.
Kısa kümeden tek seçim	fieldset içindeki radyo	Ortak name seçimi oluşturur; legend grubu adlandırır.
Bağımsız evet/hayır	checkbox	Her seçimin etiketi ve açık değeri olmalı; işaretlenmeyenler gönderilmez.
Uzun seçenek kümesi	select + option	Anlamlı gruplar için optgroup; istem seçeneğini belirsiz bırakmayın.
Serbest girişli öneri	input + datalist	Öneri kapalı kısıt değildir; sunucu gerçek değeri kabul veya reddeder.
Dosya	type="file"	POST ve genellikle multipart/form-data; accept yalnızca seçici ipucudur.

İlişkili seçenekleri `fieldset` ve `legend` ile gruplayın:

```
<fieldset>
  <legend>Attendance mode</legend>
  <label><input type="radio" name="mode" value="campus" required> On campus</label>
  <label><input type="radio" name="mode" value="online"> Online</label>
</fieldset>
```

Ortak `name`, radyo düğmelerini tek gönderilecek seçime dönüştürür; `legend` grubu adlandırır. Görsel olarak yan yana duran fakat bu ilişkiyi taşımayan denetimleri yorumlamak daha zordur.

Düğmenin türü de önemlidir. Form içinde düğme varsayılan olarak gönderim yapar. Gönderici için `type="submit"`, yalnızca istemci tarafı eylem yapan düğme için `type="button"` yazın. Olağan formlarda reset düğmelerinden kaçının; yanlış etkinleştirme çalışmayı silebilir. Formları iç içe koymayın. Enter tuşu bazı formları örtük gönderimle gönderebilir; her form yalnızca işaretçi tıklamasına bağlı olmadan tutarlı kalmalıdır.

4.4 Hangi denetimlerin gönderileceğini tahmin edin

Tarayıcı form içindeki her öğeyi göndermez. Tablo 2.4, eksik veya şaşırtıcı verileri tanılarken en sık gereken kuralları özetler.

Tablo 2.4 - Başarılı denetimlerin gönderim kuralları

Denetim koşulu	Gönderilir mi?	Neden / sonuç
Adı olan etkin denetim	Genellikle evet	Mevcut değeri o adla ilişkilendirilir.
Adı olmayan denetim	Hayır	id öğeyi hedefleyebilir fakat gönderilen alan adı oluşturmaz.
Devre dışı denetim	Hayır	Başarılı denetim değildir ve normalde kullanılamaz.
Salt okunur metin	Evet	Düzenlenemez; yine de gönderilir ve güvenilmemelidir.
İşaretsiz checkbox/radio	Hayır	Bu denetim için ad-değer çifti eklenmez.
İşaretli checkbox/radio	Evet	Açık değer gönderilir; değer yoksa `on` gibi varsayılan şaşırtabilir.
Gönderimi başlatan submit	Evet	Adı ve değeri seçilen eylemi tanımlayabilir; diğer submit düğmeleri atlanır.
Dosya seçilmemiş file	Tarayıcıya bağlı boş giriş	Sunucu boş dosya durumunu açıkça ele almalıdır.

Gönderim kuralları taşımayı açıklar, güveni değil. Gizli ve salt okunur değerler istemci tarafından değiştirilebilir. Sunucu; geçerli kullanıcı veya yetkili fiyat gibi güvenlik açısından önemli olguları güvenilir sunucu durumundan üretmelidir.

4.5 Özellikle dosyalarda kodlamayı seçin

Form gönderimi adları ve değerleri baytlara dönüştürmelidir. GET'te veri URL sorgusuna kodlanır. Olağan POST formunun varsayılanı `application/x-www-form-urlencoded`'dır; alanlar istek gövdesinde sorguya benzer ad-değer biçiminde taşınır. Boşluklar ve ayrılmış karakterler yüzde kodlaması ister.

```
<form action="/event-materials" method="post" enctype="multipart/form-data">
  <label for="agenda">Agenda document</label>
  <input id="agenda" name="agenda" type="file" accept=".pdf">
  <button type="submit">Upload agenda</button>
</form>
```

Multipart kodlama, dosya baytlarının ve sıradan metnin tek istekte taşınması için parçaları ayırır. `accept=".pdf"` seçiciye olası seçenekleri sunmaya yardım eder; güvenlik sınırı değildir. Sunucu türü, boyutu, içeriği, yetkiyi, saklama adını, kötü amaçlı yazılım politikasını ve saklama süresini uygulamalıdır. İstemci dosya adına, uzantısına, medya türüne veya yerel yoluna güvenmeyin.

4.6 Otomatik tamamlama, giriş amacı ve veri minimizasyonu

`autocomplete`, tarayıcıların ve yardımcı teknolojilerin ortak kişisel veri alanlarının amacını anlamasına yardım eder. `name`, `email`, `organization` ve `current-password` gibi belirteçler rastgele alan adlarından daha bilgilendiricidir. Gereksiz yere kapatmayın. Alanları meraktan değil görevden seçin; neden gerekli olduklarını açıklayın ve örneklerde sırları toplamayın.

`inputmode`, `autocapitalize` ve `spellcheck` gibi öznitelikler giriş yükünü azaltabilir; fakat doğrulama değil ipucudur. Parola veya tek kullanımlık kod gibi değeri yazım hizmetine göndermek uygun değilse yazım denetimini kapatın. Parola ve doğrulama alanlarında yapıştırmayı engellemeyin; parola yöneticileri ve yardımcı iş akışları buna ihtiyaç duyar.

4.7 Kısıt doğrulaması yararlıdır fakat eksiktir

`required`, `min`, `max`, `minlength`, `maxlength` ve `pattern` gibi öznitelikler tarayıcıya kısıt bildirir. Deneyimi iyileştirir ve önlenebilir istekleri azaltır; güvenlik sınırı değildir. İstemci JavaScript'i kapatabilir, işaretlemeyi değiştirebilir veya HTTP isteğini doğrudan gönderebilir. Sunucu her güvenilmeyen değeri yeniden doğrulamalıdır.

Doğrulama üç soruya cevap vermelidir: 1) Sorun nedir? 2) Sorun nerede? 3) Kullanıcı nasıl düzeltebilir? Yalnızca renk bu soruları yanıtlamaz. Belirli metin sağlayın, geçerli değerleri koruyun, mesajları denetimlerle ilişkilendirin ve başarısız gönderimden sonra odağı uygun biçimde taşıyın. Zorunlu ve isteğe bağlı durumu sözcüklerle belirtin; açıklamasız yıldız işareti belirsizdir.

4.8 JavaScript eklemeden önce hata kurtarmayı tasarlayın

Sunucu tarafından üretilen yanıt, JavaScript olmadan hata özeti ve alan mesajlarıyla formu geri döndürebilir. Sayfa başlığı başarısız durumu duyurmalı, özet formdan önce görünmeli ve her mesaj sorunu ve düzeltmeyi belirtmelidir. Güvenlik sakıncası yoksa geçerli değerleri koruyun.

```
<title>2 errors - Submit an event - Campus Hub</title>
<div role="alert" tabindex="-1">
  <h2>There are 2 errors</h2>
  <ul>
    <li><a href="#event-title">Enter an event title.</a></li>
    <li><a href="#capacity">Enter a capacity from 1 to 300.</a></li>
  </ul>
</div>

<label for="capacity">Capacity (required)</label>
<input id="capacity" name="capacity" type="number" min="1" max="300"
       required aria-invalid="true" aria-describedby="capacity-error">
<p id="capacity-error">Enter a whole number from 1 to 300.</p>
```

`aria-invalid="true"` yalnızca mevcut değerin geçersiz olduğu bilindiğinde denetimde bulunmalıdır. Açıklama düzeltmeyi bağlar. `tabindex="-1"`, özeti olağan Tab sırasına eklemeden betik veya sunucu geliştirmesinin odağı özete taşımasını sağlar. Statik sunucu yanıtı yeni sayfa olarak zaten duyurulur; dinamik eklenen hatalarda `role="alert"` özellikle önemlidir.

Çalıştırmadan önce tahmin edin: `<input id="capacity" type="number" min="1" max="300" required>` kontrolü `capacity` adıyla gönderilir mi? Hayır. Denetimin `id` değeri var, fakat `name` yoktur; bu nedenle ad-değer çifti oluşturmaz. Tarayıcı yine doğrulama yapabilir ve uyarı gösterebilir.

5. Birbiriyle ilişkili üç gösterim

Tarayıcı birkaç ilişkili modeli korur: 1) Kaynak HTML, dosyadan okunan veya alınan metindir. 2) DOM, ayrıştırılmış ve canlı belge ağacıdır. 3) Erişilebilirlik ağacı, yardımcı araçlara açılan önemli rol, ad, durum, değer ve ilişkileri içeren platforma özgü gösterimdir.

Şekil 2.7, erişilebilirlik ağacını canlı DOM ile platform gösterimini kullanan yardımcı teknoloji arasına yerleştirir.

```
HTML kaynağı -> DOM + durum -> Erişilebilirlik ağacı -> Yardımcı teknoloji
```

Roller, adlar, durumlar, değerler ve ilişkiler işletim sistemi API'leri üzerinden açılır. CSS ve JavaScript mevcut, görünür, işletilebilir veya açığa çıkarılmış olanı değiştirebilir.

Şekil 2.7. HTML ve tarayıcı durumu, platformun erişilebilirlik gösterimine katkıda bulunur.

Bu gösterimler aynı değildir. CSS içeriği gizleyebilir. HTML ayrıştırıcısı eksik yapıyı onarabilir. JavaScript DOM'u değiştirebilir. Bazı DOM düğümlerinin erişilebilirlikte görünmesi gerekmez; tek bir bileşik araç ise birkaç erişilebilir ilişki oluşturabilir.

HTML `hidden` özniteliği, CSS `display: none` ve kapalı disclosure'ın gizli durumu normalde alt içeriği hem görsel oluşturma hem erişilebilirlik ağacından kaldırır. Yalnızca yardımcı teknoloji için görsel olarak gizlenen içerikte Bölüm 3'te tanıtılan, dikkatle test edilmiş CSS kalıbını kullanın; bu amaçla `hidden` kullanmayın. İçeriği ekran dışına taşırken de odağı anlamadan hareket ettirmeyin: klavye kullanıcısı göremediği bir hedefe ulaşabilir.

Erişilebilirlik ağacı “DOM'un ekran okuyucu sürümü” değildir. İşletim sisteminin erişilebilirlik API'leri üzerinden kullanılan anlamsal arayüzdür. Tarayıcı ve platform ayrıntıları değişebilir; iyi yerel HTML birlikte çalışabilir güçlü bir temel sağlar.

5.1 Erişilebilir ad, açıklama, rol ve durum

Etkileşimli denetimlerin amacını tanımlayan bir adı olmalıdır. Düğmenin metni genellikle adını sağlar. Girdi çoğunlukla bağlı etiketinden ad alır. Görsel `alt` ile adlandırılabilir. `aria-label`, görünür metin yoksa ad sağlayabilir; ancak görünür metin genellikle tercih edilir çünkü bütün kullanıcılar aynı ifadeyi alır. Açıklama destek bilgisi verir ve adı kelimesi kelimesine tekrarlamamalıdır. Rol nesnenin türünü; durum ise işaretli, genişletilmiş, seçili, geçersiz veya devre dışı gibi bilgileri bildirir.

Konuşma girişi kullanan kişiler için görebildikleri sözcükler erişilebilir adda bulunmalıdır. Görünür etiket “Organizer email” iken `aria-label` denetimi sessizce “Contact address” diye yeniden adlandırırsa, görünür sözcüklere dayalı ses komutu başarısız olabilir. Görünür bağlı etiketi tercih edin; iyi görünür metni değiştirmek yerine ek yardım için `aria-describedby` kullanın. Önemli görünür metni CSS'nin ürettiği içerikte bırakıp tutarlı biçimde açığa çıkacağını varsaymayın; anlamlı içeriği HTML'ye koyun.

5.2 Klavye kullanımı bir mühendislik testidir

Etkileşimli öğeler arasında ilerlemek için Tab, geri gitmek için Shift+Tab, bağlantıları ve birçok düğmeyi etkinleştirmek için Enter, düğmeleri ve onay kutularını etkinleştirmek için Space kullanın. Ok tuşları, kendi ok tuşu davranışını tanımlayan denetimlerde çalışır.

Klavye incelemesi şu soruları sormalıdır:

- Her işlem erişilebilir mi? - Odak göstergesi çevresinde görünür mü? - Odak sırası okuma ve görev sırasını izliyor mu? - Kullanıcı diyalog veya menü gibi bileşenlerden çıkabiliyor mu? - Odak yalnızca açık bir gerekçe olduğunda mı taşınıyor?

Pozitif `tabindex` değerleri sürdürülmesi zor ayrı bir odak sırası oluşturur. Mantıklı kaynak sırasını ve yerel öğeleri tercih edin. Normalde odaklanamayan hata özeti gibi bir öğenin betik aracılığıyla odak alması gerektiğinde `tabindex="-1"` kullanın; onu olağan Tab sırasına sokmak için kullanmayın.

6. Erişilebilirlik uygunluktan daha geniştir

Web İçeriği Erişilebilirlik Kılavuzları (Web Content Accessibility Guidelines - WCAG) 2.2 erişilebilirliği dört ilke altında düzenler: algılanabilir, işletilebilir, anlaşılabilir ve sağlam. İngilizce baş harfleri yararlı POUR hatırlatıcısını oluşturur. Tablo 2.5 her ilkeyi Campus Hub için öğrencinin sorabileceği soruyla ilişkilendirir.

Tablo 2.5 - WCAG ilkeleri mühendislik soruları olarak

Algılanabilir	Gerekli bilgi metin alternatifleri ve altyazılar dâhil birden fazla uygun duyusal yoldan alınabiliyor mu?
İşletilebilir	Klavye, işaretçi, dokunma ve yakınlaştırmayla tuzak veya gizli odak olmadan görev tamamlanabiliyor mu?
Anlaşılabilir	Etiketler, yönergeler, gezinme, hatalar ve değişiklikler eylem için yeterince öngörülebilir mi?
Sağlam	Adlar, roller, durumlar, değerler ve ilişkiler desteklenen tarayıcılar ile yardımcı teknolojilerde korunuyor mu?

WCAG başarı ölçütleri A, AA veya AAA uygunluk düzeylerine atanır. AA yaygın bir kurumsal hedeftir; ancak uygunluk tam sayfalar ve süreçler için tanımlı ölçütlere göre değerlendirilir, tek tarama aracının ürettiği genel puan değildir. Bir ürün biçimsel bir ölçütü karşılayıp kötü deneyim sunabilir; tek bir otomatik uyarı da insan yorumuna ihtiyaç duyabilir.

Standartlar test edilebilir gereksinimler sağlar; fakat otomatik denetleyici alternatif metnin doğru amacı taşıyıp taşımadığını, yönergelerin anlaşılır olup olmadığını veya iş akışının makul bilişsel yük taşıyıp taşımadığını belirleyemez. Şu kanıtları birlikte kullanın:

- geçerli anlamsal HTML; - otomatik kontroller; - klavye testi; - tarayıcı erişilebilirlik incelemesi; - yakınlaştırma ve yeniden akış testi; - mümkünse yardımcı teknoloji testi; - engelli kişilerden ve temsili kullanıcılardan geri bildirim.

Otomatik taramayı geçmek kanıttır, erişilebilirliğin kanıtlanması değildir. Şekil 2.8, bu kanıt kaynaklarının paralel çalışmasını gösterir. Otomatik kontroller programca saptanabilir bazı hataları bulur; klavye ve yakınlaştırma testleri etkileşim ve yeniden akış sorunlarını açığa çıkarır; erişilebilirlik ağacı ve ekran okuyucu adları, rolleri, durumları ve okuma davranışını gösterir. Temsili kullanıcılar gerçek görevin anlaşılır ve tamamlanabilir olup olmadığını ortaya koyar.

Şekil 2.8. Erişilebilirlik yargısı birden çok kanıt kaynağının birleşimidir.

Yönlendirmeli laboratuvar - Erişilebilir etkinlik formu oluşturun

Amaç ve kabul ölçütleri

Week 2 başlangıcını CSS ve JavaScript olmadan erişilebilir bir etkinlik gönderme sayfası olarak tamamlayın. Sayfa yalnızca yapısıyla anlaşılır kalmalıdır. Ürününüz şunları içermelidir:

- yararlı title, dil, kodlama, açıklama ve viewport bildirimi; - hedefi main öğesi olan skip link; - bir h1 ve mantıksal biçimde iç içe başlıklar; - adlandırılmış birincil gezinme; - anlamlı action ile post kullanan form; - başlık, düzenleyici e-postası, tarih, başlangıç saati, kapasite, biçim ve açıklama için görünür etiketler; - yerel girdi türleri ve makul kısıtlar; - düzenleyici e-postası için uygun autocomplete; - ilgili gezinme bağlantısında `aria-current="page"`; - etkinlik biçimi için fieldset ve legend; - açıklamayla ilişkilendirilmiş kalıcı yardım; - açık türü olan submit düğmesi; - pozitif tabindex, yerel rolü tekrarlayan ARIA rolü, satır içi olay işleyicisi, sır veya gerçek kişisel veri bulunmaması; - en az üç tutarlı Git commit'i.

Adım 1 - Düzenlemeden önce okuyun

Depo kökünden `labs/week-02/index.html` dosyasını VS Code metin düzenleyicisinde açın. Yalnızca sekmedeki `index.html` adına güvenmeyin; tam klasör yolunu doğrulayın. SERVER terminalinde Windows'ta `dir package.json` veya macOS/Linux'ta `ls -l package.json` çalıştırın ve ardından `npm run serve:week02` komutunu verin. Süreci çalışır bırakın, `http://localhost:8000/` adresini açın ve temel sayfanın `Submit a campus event` gösterdiğini doğrulayın. Git ve testler için ikinci terminal bulundurun.

Kaynakta her TODO'yu belirleyin. Amaçlanan başlık yapısını ve form veri adlarını kâğıda çizin. Hangisinin yönerge, etiket ve kısıt olduğuna karar verin. Her anlamlı düzenlemeyi kaydedip tarayıcıyı yenileyin; yerel sunucu kaydedilmiş dosyaları gösterir, kaydedilmemiş düzenleyici içeriğini değil.

Adım 2 - Landmark ve gezinmeyi kurun

body içindeki ilk yararlı bağlantı olarak skip link, sayfa header'ı, adlandırılmış nav, `id="main-content"` olan main ve footer ekleyin. Klavyeyle skip link'i etkinleştirin ve hedefinin var olduğunu doğrulayın.

Adım 3 - Formu yerel denetimlerle kurun

Etiketleri ve yerel girdi türlerini kullanın. Sunucunun yorumlayabileceği `title`, `organizerEmail`, `date`, `startTime`, `capacity`, `format` ve `description` gibi adlar seçin. Yer tutucu metni tek yönerge olarak kullanmayın.

Adım 4 - Gönderim sözleşmesini tanımlayın

Etkinlik oluşturduğu için formun `post` kullandığını doğrulayın. Başarılı gönderimde beklenen alan adlarını sırayla yazın; devre dışı veya işaretsiz denetimlerin gönderimde neden kaybolacağını tahmin edin. Salt okunur veya arayüzde gizli olduğu için hiçbir alanı güvenilir saymayın.

Adım 5 - Kısıtları ifade edin

Açıklama dışında tüm alanları zorunlu yapın. Kapasiteyi makul aralıkla sınırlandırın. Serbest metin için yararlı bir üst uzunluk belirleyin. Açıklama yardımını `aria-describedby` ile bağlayın.

Adım 6 - Fare olmadan test edin

Sayfanın üstünden yeniden yükleyin, fareyi bir kenara bırakın ve formu tamamlayın. Odak sırasını kaydedin. En az iki yerel doğrulama mesajı üretin. %200 yakınlaştırmada hiçbir bilginin kaybolmadığını doğrulayın.

Adım 7 - Kaynağı doğrulayın ve ağacı inceleyin

Kaynağı W3C Nu HTML Checker'a doğrudan gönderin veya belge özelse onaylı yerel eşdeğer kullanın. Her kaynak hatasını düzeltin. Ardından başlık yapısını, landmark'ları, etiketleri ve kapasite denetiminin ad, rol, değer, açıklama ve zorunlu durumunu tarayıcı erişilebilirlik araçlarında inceleyin. Doğrulayıcının kanıtlayabildiği bir şeyi ve yargılayamadığı bir anlamı kaydedin.

Adım 8 - Doğrulayın, açıklayın ve commit oluşturun

Sunucu terminalini çalışır bırakın ve Terminal > New Terminal ile ikinci terminal açın. Bu ikinci terminal komut terminalidir; ilk terminal sunucuyla meşgulken istem gösterir. Depo kökünden `npm test` çalıştırın. Son çıktı `Coursework setup verification passed` bildirmelidir. Bu kontrol verilen yapı ve sunucunun çalıştığını kanıtlar; formdaki her anlamsal kararı puanlamaz.

İlk raporlanan kontrolü okuyup düzeltin, kaydedin ve komutu yeniden çalıştırın. Sayfayı kabul ölçütleriyle karşılaştırın ve bir yerel anlamsal seçimi, bir klavye gözlemini, tarayıcı doğrulamasının bir sınırını, bir gizlilik kararını ve kullanıcılarla deneyeceğiniz bir iyileştirmeyi açıklayan beş cümle yazın. Son commit'i yalnızca tarayıcı, erişilebilirlik incelemesi, doğrulayıcı ve otomatik kontrol aynı kaynakla uyuştuğunda oluşturun.

Tablo 2.6 bir kanıt kaydıdır; gözlem yapmadan işaretlenecek kontrol listesi değildir. Öğrenciler ne gördüklerini yazmalı ve gözlemin anlamını açıklamalıdır.

Tablo 2.6 - Laboratuvar erişilebilirlik gözlem kaydı

Kontrol	Kanıt ve yorum
Başlık yapısı	
Skip-link hedefi	
İlk beş odak durağı	
Kapasite denetiminin erişilebilir adı	
Yöntem ve action gerekçesi	
Beklenen başarılı alan adları	
İki yerel doğrulama sonucu	
Gizlilik nedeniyle dışlanan veri	
Otomatik kontrolün sınırı	
Nu HTML Checker sonucu ve sınırı	
Erişilebilirlik ağacı adı, rolü, durumu ve açıklaması	

Alıştırmalar

Temel Alıştırma 1 - Öğe seçimi

Her öğe için en iyi yerel HTML öğesini seçin ve nedenini açıklayın: gezinme hedefi, filtre paneli açan işlem, kendi kendine yeten haber öğesi, etkinlik kategorileri kümesi ve sayfanın kendine özgü içerik bölgesi.

Alıştırma 2 - Etiket ve ad

`label[for]`, `id` ve `name` özniteliklerinin formdaki bağımsız amaçlarını açıklayın. Bir input yalnızca `id="email"` içerirse ne bozulur?

Alıştırma 3 - Alternatif metin

Bir etkinlik kartında dekoratif konfeti deseni, görünür “Dr. Ada Kaya” adı yanında konuşmacı portresi ve fakülteye göre kayıt grafiği vardır. Her biri için alternatif metin stratejisi önerin.

Uygulama Alıştırması 4 - Anahatı düzeltin

Başlıkları istenen yazı boyutuna göre seçmeden şu yapıyı düzeltin: `<h3>Campus Hub</h3> <h1>Events</h1> <h4>Robotics evening</h4> <h2>Communities</h2>`

Alıştırma 5 - Erişilemez denetimi tanılayın

Bu neden yeterli bir düğme değildir ve en küçük doğru değişiklik nedir? `<div class="button" onclick="submitEvent()">Submit</div>`

Alıştırma 6 - Tablo ilişkileri

Üç odanın kapasitesini, binasını ve uygunluğunu anlatan anlamsal tablo işaretlemesi tasarlayın. Caption, sütun başlıkları ve satır başlıklarını belirtin.

Zorluk Alıştırması 7 - Doğrulama sınırı

Tarayıcı boş başlığı reddediyor; saldırgan ise API'ye boş başlık göndermeyi başarıyor. Her iki gözlemin nasıl doğru olabildiğini açıklayın ve tarayıcı, sunucu sınırı ve veri tabanı arasındaki doğrulama sorumluluklarını belirtin.

Alıştırma 8 - Erişilebilirlik ağacı akıl yürütmesi

Görsel olarak gizlenmiş checkbox DOM'da kalıyor fakat `display: none` ile erişilebilirlik ağacından kaldırılıyor. Biçimlendirilmiş bir `div` görünür ve tıklanabilir; fakat rolü veya klavye davranışı yok. Farklı kullanıcılar ve araçlar ne görebilir? Yerel HTML onarımlarını önerin.

Alıştırma 9 - Gönderilen veriyi tahmin edin

```
<form action="/profiles" method="post">
<input name="displayName" value="Ada">
<input name="theme" value="dark" disabled>
<input name="studentId" value="CE-104" readonly>
<input type="checkbox" name="newsletter" value="yes">
<button type="submit" name="intent" value="save">Save</button>
</form>
```

Kullanıcı Ada giriyor, checkbox'ı boş bırakıyor ve Save düğmesine basıyor. Tarayıcının göndereceği ad-değer çiftlerini yazın; diğer denetimlerin neden katkı yapmadığını açıklayın.

Alıştırma 10 - Test stratejisi seçin

Bir tarayıcı aracı ders kayıt formunda erişilebilirlik hatası bildirmiyor. Bunun neden yetersiz olduğunu açıklayın. En az dört farklı kanıt türü toplayan kısa bir test planı tasarlayın; her yöntemin güvenilir biçimde yanıtlayabildiği ve yanıtlayamadığı bir soruyu belirtin.

Alıştırma 11 - Sözdizimini ve anlamı onarın

```
<p class=event note>Register <strong>before Friday</p></strong>
<input id="email" type="email" required="false" placeholder="Email">
<p id="help">Use your university address & include the department.</p>
```

Bağımsız sorunları belirleyin, kurallara uygun onarım yazın ve tarayıcının özgün kaynaktan neyi çıkaramayacağını açıklayın.

Alıştırma 12 - Denetimleri ve kodlamayı seçin

Etkinlik araması, altı haneli tek kullanımlık kod, 1-20 arası koltuk miktarı ve isteğe bağlı PDF gündem yüklemesi için HTML form sözleşmesi tasarlayın. Yöntem, denetim türleri, klavye ipuçları, adlar, kısıtlar ve kodlamayı seçin. Neden dört işlemin tek formda bulunmasının zorunlu olmadığını açıklayın.

Alıştırma 13 - Hata kurtarma tasarlayın

Sunucu `capacity=500` değerini reddedip formu geri döndürüyor. Sayfa başlığını, hata özetini, bağlantı hedefini, alan özniteliklerini, alan mesajını, değeri koruma kararını ve odak planını belirtin. JavaScript hatayı mevcut sayfaya eklerse ne değişir?

Alıştırma 14 - Karmaşık tabloyu modelleyin

Bir zaman çizelgesinde Monday ve Tuesday adlı iki sütun grubu, her birinin Morning ve Afternoon sütunları vardır. Her satır oda başlığıyla başlar. `caption`, `thead`, `tbody`, `colgroup` ve uygun `scope` değerleriyle ilişkileri tasarlayın. Tabloyu ne zaman iki basit tabloya böleceğinizi veya açık `id` ve `headers` ilişkileri kullanacağınızı belirtin.

Çözümlü cevaplar

Cevap 1

Gezinme hedefi için geçmişe, bağlantı menülerine, kopyalamaya ve yerel klavye davranışına katılan `href` değerli `a` kullanın. Filtre paneli açma işlemi mevcut arayüzde eylem yaptığı için `button` olmalıdır. Kendi kendine yeten haber `article`, sırası önemli olmayan kategoriler `li` içeren `ul`, sayfanın benzersiz içeriği `main` olmalıdır. CSS görünüşü değiştirebilir; anlamı silmez.

Cevap 2

`id` öğeyi belgede benzersiz tanımlar. `label for="email"` görünür etiket metnini denetimle ilişkilendirir. `name`, form verisi oluşturulurken kullanılan anahtardır. Yalnızca `id="email"` varsa bağlı etiket çalışabilir; fakat denetim `email` adlı alan göndermez. Değerleri aynı olabilir, görevleri değişmez.

Cevap 3

Dekoratif konfeti `alt=""` kullanmalıdır. Yanındaki görünür ad tüm kimliği sağlıyorsa portre de boş alternatif kullanabilir; ek bilgi taşıyorsa yalnızca o bilgi yazılır. Grafik kısa sonuç özeti ve kesin değerler için yakında tablo veya metin açıklaması ister. Yalnızca “chart” eşdeğer değildir.

Cevap 4

Savunulabilir onarım: Campus Hub için `h1`, Events ve Communities için `h2`, Events altında Robotics evening için `h3`. CSS boyutu daha sonra değiştirir.

```
<h1>Campus Hub</h1>
<h2>Events</h2>
<h3>Robotics evening</h3>
<h2>Communities</h2>
```

Cevap 5

`div` yerel düğme rolüne, odaklanabilirliğe, klavye etkinleştirmesine, devre dışı durumuna ve form anlamına sahip değildir. En küçük onarım `<button type="button">Submit</button>` olur; davranış ayrı bağlanır. Form gönderiyorsa `type="submit"` kullanın.

Cevap 6

Caption tabloyu adlandırır. Room, Capacity, Building ve Availability sütun başlıklarını `thead` içinde `th scope="col"` ile yazın. Her gövde satırında oda adı `th scope="row"`, diğerleri `td` olur. Görsel hizalama ile aynı ilişkiler yazılıma açılır.

Cevap 7

Tarayıcı doğrulaması kullanıcının denetimindeki istemcide çalışır; saldırgan belgeyi değiştirebilir veya API'yi doğrudan çağırabilir. Tarayıcı anında geri bildirim sağlar. Sunucu her isteği güvenilmez saymalı, değeri normalize edip doğrulamalı ve tutarlı hata döndürmelidir. Veri tabanı `NOT NULL` ve uygun uzunluk kontrolü gibi kalıcı değişmezleri uygulamalıdır.

Cevap 8

`display:none` checkbox'ı görsel düzenden ve normalde erişilebilirlik ağacından kaldırır. Tıklanabilir `div` fare kullanıcısına görünebilir; fakat yerel rolü ve Tab davranışı yoktur. İkili seçim için görünür etiketli gerçek checkbox, eylem için gerçek button kullanın.

Cevap 9

Gönderim `displayName=Ada`, `studentId=CE-104` ve `intent=save` içerir. Etkin ve adlandırılmış displayName gönderilir; salt okunur studentId düzenlenemese de gönderilir. Devre dışı theme ve işaretsiz newsletter gönderilmez. Etkin submit düğmesi kendi ad-değer çiftini ekler. Bu kurallar veri oluşturmayı açıklar; sunucu yine her alanı yetkilendirip doğrulamalıdır.

Cevap 10

Hatasız tarama yalnızca aracın bildiği kuralları, incelediği sayfa durumunda bulamadığını gösterir. Anlamlı etiketleri, klavye sırasını, yeniden akışı, ekran okuyucu geri bildirimini veya kayıt görevinin tamamlanabilirliğini kanıtlamaz. Otomatik standart kontrolü, klavye testi, yakınlaştırma, erişilebilirlik ağacı/ekran okuyucu ve mümkünse temsili kullanıcı testini birleştirin.

Cevap 11

Tırnaksız class değeri `event`'te durur; `note` ayrı Boolean öznitelik olur. `p` ile `strong` üst üste biner. `required="false"` Boolean durumu yine açar. Bağlı görünür etiket ve `name` yoktur; placeholder kalıcı etiket değildir. & gerektiğinde `&` yazılmalıdır.

```
<p class="event note">Register <strong>before Friday</strong>.</p>
<label for="email">University email</label>
<input id="email" name="email" type="email" aria-describedby="help">
<p id="help">Use your university address &amp; include the department.</p>
```

Cevap 12

Etkinlik araması GET ve `type="search" name="q"` kullanır. Tek kullanımlık kod bir miktar değil tanımlayıcıdır: metin girişi, `inputmode="numeric"`, `autocomplete="one-time-code"`, altı rakam deseni ve maxlength 6. Koltuk sayısı `type="number"`, min 1, max 20, step 1 olabilir. PDF yüklemesi `POST` ve `multipart/form-data` ister; sunucu içeriği, türü, boyutu ve yetkiyi doğrular. Arama ile yüklemeyi tek formda birleştirmek yöntem ve kodlama anlamlarını bulanıklaştırır.

Cevap 13

“1 error - Submit an event - Campus Hub” gibi özel başlık kullanın. Formdan önce “There is 1 error” başlıklı özet ve `#capacity` bağlantısı koyun. `id="capacity"`, `aria-invalid="true"` ve `aria-describedby="capacity-error"` kullanın. Mesaj “Enter a whole number from 1 to 300.” diyebilir. 500 değerini düzeltme için koruyun; güvenlik duyarlı değerler farklı ele alınabilir. Yeni sunucu sayfasında başlık hatayı bildirmeli ve odak özete taşınmalıdır. JavaScript mevcut sayfaya eklerse canlı duyuru, odak yönetimi ve yinelenen kimliklerden kaçınma gerekir.

Cevap 14

Zaman çizelgesini adlandıran caption ekleyin. `thead` ilk satırda oda köşe hücresiyle birlikte `Monday` ve `Tuesday` için iki `th scope="colgroup" colspan="2"` kullanır; ikinci satırda Morning ve Afternoon `th scope="col"` olur. Gövdede oda adı `th scope="row"`, ardından dört veri hücresi gelir. Kullanıcılar günleri ayrı karşılaştırıyorsa veya dar görünümde ızgara anlaşılmazsa tabloyu bölün. `scope` yetersizse başlıklara benzersiz `id`, veri hücrelerine `headers` listesi verin.

Hata ayıklama kliniği

Öğrenci “Email metni alanın hemen üzerinde, demek ki etiketli” diyor. İnceleme `<p>Email</p><input type="email" name="email">` gösteriyor. Görsel yakınlık programatik ilişki değildir; denetimin erişilebilir adı yoktur. Paragrafı `<label for="email">Email</label>` ile değiştirin ve input'a `id="email"` ekleyin veya input'u label içine alın. Etiketi etkinleştirip odağı ve erişilebilirlik ağacını yeniden inceleyin; düzeltme yerine `aria-label` eklemeyin.

Geriye bakmadan cevaplayın

1. Anlamsal HTML ne iletir? 2. Başlık düzeyi neden varsayılan yazı boyutundan seçilmez? 3. `article` ne zaman `section`dan uygundur? 4. Bağlantı ile düğme arasındaki fark nedir? 5. Placeholder neden yeterli etiket değildir? 6. Form denetiminin `name` özniteliği ne yapar? 7. Sunucu tarayıcı doğrulamasını neden tekrarlamalıdır?

8. Erişilebilir adda hangi bilgi bulunur? 9. `alt=""` ne zaman doğrudur? 10. Erişilebilirlik ağacı ne içerir? 11. Otomatik testin iki sınırını yazın. 12. Pozitif tabindex neden kaçınılmalıdır? 13. Devre dışı ve salt okunur denetimler gönderimde nasıl ayrılır? 14. HTML formunda GET ne zaman POST'tan uygundur? 15. Dört WCAG ilkesini yazın ve birinin diğerlerini garanti etmediğini açıklayın. 16. Etiket ile öğe arasındaki fark nedir? 17. `required="false"` neden yine zorunlu yapar? 18. Öğrenci numarası için `type="number"` neden genellikle yanlıştır? 19. Dosya yüklemesinde hangi kodlama gerekir? 20. Sunucu tarafından oluşturulan erişilebilir hata yanıtının dört parçasını yazın.

Quiz cevapları ve yanılgı notları

1. Roller ve ilişkiler. 2. Düzeyler belge iç içeliğini, CSS ise görünüşü belirler. 3. Bağımsız anlam taşıyan haber veya etkinlik kartında article; çevre belgenin tematik bölümü için section. 4. Link kaynağa gider, button eylem yapar. 5. Placeholder kaybolur ve kalıcı ad değildir. 6. Gönderilen değerin anahtarını sağlar. 7. İstemci güven sınırı dışındadır ve atlanabilir. 8. Denetimin amacının kısa açıklaması; genellikle görünür etiket veya metinden gelir. 9. Dekoratif ya da yinelenen görseli erişilebilirlik deneyiminden çıkarmak için.

10. Roller, adlar, durumlar, değerler ve ilişkiler. 11. Otomatik araçlar alternatif metnin anlamını veya tüm iş akışının anlaşılabilirliğini güvenilir biçimde yargılayamaz. 12. Ayrı bir odak sırası oluşturur ve kaynak sırasından kopar. 13. Disabled gönderilmez; readonly metin değeri gönderilebilir; ikisi de sunucu güveni oluşturmaz. 14. Durum değiştirmeyen alma/filtreleme ve paylaşılabilir URL gerektiğinde. 15. Algılanabilir, işletilebilir, anlaşılabilir, sağlam; farklı hata türlerini kapsarlar. 16. Etiket sınır işaretlemesidir; öğe ayrıştırılmış nesnenin tamamıdır. 17. Boolean öznitelik mevcut olduğu için. 18. Kimlik miktar değildir; baştaki sıfırlar kaybolabilir. 19. Dosyada `multipart/form-data`. 20. Hata bildiren başlık, özet ve hedef bağlantıları, alan mesajı/ilişkisi ve uygun odak yönetimi.

17. `required` bir Boolean özniteliktir. Varlığı, “false” gibi görünen dizeden bağımsız olarak doğru demektir; kapatmak için özniteliği kaldırın. 18. Bu değerler aritmetik miktar değil tanımlayıcıdır. Sayısal input biçimi veya baştaki sıfırları reddedebilir ve miktar davranışı ima eder; inputmode ve alan doğrulaması olan metin benzeri denetim kullanın. 19. POST formunda `multipart/form-data`; seçici ve accept sunucunun baytları doğrulama sorumluluğunu kaldırmaz. 20. Hataya özel sayfa başlığı, bağlantılı hata özeti, alan mesajları ve geçersiz durum/odak yönetimi gerekir. Güvenli değerleri korumak ve düzeltmeyi açıklamak da önemlidir.

Kendini değerlendirme

17-20: Yapıyı koruyarak belgeyi biçimlendirmeye hazırsınız. 12-16: Sözdizimini, formları ve erişilebilirlik ağacını yeniden inceleyip klavye testini tekrarlayın. 0-11: Laboratuvar formunu yerel öğelerle yeniden kurun ve her seçimi bir arkadaşınıza veya öğretim asistanına açıklayın.

Oturum özeti

2. Oturumun çalışma kaynağı `labs/week-02/index.html` dosyasıdır; VS Code onu düzenler, yerel Node.js sunucusu sunar, tarayıcı görüntüler. Temel döngü düzenle, kaydet, yenile ve incele adımlarından oluşur. Kaydedilmemiş dosya veya farklı `index.html` sunulan sayfayı değiştiremez. HTML yapı, ilişki, denetim ve anlamı açıklar. Kurallara uygun sözdizimi etiketleri, öğeleri, öznitelikleri, Boolean varlığını, boş öğeleri, iç içeliği ve karakter başvurularını ayırır. Belge dili, başlık, kodlama ve viewport meta verileri tüm sayfa için anlam ve çalışma varsayımları sağlar.

Yerel öğeler özelleştirmeden önce denenmiş anlam ve davranış sağlar. Başlıklar hiyerarşi, landmark'lar gezinilebilir bölgeler oluşturur. Bağlantılar gezinir, düğmeler eylem yapar. Görsellerin alternatifleri mekanik tanımdan değil amaçtan seçilir. Veri tabloları caption ve açık başlık ilişkileri ister. Form etiketleri, tanımlayıcılar ve gönderilen adlar farklı sorumluluklara sahiptir. GET ve POST farklı niyetleri, başarılı denetim kuralları ise hangi alanların gönderileceğini ifade eder.

Sıradan metin alanları ve dosya baytları farklı form kodlamaları ister; multipart ipuçları sunucu doğrulamasının yerine geçmez. Tarayıcı doğrulaması deneyimi iyileştirir ama sunucu güveni kurmaz. Erişilebilir hata kurtarma, hataya özel başlık, özet, bağlantılı alan mesajları, korunan güvenli değerler, durum ve bilinçli odak davranışını birleştirir. Erişilebilirlik çalışması Algılanabilir, İşletilebilir, Anlaşılabilir ve Sağlam içeriği; otomatik ve insan kanıtını birlikte ele alır.

Sözlük

Tablo 2.7, eğitmenlerin geri çağırma sorularında yeniden kullanabileceği ve öğrencilerin laboratuvar kanıtlarını açıklarken başvurabileceği terimleri toplar.

Tablo 2.7 - 2. Oturum sözlüğü

Erişilebilirlik ağacı	Yardımcı teknolojilere açılan rol, ad, durum, değer ve ilişki gösterimi.
Erişilebilir ad	Yazılımın arayüz öğesini kullanıcıya tanıttığı metin.
Erişilebilir açıklama	Biçim ipucu veya kısıt gibi adı tamamlayan destek bilgisi.
ARIA	Yerel HTML'nin ifade edemediği örüntüler için erişilebilirlik anlamı ekleyen Accessible Rich Internet Applications özellikleri.
Boolean öznitelik	Varlığı doğru, yokluğu yanlış temsil eden; "false" dizesiyle kapanmayan öznitelik.
Karakter başvurusu	İşaretleme sanılabilecek karakteri temsil eden `&` veya `<` gibi HTML sözdizimi.
İçerik modeli	Bir öğenin hangi tür içeriği içerebileceğini belirleyen HTML kuralları.
Kısıt doğrulaması	required ve aralık gibi form kısıtlarını değerlendiren tarayıcı mekanizmaları.
Öğe	Anlamı, öznitelikleri, izinli içeriği ve belge ağacındaki ilişkileri olan ayrıştırılmış HTML parçası.
Form kodlaması	Başarılı denetimleri `application/x-www-form-urlencoded` veya `multipart/form-data` gibi serileştiren gösterim.
Odak	Klavye girdisinin geçerli hedefi.
Landmark	nav veya main gibi büyük gezinilebilir sayfa bölgesi.
Yerel anlam	Standart HTML öğesinin sağladığı anlam ve davranış.
Etiket	HTML öğesini başlatan veya bitiren kaynak işaretlemesi; öğenin tamamı değildir.
Anlamsal HTML	İçeriğin anlamı ve ilişkisine göre öğe seçimi.
Başarılı denetim	Form kuralları altında adı ve mevcut değeri gönderilen denetim.
Metin alternatifi	Metin olmayan içeriğin ilgili amacını veya bilgisini ileten metin.
Boş öğe	Çocuk içerik ve kapanış etiketi taşıyamayan `img` veya `input` gibi HTML öğesi.
WCAG	Algılanabilir, İşletilebilir, Anlaşılabilir ve Sağlam ilkeleri etrafında düzenlenen Web İçeriği Erişilebilirlik Kılavuzları.

Birincil başvurular ve ileri okuma

1. WHATWG, HTML Living Standard - Web developer edition. 2. W3C, Web Content Accessibility Guidelines 2.2. 3. W3C, WAI-ARIA Authoring Practices. 4. MDN, HTML: A good basis for accessibility. 5. MDN, Web forms. 6. MDN, Semantic HTML. 7. Manuel Matuzovic, Web Accessibility Cookbook. 8. MDN, Basic HTML syntax. 9. W3C Web Accessibility Initiative, Forms Tutorial. 10. Jennifer Niederst Robbins, Learning Web Design, 6. baskı. 11. Adam Silver, Form Design Patterns. 12. Heydon Pickering, Inclusive Components.

Neler daha sık değişir?

HTML'nin temel anlamları ve WCAG 2.2 gereksinimleri kararlıdır. Tarayıcı doğrulama mesajları, erişilebilirlik ağacı panelleri ve yardımcı teknoloji ayrıntıları platforma ve sürüme göre değişir. Teknik inceleme tarihi: 22 Eylül 2026.

Oturum yansıtması

- Formunuzda en çok özel çalışmayı hangi yerel HTML öğesi önledi? - Hangi klavye gözlemi tasarımınızı değiştirdi? - Hangi erişilebilirlik iddiası hâlâ temsili bir kullanıcıdan kanıt gerektiriyor?

Oturum 3 - Basamaklı Stil Sayfaları, yerleşim ve duyarlı tasarım

Temel soru: Basamaklı Stil Sayfaları (Cascading Style Sheets - CSS), tek bir anlamsal belgenin kırılganlaşmadan farklı ekranlara, tercihlere, dillere ve içerik miktarlarına uyum sağlamasına nasıl yardım eder?

Tarayıcı, Bölüm 2 belgesini yazar CSS'si olmadan görüntüleyebilir. Başlıklar büyük, paragraflar blok, bağlantılar tanınabilir ve form denetimleri platform kurallarını kullanır. Bu varsayılanlar bilinçli olarak mütevazıdır; önemli başlangıç noktası şudur: CSS belgenin anlamını oluşturmaz, anlamlı belgenin sunumuna katılır.

Bu bölüm stil sayfasındaki ilk karakterden başlayıp basamaklandırma, tipografi, renk, kutular, konumlandırma, Flexbox, Grid, medya sorguları, konteyner sorguları, erişilebilirlik tercihleri ve tarayıcı hata ayıklamasına ilerler. Canlı derste Tablo 3.1'de işaretlenen ana yol izlenmelidir; ek bölümler kitabı başvuru kaynağı yapar.

Oturum haritası

Tablo 3.1 önkoşulları, beklenen çalışmayı ve kanıtları özetler.

Tablo 3.1 - Oturum kapsamı, hazırlık ve kanıtlar

Önkoşullar	1-2. Oturumlar; anlamsal Campus Hub HTML
Ders süresi	Yaklaşık 3 saat atölye ve 2 saat laboratuvar
Bireysel çalışma	Referans bölümleri ve alıştırmalarla yaklaşık 5 saat
Ana ürün	Duyarlı, erişilebilir Campus Hub kabuğu
Değerlendirme	Üç görünüm alanı kontrolü, klavye kanıtı, taşma denetimi, alıştırmalar ve quiz

Önerilen öğretim yolu CSS sözdizimi ve yükleme, seçici eşleşmesi, cascade, kutu modeli, normal akış, Flexbox, Grid, duyarlı kısıtlar ve rehberli laboratuvardır. Tipografi, renk, konumlandırma, konteyner sorguları ve genişletilmiş hata ayıklama atölye öncesi veya sonrası verilebilir.

6. Okunabilir ve uyarlanabilir birimler, renkler, tipografi, arka planlar, kenarlıklar ve form stilleri seçin. 7. Flexible Box Layout (Flexbox) ile sağlam tek boyutlu, Grid ile iki boyutlu yerleşimler oluşturun. 8. Medya sorgularını ve bileşen düzeyindeki konteyner sorgularını cihaz adlarından değil içerik kısıtlarından türetin. 9. Odak görünürlüğünü, kontrastı, hedef boyutunu, kaynak sırasını, yakınlaştırmayı, yeniden akışı, zorunlu renkleri ve azaltılmış hareket tercihlerini koruyun. 10. Eksik bildirimleri, cascade çakışmalarını, taşmayı ve yerleşim hatalarını geliştirici araçlarıyla tanılayın. 11. CSS'yi sonraki değişiklikler yerel ve öngörülebilir kalacak biçimde düzenleyin.

1. CSS sunum ve kısıtlar için bir dildir

CSS sabit bir ekran görüntüsü çizmez. Bildirimsel bir dildir: yazar istenen ilişkileri açıklar, tarayıcı geçerli koşullar için sunumu belirler. Tarayıcı CSS kurallarını belge yapısı, yazı tipleri, kullanılabilir alan, kullanıcı tercihleri, yazım yönü, yakınlaştırma ve gerçek içerikle birleştirir. Yalnızca tek görünüm alanında ve tek paragrafta çalışan tasarım duyarlı değildir; tek test durumunun resmidir.

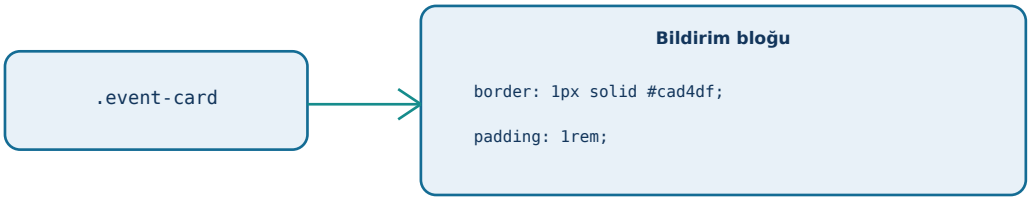
Profesyonel CSS birçok geçerli durumun yerleşebileceği kısıtlar kurar. CSS eklemeden önce Bölüm 2 sayfasını dar genişlikte, %200 yakınlaştırmada, uzun etkinlik başlığıyla ve klavye odağıyla test edin. Normal HTML akışı sayesinde zaten çalışanları kaydedin; CSS bu davranışı yok etmeden iyileştirmelidir. Bu bölümün kısa örnekleri birbirinden kopuk gösterimler değil, tek bir birikimli sayfa oluşturur.

1.1 CSS kuralını tam cümle gibi okuyun

`styles.css` dosyasına eklenen ilk kuralı düşünün:

```
.event-card { border: 1px solid #cad4df; padding: 1rem; }
```

`.event-card` seçicisi hangi öğelerin eşleşebileceğini belirler. Süslü parantezler bildirim bloğudur. Her bildirim bir özellik, iki nokta, değer ve genellikle noktalı virgülden oluşur. `padding: 1rem;` içinde `padding` özelliktir, `1rem` değerdir. İki nokta CSS sözdizimine aittir; adın veya değerin parçası değildir.



Şekil 3.1. CSS seçiciyi özellik ve değer bildirimleri içeren blokla ilişkilendirir.

```
.event-card { padding: 1rem; }

Seçici: .event-card
Bildirim bloğu: { ... }
Özellik: padding
Değer: 1rem
```

Şekil 3.1. Bir CSS kuralı, seçiciyi özellik-değer önerileri içeren bildirim bloğuna bağlar.

Özellik denetlenen yönü açıklar. Değer, özelliğin kabul ettiği dil bilgisini kullanır. `padding: teal` anlamlı değildir; `teal` geçerli bir renk olsa da padding uzunluk, yüzde veya ilgili anahtar sözcük bekler. Geliştirici araçları çoğu zaman bu değeri çözemediğinde bildirimi üstü çizili gösterir veya gizler.

Noktalı virgül bildirimleri ayırır. Son noktalı virgül teknik olarak isteğe bağlıdır; fakat sonraki eklemeleri daha güvenli kıldığı için koruyun. CSS yorumları `/*` ile başlar, `*/` ile biter. Bazı programlama dillerindeki `//` CSS yorumu değildir.

CSS birçok hatadan sonra devam edecek şekilde tasarlanmıştır. Tarayıcı bir bildirimi anlamazsa genellikle yalnızca onu yok sayar ve kuralın kalanını işler. Bu dayanıklılık aşamalı geliştirmeyi destekler; fakat yazım hatasını sessiz gösterebilir. `colr: navy` stil sayfasını durdurmaz, yalnızca color ayarlamaz.

1.2 Kurallar, at-rule'lar ve shorthand'ler farklı yapılardır

At-rule `@` ile başlar ve daha büyük bir koşulu veya işlemi yönetir. Bu bölümde `@layer`, `@media`, `@container` ve `@supports` örnekleridir. Bazıları noktalı virgülle, bazıları blokla biter; her birinin dil bilgisini ayrıca okuyun.

Shorthand özellik birkaç ilişkili longhand özelliği birlikte ayarlar. `border: 1px solid #cad4df` dört kenarın genişlik, stil ve rengini önerir. Shorthand kısadır; fakat atlanan longhand'leri başlangıç değerlerine sıfırlar. Bu sıfırlama amaçlanıyorsa kullanın. Hata ayıklarken Computed paneli sonuç longhand değerlerini gösterir.

1.3 CSS'yi HTML'ye bilinçli bağlayın

```
<link rel="stylesheet" href="styles.css">
```

`rel="stylesheet"` ilişkiyi, `href="styles.css"` ise HTML belgesinin adresine göre çözülen yolu belirtir. Tarayıcı CSS kaynağını ister, ayrıştırır ve uygun kuralları uygular. Network panelinde CSS medya türüyle başarılı istek görünmelidir. 404, yanlış dosya adı veya yanlış belgeye göre göreli yol, CSS doğru olsa bile sayfayı stilsiz bırakabilir.

CSS `style` öğesinde veya `style` özniteliğinde de bulunabilir. `style` öğesi kendi kendine yeten gösterimde yararlı olabilir; `style` özniteliği uygulamalarca üretilebilir fakat yaygın satır içi stil kararları çoğaltır, seçicileri atlar ve bakımı zorlaştırır. Proje, birden çok belgeye hizmet edebilen ve bağımsız önbelleğe alınabilen dış stil sayfası kullanır.

`@import` başka stil sayfası yükleyebilir; ancak ek seri keşif istekleri oluşturabilir ve bağımlılık sırasını gizleyebilir. Proje mimarisi özellikle gerektirmedikçe ana stil sayfalarında HTML `link` öğelerini tercih edin.

1.4 Tarayıcının zaten bir stil sayfası vardır

Tarayıcılar yazar stil sayfasından önce kullanıcı aracısı stil sayfası uygular. Stilsiz başlığın kalın görünmesi ve düğmenin platform denetimine benzemesi bundandır. Kullanıcılar kendi tercihlerini veya stillerini de sağlayabilir. Cascade bu kaynakları uzlaştırır; yazar CSS'si tek katılımcı değildir.

Reset, tasarımın yerine koymaya hazır olduğu varsayılanları kaldırmalıdır. Tüm kenar boşluklarını, odak göstergelerini, liste işaretlerini ve denetim görünüşünü silmek yararlı affordance'ları yok edebilir. Proje öngörülebilir kutu boyutlandırma, duyarlı medya ve birkaç bilinçli boşluk kararı kullanan küçük bir temel uygular; her öğeyi görsel olarak boşaltmaz.

2. Seçiciler eşleşir; cascade karar verir

Seçici “hangi öğeleri hedefliyor?” sorusunu yanıtlar. Belgeyi dolaşıp HTML'yi değiştirmez. Tarayıcı seçiciyi DOM'a uygular, eşleşen küme oluşturur ve her öğedeki bildirimleri değerlendirir.

```
<article class="event-card"><h3>...</h3></article>
.event-card       → article ile eşleşir
.event-card h3    → içteki h3 ile eşleşir
.event-card.featured → iki sınıflı tek öğe
```

Boşluk alt öğe ilişkisini belirtir; boşluksuz iki sınıf tek öğede birlikte eşleşir. Şekil 3.2 bu ayrımı görsel olarak gösterir.

2.1 Uzun seçicilerden önce seçici okuryazarlığı

Tablo 3.2, başlangıç düzeyindeki web programcısının okuyabilmesi gereken seçici ailelerini tanıtır.

Tablo 3.2 - CSS seçici aileleri ve ifade ettikleri ilişkiler

Aile	Örnek	Anlam
Tür	article	Her article öğesi
Sınıf	.event-card	Sınıf listesinde event-card olanlar
ID	#events	Belge ID'si events olan öğe
Öznitelik	input[required]	required özniteliği olan input
Öznitelik değeri	input[type="email"]	type değeri email olan input
Pseudo-class	a:hover, :focus-visible	Durum veya yapısal koşul
Pseudo-element	.event-card::before	Eşleşen öğeye bağlı üretilen kutu
Alt öğe	.event-card h3	Kart içindeki herhangi h3
Çocuk	.site-navigation > ul	Doğrudan çocuk ul
Sonraki kardeş	h2 + p	h2'den hemen sonraki p
Kardeş	h2 ~ p	h2'den sonraki kardeş p
Seçici listesi	h1, h2, h3	Her biri ayrı değerlendirilen birleşim

`.event-card` ile `h3` arasındaki boşluk alt öğe birleşimidir. Buna karşılık `.event-card.featured` boşluksuzdur ve iki sınıfa sahip tek öğeyi eşleştirir. Sınıflar genellikle en yeniden kullanılabilir stil kancalarıdır. ID'ler geçerli seçicilerdir; yüksek özgüllükleri ve belge tekilliği nedeniyle rutin bileşen stilinden çok parça hedefleri, etiketler ve betik kimlikleri için uygundur.

`:hover`, `:focus-visible`, `:checked`, `:disabled`, `:valid` ve `:invalid` gibi pseudo-class'lar durumları anlatır. Temel bilgiyi yalnızca hover ile göstermeyin; dokunma, klavye ve yardımcı giriş bunu üretmeyebilir. `::before` ve `::after` dekoratif kutular ekleyebilir; üretilen içerik temel metin için güvenilir yer değildir.

`:where()` özgüllüğe sıfır katkı yapar; düşük öncelikli varsayılanlarda yararlıdır. `:is()`, `:not()` ve `:has()` en özgül argümanın özgüllüğünü alır. `:has()` daha önce ifade edilemeyen ilişkilerde kullanılabilir; fakat belirsiz belge modelini zor bir seçicinin arkasına saklamayın.

2.2 Cascade çakışan bildirimleri çözer

Aynı öğede aynı özellik için birden çok bildirim değer önerdiğinde cascade çakışmayı çözer. “Son bildirim kazanır” kuralı eksiktir; önce uygunluk, kaynak ve önem, katman, özgüllük, scope yakınlığı, sonra sıra değerlendirilir.

```
Uygun kaynak + katman
        ↓
Özgüllük → Scope yakınlığı → Sıra → Kazanan
```

Şekil 3.3. Cascade, ilgili bildirimleri sıralı bir karar yolundan geçirir.

Başlangıç düzeyinde şu sırayla sorun: 1) Bildirim bu öğe, belge, koşul ve ortam için uygun mu? 2) Hangi kaynak ve önem düzeyi öncelikli? 3) Aynı düzeyde hangi cascade katmanı öncelikli? 4) Hangi seçicinin özgüllüğü daha büyük? 5) `@scope` ve özgüllük eşitse hangi scope kökü daha yakın? 6) Öncekiler eşitse stil sırasındaki hangi bildirim daha sonra geliyor? Geçişler ve animasyonlar özel konumlara sahiptir.

Normal yazar bildirimlerinde katmansız kurallar katman içindeki normal kuralları aşar; normal katmanlarda daha sonra tanımlanan katman daha önceliklidir. `!important` katman sırasını tersine çevirir; önemli kullanıcı tercihleri önemli yazar bildirimlerini aşar. Keyframe animasyonları normal bildirimleri aşar, geçiş hâlindeki özellik geçiş süresince en yüksek önceliğe sahiptir. Satır içi stil yazar stilidir ve özel özgüllük davranışı taşır; mimarinin rutini yapmayın.

Kaynak sırası yalnızca tüm önceki boyutlar eşitse kazanır. Tabloyu `!important` kullanma lisansı olarak ezberlemeyin; önce Styles panelinde hangi bildirimin neden kazandığını inceleyin. Kaybeden bildirimler genellikle üstü çizili görünür ve dosya ile satır numarasını gösterir.

2.3 Özgüllük artırılacak puan değildir

Özgüllük üç kategoriyle karşılaştırılır: ID; sınıf, öznitelik ve pseudo-class; tür ve pseudo-element. Önce ID kategorisi, eşitse sınıf-benzeri, o da eşitse tür kategorisi karşılaştırılır. `1-0-0` ve `0-12-0` ondalık sayı değildir; bir ID ilk karşılaştırmayı kazanır.

```
/* 0-1-0: yeniden kullanılabilir kanca */
.event-card__title { color: var(--color-heading); }
/* 1-2-3: tek konuma bağlanmış */
#main-content section.events article.event-card h3 { color: navy; }
```

Bileşen stilinde sığ sınıf seçicilerini tercih edin. Kaynak sırasını ve açık cascade katmanlarını bilinçli kullanın. `!important`'ı rutin kaçış olarak kullanmayın; çakışma düzeyini değiştirir ve büyüyen override yarışına yol açar. Üçüncü taraf CSS'yi her seçiciyle savaşmak yerine bilinçli düşük öncelikli katmana koyun.

2.4 Kalıtım seçici eşleşmesi değildir

`color` ve `font-family` gibi özellikler normalde kalıtılır; `margin`, `border` ve `width` normalde kalıtılmaz. Çocukta doğrudan hedeflenen bildirim, ata seçicisi daha özgül olsa bile kalıtılmış değeri kazanır; çünkü doğrudan ve kalıtılmış bildirimler arasında özgüllük karşılaştırılmaz.

```
body { color: #172033; font-family: system-ui, sans-serif; }
```

CSS-genişlikteki anahtar sözcükler kararı açıklaştırır: `inherit` normalde kalıtılmayan özellikte bile ebeveynin hesaplanmış değerini alır; `initial` belirtimin başlangıç değerini kullanır; `unset` kalıtılanlarda inherit, diğerlerinde initial gibi davranır; `revert` mevcut kaynağın katkısını geri alır; `revert-layer` geçerli cascade katmanının katkısını geri alır. `all: unset`, bileşenin ihtiyaç duyduğu display veya etkileşim kolaylıklarını kaldırabilir.

2.5 Özel özellikler kararları cascade boyunca taşır

```
:root {
  --color-ink: #172033;
  --color-accent: #146c94;
  --space-2: 0.5rem;
  --space-4: 1rem;
  --radius: 0.5rem;
}
```

Özel özellikler tekrar eden renk, boşluk ve yarıçap kararlarını tek yerde tutar. Bileşenler `var(--color-accent)` gibi değerleri kullanarak anlamlı bir isim üzerinden değişir; daha sonraki sayfalarda fallbacks, kapsam ve hesaplama ayrıntılarını inceleyeceğiz.

```
:root { --color-ink: #172033; --radius: 0.5rem; }
.event-card { color: var(--color-ink); border-radius: var(--radius); }
```

Özel özellik adları iki tireyle başlar, büyük-küçük harfe duyarlıdır, cascade'e katılır ve varsayılan olarak kalıtılır. `var()` işlevi, özellik dil bilgisinin izin verdiği yerde değerini yerleştirir. `var(--card-surface, white)` gibi fallback, başvurulan özel özellik eksik veya yerleştirme sırasında geçersiz olduğunda kullanılır; desteklenmeyen rastgele değer için uyumluluk yedeği değildir. `--color-accent`, `--blue`'dan daha iyi amaç anlatır.

Token'lar kararları merkezileştirir fakat kendiliğinden tutarlı tasarım sistemi oluşturmaz; amaçlarını belgeleyin ve kullanıldıkları her kombinasyonu test edin.

3. Kutular bildirilen değerleri geometriye dönüştürür

Görsel yerleşim soruları çoğunlukla kutu sorusudur: bu hangi kutu, ne kadar büyüyebilir, hangi kapsayıcı blok referans sağlıyor ve normal akışta kalıyor mu?

3.1 `display` dış ve iç davranışı kurar

Bir öğenin `display` değeri ana kutusunun ebeveyn yerleşimine katılımını ve yerleşim kapsayıcısıysa çocuklarının nasıl düzenlendiğini belirler. `block` kullanılabilir satır içi alanı alıp yeni satır başlatır; `inline` satır yerleşimine katılır; `inline-block` satır içinde yer alırken içi blok kapsayıcı gibi davranır; `flex` ve `grid` çocukları için blok düzeyi yerleşim kapsayıcısı oluşturur; `none` üretilen kutuları ve normalde erişilebilirlik ağacındaki içeriği kaldırır.

HTML anlamı ile CSS `display` yalnızca tarayıcı varsayılanları üzerinden ilişkilidir. `nav` Grid ile, `span` block olarak düzenlenebilir; bu, öğelerin anlamını değiştirmez.

3.2 İçerik, padding, border ve margin kutu modelini oluşturur

Standart kutu modeli content, padding, border ve margin bölgelerinden oluşur. Şekil 3.4 bu dört bölgeyi sınırlarını ayıracak biçimde iç içe gösterir.

```
margin
border
padding
content

box-sizing: border-box → content + padding + border
margin dışarıda kalır
```

Şekil 3.4. Padding, border ve margin öğenin content kutusunu çevreler.

Varsayılan `box-sizing: content-box` ile bildirilen width veya inline-size content kutusuna uygulanır; padding ve border dışarı eklenir. Yaygın temel şudur:

```
*, *::before, *::after { box-sizing: border-box; }
```

Böylece bildirilen boyut content, padding ve border'ı içerir; margin dışarıda kalır. Bu her kutunun sığacağı anlamına gelmez: 20rem sabit border kutusu 17rem kapsayıcıya sığmaz; bölünemeyen dize de daha büyük minimum dayatabilir.

Padding border'ın içindedir ve arka planı devralır. Margin border'ın dışındaki saydam boşluktur. Border boyuta katkı verir ve sınır gösterebilir; yalnızca soluk border'a dayanan fark algılanamayabilir. `outline` kutu modeli alanı kaplamaz; yerleşimi kaydırmadan odak göstergesi için yararlıdır.

3.3 Normal akış dayanıklı varsayılandır

Blok içerik normalde belge sırasını izler ve içerik miktarını karşılamak için blok yönünde genişler. Satır içi içerik satır yerleşimine katılır. Normal akış bilinmeyen metin uzunluğu, yazı tipi değişimi, çeviri ve yakınlaştırma durumlarının çoğuyla iyi çalışır. Öğeyi akıştan çıkarmak komşular için olağan alan ayırmayı durdurur; sabit koordinatlı absolute kart metin büyüdüğünde üst üste binebilir. Akışta kalan kart çevre yerleşimine boyutunu bildirir. Önce normal akışla başlayın, gerçek ilişki gerektiriyorsa başka modele geçin.

3.4 Boyutlandırma intrinsic içerikle başlar

Intrinsic boyut içeriğin kendisinden gelir. Bölünemeyen sözcük minimum katkı, görsel intrinsic boyutlar, paragraf ise satırların sarılmayacağı tercih genişliği taşır. `min-inline-size`, `max-inline-size`, `min-block-size` ve `max-block-size` tek boyut zorlamadan sınırlar ifade eder.

```
.page-shell {
  inline-size: min(100% - 2rem, 72rem);
  margin-inline: auto;
}
```

`min-content`, `max-content` ve `fit-content()` yararlı intrinsic boyut fikirleridir; küçük, büyük ve duyarlı sözcüklerinin eş anlamlısı değildir. Gerçek içerikle test edin. Flex ve Grid öğelerinde otomatik minimum boyut şaşırtabilir; güvenliyse `min-inline-size: 0` öğenin min-content katkısının altına küçülmesine izin verir.

Görsel ve video gibi replaced element'ler metin düzeninin dışından intrinsic boyut ve en-boy oranı getirir. HTML width ve height öznitelikleri kaynak yüklenmeden doğru oran için alan ayırabilir. CSS çizilen kutuyu kısıtlar. Oran tasarımın parçasıysa `aspect-ratio`; içerik bilinçli boyutlu kutuya sığarken `object-fit: cover` veya `contain` kullanılır. `cover` kırpması kullanıcının ihtiyaç duyduğu bilgiyi silmemelidir.

3.5 Mantıksal özellikler yazım sistemini izler

left, right, top ve bottom fiziksel yönlerdir. CSS yazım kipine göre mantıksal yönler de tanımlar. Inline eksen metin çizgisini, block eksen satır ve blokların biriktiği yönü izler. İngilizcede inline genellikle yatay ve soldan başlar; Arapça veya Farsçada sağdan başlayabilir. Bazı yazım sistemleri dikey akış kullanır.

Şekil 3.5, `margin-inline`, `padding-block`, `inline-size` ve `inset-inline-start` özelliklerinin left, right, width ve top varsayımlarından neden daha uyarlanabilir olduğunu gösterir.

```
.event-card {
  padding-block: 1.5rem;
  padding-inline: 1rem;
  border-inline-start: 0.25rem solid var(--color-accent);
}
```

Bu border yön değiştiğinde inline başlangıç kenarını izler. Mantıksal özellikler içeriği çevirmez veya hatalı HTML `dir` değerini düzeltmez; mevcut yön bilgisini yerleşime saygılı kılar.

3.6 Margin birleşmesi ve overflow kanıt sağlar

Normal akışta bitişik blok ekseni margin'leri tanımlı koşullarda birleşebilir; sonuç toplam değil büyük margin olabilir. Padding, border, yeni biçimlendirme bağlamı veya yerleşim kapsayıcısı koşulları değiştirir. Boşluk kayboluyorsa daha büyük margin eklemek yerine kutuları inceleyin.

Overflow çoğu zaman yanlış kısıtı gösterir: kullanılabilir inline boyuttan geniş sabit boyut; uzun URL gibi bölünemeyen içerik; küçülmeyi engelleyen Flex/Grid otomatik minimumu; duyarlı maksimumu olmayan görsel; padding veya kaydırma çubuğu olan sayfada 100vw boyutlu çocuk; ya da kaydırılabilir alan dışına taşan dönüştürülmüş/konumlandırılmış nesne.

Overflow davranışını yerel seçin. Geniş veri tablosu gibi iki boyutlu kaydırmanın gerçekten gerektiği yerde `overflow: auto` kaydırma kapsayıcısı oluşturabilir. `overflow: clip` veya `hidden` bilinçli görsel efektlerde doğru olabilir; fakat odak veya temel içerik erişilemez olmamalıdır. Sayfa genelindeki belirtiyi `overflow-x: hidden` ile saklamayın; kapsayıcısını aşan kutuyu bulup nedeni düzeltin.

3.7 Konumlandırılmış yerleşim referansı ve akışı değiştirir

Konumlandırma nesne kutuya veya görünüm alanına sabitlenmek zorunda olduğunda yararlıdır; her kipin referansı farklıdır: `static` normaldir; `relative` akış alanını korur ve görsel ofset verir; `absolute` akıştan çıkar ve uygun kapsayıcı bloğa göre yerleşir; `fixed` genellikle görünüm alanına göre yerleşir; `sticky` kaydırma eşiğine kadar akışta kalır.

Sticky veya fixed başlıklar başlıkları, skip-link hedeflerini veya klavye odağını örtebilir. Yüksek yakınlaştırmada test edin; bağlı içerik için gerektiğinde `scroll-margin-block-start` kullanın. `z-index` küresel sayı çizgisi değil stacking context içinde çalışır. Positioning, opacity, transform ve isolation ayrı bağlamlar oluşturabilir. `z-index: 9999` etkisizse daha büyük sayı seçmek yerine üst bağlamları inceleyin. Transform da görsel geometriyi değiştirir, normal akış alanı ayırmaz; yerleşimin yerine geçmez.

4. Değerler türü, ölçeği, rengi ve görsel hiyerarşiyi ifade eder

CSS değerleri tiplidir. Bir özellik anahtar sözcüğü, sayı, uzunluk, yüzde, renk, görsel, konum, işlev veya dil bilgisinde tanımlanan bileşim kabul edebilir. Tarayıcı değeri birkaç aşamadan geçirir: bildirim specified değeri verir; cascade ve kalıtım computed değere katkı yapar; yerleşim used değeri çözer; çizim sonunda cihaz piksellerine yuvarlayabilir. Bu nedenle Computed paneli stil sayfasında yazılandan farklı görünebilir.

4.1 Birimler ilişkileri iletir

Tablo 3.4 birimleri birbirinin yerine geçen sayılar olarak değil, ifade ettikleri ilişkiye göre karşılaştırır.

Tablo 3.4 - CSS birimleri ve duyarlı boyutlandırma işlevleri

Birim/işlev	İlişki ve kullanım
px	CSS referans pikseli; ince border ve hassas eşikler için
rem	Kök em; html'nin computed font-size'ının katı
em	Yerel font ilişkili uzunluk; font-size bildiriminde ebeveyn boyutuna göre
%	Özelliğe özgü referansa göre; her zaman ebeveyn genişliği değildir
vw, vh	Görünüm alanı genişlik/yüksekliğinin yüzde biri
svh, lvh, dvh	Küçük, büyük ve dinamik görünüm yüksekliği
ch	0 glifinin ilerleme ölçüsü; satır uzunluğu yaklaşımı
fr	Grid'de esnek olmayan izlerden sonra kalan alan payı
min(), max()	Kullanılan değerde karşılaştırılabilir sınır seçimi
clamp()	Tercih edilen değeri alt-üst sınırlar arasında tutma

rem ne demektir?

`rem`, root em'in kısaltmasıdır. HTML belgesinde kök öğe `html`'dir. `rem` ile yazılan uzunluk, html öğesinin computed font-size değerini alır ve önündeki sayıyla çarpar. Kök boyut R ise `x rem = x × R` hesabı yapılır.

Birçok tarayıcı değiştirilmemiş ayarlarda başlangıç kök boyutunu 16px hesaplar; bu sınıf içi örnektir, CSS garantisi değildir. Kullanıcı daha büyük metin seçebilir, işletim sistemi farklı ayar kullanabilir veya yazar kökü değiştirebilir. Her zaman ezberlenen 16px yerine computed kök boyutundan akıl yürütün.

Kök 16px ise: 0.5rem = 8px, 1rem = 16px, 1.5rem = 24px, 2rem = 32px. Kök 20px olursa aynı bildirimler 10px, 20px, 30px ve 40px olur. Stil sayfası değişmez; ortak referans değişir.

```
rem: tek belge referansı → html 16px → 0.5rem=8px, 1rem=16px, 2rem=32px
em: bileşen zinciri → parent 16px → card 1.25em=20px → padding 1em=20px → tag 0.8em=16px
```

Şekil 3.6. Her rem uzunluğu tek kök referansı kullanır; em bileşenin yerel font boyutu zincirini izleyebilir.

Şekil 3.6'daki temel ilişki 16px sayısı değil, bütün rem uzunluklarının aynı köke dönmesidir. Bu, iç içe bileşenler arasında boşluk ve tipografi ölçeğini düşünmeyi kolaylaştırır.

rem bildirimini hesaplama olarak okuyun: `body { font-size: 1rem; } .event-card { padding: 1rem; gap: 0.75rem; } .event-card__title { font-size: 1.5rem; }`. Kök 16px ise gövde 16px, padding 16px, gap 12px ve başlık 24px olur. Başlığın karta iç içe olması hesabı değiştirmez. Styles paneli 1.5rem'i, Computed paneli 24px'i gösterebilir; ilki ilişkiyi, ikincisi tarayıcının şu an kullandığı kanıtı anlatır.

rem ve em farklı soruları yanıtlar. rem “belgenin kök metnine göre ne kadar?” der; em çoğunlukla “bu bileşenin metnine göre ne kadar?” der. `font-size` bildirimindeki em ebeveynden kalıtılan boyuta göre çözülür; padding, gap veya border-radius gibi diğer özelliklerde öğenin computed font-size'ı kullanılır.

Başlangıç politikası: ana tipografi, sayfa/bileşen boşluğu ve kullanıcının kök ayarını izleyen boyutlarda rem; ikon, rozet veya düğme iç boşluğu gibi bileşen metniyle bilinçli ölçeklenen ayrıntılarda em; ince border gibi CSS referans pikselinin uygun olduğu yerde px; ilişkisi farklı olan yüzde, viewport birimi, container birimi, ch, Grid fr veya sizing işlevleri.

`html { font-size: 62.5%; }` kısayolu başlangıç boyutu 16px olduğunda 10px üretir; fakat kullanıcı ayarı farklı olabilir ve normal metni küçültebilir. Bu ders zihinsel aritmetik için kökü değiştirmez. 200% metin büyütmede, tarayıcı yakınlaştırmasında ve dar yeniden akışta gerçek sayfayı test edin. Amaç rem kullanmak değil, içeriğin ve işlevin erişilebilir kalmasıdır.

```
.event-card { font-size: 1.25em; padding: 1em; }
.event-card__tag { font-size: 0.8em; }
```

Kartın ebeveyni computed font-size 16px ise kartın 1.25em değeri 20px olur. Padding, kartın computed boyutunu kullandığı için 1em = 20px'tir. Etiket karttan 20px devralır; 0.8em = 16px olur. Bu yerel ölçekleme bağımsız rozet, ikon veya düğmede bilinçli olabilir; iç içe bileşenlerde boyutları fark etmeden katlayabilir. İstenen referans belge ölçeğiyse rem kullanmak bu bileşik sürprizi önler.

Başlangıç politikası: ana tipografi, sayfa/bileşen boşluğu ve kök ayarını izleyen boyutlarda rem; kendi bileşen metniyle bilinçli büyüyen ayrıntılarda em; ince border gibi CSS referans pikselinin uygun olduğu yerde px; ilişkisi farklıysa yüzde, viewport/container birimleri, ch, Grid fr veya sizing işlevleri.

Bu politika birimi evrensel olarak üstün yapmaz. Tarayıcı yakınlaştırması px ile yazılmış içeriği de büyütür. rem'in yararı kök yazı ayarına açık bağdır; erişilebilirlik yine gerçek oluşturulmuş sayfada test ister.

Kullanıcının başlangıç noktasını koruyun

```
html { font-size: 62.5%; }
```

Tarayıcının başlangıç boyutu 16px olduğunda 62.5% 10px üretir; fakat bu garanti değildir ve sonraki kurallar büyütmezse normal metni küçültür. Bu ders ondalık aritmetiği kolaylaştırmak için kökü değiştirmez; 62.5% zorunlu şablon değildir. Göreli birimler katı ilişkileri azaltır, fakat kırpılmış metni, sabit yükseklikli denetimleri veya büyütmede bozulan yerleşimleri önlemez.

Sayfayı %200 metin büyütmede, tarayıcı yakınlaştırmasında ve dar yeniden akış genişliklerinde doğrulayın. Amaç rem'in bulunması değil, içerik ve işlevin erişilebilir kalmasıdır.

Kontrol noktası - geliştirici araçlarını açmadan hesaplayın

Kök computed font-size 18px'tir. Kart padding'i 1.25rem, başlık font-size'ı 1.5rem'dir. Kart, computed boyutu 20px olan ebeveynde 1.1em font-size kullanır; yerel rozet padding-inline: 0.5em'dir. Hangi değerleri öngörürsünüz?

Cevap: padding 1.25 × 18 = 22.5px; başlık 1.5 × 18 = 27px. Kartın kendi font-size'ı 1.1 × 20 = 22px'tir. Rozet bu 22px'i devralıyorsa 0.5 × 22 = 11px'tir. Yaygın hata her şeyi 18px'ten hesaplamaktır: yalnızca rem kökü kullanır; em yerel font-size zincirini izler.

Fiziksel cm, mm, in ve pt birimleri ekranlarda CSS referans birimlerine bağlanır; fiziksel cetvel doğruluğu vaat etmez. Gerçek fiziksel tasarım gerektiren baskı çıktısında uygundur. Sıradan paragraf metnini yalnızca viewport birimleriyle ayarlamayın; uç görünüm boyutları metni okunamayacak kadar küçültebilir veya büyütebilir. Sınırlandırılmış akışkan boyut daha güvenlidir:

```
h1 { font-size: clamp(2rem, 1.4rem + 3vw, 4rem); }
```

Minimum ve maksimum uçları korur, orta terim kademeli değişim sağlar. Formülü erişilebilir varsaymadan yakınlaştırmayı ve kullanıcı yazı ayarlarını test edin.

4.2 Renk gösterimi renk kullanılabilirliğinden ayrıdır

CSS adlandırılmış renkleri, hex, rgb(), hsl() ve yeni renk işlevlerini kabul eder. Campus Hub kısa ve tanıdık olduğu için hex kullanır; gösterim kaliteyi belirlemez. `#146c94` ve eşdeğer rgb() aynı sRGB rengidir. Alpha saydamlığı kontrol eder; yarı saydam ön planın son kontrastı arkasındaki tüm katmana bağlıdır. `currentColor` öğenin computed rengini yeniden kullanır; sistem renk anahtar sözcükleri forced-color modlarıyla iş birliği yapabilir.

Tablo 3.5 görsel kararlarla erişilebilirlik sonuçlarını bağlar.

Tablo 3.5 - Renk, kontrast ve renk dışı kanıt

Karar	Gerekçe
Sıradan metin kontrastı	WCAG 2.2 AA normalde gerçek arka plana karşı en az 4.5:1 ister.
Büyük metin kontrastı	Uygun büyük metin normalde en az 3:1 ister; düşük eşik için metni büyütmeyin.
Arayüz/grafik sınırı	Anlamlı görsel bilgi ilgili ölçütte komşu renklere karşı normalde en az 3:1 olmalıdır.
Hata, durum, seçim	Renk durumu güçlendirebilir ama tek ipucu olamaz; metin, şekil veya simge ekleyin.
Odak göstergesi	Bileşen ve çevre yüzeyinde görünür kalmalı; gerçek renk ve durumlarla test edilmelidir.

Kontrast oranı estetik yargı değil ölçümdür. Uyumlu çift küçük boyutta yorucu olabilir; opacity veya arka plan görseli değişince güçlü görünen çift başarısız olabilir. Oluşturulmuş birleşimi ölçün ve yüksek kontrast/forced-color ayarlarını test edin.

4.3 Tipografi yalnızca süs değil, okumayı denetler

```
body { font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
       font-size: 1rem; line-height: 1.6; }
```

Font stack sıralı alternatifler sağlar; tarayıcı metni temsil edebilen ilk yüzü seçer. Font bulunabilirliği, glif kapsamı, ağırlık sentezi ve ölçüler satır kırılmasını ve bileşen yüksekliğini değiştirebilir. Birden çok platformda ve ders dillerinde test edin. Alt öğelerin kendi boyutlarıyla oranı izlemesi için sıradan metinde birimsiz line-height kullanın. Çok sıkı satırlar çakışabilir, çok geniş satırlar ilişkili metni ayırabilir; gerçek yazı tipi, dil, genişlik ve kullanıcı ayarlarıyla test edilen başlangıç aralığı çoğu zaman 1.4-1.7'dir.

Okunabilir paragraf için ölçüyü de kontrol edin. Satır başına yaklaşık 45-75 karakter başlangıç aralığıdır, uygunluk kuralı değildir. Dar dijital sütunlarda düzensiz boşluklar üretiyorsa iki yana yaslamadan kaçının. Link alt çizgilerini, güçlü ve renk dışı başka affordance yoksa koruyun. Web fontları kimliği güçlendirir fakat ağ isteği ve geç metrik değişimi ekler; gereken ağırlıkları kullanın, fallback seçin, performans ve yerleşim hareketini değerlendirin.

4.4 Arka plan, border ve gölge içeriği desteklemelidir

```
.event-card {
  background: white;
  border: 1px solid var(--color-border);
  border-radius: var(--radius);
  box-shadow: 0 0.25rem 1rem rgb(18 53 91 / 8%);
}
```

Arka plan görselleri konumlandırılabilir ve tekrar edilebilir; gradient'ler üretilmiş görsellerdir. Sunumdur: temel içerik ve alternatif metin HTML'de kalır. Border güvenilir sınır sağlar, hafif gölge derinlik ekler fakat tek ayrım değildir. Büyük bulanık gölgeler boyama maliyetini artırabilir ve metin kontrastını düşürebilir.

4.5 Form denetimleri platform davranışını korur

Form denetimleri tipografiyi aynı biçimde kalıtmayabilir; iç çizimleri işletim sistemi ve tarayıcıya göre değişir. Her yerel özelliği kaldırmak yerine küçük, test edilebilir temel ile başlayın:

```
button, input, select, textarea { font: inherit; }
button, select { min-block-size: 2.75rem; padding-block: 0.6rem; padding-inline: 0.9rem; }
```

Minimum block boyutu test edilen font ayarlarında rahat hedef oluşturur; tek CSS bildirimi WCAG hedef boyutu uygunluğunu kanıtlamaz. Oluşturulmuş hedefi ve çevresini ölçün. Tanınabilir durumları, klavye davranışını, etiketleri ve yerel anlamı koruyun. `appearance: none` yazara daha fazla sorumluluk aktarır; gerekli affordance'lar yeniden kurulup test edilmeden kullanmayın.

5. İlişkiden bir yerleşim modeli seçin

Normal akış, Flexbox ve Grid birlikte çalışan araçlardır; yenisi öncekinin yerini alan nesiller değildir. Önce içerik ilişkisini tanımlayın, sonra onu ifade eden en küçük modeli seçin. Tablo 3.6 bu seçimi özetler.

Tablo 3.6 - CSS yerleşim modelini seçme

İlişki	Başlangıç modeli	Neden
Okuma sırasındaki başlık/paragraf	Normal akış	İçerik açık izler ve özel track olmadan büyür.
Gezinme, toolbar, düğme grubu	Flexbox	Tek ana eksen dağıtımı ve hizalamayı yönetir.
Kart galerisi, sayfa bölgeleri, çizelge	Grid	Satır ve sütunlar koordineli track'ler ister.
Çevre metinli görsel	Float, dar kullanım	Öğe çevresinde metin sarma için; tüm sayfa yerleşimi için değil.
Gerçek tablosal veri	Anlamsal HTML table	İlişkiler tablo anlamı gerektirir.

Şekil 3.7 Flexbox'ın ana eksen, Grid'in satır ve sütun koordinasyonunu karşılaştırır.

```
Flexbox → tek ana eksen
Grid → koordineli satırlar ve sütunlar
```

Şekil 3.7. Flexbox bir ana ekseni; Grid satırları ve sütunları koordine eder.

5.1 Flexbox öğeleri ana eksende dağıtır

`display: flex` flex kapsayıcısı oluşturur; doğrudan çocuklar flex öğesi olur. Ana eksen `flex-direction`'ı, çapraz eksen ona dik yönü izler. Varsayılan yatay yazımda row inline ekseni izler; yön ve yazım kipinin etkisi nedeniyle row'u “soldan sağa” diye ezberlemeyin.

```
.site-navigation ul { display: flex; flex-wrap: wrap;
  align-items: center; gap: 0.5rem 1rem; }
```

Wrap bir dayanıklılık kararıdır. Çevrilmiş etiketler, kullanıcı font ayarları veya yakınlaştırma olmadan yatay taşmaya neden olabilir. `justify-content` ana eksende boş alanı dağıtır; `align-items` çapraz eksende hizalar. Bu özellikler kutuları hizalar, denetim iç padding'inin yerine geçmez.

Her flex öğesi grow, shrink ve basis bileşenlerine sahiptir. `flex: 1 1 16rem`, büyüyebileceği, küçülebileceği ve 16rem tabanından esneyeceği anlamına gelir. Açılımını anlamadan `flex: 1` kullanmayın. Uzun sözcük veya intrinsically geniş çocuk otomatik minimum nedeniyle küçülmeye direnebilir; doğru içerik politikası buysa sarma veya `min-inline-size: 0` kullanın. Auto margin belirli ilişkide ana eksen alanını emebilir; boş spacer öğesinden daha açıktır. `gap` kapsayıcıya aittir ve ilk/son çocuk margin kuralları olmadan tutarlı ayrım sağlar.

5.2 Grid iki boyutta track'leri koordine eder

```
.event-grid { display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 17rem), 1fr));
  gap: 1rem;
}
```

İç `min(100%, 17rem)`, kapsayıcı 17rem'den darsa track'in küçülmesine izin verir. `minmax()` alt ve üst sınırı belirler. `auto-fit` sığan tekrar sayısını oluşturur ve boş track'leri çöker. `1fr`, sabit ve intrinsic katkılar hesaba katıldıktan sonra kalan alanı paylaşır. Bu kural cihaz adı yazmadan farklı sütun düzenleri üretir.

Açıkça bildirilen track'ler explicit grid'dir. Daha fazla öğe olduğunda auto-placement implicit grid'de satır veya sütun oluşturabilir; bilinçli politika gerekiyorsa `grid-auto-rows` veya `grid-auto-columns` kullanın. Öğeler satır numarası veya adlandırılmış alanla yerleştirilebilir; öğe sayısı değişen koleksiyonlarda auto-placement çoğu zaman daha dayanıklıdır. `subgrid`, iç grid'in seçili eksende ebeveyn track boyutunu kullanmasını sağlar; gerçek ortak track ilişkisini çözüyorsa kullanın.

5.3 Hizalama sıralama değildir

Flexbox ve Grid `order`, açık grid çizgileri veya adlandırılmış alanlarla görsel konumu değiştirebilir. Önemli sıra deneyimlerinde DOM sırası okuma ve klavye odağını yönetir. Yanlış HTML sırasını görsel yeniden sıralamayla düzeltmeyin; önce tutarlı kaynak sırası seçin, sonra sorumlu yerleştirin ve doğrusal sayfayı test edin.

`justify-*` ve `align-*` farklı eksenlerde çalışır; etki yerleşim yöntemine göre değişir. Bir özellik etkisiz görünürse o eksende boş alan olup olmadığını ve özelliğin kapsayıcıya mı öğeye mi uygulandığını sorun.

6. Duyarlı tasarım bağlamı girdi olarak ele alır

Duyarlı tasarım cihaz boyutları listesi değildir. Sayfa yeniden boyutlandırılan pencerede, bölünmüş ekranda, gömülü panelde, tarayıcı yakınlaştırmasında, farklı font ayarlarında, dikey/yatay yönde ve stil yazılırken var olmayan cihazlarda çalışmalıdır. Esnek içerik ve dar kısıtlarla başlayın; gerçek ilişkiye yer açıldığında geliştirin.

6.1 Görünüm alanı davranışı HTML'de başlar

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Bu bildirim mobil tarayıcıdan başlangıç ölçeğinde yerleşim görünüm alanını aygıtın CSS piksel genişliğiyle eşleştirmesini ister. Sabit yerleşimi kendiliğinden duyarlı yapmaz. `user-scalable=no` veya kısıtlayıcı maximum-scale ile kullanıcı yakınlaştırmasını kapatmayın; yakınlaştırma temel kullanıcı yeteneğidir.

6.2 Medya sorguları ortama yanıt verir

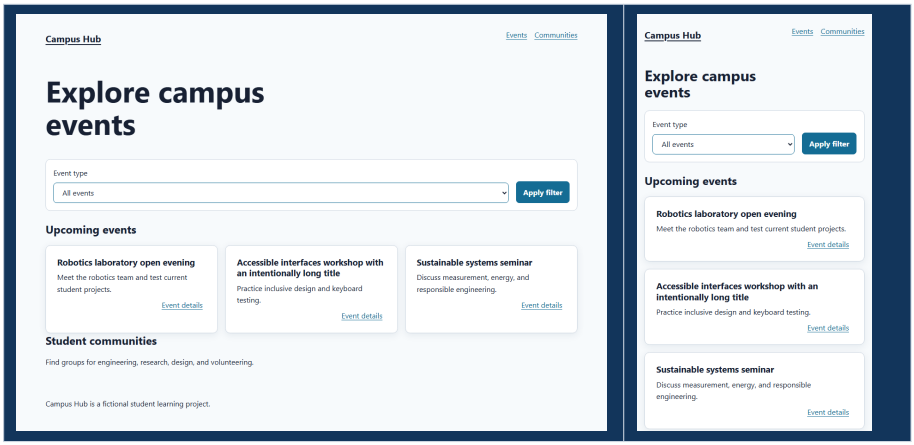
```
.page-shell { width: min(100% - 2rem, 72rem); margin-inline: auto; }
@media (min-width: 48rem) {
  .dashboard { grid-template-columns: minmax(0, 2fr) minmax(16rem, 1fr); }
}
```

Yararlı tek sütun düzeniyle başlayın. Medya sorgusunu ancak içerik geliştirmeyi destekleyecek kadar alan olduğunda ekleyin. Breakpoint belirli cihaz modeline değil yerleşim içeriğine dayanır. Yeni cihaz ve bölünmüş pencere tanıdık etikete uymasa da çalışmalıdır.

Şekil 3.8 aynı kart içeriğini dar ve geniş kısıtlarda karşılaştırır; Şekil 3.9 masaüstü, tablet ve telefon boyunca aynı anlamın yalnızca yerleşimle değiştiğini gösterir.

Şekil 3.9. Duyarlı tasarım içerik ve okuma sırasını korurken yerleşimi kullanılabilir görünüm alanına uyarlar.

Tam örnek, geniş tarayıcı çıktısını Şekil 3.10'da üretir. Görünüm alanı daraldığında aynı kaynak, ayrı mobil belge veya çoğaltılmış içerik olmadan Şekil 3.11'i üretir.



Şekil 3.10. Geniş görünüm alanında tamamlanmış CSS örneği filtre ve etkinlik kartlarını geliştirilmiş bileşen yerleşimi için yeterli alanla sunar.

Şekil 3.11. Dar görünüm alanında aynı HTML ve CSS filtre ile etkinlik kartlarını okunabilir tek sütuna yeniden akıtır.

Aralık sözdizimini veya yerleşik min-width/max-width sözdizimini tutarlı kullanın. Width sorguları donanımı değil görünüm alanını anlatır. Medya özellikleri yönü, hover yeteneğini, işaretçi hassasiyetini, renk şemasını, kontrast ve hareket tercihlerini de ifade edebilir. Tek özellikten kişi veya cihaz sınıfı çıkarmayın; dizüstünün hem dokunması hem faresi olabilir.

6.3 Container query bileşenleri bağlama duyarlı yapar

Medya sorgusu görünüm alanını bilir. Yeniden kullanılabilir bileşen çoğu zaman ebeveyninin verdiği alanı bilmelidir. Aynı etkinlik kartı aynı görünüm genişliğinde ana bölgede geniş, kenar çubuğunda dar görünebilir.

```
.event-card { container-name: event-card; container-type: inline-size; }
@container event-card (min-width: 20rem) {
  .event-card__content { grid-template-columns: 1fr auto; align-items: end; }
}
```

`container-type: inline-size`, inline-size sorguları için gereken boyut kapsamasını kurar. Adlandırılmış sorgu kapsayıcının kendisine değil çocuklarına uygulanır. Şekil 3.12 tek bileşenin, kapsayıcıları farklı inline-size sağladığı için farklı tepki verdiğini gösterir.

```
Tek görünüm: 1200px
Ana kapsayıcı: 25rem → içerik + eylem yan yana
Kenar çubuğu: 15rem → eylem alta
Aynı bileşen yalnızca geniş kapsayıcıda 20rem eşiğini geçer.
```

Şekil 3.12. Tek görünüm genişliğinde bir bileşen kendi kapsayıcısının inline-size değerine uyum sağlar.

Container query medya sorgusunu tamamlar. Sayfa düzeyi ortam değişiklikleri ve kullanıcı tercihleri için medya sorgusu; bileşenin iç düzenini kendi ayrılan alanı yönetiyorsa container query kullanın. `cqi` gibi konteyner birimleri sorgu kapsayıcısına göre hesaplanır; sıradan rem, yüzde ve intrinsic boyut çoğu durumda daha kolay düşünülür.

6.4 Görseller ve gömülü medya

```
img, svg, video { max-inline-size: 100%; block-size: auto; }
```

Sıradan görsellerin kapsayıcıyı aşmasını önleyin. Mümkünse oranı veya boyutu ayırın; geç yüklenen medya gereksiz yerleşim hareketi üretmesin. CSS görsel kutusunun sunumunu kısıtlar; HTML içerik ve yükleme koşullarına uygun kaynağı seçer. `srcset` ve `sizes` çözünürlük seçimi, `picture` ise gerçek crop/kompozisyon farklarında sanat yönetimi sağlar. Alternatif metin veya belge anlamı gereken içerik için CSS background görseli kullanmayın.

6.5 Kullanıcı tercihleri duyarlı girdilerdir

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    scroll-behavior: auto;
    animation-duration: 0.01ms;
    animation-iteration-count: 1;
  }
}
```

Bu geniş kural temel politikadır; hareket temel durumu iletiyorsa uyarlayın. Renk şeması ve forced colors tercihlerini de düşünün. Etkileşimi yalnızca hareket, hover veya renge bağlamayın. `prefers-color-scheme` açık/koyu palet seçebilir; her token bileşimi yine kontrast testinden geçmelidir. `color-scheme`, yerel denetim ve kaydırma çubuklarının desteklenen şemayla iş birliği yapacağını tarayıcıya bildirir.

Forced-colors: active durumunda kullanıcı aracısı renkleri yazar renklerinin yerine koyabilir. Border, odak ve durumu sistem renk anahtar sözcükleri veya forced-color uyumlu tekniklerle koruyun; modu savaşarak geçersiz kılmayın. `prefers-reduced-motion` ile temel geri bildirimi kaldırmayın; gereksiz hareketi anlık duruma dönüştürün ve ilerleme, hata, tamamlanma bilgisini algılanabilir tutun.

7. Erişilebilir sunum gerçek kullanıcı koşullarında ayakta kalır

Erişilebilirlik yerleşimden sonra eklenen bir tema değildir. CSS anlamsal içeriği okunamaz, erişilemez, yanıltıcı veya harekete duyarlı hâle getirebilir. Her sunum kararını insanların gerçekten algıladığı ve işlettiği biçimlerle test edin.

7.1 Etkileşim durumları algılanabilir olmalıdır

Bağlantı ve denetimlerin gerektiğinde ayırt edilebilir hover, focus, active, disabled, valid ve invalid durumları olmalıdır. Odak outline'ını yerine tüm yüzeylerde en az görünür bir alternatif olmadan kaldırmayın:

```
:focus-visible {
  outline: 3px solid #f0a202;
  outline-offset: 3px;
}
```

`:focus-visible`, tarayıcının göstergenin uygunluğunu belirlemesine izin verirken klavye tarzı odağı vurgular. Test edin; tarayıcı etkileşimlerinin aynı sezgiyi izleyeceğini varsaymayın. Sticky başlık, çerez paneli ve sabit toolbar odaklı denetimi örtebilir. Dar genişlikte ve yüksek yakınlaştırmada Tab ile ilerleyin. Farklı yüzeylerde kontrast için iki renkli veya ayrılmış outline yararlı olabilir. Devre dışı denetim okunabilir kalmalı; doğrulama yalnızca kırmızı ve yeşile dayanmamalıdır. Hover önemli bilgiyse klavye odağına eşdeğer davranış verin.

7.2 Hedef boyutu yerleşimden sonra ölçülür

WCAG 2.2 AA genel olarak işaretçi hedeflerinin en az 24×24 CSS piksel olmasını veya ölçütün aralık/bağlam istisnalarından birini karşılamasını ister. Bu asgari uygunluktur, rahat tasarım hedefi değildir. Campus Hub denetimleri test edilen kök ayarlarda 2.75rem minimum block-size ve komşu hedefler arasında boşluk kullanır. Satır içi bağlantı ve tarayıcı yerel denetimlerinde istisna olabilir. Bildirilen CSS boyutunun tıklanabilir son alanla aynı olduğunu varsaymayın; oluşturulmuş kutuyu ve dokunma/yanlış işaretçi kullanımını test edin.

7.3 Yakınlaştırma, metin aralığı ve yeniden akış yerleşim testleridir

%400 tarayıcı yakınlaştırmada başlangıçta 1280 CSS pikseli olan sayfa yaklaşık 320 CSS pikseli görünüm alanı sunabilir. WCAG yeniden akış, veri tablosu gibi gerçekten iki boyut isteyen içerik istisnaları dışında çoğu içeriğin iki boyutlu kaydırma olmadan okunabilir kalmasını bekler. Dar genişliği ve yakınlaştırmayı birlikte test edin; benzer fakat aynı olmayan arızaları açığa çıkarırlar.

Kullanıcı satır yüksekliğini, paragraf, harf ve sözcük aralığını değiştirebilir. Sabit yükseklik ve kırpılmış metinden kaçının. Yalnızca özgün metin, font ve boşlukla çalışan kart dayanıklı değildir.

7.4 Gizleme ve görsel sıra erişimi değiştirebilir

`display: none` ve HTML `hidden` normalde içeriği hem görsel yerleşimden hem erişilebilirlik ağacından kaldırır. `visibility: hidden` yerleşim alanını korurken öğeyi ve çocuklarını gizler. `opacity: 0` görünmez ama odaklanabilir kontrol bırakabilir. Ekran dışına konumlandırma da beklenmeyen kaydırma oluşturabilir. Gizleme mekanizmasını tarif kopyalamak yerine amaçlanan duruma göre seçin. CSS üretilmiş içerik, background görseli ve görsel yeniden sıralama eksik anlamsal sırayı onarmaz. CSS olmadan, klavye odağıyla ve doğrusal okuma sırasıyla test edin.

7.5 Hareket, kontrast ve forced colors birleşik kanıt ister

Hover, focus, disabled ve invalid dâhil her durumda metin ve metin dışı kontrastı ölçün. Anlamı renk olmadan doğrulayın. Azaltılmış hareket ve forced colors emülasyonu yapın; mümkünse gerçek işletim sistemi modunu da kullanın. Otomatik kontroller bazı kontrast ve hedef sorunlarını bulabilir; klavye, yakınlaştırma/yeniden akış, tercih ve insan okuma testleri yine gerekir.

Çalıştırmadan önce tahmin edin: kart 280px genişlikli kapsayıcıya konursa ne olur? `.card { width: 20rem; padding: 2rem; }` tipik 16px kökle iç alan hesaba katılmadan 320px'tir. border-box padding'i bu 320px içine alır; content-box dış kutuyu daha da büyütür. Her iki durumda sabit genişlik 280px kapsayıcıdan taşabilir. Dayanıklı alternatif `width: min(100%, 20rem)`'dir.

8. CSS mimarisi değişikliği ucuzlaştırmalıdır

CSS varsayılan olarak küreseldir: seçici belgenin her yerinde eşleşebilir ve özel özellik birçok çocuğa akabilir. Bakım, bu gücü cascade yokmuş gibi davranarak değil kontrol ederek gelir. Küçük ve sürdürülebilir proje şu açık katman sırasını kullanabilir:

```
@layer reset, base, layout, components, utilities;
```

1) reset, projenin bilinçli sahip olduğu küçük birlikte çalışabilirlik temelini; 2) base, token'ları ve düşük özgüllüklü öğe varsayılanlarını; 3) layout, sayfa kabuklarını ve yeniden kullanılabilir uzamsal ilişkileri; 4) components, event-card gibi bütünlüklü arayüz nesnelerini; 5) utilities, dar ve belgelenmiş tek amaçlı istisnaları içerir. Normal bildirimin sonraki katmanı, daha yüksek özgüllük gerektirmeden önceki katmanı aşar. İçerikleri yüklemeden sırayı ilan edin. Katmansız normal yazar kuralları katmanlı normal kuralları aştığı için kazara katmansız dosya mimariyi devre dışı bırakabilir.

Adlar amacı ve sahipliği anlatmalıdır. `.event-card`, `.blue-box-left`'ten daha dayanıklıdır. Her seçicide bir sayfanın tüm iç içeliğini kodlamayın. `.event-card__title` gibi adlandırma ekip aracıdır, CSS sözdizimi değildir. Native CSS nesting ilişkili seçicileri gruplayabilir; derin iç içelik uzun descendant seçicilerle aynı bağımlılığı yeniden üretir. Son seçiciyi açık tutun ve işlevsel pseudo-class'ların özgüllüğünü anlayın.

Yorumlar açık sözdizimini İngilizceye çevirmek için değil, anlaşılmayan kararı, uyumluluk sınırını veya kısıtı açıklamak için yazılmalıdır. Bir özelliğin korumalı fallback'e ihtiyacı varsa progressive enhancement için `@supports` kullanın:

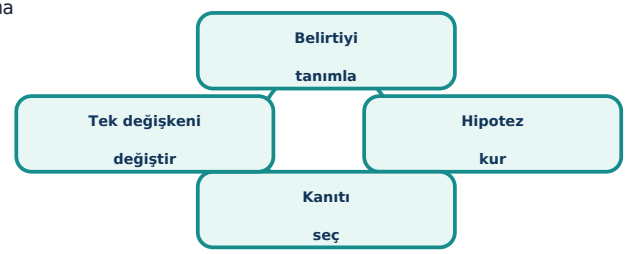
```
.dashboard { display: block; }
@supports (display: grid) {
  .dashboard { display: grid; }
}
```

Desteklenen tarayıcılarda modern Flexbox ve Grid bu fallback'i normalde gerektirmez; örnek karar kalıbını gösterir. Yeni özellik yayımlamadan önce güncel uyumluluk ve kitle gereksinimlerini kontrol edin. Eski tarayıcının yok saydığı bildirim ancak önceki sunum kullanılabilir kalıyorsa güvenli geliştirmedir.

9. CSS'yi değer tahminiyle değil kanıt izleyerek hata ayıklayın

Sayfa yanlış göründüğünde dört soruyu ayırın: 1) Stil sayfası yüklendi mi? 2) Seçici amaçlanan öğeyle eşleşti mi? 3) Bildirim cascade'i kazanıp geçerli değer olarak hesaplandı mı? 4) Yerleşim beklenen alan ve içerik varsayımlarına sahip mi? Şekil 3.13 bu soruları tarayıcı kanıtına bağlar.

```
Network → CSS yüklendi
Styles → seçici eşleşti, kazanan üstü çizili değil
Computed → kullanılan değer
Layout → kutu, track, taşma
```



Şekil 3.13. CSS hata ayıklama kaynak yüklemeden eşleşme, cascade, computed değer ve yerleşim kanıtına ilerler.

9.1 Doğru tarayıcı panellerini açın

1) Campus Hub sayfasını açın. 2) F12 veya Windows/Linux'ta Ctrl+Shift+I, macOS'ta Command+Option+I kullanın; kısayol değişebilir, tarayıcı menüsünde Developer tools bulunur. 3) Network panelini seçip yenileyin; CSS ile filtreleyin veya `styles.css` bulun. Başarılı durum ve CSS yanıtı dosyanın istendiğini kanıtlar. Satır yoksa HTML linkini, 404 ise çözülen adresi ve dosya adını inceleyin.

4) Elements/Inspector'da öğe seçiciyi etkinleştirip görünüşü beklenmeyen kartı veya denetimi seçin. 5) Styles/Rules'ta eşleşen seçicileri inceleyin. Üstü çizili özellik cascade'i kaybetmiş veya shorthand tarafından ezilmiştir; uyarı simgesi geçersiz sözdizimi gösterebilir. 6) Computed'da özelliği arayıp kazanan bildirime kadar açın; box-model görselinde content, padding, border ve margin'i okuyun.

7) Tarayıcı destekliyorsa Layout panelinde Flexbox/Grid overlay'ini açın; çizgi numaraları, track boyutları, gap ve öğe kaplamalarını görünür yapın. 8) Responsive-design modunda kesin CSS pikseli görünüm alanı belirleyin; olağan sayfada yakınlaştırmayı ayrıca test edin. Geliştirici araçlarındaki düzenlemeler geçici deneydir; nedeni açıklayan en küçük düzeltmeyi `styles.css` içine yazın, kaydedin, yenileyin ve kanıtı yeniden doğrulayın.

9.2 Yatay taşmayı sistematik izleyin

Sayfanın yatay kaydırma çubuğundan başlayıp kanıtı dışa doğru inceleyin: 1) görünüm genişliği ve yakınlaştırmayı kaydedin; 2) aday kutuları geçici outline ile çizin; 3) inline-end kenarı kapsayıcısının dışına çıkan ilk kutuyu bulun; 4) bildirilmiş boyut, intrinsic minimum, margin, transform, konum ofseti ve bölünemeyen çocukları inceleyin; 5) nedeni yerel düzeltin, ardından uzun metin, başka yazım yönü ve tüm zorunlu görünüm alanlarını test edin. Üretim stiline tanısal outline bırakmayın. Scrollbar kayboldu diye başarılı saymayın; içerik ve odak göstergelerinin erişilebilir olduğunu doğrulayın.

Yönlendirmeli laboratuvar - Duyarlı Campus Hub kabuğunu oluşturun

Başlamadan önce Fall 2026 ders şablonundan oluşturduğunuz kişisel depoda çalışın. Kök, `package.json`, `labs` ve `scripts` içeren klasördür. VS Code'da Terminal > New Terminal açın ve PowerShell'de geçerli konumu gösterin:

```
Get-Location
cd "C:\path\to\web-programming-coursework-2026_Fall-YOUR_STUDENT_NUMBER"
```

Yol kendi klonladığınız `web-programming-coursework-2026_Fall-YOUR_STUDENT_NUMBER` klasörü değilse gerçek yolunuza geçin. Yer tutucuyu aynen kopyalamayın; kendi öğrenci numaranızı kullanın.

npm komutu çalıştırmadan önce proje manifestinin görünür olduğunu doğrulayın: `Get-Item package.json`. Ders iş akışı istiyorsa temiz başlangıç noktasını commit edin; aksi durumda öğretim elemanının starter commit'inden başlayın.

```
git switch -c chapter-03-responsive-campus-hub
```

İki sıradan terminal sekmesi açın. İlk sekme uzun süre çalışan sunucu sürecini yürüttüğü için server terminalidir; ikinci sekme test ve Git komutları için boş kalır. Bunlar farklı tür yazılım pencereleri değildir; geçerli görevleri farklıdır. İlk terminalde çalıştırın:

```
npm run serve:week03
```

Süreci çalışır bırakın ve komutun yazdırdığı adresi, normalde `http://localhost:8000/`, tarayıcıda açın. npm `package.json` bulamıyorsa terminal yanlış klasördedir. Tarayıcı bağlanamıyorsa server terminalinin çalıştığını ve bağlantı noktasının yazdırılan adresle aynı olduğunu kontrol edin.

Amaç ve kabul ölçütleri

Sağlanan anlamsal Campus Hub sayfasını anlamını değiştirmeden stillendirin. Sonuç harici stylesheet, renk/boşluk/radius/okunabilir genişlik token'ları, tutarlı border-box, mantıksal kaynak ve odak sırası, etkinlik koleksiyonu için Grid, gezinme/eylemler için Flexbox, yerel klavye davranışını koruyan filtre, bir component container query, 320/768/1440 CSS pikselinde yatay kaydırmasız düzen, %200 yakınlaştırma, güçlü klavye odağı, yeterli pointer hedefleri, uzun başlık ve etiket sarması, reduced motion, forced colors ve renk dışı anlam içermelidir.

`!important`, ID seçicileri, pozitif tabindex ve sabit sayfa yüksekliği kullanmayın. Sağlanan kontrolü ve manuel ölçütleri geçirin; en az üç anlamlı commit oluşturun.

Adım 1 - Stilsiz belgeyi inceleyin

`labs/week-03/index.html` dosyasını açıp başlık, landmark, event-card ve filtre formu yapısını okuyun. Tarayıcıda Tab'a art arda basıp odak sırasını yazın. Responsive-design modunda görünümü 320 CSS pikseline daraltın. Seçiciyi kolaylaştırmak için HTML'yi değiştirmeyin.

Beklenen kanıt: içerik anlamlı sırayı korur, formun görünür etiketleri vardır ve sayfa tarayıcı varsayılanlarını kullanır. Başlangıç stil sayfası yalnızca görev yorumunu içerdiği için henüz Şekil 3.9 ve 3.10'a benzemez.

Adım 2 - Token'ları ve genel kısıtları kurun

`labs/week-03/styles.css` dosyasını açın. Katman bildirimini, amaç adlandırılmış özel özellikleri, kalıtılan body tipografisini, evrensel border-box temelini, duyarlı medya boyutunu, ortalanmış sayfa kabuğunu ve görünür `:focus-visible` kuralını ekleyin. Kaydedip yenileyin. Geliştirici araçlarında event card seçin; Computed box model'in border-box olduğunu doğrulayın. Klavyeyle bir bağlantı seçip outline'ın gerçek arka planda görünür olduğunu kontrol edin. Temel işi commit edin:

```
git add labs/week-03
git commit -m "Establish CSS tokens and resilient baseline"
```

Adım 3 - Bileşen yerleşimlerini kurun

Gezinmede sarılan Flexbox listesi, etkinlik kartlarında duyarlı Grid track'leri kullanın. Kartlara iç boşluk ve ince sınır verin; article anlamını koruyun. Filtreyi sarılan Flexbox ile stillendirin ve kontrollerin tipografiyi devralmasına izin verin. Yerel select veya button görünüşünü kaldırmayın. Flexbox'ın gezinmede hangi ekseni, Grid'in etkinlik koleksiyonunda hangi track'leri yönettiğini açıklayın. Dar ve geniş boyutlarda ilişkiler çalıştıktan sonra ikinci commit'i yapın.

Adım 4 - İçerik güdümlü geliştirme ekleyin

İçeriğin ek sütunları desteklediği breakpoint'te sayfa header'ı veya dashboard için kolonlara izin verin ve genişliği gerekçelendirin. Her event card'ı inline-size query kapsayıcısı yapın. Kart içeriği eşiği desteklediğinde ayrıntı bağlantısını metadata yanında hizalayın; mantıksal okuma sırasını koruyun. Sayfayı yeniden boyutlandırıp kartın eşik öncesi ve sonrası kapsayıcı genişliğini kaydedin.

Adım 5 - Taşma denetimi

320, 768 ve 1440 CSS pikseli ile %200 yakınlaştırmayı test edin. Bilerek uzun başlık ekleyin. Yatay kaydırma görünürse CSS değiştirmeden önce Layout inspector ile tam kutuyu bulun. Uzun bölünemeyen URL yapıştırın, belge yönünü geçici olarak rtl yapın ve mantıksal boşluğun inline başlangıç/bitişi izlediğini kontrol edin. Geçici içeriği nedeni ve düzeltmeyi kaydettikten sonra kaldırın; denetlenen düzen kararlı olduğunda üçüncü commit'i yapın.

Adım 6 - Tercihleri ve etkileşimi doğrulayın

Sayfada Tab ile ilerleyip görünür ve örtülmemiş odak kanıtını kaydedin. Oluşturulmuş filtre hedeflerini ölçün. Tarayıcı araçlarında reduced motion ve forced colors emülasyonu yapın; mümkünse işletim sisteminin yüksek kontrast modunu da deneyin. Her durumun renk olmadan anlaşılır kaldığını kontrol edin. İkinci terminalde depo kökünden `npm test` çalıştırın.

```
npm test
```

Başarılı komut `Coursework setup verification passed` bildirir. Bu, depo yapısı, Node.js sürümü, başlangıç belgeleri ve yerel sunucuların çalıştığını doğrular; her CSS kararını puanlamaz. Başarısızsa raporlanan dosya veya ortam sorununu okumadan düzenleme yapmayın.

Sonra kabul ölçütlerini manuel değerlendirin ve commit edilmiş çalışmanızı Ek A'daki test edilmiş referansla karşılaştırın. Şekil 3.10 geniş, Şekil 3.11 dar çıktıyı gösterir. İşletim sistemleri arasında font çizimi farklı olabilir; ilişkiler, durumlar ve responsive davranış aynı olmalıdır. Tablo 3.7 ile her görünüm alanı ve kullanıcı tercihi için yeniden üretilebilir kanıt kaydedin; yalnızca “çalışıyor” yazmayın.

Tablo 3.7 - Responsive yerleşim laboratuvarı gözlem kaydı

Kontrol	Kanıt ve yorum
Stylesheet isteği	
Eşleşen seçici ve computed değer	
320px taşma	
768px yerleşim	
1440px okunabilir satır uzunluğu	
%200 yakınlaştırma	
Uzun başlık sarması	
Sağdan sola mantıksal boşluk	
Container-query eşiği	
Odak görünürlüğü	
Hedef boyutu ve boşluğu	
Reduced-motion davranışı	
Forced-color davranışı	

Alıştırmalar

Temel Alıştırma 1 - Cascade kararı

`.card p { color: navy; }`, `.message { color: teal; }` ve `<p class="message">` için aynı kaynak ve katmanda hangi bildirim kazanır? Özgüllük karşılaştırmasını gösterin.

Alıştırma 2 - Kutu hesabı

content-box altında width 240px, her yanda 16px padding, 2px border ve 12px margin olan öğenin dış inline boyutunu hesaplayın. border-box için tekrarlayın.

Alıştırma 3 - Font ilişkili birimleri hesaplayın ve seçin

Kök computed font-size 20px. Kart padding: 1.25rem, başlık font-size: 1.6rem kullanıyor. İki değeri hesaplayın. Body satır uzunluğu, bileşen padding'i, kendi metniyle ölçeklenen rozet padding'i, tam genişlik medya ve akışkan fakat sınırlı başlık için birim/işlev seçip gerekçelendirin.

Uygulama Alıştırması 4 - Taşmayı onarın

`width: 100vw; padding: 2rem` sayfa içinde neden yatay kaydırma üretebilir? Dayanıklı alternatifi önerin.

Alıştırma 5 - Flexbox mı Grid mi?

Sarılan gezinme çubuğu, iki boyutlu zaman çizelgesi, dikey düğme grubu ve duyarlı kart galerisi için yerleşim sistemini seçin ve her seçimi açıklayın.

Alıştırma 6 - Odak gerilemesi

`*:focus { outline: none; }` içeren reset neden başarısızdır? Yalnızca renk değişimine dayanmayan onarım verin.

Zorluk Alıştırması 7 - Kaynak sırası

Tasarımcı geniş ekranda kenar çubuğunu ana etkinlik listesinden önce, dar ekranda sonra göstermek istiyor. Kafa karıştırıcı odak/okuma sırası oluşturmadan DOM sırası ve CSS stratejisi nasıl seçilir?

Alıştırma 8 - Responsive akıl yürütme

Sığabildiği kadar kart oluşturan, 17rem minimumu tercih eden, kapsayıcıyı aşmayan ve kalan alanı eşit paylaşan Grid bildirimini yazın. Her işlevi ve birimi açıklayın.

Alıştırma 9 - Stilsiz sayfayı tanılayın

HTML `/pages/index.html` içinde `styles.css` linkli; Network `http://localhost:8000/pages/styles.css` için 404 gösteriyor. Gerçek dosya `/assets/styles.css`. CSS neden uygulanamaz? İki doğru onarım verin.

Alıştırma 10 - Seçicileri bilinçli eşleştirin

`<article class="event-card featured"><h3>Robotics night</h3></article>` için `.event-card`, `.event-card.featured`, `.event-card .featured`, `article > h3` ve `article::before` seçicilerinin eşleşmesini açıklayın.

Alıştırma 11 - Kalıtım ve CSS-genişlik anahtar sözcükleri

Kart body'den color: navy kalıtıyor; bileşen kuralı `.event-card { color: teal; }` doğrudan yazıyor. Sıradan kart metni hangi rengi alır? Kartta color: inherit, color: initial ve color: unset ne anlama gelir?

Alıştırma 12 - Konum ve stacking

Modal çocukta z-index: 9999 olmasına rağmen transform uygulanmış kenar çubuğunun altında kalıyor. Daha büyük sayının neden yetmeyebileceğini ve mimariyi değiştirmeden önce geliştirici araçlarında hangi kanıtı inceleyeceğinizi açıklayın.

Alıştırma 13 - Medya mı container query mi?

Aynı event card ana bölgede 25rem, kenar çubuğunda 15rem; görünüm alanı her iki durumda 1200px. İç düzeni değiştirmek için medya mı container query mi? Minimal kural ve containment gereğini yazın.

Alıştırma 14 - Erişilebilir kontrol stilleri

Tasarım select okunu `appearance: none` ile kaldırıyor, focus outline'ı siliyor, soluk gri metin ve 16px tıklama hedefi veriyor. Ayrı kullanılabilirlik/erişilebilirlik risklerini bulun. CSS tek başına uygunluk kanıtlamadan daha güvenli temel önerin.

Alıştırma 15 - Geliştirici aracı kanıt planı

`padding-inline: 1rem` üstü çizili görünüyor. Dosyanın yüklenip yüklenmediğini, seçicinin eşleşmesini, bildirimin geçerliliğini, kazanan bildirimi ve tarayıcının kullandığı padding'i belirleyen kanıt dizisini yazın.

Alıştırma 16 - Mimari inceleme

```
#main-content section.events article.event-card h3.title {
  color: purple !important;
}
```

Bakım maliyetlerini açıklayın ve featured kart başlığının rengini değiştirmesine izin veren katmanlı, düşük özgüllüklü alternatif önerin.

Çözümlü cevaplar

Cevap 1

`.card p` bir sınıf ve bir tür seçicisi içerir; `.message` yalnızca bir sınıf içerir. Sınıf kategorisi eşit, tür kategorisi `.card p` lehine olduğu için navy kazanır; tüm önceki boyutlar eşitse sonraki sıra daha düşük özgüllüğü yenemez.

Cevap 2

content-box dış boyut margin ile 240 + 32 + 4 + 24 = 300px; marginsiz border kutusu 276px'tir. border-box bildirilen 240px padding ve border'ı içerir; margin eklenince 240 + 24 = 264px olur. Margin hiçbir zaman border-box genişliğine dahil değildir.

Cevap 3

1.25rem = 25px, 1.6rem = 32px. Body satır uzunluğunda `max-inline-size: 68ch`, kök ölçeği izleyen bileşen padding'inde rem, rozet iç boşluğunda yerel em, medya containment'te max-inline-size: 100% ve sınırlı başlıkta rem ile akışkan terimi birleştiren clamp seçin. Her karar gerçek içerik, font, kullanıcı ayarı, yakınlaştırma ve yeniden akışla test edilmelidir.

Cevap 4

100vw kapsayıcının kalan iç genişliğini değil görünüm alanını anlatır. content-box padding'i dışarı ekler; dikey kaydırma çubuğu da fark oluşturabilir. Normal block sizing veya kapsayıcı içinde `width: 100%`, `border-box` ve ebeveyn padding'i tercih edin.

Cevap 5

Sarılan gezinme için Flexbox; satır ve sütunları hizalayan zaman çizelgesi için Grid; dikey düğme grubu için column Flexbox; track oluşturma ve kolon hizalaması gereken kart galerisi için responsive Grid. Bunlar içerik ilişkisine dayalı varsayılanlardır.

Cevap 6

Reset görünür klavye konumunu kaldırır. Kullanıcı kontrole ulaşsa da girdinin nereye gideceğini bilemez. Kuralı kaldırın ve :focus-visible için çevreyle kontrast oluşturan kalın outline/offset kullanın; her yüzeyde test edin.

Cevap 7

Genellikle ana içerik sonra tamamlayıcı kenar çubuğu olacak tutarlı DOM sırası seçin. Geniş ekranda named Grid areas ile görsel konumu değiştirin; DOM sırasını değiştirmeyin. Her iki genişlikte doğrusal okuma ve Tab sırasını test edin.

Cevap 8

```
.cards { display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 17rem), 1fr));
  gap: 1rem;
}
```

repeat tekrar eden track'leri, auto-fit sığan sayıyı, minmax alt/üst sınırı, iç min daha dar kapsayıcıda taşmama davranışını, 1fr kalan alanın eşit paylaşımını sağlar.

Cevap 9

`href /pages/index.html` adresine göre çözüldüğü için `styles.css`, `/pages/styles.css` olur ve 404 alır. Doğru CSS yüklenmediğinden seçici veya cascade incelenemez. Onarımlar: `<link rel="stylesheet" href="/assets/styles.css">` ya da belge göreli `<link rel="stylesheet" href="../assets/styles.css">`. Önce Network'te başarılı CSS yanıtı isteyin.

Cevap 10

`.event-card` ve `.event-card.featured` article'ı eşleştirir; boşluksuz iki sınıf aynı öğede bulunur. `.event-card .featured` alt öğe istediği için eşleşmez. `article > h3` doğrudan çocuk h3'ü eşleştirir. `article::before` üretilmiş pseudo-element'tir; HTML çocuğu değildir ve uygun bildirim/content olmadan görünür kutu üretmez.

Cevap 11

Kartın doğrudan teal değeri kazanır; doğrudan color yoksa sıradan metin teal'i kalıtır. `color: inherit` ebeveynden navy alır; `initial` özelliğin belirtim başlangıç değeridir; color kalıtıldığı için `unset` inherit gibi davranıp navy alır. `initial`, HTML öğesinin tarayıcı varsayılanı anlamına gelmez.

Cevap 12

z-index stacking context içinde karşılaştırılır. Transform edilmiş ata ayrı bağlam oluşturabilir; 9999 çocuk değeri kendi bağlamından çıkıp kardeş bağlamı aşamaz. Elements panelinde modal/kenar çubuğu atalarını ve transform, positioned + non-auto z-index, opacity gibi bağlam oluşturan özellikleri inceleyin.

Cevap 13

Kartın kendi tahsis edilen alanı değiştiği için container query seçin:

```
.event-card { container-type: inline-size; }
@container (min-width: 20rem) {
  .event-card__content { grid-template-columns: 1fr auto; }
}
```

Cevap 14

appearance: none yerel ok ve davranış sorumluluğunu yazara aktarır; focus outline'ını silmek klavye konumunu gizler; soluk gri metin kontrastı düşürebilir; 16px hedef rahat değildir. Yerel görünüşü koruyun veya tüm affordance'ları yeniden kurup test edin; focus-visible kullanın, kontrast ölçün, en az uygun hedef boyutu ve komşu boşluk sağlayın.

Cevap 15

Önce Network paneli açıkken yenileyip stil sayfasının başarıyla döndüğünü doğrulayın. Sonra öğeyi seçip Styles'ta kuralı bulun; yoksa seçici eşleşmemiş veya koşul etkin değildir. Uyarı veya geçersiz değeri kontrol edin. Üstü çizili bildirimi açın veya Computed'da padding-inline arayıp kazanan shorthand/longhand ve kaynağını bulun. Son olarak box-model ve computed padding-inline-start/end değerlerini okuyun. `!important` eklemekten önce bildirimin geçerli, uygun ve yüklü olduğunu öğrenin.

Cevap 16

Seçici başlığı ID, çok sayıda ata, öğe adı ve iki sınıfa bağlar. Yüksek özgüllük yeniden kullanım ve varyasyonu pahalılaştırır; `!important` ayrı öncelik düzeyi açıp yeni önemli override'lar doğurur. Düşük özgüllüklü bileşen katmanında varsayılan ve modifier yazın:

```
@layer components {
  .event-card__title { color: var(--event-title-color, var(--color-heading)); }
  .event-card--featured { --event-title-color: var(--color-accent-strong); }
}
```

Başlık kuralı yeniden kullanılabilir kalır; featured kart cascade üzerinden belgelenmiş bileşen kararını değiştirir. Daha fazla ata adı seçiciyi güvenli değil daha bağlı yapar.

Hata ayıklama kliniği

%400 yakınlaştırmada etkinlik kartı footer'ın üstüne biniyor. Kart `height: 18rem`, açıklama `overflow: hidden` kullanıyor. Sabit yükseklik belirli metin boyutu ve miktarını varsayıyor; yakınlaştırma alan gereksinimini artırırken kart büyüyemiyor ve metin kırpılıyor. Sabit height'ı kaldırın, normal blok büyümesine izin verin; yalnızca görsel ritim için gerekirse min-height kullanın. Uzun içerik ve yakınlaştırmayla tekrar test edin; kırpmayla arızayı saklamayın.

Geri çağırma quiz'i

1. CSS kuralında selector, declaration block, property ve value nedir? 2. Geçersiz bildirim neden tüm stil sayfasını durdurmaz? 3. Harici stylesheet yolu nasıl çözülür ve yüklemeyi hangi panel kanıtlar? 4. `.card.featured` ile `.card .featured` farkı nedir? 5. Seçici ne yapar, cascade ne yapar? 6. Kaynak sırası ne zaman karar verir? 7. Özgüllük neden ondalık puan değildir?

8. Kalıtım doğrudan çocuk eşleşmesinden nasıl ayrılır? 9. box-sizing border-box olduğunda ne değişir? 10. Normal akış ve intrinsic sizing neden değerlidir? 11. Inline ve block eksenleri neyi temsil eder? 12. Büyük z-index neden yine başka bileşenin altında kalabilir? 13. Flexbox ne zaman Grid'den uygundur? 14. `minmax(min(100%, 17rem), 1fr)` neyi korur? 15. Bileşen medya sorgusu yerine ne zaman container query kullanır? 16. Görsel yeniden sıralama neden risklidir? 17. `overflow-x: hidden` neden kötü ilk onarımdır? 18. Focus tasarımı `:focus-visible` bildiriminden başka ne sağlamalıdır? 19. Zoom, reflow ve target-size testleri farklı hangi kanıtları verir? 20. Reduced-motion ve forced-color tercihleri tasarımı nasıl etkilemelidir?

Quiz cevapları ve yanılgı notları

1. `.card { padding: 1rem; }` içinde `.card` seçici, süslü parantez bildirim bloğu, padding özellik, 1rem değerdir. 2. CSS hata kurtarma ile desteklenmeyen bildirim yerel olarak yok sayılır; bu yazım hatasının zararsız olduğu anlamına gelmez. 3. Göreli href HTML adresinden çözülür; Network istek URL'si, durum kodu ve CSS yanıtını kanıtlar. 4. `.card.featured` iki sınıflı tek öğeyi, `.card .featured` card içinde featured sınıflı alt öğeyi eşleştirir. 5. Seçici eşleşen öğeleri belirler; cascade her özellik için rakip bildirimleri çözer. 6. Kaynak sırası ancak uygunluk, origin/importance, katman, özgüllük ve scope eşitse karar verir. 7. Özgüllük ID, sınıf-benzeri ve tür-benzeri kategorileri sıralı karşılaştırır; ondalık sayı değildir.

8. Doğrudan çocuk bildirimi cascade'e katılır; kalıtım ebeveynden gelir ve çocuk kendi değerini kurmadığında kullanılır. 9. width/height content, padding ve border'ı içerir; margin dışarıda kalır. 10. Normal akış alan ayırır, intrinsic sizing gerçek içeriği kısıta katar. 11. Inline eksen satırı, block eksen satır/blok birikimini izler. 12. z-index stacking context içinde karşılaştırılır; çocuk atasından çıkamaz. 13. Tek ana eksende Flexbox; satır-sütun track koordinasyonunda Grid. 14. İç min() dar kapsayıcıda 17rem tercihini bırakır; minmax() büyümeye izin verir, 1fr kalan alanı paylaşır.

15. Bileşenin ayrılan alanı görünüm alanından önemliyse container query kullanın; uygun kapsayıcı kurun ve sorgunun kapsayıcıyı değil çocukları stillendirdiğini unutmayın. 16. Görsel sıra DOM okuma ve klavye sırasından ayrılıp doğrusal olarak kafa karıştırabilir. 17. Global clipping kanıtı saklar ve içerik/odak göstergesini erişilemez kılabilir; ilk büyük kutuyu bulun. 18. Odak göstergesi görünür, yeterli kontrastlı, örtülmemiş ve anlamlı sırada olmalıdır. 19. Zoom metin büyütmeyi, reflow iki boyutlu kaydırmasız okunabilirliği, target-size son etkileşim alanı ve boşluğu test eder. 20. Temel olmayan hareketi azaltın/değiştirin; forced colors'ta kullanıcı renklerinin yerini almasına izin verin, sınır/odak/durumu renk dışı ipuçlarıyla koruyun.

Kendini değerlendirme

17-20: Dayanıklı arayüze davranış eklemeye hazırsınız. 13-16: Not kullanmadan açıklayamadığınız cevapları inceleyin. 8-12: Cascade, boyutlandırma ve responsive incelemelerini tekrarlayın. 0-7: Kabuğu normal akıştan yeniden kurun, sonra her kararla Flexbox ve Grid ekleyin.

Oturum özeti

CSS sabit ekran koordinatları değil uyarlanabilir kısıtlar sağlar. Sözdizimi seçicileri türlenmiş özellik değerlerine bağlar; hata kurtarma geçersiz bildirimlerin yerel kalmasına izin verir. Stil sayfası yüklenmeli ve seçici eşleşmelidir; ancak ondan sonra cascade anlamlıdır. Cascade origin, önem, katman, özgüllük, scope ve sırayı değerlendirir. Düşük özgüllüklü bileşen kuralları bakımı kolaylaştırır. Kalıtım ve custom property kararları belge ağacına taşır. Display, intrinsic sizing, kutu modeli ve containing block boyut/süreç sürprizlerini açıklar. Normal akış gerçek ilişki gerektirmedikçe korunmalıdır.

Mantıksal özellikler yazım yönünü izler. rem kökün computed font-size'ını, em bilinçli yerel bileşen ölçeklenmesini ifade eder. Tipografi, renk, denetim, arka plan ve border bildirime bakılarak değil oluşturulmuş birleşim olarak test edilmelidir. Flexbox bir ana eksen, Grid satır ve sütunları koordine eder. Medya sorgusu ortama, container query bileşenin ayrılan alanına yanıt verir. DOM sırası görsel yerleşimden bağımsız tutarlı olmalıdır. Odak, hedef boyutu, zoom, reflow, uzun içerik, yön, reduced motion ve forced colors test girdileridir.

Tarayıcı geliştirici araçları görünür arızayı yükleme, eşleşme, cascade, computed değer, kutu ve yerleşim kanıtına bağlar. Overflow tanısal kanıttır; küresel olarak gizlenmemelidir.

Sözlük

Tablo 3.8, inceleme sırasında cascade ve yerleşim kararlarını açıklamak için gereken sözcükleri toplar.

Tablo 3.8 - 3. Oturum sözlüğü

Kutu modeli	Content, padding, border ve margin bölgeleri.
At-rule	@ ile başlayan koşul/gruplama/işlem yapısı.
Cascade	Rakip CSS bildirimlerini çözen algoritma.
Cascade katmanı	Özgüllükten bağımsız öncelik grubu.
Computed değer	Cascade ve değer işleme sonrasındaki, son yerleşim öncesi değer.
Containing block	Başka kutunun boyut/konumunu çözmek için referans kutu.
Container query	Görünüm alanı yerine sorgu kapsayıcısına dayalı koşul.
Custom property	--color-accent gibi yazar tanımlı cascade özelliği.
Bildirim	padding: 1rem gibi özellik-değer önerisi.
em	Font ilişkili CSS uzunluk birimi.
Flexbox	Öncelikle tek eksende dağıtım/hizalama modeli.
Grid	Satır ve sütunları koordine eden model.
Kalıtım	Seçili değerlerin alt öğelere aktarılması.
Intrinsic boyut	Yazar dış boyutundan çok içerikten gelen katkı.
Mantıksal özellik	Fiziksel yönler yerine block/inline akışına göre özellik.
Medya sorgusu	Genişlik veya kullanıcı tercihi gibi ortama dayalı koşul.
Normal akış	Blok ve inline kutuların varsayılan belge yerleşimi.
Pseudo-class	:focus-visible gibi durum/yapısal koşul seçicisi.
Pseudo-element	::before gibi stillendirilebilir üretilmiş/soyut parça.
rem	Belge kökünün computed font-size'ına eşit root em.
Seçici	Kuralın uygulanabileceği öğeleri belirleyen desen.
Stacking context	z-index'in karşılaştırıldığı yerel boyama bağlamı.
Özgüllük	İlgili cascade düzeyinde rakipleri karşılaştıran seçici değeri.

Birincil başvurular ve ileri okuma

1. W3C, CSS Snapshot 2026. Modern CSS'nin tek numaralı “CSS3” değil, koordineli modüller ailesi olduğunu anlamak için. 2. MDN, CSS fundamentals curriculum. Sözdizimi, seçiciler, kutular, çakışmalar, değerler, formlar ve hata ayıklama için. 3. MDN, CSS text styling curriculum. Tipografi, bağlantılar, listeler ve web fontları için. 4. MDN, CSS layout curriculum. Normal akış, konumlandırma, Flexbox, Grid ve responsive pratik için. 5. W3C, CSS Cascading and Inheritance Level 5. Katman ve değer işlemenin normatif ayrıntıları için. 6. W3C, Selectors Level 4. Seçici dili ve işlevsel pseudo-class'lar için. 7. W3C, CSS Flexible Box Layout. Flex boyutlandırma ve hizalama için. 8. W3C, CSS Grid Layout Level 2. Track, placement, auto-placement ve subgrid için. 9. W3C, CSS Containment Level 3. Container query davranışı için. 10. W3C WAI, Understanding WCAG 2.2. Kontrast, reflow, odak, hedef boyutu ve tercih sunumu için. 11. Harvard University, CS50 Web Week 0: HTML and CSS. Ders-proje ilerlemesini karşılaştırmak için. 12. Stanford University, CS142 CSS lecture notes. CSS kuralı, seçici, renk, kutu ve yerleşim özeti için. 13. Jennifer Niederst Robbins, Learning Web Design, 6. baskı. CSS, tipografi, renk, kutu ve responsive düzen için. 14. Keith J. Grant, CSS in Depth, 2. baskı. Cascade, birimler, akış, kutular ve bileşen sistemleri için.

Neler daha sık değişir?

Cascade, akış, Flexbox ve Grid ilkeleri kararlıdır. Seçiciler, container query, nesting, renk uzayları, viewport birimleri, tercih medya özellikleri ve geliştirici araçları gelişmeye devam eder. Uyumluluk kararlarını “CSS3” gibi hatırlanan etiket yerine projenin test edilmiş tarayıcı politikası ve güncel destek verisiyle verin. Teknik inceleme tarihi: 21 Ekim 2026.

Oturum yansıtması

- Hangi CSS bildirimi özel durumların çoğunu ortadan kaldırdı? - Hangi overflow tanısı zihinsel modelinizi değiştirdi? - Çeviri veya kullanıcı zoom'u altında düzeni hangi kısıt daha güvenli yapar?

15. W3C, CSS Values and Units Module Level 4. Kök-temelli ve yerel font-temelli uzunluk birimlerinin tanımlarını rem ve em hesaplarını doğrulamak için kullanın. 16. W3C Web Accessibility Initiative, Understanding Resize Text. Göreli boyutlandırmayı, içerik veya işlev kaybı olmadan büyütülmüş metin çıktısıyla ilişkilendirmek için kullanın.

Neler daha sık değişir?

Cascade, normal akış, Flexbox ve Grid ilkeleri kararlıdır. Seçiciler, kapsayıcı sorguları, nesting, renk uzayları, görünüm alanı birimleri, tercih medya özellikleri ve tarayıcı geliştirici araçları gelişmeye devam eder. Uyumluluk kararlarını hatırlanan “CSS3” etiketiyle değil, projenin test edilmiş tarayıcı politikası ve güncel destek verisiyle verin. Teknik inceleme tarihi: 21 Ekim 2026.

Oturum yansıtması

• Hangi CSS bildirimi özel durumların çoğunu ortadan kaldırdı? • Hangi overflow tanısı zihinsel modelinizi değiştirdi? • Çeviri veya kullanıcı yakınlaştırması altında düzeni hangi kısıt daha güvenli yapar?

Öğretim elemanı rehberi

Önerilen beş saatlik sıra

1. Başlangıç kanıtı - 20 dakika: Stilsiz Bölüm 2 belgesiyle tamamlanmış sayfayı karşılaştırın. CSS devre dışıyken hangi anlamın kaldığını sorun. 2. Dil ve cascade - 50 dakika: Şekil 3.1'i açıklayın, seçici eşleşmelerini canlı düzenleyin ve bir çakışmayı Şekil 3.3 ile geliştirici araçlarında izleyin. 3. Kutular ve değerler - 55 dakika: Bir kartı iki box-sizing modeliyle ölçün; incelenen computed font-size değerlerinden rem ve em hesaplayın; intrinsic taşmayı ve mantıksal eksenleri gösterin. 4. Ara - 10 dakika.

5. Yerleşim stüdyosu - 60 dakika: Önce normal akışla bir kart listesi kurun; ardından aynı içeriği Flexbox ve Grid ile düzenleyin. Öğrencilerden hangi ilişkinin her modeli haklı çıkardığını açıklamalarını isteyin. 6. Responsive ve erişilebilirlik kanıtı - 45 dakika: Görünüm alanını ve kapsayıcı genişliğini değiştirin; zoom, reflow, klavye odağı, uzun içerik, reduced motion ve forced colors durumlarını kaydedin. 7. Hata ayıklama ve aktarım - 30 dakika: Her grup tek bir CSS arızasını yükleme, eşleşme, cascade, computed değer, kutu ve yerleşim kanıt zinciriyle sunar. 8. Yansıtma ve çıkış bileti - 20 dakika: Öğrenciler bir kuralı, bir kanıtı ve bir sonraki testlerini yazar.

Öğretim notları

Önce sonuç tahminini alın; sonra tarayıcı araçlarında kanıt gösterin. Öğrenciler “tarayıcı bunu görmezden geliyor” dediğinde Styles panelindeki üstü çizili bildirim, Network yanıtı ve Computed değeri arasında ayrım yaptırın. Ekran görüntüsünü tek başına kanıt saymayın: kaynak dosya, istek URL'si, durum kodu ve hesaplanmış sonuç birlikte incelenmelidir. Görsel yerleşim ile DOM sırasını özellikle karşılaştırın; bir öğeyi yalnızca güzel göründüğü için yeniden sıralamanın okuma ve klavye sonuçlarını tartışın.

Kabul ölçütü

Oturum sonunda öğrenci bir CSS bildiriminin nereden geldiğini açıklayabilmeli, bir bileşenin neden taşma yaptığını kanıtla gösterebilmeli, Flexbox ile Grid arasında içerik ilişkisine dayalı seçim yapabilmeli ve aynı sayfayı farklı dil, yakınlaştırma ve kullanıcı tercihlerinde inceleyebilmelidir. Değerlendirme, yalnızca piksel benzerliğine değil, bu kararları açıklayan kanıta dayanır.

5. Yerleşim ve responsive sistemler - 55 dakika: Gezinmeyi Flexbox, kartları Grid ve bir kapsayıcı sorgusu geliştirmesiyle kurun. Öğrencilerden her yeniden boyutlandırmadan önce değişikliği tahmin etmelerini isteyin. 6. Erişilebilir sunum - 30 dakika: Odağı, hedef boyutunu, %200 yakınlaştırmayı, 320 CSS pikselini, reduced motion ve forced colors durumlarını test edin. 7. Laboratuvar - 120 dakika: Altı kontrol noktasıyla Tablo 3.7'yi biçimlendirici kanıt olarak kullanın; son 15 dakikayı akran açıklaması ve commit incelemesine ayırın.

Yanılgıları ortaya çıkaran gösterimler

• Özgüllükten önce stylesheet yolunu bozun. Öğrenciler Network panelini kullanarak yüklenmeyen bir kuralın cascade'i kaybedemeyeceğini görmelidir. • Şekil 3.2'nin belge ağacında `.card.featured` ile `.card .featured` seçicilerini karşılaştırın. • Düşük özgüllüklü kuralı daha sonraki katmana, yüksek özgüllüklü kuralı daha erken katmana yerleştirin; öğrenciler özgüllüğün ilk cascade boyutu olmadığını görsün. • Flex veya Grid öğesine uzun, bölünmeyen dize verin; `min-inline-size: 0` eklemeden önce otomatik minimumu inceleyin. • Tarayıcının varsayılan font boyutunu değiştirin, kökün computed boyutunu inceleyin ve Campus Hub'ı yenilemeden önce rem/em ölçülerinin hangilerinin değişeceğini tahmin ettirin. • Aynı kartı tek görünüm alanında geniş ve dar kapsayıcılara yerleştirerek container query ile media query'yi ayırt edin. • Odak stilini geçici olarak kaldırıp forced colors'ı etkinleştirin; öğrencilerden tasarımın “erişilebilir göründüğünü” söylemek yerine kanıtı açıklamalarını isteyin.

Dinlenecek yaygın yanılgılar

• “CSS3, CSS'nin güncel tek sürümüdür.” Modern CSS bağımsız seviyelere ve olgunluklara sahip modüllerden oluşur. • “Son bildirim kazanır.” Yalnızca önceki cascade boyutları eşitlendiğinde kazanır. • “Kalıtım, seçicinin her alt öğeyle eşleşmesi demektir.” Kalıtım ve doğrudan eşleşme farklı değer yollarıdır. • “border-box taşmayı önler.” Boyut hesabını değiştirir; her sabit veya intrinsic minimumu geçersiz kılmaz. • “1rem her zaman 16px'tir.” 16px yaygın başlangıçtır; birim computed kök font boyutunu izler. • “rem otomatik olarak erişilebilirlik sağlar.” Yalnızca yeniden boyutlandırma, zoom ve reflow testleri kırpma veya işlev kaybını ortaya çıkarır.

• “Flexbox, Grid'in eskisidir” veya “Grid, Flexbox'ın yerini alır.” İkisi farklı boyutsal ilişkileri ifade eder ve normal akışla birlikte çalışır. • “Responsive; telefon, tablet, masaüstü demektir.” Gerçek girdiler içerik, kapsayıcılar, zoom, giriş yöntemi, tercihler ve pencere yapılandırmasıdır. • “Büyük z-index küresel olarak en üsttedir.” Stacking context'ler yereldir. • “Otomatik erişilebilirlik denetiminden geçmek CSS'nin erişilebilir olduğunu kanıtlar.” Bu yalnızca kısmi kanıt sağlar.

Uyarlamalar ve alternatifler

Zamanlı yazma assistive workflow için anlamsız bir engelse öğrencilere tamamlanmış kaynak dosyaları sağlayın; açıklamalarını, teşhislerini ve uyarlamalarını değerlendirin. Her diyagramın metinsel açıklamasını çevreleyen düzyazıda verin; tarayıcı zoom'una ve kişisel stillere izin verin; platformlar arasında piksel özdeşliğini puanlamayın. İşaretleme aygıtı kullanamayan öğrenci gerekli her görevi klavye veya eşdeğer giriş yöntemiyle tamamlayabilmelidir.

Biçimlendirici rubrik

• Kavramsal akıl yürütme - %30: Eşleşme, cascade, kutular ve yerleşim seçimlerini doğru sınırlarla açıklar. • Responsive dayanıklılık - %25: Gerekli genişlik, zoom, uzun içerik, yön ve container testlerini taşmayı gizlemeden geçer. • Erişilebilir sunum - %20: Odağı, kontrastı, hedef kullanılabilirliğini, kaynak sırasını, tercihleri ve algılanabilir durumları korur. • Mühendislik kanıtı - %15: Network, Styles, Computed, kutu ve yerleşim araçlarını kullanıp yeniden üretilebilir gözlemler kaydeder. • Sürdürülebilirlik - %10: Amaçlı adlar, düşük özgüllük, belgelenmiş katmanlar, tutarlı commit'ler ve yasaklanmış kısa yolların yokluğunu gösterir.

Oturum 4 - JavaScript'i ilk ilkelerden öğrenmek

JavaScript, durağan Campus Hub sayfasını programa dönüştürür. Bu bölüm ilk kod satırından önce başlar. Hiçbir programlama çalışma zamanı kurmamış, proje klasörü açmamış, terminal kullanmamış veya hata mesajı yorumlamamış olabileceğiniz varsayılır. Bilgisayarı hazırlayacak, Node.js ile bilinçli olarak küçük bir program çalıştıracak, bir diğerini tarayıcıda çalıştıracak ve kaynak kodu çıktısından ayırmayı öğreneceğiz. Ancak bundan sonra değerleri, adları, kararları, tekrarı, fonksiyonları, dizileri, nesneleri, hataları ve testleri tanıtacağız.

Document Object Model (DOM) değiştirme Bölüm 5'te başlar. Sınırı açık tutmak yararlıdır: önce JavaScript dilini, sonra sayfayı değiştirmek için tarayıcı olanaklarını öğreniriz. Daha önce programlama yapan okuyucu bile ortam kontrollerini tamamlamalıdır; sonraki laboratuvarlar aynı klasörlere, komutlara ve kanıta dayanır.

Oturum haritası

Tablo 4.1 her öğrenme evresini, öğrencinin ve öğretim elemanının ilerlemeden önce inceleyebileceği kanıtla eşleştirir.

Tablo 4.1 - Oturum 4 öğrenme sırası ve kanıtı

Evre	Ana soru	Kanıt
Yönel	JavaScript nerede çalışır?	Tarayıcı ve Node.js ayırt edilebilir sonuç üretir.
Temsil et	Program ne tür değer tutabilir?	Tahmin edilmiş türler ve hesaplanmış ifadeler.
Karar ver ve tekrarla	Yürütme yolu nasıl seçer veya yineler?	İzlenmiş dallar ve döngü adımları.
Soyutla	Fonksiyon girdiyi nasıl sonuca dönüştürür?	Çağrı, dönüş ve kapsam izleri.
Modelle	Diziler ve nesneler Campus Hub verisini nasıl temsil eder?	Filtrelenmiş ve özetlenmiş etkinlik kayıtları.
Doğrula	Programın doğru olduğunu nasıl biliriz?	Başarılı testler ve incelenmiş tarayıcı çıktısı.

Önkoşullar

Dosya açma ve web adresini ziyaret etme konusunda sıradan bir aşinalık yeterlidir. Bölüm 0 tam yönlendirme kaynağıdır; ancak bu bölüm açıklanamayan bir araç yüzünden kimsenin durmaması için JavaScript'e özgü hazırlığı tekrarlar. Önceki programlama deneyimi varsayılmaz.

Öğrenme çıktıları

Bu bölümün sonunda şunları yapabilmelisiniz: editör, tarayıcı, Git, Node.js ve npm'i kurmak veya doğrulamak; proje klasörünü, terminalin geçerli dizinini, kaynak dosyasını, komutu ve çıktıyı ayırt etmek; Node.js ve tarayıcıda ilk JavaScript dosyasını oluşturup çalıştırmak; ECMAScript dili ile tarayıcı ve Node.js ana bilgisayar olanaklarını ayırmak; ilkel değerleri, dizileri, nesneleri ve fonksiyonları oluşturup incelemek; yeniden atanması gerekmeyen bağlamalarda const, gerekenlerde let kullanmak; ifadeleri, deyimleri, dalları, döngüleri ve fonksiyon çağrılarını izlemek; strict equality, truthiness, conversion, null ve undefined'i açıklamak; açık girdileri, dönüşleri ve yararlı hataları olan fonksiyonlar tasarlamak; kaynak veriyi yanlışlıkla değiştirmeden dizileri dönüştürmek; bir adı yeniden bağlamakla bir nesneyi değiştirmek arasındaki farkı açıklamak; syntax, reference, type ve assertion hatalarını kanıta göre teşhis etmek; Node.js ile küçük otomatik testler yazıp çalıştırmak.

0. Bilgisayarı hazırlayın ve ilk JavaScript'inizi çalıştırın

Programlama dili, tahminler yapıp bunları çalışan programa karşı kontrol ederek öğrenilir. Salt okuma bu geri bildirimi sağlayamaz. Bu yüzden JavaScript sözcük dağarcığından önce, düzenlediğiniz dosyadan gözlemleyebileceğiniz çıktıya giden küçük ve güvenilir yolu kuracağız.

Şekil 4.1 eksiksiz başlangıç iş akışını gösterir. Kutular ardışık adımlardır; alternatif teknolojiler değildir: Visual Studio Code kaynak dosyasını düzenler, Node.js onu çalıştırır, terminal çıktıyı gösterir, Git yararlı değişikliği kaydeder.

Şekil 4.1. İlk program gözlemlenebilir düzenle-kaydet-çalıştır-kontrol et-kaydet döngüsünden geçer.

0.1 Kurmadan önce beş aracın sorumluluğunu anlayın

Visual Studio Code (VS Code), proje klasörünü açıp kaynak dosyaları yazdığımız metin editörüdür. Node.js, tarayıcı dışında JavaScript dosyasını okuyup çalıştırabilen çalışma zamanıdır. npm, Node.js ile kurulan paket yöneticisidir; package.json içindeki talimatları okur ve npm test gibi adlandırılmış betikleri çalıştırır. npm JavaScript dili veya web sunucusunun kendisi değildir.

Tarayıcı, istemci tarafı JavaScript'i çalıştırır ve Document Object Model (DOM), olaylar ve ağ istekleri gibi tarayıcı Application Programming Interfaces (API'ler) sağlar. Güncel Chromium tabanlı tarayıcıyı kullanır, önemli çalışmaları Firefox'ta da test ederiz. Git proje sürümlerini kaydeder; GitHub uzak kopyayı saklayıp işbirliğini kolaylaştırır, fakat Git ve GitHub JavaScript çalıştırmaz.

Tablo 4.2 - JavaScript çalışma istasyonu araçları ve doğrulama kanıtı

Araç	Bu bölümdeki amacı	Doğrulama kanıtı
VS Code	Projenin tamamını düzenlemek	Explorer package.json ve labs klasörünü gösterir.
Node.js 24 Long-Term Support (LTS)	JavaScript ve ders araçlarını çalıştırmak	node --version v24 ile başlayan değer yazdırır.
npm	Projenin tanımladığı komutları çalıştırmak	npm --version bir sürüm numarası yazdırır.
Tarayıcı	İstemci kodunu ve Developer Tools'u çalıştırmak	Console 2 + 3 yazınca 5 gösterir.
Git	Değişiklikleri kaydetmek ve incelemek	git --version bir sürüm numarası yazdırır.

Ders, Node.js 24 Long-Term Support (LTS) serisinin kullanılabilir en yeni düzeltme sürümüyle test edilir. LTS, kısa ömürlü current sürümüne göre daha uzun bakım alan sürüm serisidir. Resmî Node.js indirme sayfasından sunulan en yeni 24.x düzeltmesini kurun; ekran görüntüsündeki eski yama numarasını aramayın.

0.2 Windows'ta Node.js'i adım adım kurun

node --version zaten v24.x.x bildiriyorsa Bölüm 0.4'e geçin. Aksi hâlde tarayıcıda https://nodejs.org/en/download adresini açın; en yeni LTS 24.x sürümünü ve bilgisayarınız için Windows Installer'ı seçin. Çoğu Windows bilgisayarı x64 kullanır; ARM tabanlı Windows ARM64 yükleyicisi ister. Kurulumda npm'yi ve Node.js'i PATH'e ekleyen seçeneği koruyun. PATH, node gibi bir komut yazdığınızda işletim sisteminin aradığı klasörler listesidir. Kurulum bitince eski Command Prompt, PowerShell ve VS Code terminal pencerelerini kapatın; yeni terminalde node --version ve npm --version çalıştırın.

macOS'ta resmî .pkg yükleyicisini, Linux'ta kurumun onayladığı yöntemi kullanın. Hangi yöntem seçilirse seçilsin, yeni terminal Node.js 24 ve çalışan npm komutunu göstermelidir. Çevrim içi playground'ı bu kurulumun yerine kullanmayın; yerel klasörleri, modülleri, testleri, sunucu sürecini ve Git akışını öğrenmek için gerçek çalışma alanı gerekir.

0.3 Visual Studio Code'u kurun veya doğrulayın

https://code.visualstudio.com/Download adresindeki resmî indirme sayfasını açın. Windows'ta çoğu öğrenci için User Installer uygundur; teklif edilirse code komutunu PATH'e ekleyen seçeneği koruyun ve terminalleri yeniden açın. VS Code'da File > Open Folder'ı seçip kişisel coursework repository'nizi açın. Explorer'ın üst düzeyinde package.json, labs ve scripts görünmelidir. Yalnızca events.js açmak eşdeğer değildir; npm komutları package.json içeren proje köküne bağlıdır. Workspace Trust görünürse yalnızca kendi klonunuza veya yetkili ders deposuna güvenin; tanımadığınız kodu Restricted Mode'da inceleyin.

0.4 Doğru terminali açın ve geçerli dizini kanıtlayın

VS Code'da Terminal > New Terminal'ı seçin. Terminal paneli genellikle editörün altında açılır. İmleçten önceki metin komut istemidir; örneğin PowerShell şu yolu gösterebilir:

```
PS C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall>
```

PS'yi, yolu veya sondaki > işaretini komutla birlikte yazmayın; bunlar çalışan kabuk ve geçerli klasör için kanıttır. Her komutu tek satırda çalıştırın:

```
dir package.json
node --version
npm --version
git --version
```

İlk komut package.json bulunamadı derse terminal depo kökünde değildir. File > Open Folder ile doğru klasörü açın veya yolu cd ile değiştirin:

```
cd "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall"
```

Şekil 4.2, diskteki dosyanın, kabukta yazılan komutun ve çalışma zamanı çıktısının farklı şeyler olduğunu gösterir.

0.5 İlk Node.js programını oluşturun ve çalıştırın

VS Code Explorer'da labs klasörünü, ardından week-04 klasörünü genişletip hello.js dosyasını açın. Dosya yoksa week-04 üzerinde sağ tıklayıp New File'ı seçin, hello.js yazın ve Enter'a basın. .js uzantısı dosyanın JavaScript kaynak kodu içerdiğini belirtir. Aşağıdaki programı tam olarak yazın ve Ctrl+S (macOS'ta Command+S) ile kaydedin:

```
const courseName = "Web Programming";
const sum = 2 + 3;

console.log(`Hello from ${courseName}!`);
console.log(`Two plus three is ${sum}.`);
```

Çalıştırmadan önce noktalama işaretlerini okuyun. const, daha sonra farklı bir değer atanmayacak adı bildirir; courseName ve sum programcının seçtiği, büyük/küçük harfe duyarlı adlardır; = burada eşitlik değil, sağdaki değeri soldaki ada atamadır; 2 + 3, 5 üreten bir ifadedir; backtick içindeki template literal, değerlendirilen ifadeyi metne yerleştirir; console.log(...) ana bilgisayarın console nesnesinin log fonksiyonunu çağırır; nokta console içinden log özelliğini seçer; noktalı virgül bu dersin yazımında deyimin amaçlanan sonunu belirtir.

Depo kökünden dosyayı Node.js ile çalıştırın:

```
node labs/week-04/hello.js
```

Beklenen çıktı: Hello from Web Programming! Two plus three is 5.

Dosyayı düzenlemek tek başına programı çalıştırmaz. Önce kaydedin; komut, Node.js'ten diskteki kaydedilmiş sürümü okumasını ister. Kaynak kod, shell komutu, çalışma zamanı süreci ve terminal çıktısını birbirine karıştırmayın.

Beklenen terminal çıktısı aşağıdadır. Bu iki satırı komut olarak yazmayın.

```
Hello from Web Programming!
Two plus three is 5.
```

2 + 3'ü 2 * 3 olarak değiştirin, kaydedin, yeni sonucu tahmin edin ve aynı komutu yeniden çalıştırın. Çıktı hâlâ 5 ise dosya kaydedilmemiş veya farklı dosya çalıştırılmış olabilir. Bu küçük deney düzenle, kaydet, tahmin et, çalıştır, gözle ve açıkla döngüsünü kurar.

0.6 JavaScript'i tarayıcıda çalıştırın

Node.js ve tarayıcı JavaScript'i uygular; ancak farklı ana bilgisayar ortamlarıdır. hello-browser.html tarayıcının göstereceği HTML belgesidir. hello-browser.js ise tarayıcı JavaScript motorunun çalıştıracağı kaynak dosyasıdır. Bu iki dosya aynı klasörde bulunsa bile tarayıcı onları kendiliğinden birleştirmez; HTML içindeki script öğesi aralarındaki açık bağlantıyı kurar.

```
<script src="hello-browser.js"></script>
```

HTML yapıyı ve anlamı tanımlar; JavaScript hesaplama ve etkileşim talimatlarını içerir. Bu ayrım, HTML'nin JavaScript devre dışıyken bile okunabilir kalmasını, aynı JavaScript dosyasının birden çok sayfada kullanılabilmesini ve her dosyanın ayrı ayrı test edilmesini sağlar. Tablo 4.3 iki dosyanın sorumluluklarını ve kanıtlarını özetler.

Tablo 4.3 - hello-browser.html ile hello-browser.js arasındaki görev ayrımı

Dosya	Görevi	Gözlenebilir sonuç
hello-browser.html	Belgenin yapısını ve script bağlantısını sağlar.	Başlık, açıklama ve script öğesi sayfada görünür veya kaynakta incelenir.
hello-browser.js	Tarayıcı JavaScript motoruna çalıştırılacak talimatları verir.	İki mesaj Developer Tools > Console'da görünür.

src göreli Uniform Resource Locator (URL)'dir; hello-browser.html'den komşu hello-browser.js dosyasını tanımlar. Ayrı dosya kullanılsa bile kapanış script etiketi gereklidir.

Şekil 4.3. Tarayıcı HTML dosyasını alır, script bağlantısını keşfeder, JavaScript dosyasını ayrı ister ve sonra çalıştırır.

Depo kökünden yerel sunucuyu başlatın ve o terminali açık bırakın:

```
npm run serve:week04
```

http://localhost:8000/ içeren satırı gördüğünüzde sunucu tarayıcı isteklerini bekliyordur; yeni istem görünmemesi normaldir. İkinci terminal açın. Tarayıcıda http://localhost:8000/hello-browser.html adresine gidin; F12 veya Ctrl+Shift+I ile Developer Tools'u açıp Console'u seçin. JavaScript dosyasının iki mesajı orada görünmelidir.

Şekil 4.3'te tarayıcı önce HTML'i ister, script öğesini ayrıştırdıktan sonra JavaScript için ikinci istek yapar; ancak bundan sonra motor kaynağı çalıştırıp Console'a yazabilir. HTML dosyası harici JavaScript'i fiziksel olarak içermez. Console görünür sayfanın değil, geliştirici kanıtının panelidir.

0.7 İlk hataları okuyun; baştan kurmaya başlamayın

Hata mesajı, denenmiş bir işlem hakkındaki rapordur. Değişiklik yapmadan önce okuyun. `node` bulunamıyorsa Node.js kurulumunu, terminali kapatıp yeni terminal açmayı ve node --version'ı kontrol edin. Cannot find module ...hello.js için geçerli klasörü ve dosya yazımını karşılaştırın. npm package.json için ENOENT bildiriyorsa dir package.json ile depo köküne dönün. Sunucu komutu sonrasında istem görünmüyorsa sunucu normal çalışıyordur; test ve Git için ikinci terminal kullanın. Tarayıcı siteye ulaşamıyorsa server terminalini okuyup yazdırılan URL'yi aynen açın. HTML var ama Console mesajı yoksa iki dosyayı kaydedin, yenileyin, Console ve Network'te hello-browser.js isteğini inceleyin. 404 için src, büyük-küçük harf, dosya adı ve klasörü karşılaştırın. SyntaxError satırında eksik tırnak, parantez, köşeli ayraç veya backtick'i ve önceki satırı inceleyin. EADDRINUSE 8000 başka sürecin portu tuttuğunu belirtir; önceki sunucu terminaline dönün veya Ctrl+C ile durdurun.

Her hatada yazılımı yeniden kurmayın. Kurulum eksik aracı onarır; yanlış dosya adı, kaydedilmemiş kaynak, yanlış klasör veya sözdizimi hatasını onarmaz. Kanıtın belirttiği sınırı teşhis edin.

1. Program, makine için sıralı bir açıklamadır

Campus Hub bilgiyi ve sunumu içerir; HTML ve Cascading Style Sheets (CSS) kalan koltukları hesaplamaz. Program sıralı talimatlar ve kararlar sağlar. Kapasite 80, kayıt 62 ise 80 - 62'yi hesaplayabilir, sonucu adlandırabilir, etkinliği açık olarak sınıflandırabilir ve özet üretebilir:

```
const capacity = 80;
const registered = 62;
const remaining = capacity - registered;

console.log(remaining);
```

Beklenen Console çıktısı sayı 18'dir. Kod kaynak metindir; 18 çalışma zamanı kanıtıdır. Bu derste deyim sınırlarını görünür tutmak için noktalı virgülleri tutarlı yazarız.

Şekil 4.4 dili, onu çalıştıran iki ortamdan ayırır: her ikisi çıkarma, bağlama, dizi ve fonksiyonları bilir; tarayıcı document, Node.js ise process, dosya ve sunucu API'leri gibi host nesneleri sağlar. Bu host nesneleri çekirdek dilin parçası değildir.

Şekil 4.4. ECMAScript bir host içinde çalışır; tarayıcı ve Node.js farklı ortam API'leri ekler.

1.1 ECMAScript ve JavaScript

ECMAScript standartlaştırılmış dil belirtimidir. JavaScript, bu dilin uygulamaları ve kodun çalıştığı çevre platformu için kullanılan addır. Standart değerleri, işleçleri, deyimleri, fonksiyonları, nesneleri ve yerleşik olanakları tanımlar. HTML, DOM, tarayıcı depolaması ve Node.js dosya erişimi host'lardan gelir.

1.2 İlk dosyanız: doğru dizinden

Kişisel coursework repository'nizde labs/week-04/events.js dosyasını VS Code'da açın. package.json içeren terminalden `npm run serve:week04` çalıştırın. Bu terminali açık bırakın; buna server terminali denir. http://localhost:8000/ adresini ziyaret edip Developer Tools > Console'u seçin. Başlangıç kodu üç etkinlik nesnesini tablo olarak yazdırır. Sayfa yüklenmezse önce server terminalini, tablo yoksa Console ve Network'te events.js isteğini ve JavaScript hatasını inceleyin.

1.3 Yürütme sırası, yorumlar ve büyük-küçük harf

Senkron programlarda JavaScript deyimleri genellikle yukarıdan aşağı değerlendirir. Daha sonra gelen deyim önceki bağlamayı kullanabilir; console.log(sum)'i const sum = 2 + 3'ün üzerine taşımak başlatılmadan önce okumaya çalıştığı için hata verir. eventTitle, eventtitle ve EventTitle farklı adlardır. Değişken ve fonksiyonlarda lower camel case kullanın; örneğin seatsRemaining. Yorumlar kodun açıkça anlatamadığı amacı açıklamalıdır; eski kodu saklamak için yorum kullanmayın, Git geçmişi zaten bunu kaydeder.

```
// One-line comment
const remaining = Math.max(0, capacity - registered);

/* A block comment can span
   more than one source line. */
```

2. Değerlerin türleri, bağlamaların adları vardır

Değer, programın üzerinde çalışabildiği veridir; tür, değerle ilişkili işlem ve davranışları açıklar. JavaScript'in yedi ilkel türü vardır: string, number, bigint, boolean, undefined, symbol ve null. Diğer her şey nesnedir; diziler ve fonksiyonlar özelleşmiş nesnelerdir. JavaScript dinamik türlenir: tür çalışma zamanında değere aittir, değişken adı kalıcı tür edinmez. Yine de 18 sayısı ile “18” metni aynı değildir.

Kaynakta doğrudan yazılan 18, true veya "open" bir literal'dir. Değerler ifadelerden, girdiden, fonksiyon çağrılarından veya ağdan da gelebilir. Tablo 4.5 bu bölümdeki sınırları toplar: string değişmez metindir; number kayan noktalı sayıdır; bigint güvenli Number aralığının ötesindeki tamsayıdır ve Number ile doğrudan karıştırılamaz; boolean koşullarda kullanılır; undefined değer sağlanmadığında; null bilinçli yoklukta kullanılır; symbol benzersiz ilkel anahtardır; object anahtar özellikleri olan değişebilir koleksiyondur.

```
typeof "Robotics night"; // "string"
typeof 18; // "number"
typeof true; // "boolean"
typeof undefined; // "undefined"
typeof null; // "object" (tarihsel sonuç)
typeof []; // "object"
typeof function () {}; // "function"
```

typeof'i gözlem aracı olarak kullanın; tam değer teorisi değildir. Diziyi Array.isArray(value) ile, null'ı value === null ile test edin.

2.1 İfadeler değer üretir

capacity - registered, event.title ve seatsRemaining(event) değerlendirilerek değer üreten ifadelerdir. Bildirim veya if deyimi ise yürütmede eylem gerçekleştirir. Küçük ifadelerden sonuca ilerleyin; parantezleri `(registered / capacity) * 100` gibi açık gruplama için kullanın.

2.2 const ve let

Bildirim, adı değere bağlar:

```
const eventTitle = "Robotics night";
let registered = 62;
registered = registered + 1;
```

const varsayılandır; adın başka değere atanmasını engeller. Algoritma yeniden atama gerektiriyorsa let kullanın. Yeni ders kodunda var kullanmayın; function scope ve hoisting ilk blok kapsamı modelini bulanıklaştırır. const nesneyi dondurmaz:

```
const event = { registered: 62 };
event.registered = 63; // mutation mümkündür
// event = {}; // rebinding başarısız olur
```

Bağlama aynı nesneye bağlı kalır; nesne içindeki özellik değişmiştir. Bu, yeniden bağlama ile mutation arasındaki farktır.

2.3 Dizeler ve template literal'lar

Tek ve çift tırnak string oluşturur. Backtick template literal'dır ve ${...} içinde ifade çalıştırabilir:

```
const message = `${eventTitle}: ${remaining} seats remaining`;
```

Interpolation sonucu metne çevirir; + zincirlerinde aynı işlecin toplama veya birleştirme anlamına gelmesi gibi belirsizliği azaltır. String'ler değişmezdir; işlemler yeni string üretir. length UTF-16 code unit sayısını, [0] ilk karakteri (sıfır tabanlı) verir. Escape dizileri satır sonu ve tırnak gibi karakterleri temsil eder.

3. İşleçler ilişkiler kurar

+, -, *, /, %, üstelik işleci aritmetik; <, <=, >, >= karşılaştırma işleçleridir. Uygulama karşılaştırmalarında strict equality === ve !== kullanın. = değeri bağlamaya saklar; === soru sorup boolean üretir. 18 === 18 true, 18 === "18" false, 18 == "18" ise gevşek eşitlik dönüşüm yaptığı için true'dur. Bu ders açıkça gerekmedikçe strict equality kullanır.

3.1 Dönüşüm görünür olmalıdır

Form denetimleri ve URL parametreleri çoğu zaman string gelir:

```
const enteredCapacity = "80";
const capacity = Number(enteredCapacity);

if (!Number.isFinite(capacity)) {
  throw new TypeError("Capacity must be a finite number.");
}
```

Number dönüşümü açık eder; Number.isFinite NaN ve sonsuzlukları reddeder. NaN === NaN false'tur; özel test için Number.isNaN kullanın. Number(boş-string) 0, Number(eighty) NaN üretir. Boş değer reddedilecekse önce özgün girdiyi, sonra dönüştürülmüş sonucu doğrulayın. parseInt ve parseFloat '12 seats' içindeki 12 gibi sayısal ön ekleri kabul eder; bunu yalnızca veri sözleşmesi özellikle istiyorsa kullanın.

3.2 Truthiness ve short-circuiting

Koşullar ifadeyi boolean'a dönüştürür. false, 0, -0, 0n, boş string, null, undefined ve NaN falsy'dir; boş dizi ve nesne dahil nesneler truthy'dir. && ilk falsy'de, || ilk truthy'de durur; ?? yalnızca sol taraf null veya undefined ise sağı kullanır:

```
const label = suppliedLabel ?? "Untitled event";
```

0, false veya boş string geçerli veri olabilir; bu durumda || anlamlı değeri yanlışlıkla değiştirebilir.

4. Kontrol akışı işi seçer ve tekrarlar

if deyimi bir dal seçer; aşağıdaki zincirde yalnızca ilk eşleşen dal çalışır:

```
if (remaining === 0) {
  status = "full";
} else if (remaining <= 10) {
  status = "almost full";
} else {
  status = "open";
}
```

Bu üç bağımsız test değil, karar akışıdır. İlk koşul başarılı olunca kalan dallar atlanır ve zincirden sonraya geçilir. if (...) içi koşuldur; süslü ayraç bloktur. Girintileme yürütmeyi yönetmez, fakat insan okuyucuya blok yapısını gösterir. Bir dalı izlerken girdiyi, erişilen her koşulun boolean sonucunu, seçilen dalı ve zincir sonrasındaki değeri yazın.

Şekil 4.6. if-else zinciri koşulları sırayla sınar ve yalnızca ilk eşleşen dalı çalıştırır.

4.1 Döngüler değişen durumla tekrar eder

İter edilebilir koleksiyondaki her değere ihtiyaç duyduğunuzda for...of kullanın:

```
for (const event of events) {
  console.log(event.title);
}
```

Her yinelemede event bir sonraki dizi değerine bağlanır ve gövde o değer için bir kez çalışır. Üç etkinlik varsa gövde üç kez çalışır; döngü kendisi yeni dizi oluşturmaz.

```
for (let index = 0; index < events.length; index += 1) {
  console.log(index, events[index].title);
}
```

Sayaçlı döngü başlatmayı, devam koşulunu ve güncellemeyi başlıkta görünür kılar. index sıfırdan başlar; her gövde sonrasında bir artar. < yerine <= yazmak son geçerli öğe length - 1 iken bir sonraki indekse erişmeye çalışır. İndeks önemliyse sayaçlı for, tekrar sayısı önceden bilinmiyorsa while kullanın. Her döngünün sonunda koşulu false yapacak ilerleme açık olmalıdır. break en yakın döngüden çıkar, continue sonraki yinelemeye atlar. Dizi değerleri için for...in kullanmayın; enumerable property key'leri gezer.

5. Fonksiyonlar hesaplamaya ad ve sınır verir

```
function seatsRemaining(event) {
  return Math.max(0, event.capacity - event.registered);
}

const remaining = seatsRemaining(events[0]);
```

Fonksiyon tanımı gövdeyi çalıştırmaz; parantezli çağrı ifadesi kullanıldığında çalışır. events[0] argümandır, event yerel parametredir, return çağrıyı bitirip bir değeri çağrı yerine gönderir. return yoksa sonuç undefined olur.

Şekil 4.7. Fonksiyon çağrısı argümanları yerel parametrelere bağlar ve bir sonucu çağrı yerine döndürür.

5.1 Bildirimler, ifadeler ve arrow function'lar

```
function double(value) { return value * 2; }
const doubleAgain = function (value) { return value * 2; };
const doubleBriefly = (value) => value * 2;
```

Bu biçimler ileri durumlarda this, oluşturma ve hoisting açısından aynı değildir. Bu derste adlandırılmış alan işlemleri için function declaration, dizi dönüşümleri gibi callback'ler için kısa arrow function kullanın. Fonksiyon sözleşmesi kabul edilen girdileri, dönüşü, olası hatayı ve gözlemlenebilir yan etkileri belirtmelidir. Pure function aynı girdiye aynı sonucu verir ve dış durumu değiştirmez; seatsRemaining gibi hesaplamalar böylece kolay test edilir.

5.2 Scope bir adın nerede kullanılabileceğini açıklar

```
const course = "Web Programming";

function describeEvent(event) {
  const remaining = seatsRemaining(event);
  return `${course}: ${event.title} has ${remaining} seats`;
}
```

JavaScript lexical scope kullanır: let veya const ile { ... } içinde bildirilen bağlama o blok dışında yoktur. Fonksiyon çevresindeki kapsamları okuyabilir; çağıran kod yerel bağlamaları okuyamaz. Veriyi yerel tutun ve gerekli girdileri açıkça geçirin; aşırı küresel durum testlerin görünmez bağlama bağımlı olmasına yol açar.

6. Diziler sıralı koleksiyonları modeller

```
const categories = ["technology", "arts", "community"];
categories[0];     // "technology"
categories.length; // 3
```

Diziler sıfırdan başlayan indeksli dizilerdir. Var olmayan indeksi okumak undefined üretir. push() diziyi değiştirir; [...categories] sığ bir kopya üretir. İlk öğe 0, son öğe length - 1'dir. Güvenilmeyen sınırda Array.isArray kullanın; typeof diziler için de object döndürür.

6.1 Dönüşümler amacı ifade eder

filter öğeleri seçer, map her öğeyi dönüştürür, find ilk eşleşeni döndürür, some ve every boolean sorularını yanıtlar:

```
const featuredTitles = events
  .filter((event) => event.featured)
  .map((event) => event.title);
```

İlk callback boolean, ikincisi seçilen her etkinlik için yeni değerdir. İki yöntem de yeni dizi döndürür fakat içindeki nesneleri derin kopyalamaz. reduce yalnızca biriktirme gerçekten anlaşılırsa kullanılmalı; durum değişimini açıkça anlatıyorsa döngü daha iyi olabilir.

7. Nesneler adlandırılmış gerçekleri modeller

```
const event = {
  title: "Robotics night",
  capacity: 80,
  registered: 62,
};

event.title;
event["capacity"];
```

Nokta gösterimi kaynakta bilinen özellik adı için kısa, köşeli ayraç gösterimi dinamik anahtarlar veya identifier sözdizimine uymayan adlar için gereklidir. Eksik özellik genellikle undefined döndürür; mevcut olmayan özelliğe atama onu oluşturur. Bu nedenle event.capcity yazım hatası hemen syntax hatası yerine eksik veri gibi görünebilir. Sınırda nesne şeklini doğrulayın.

```
const { title, capacity } = event;
const updated = { ...event, registered: 63 };
```

Destructuring adlandırılmış özellikleri çıkarır; object spread seçili değişikliklerle sığ kopya oluşturur.

7.1 İlkel kopyalama ve nesne kimliği

```
const first = { registered: 62 };
const second = first;
second.registered = 63;
console.log(first.registered); // 63
```

İki bağlama aynı nesneye gönderme yapar; birinden yapılan mutation diğerinden görünür. Spread yeni dış nesne oluşturur, fakat iç içe nesneler açıkça kopyalanmadıkça paylaşılır. Şekil 4.8 ilkel sayı atamasındaki bağımsız kopyayı nesne atamasındaki ortak kimlikle karşılaştırır.

event.organizer?.email gibi optional chaining, ?. öncesindeki değer null veya undefined ise durup undefined üretir. E-posta adresinin doğru olduğunu doğrulamaz ve sözleşmenin zorunlu tuttuğu özelliği gizlemek için kullanılmamalıdır.

8. Hatalar hükümsüz karar değil kanıttır

SyntaxError kaynak ayrıştırılamadığında; ReferenceError ad kullanılamadığında; TypeError alınan değere işlem uygun olmadığında; AssertionError gözlenen davranış test beklentisiyle uyuşmadığında oluşur. Logical error ise program çalıştığı hâlde yanlış davranıştır. Stack trace etkin fonksiyon çağrılarını gösterir; dış kütüphanedeki ilk satır uygulama kusurunun kaynağı olmayabilir.

İlk ilgili stack frame'den başlayın. Hata adını, mesajını, dosya ve satırı, denenmiş işlemi, gerçek girdiyi ve beklenen sözleşmeyi kaydedin. Şekil 4.9 gözlenen arızadan ihlal edilen sözleşmeye ve doğrulanmış onarıma giden yolu gösterir.

```
if (!Number.isFinite(event.capacity)) {
  throw new TypeError("Event capacity must be a finite number.");
}
```

Her hatayı susturmak için catch kullanmayın. Yalnızca bağlam ekleyebiliyor, kurtarabiliyor, uygun sınır yanıtına çevirebiliyor veya temizlik yapabiliyorsanız yakalayın.

9. Küçük testler davranışı tekrarlanabilir yapar

Test girdi verir, sonucu gözlemler ve beklentiyle karşılaştırır. Pure function'lar açık argümanlara bağlı oldukları için özellikle test edilebilirdir.

```
import test from "node:test";
import assert from "node:assert/strict";
import { seatsRemaining } from "./events.js";

test("calculates available seats", () => {
  const event = { capacity: 80, registered: 62 };
  assert.equal(seatsRemaining(event), 18);
});
```

test(...) adlandırılmış vaka kaydeder; callback vakayı yürütür; assert.equal gözlenen ve beklenen değeri strict equality ile karşılaştırır. Başarısızsa test adını, gerçek değeri, beklenen değeri, işleci ve ilk uygulama frame'ini okuyun. Beklentiyi yalnızca hata kaybolsun diye değiştirmeyin.

Hafta 4 testlerini depo kökünden çalıştırın:

```
npm test
```

Başarılı test yalnızca test edilen örnekler için kanıttır. Normal, sınır ve geçersiz girdileri ekleyin; test adı davranışı belirtmeli, hata anlaşmazlığı kolay bulunur kılmalıdır.

Yönlendirmeli laboratuvar - Campus Hub etkinlik uygunluğunu modelleyin

Amaç ve kanıt

labs/week-04/events.js dosyasını tamamlayarak etkinlik kayıtlarının doğrulanmasını, sınıflandırılmasını, filtrelenmesini ve özetlenmesini sağlayın. Gerekli kanıt başarılı test çalıştırması, doğru tarayıcı özeti, temiz Console ve yalnızca amaçlanan Hafta 4 değişikliklerini içeren Git commit'idir.

Adım 1 - Temeli kurun

Kişisel coursework repository kökünde package.json'ın bulunduğunu kanıtlayın, npm test çalıştırın ve npm run serve:week04 başlatın. http://localhost:8000/ adresini açın; server terminalini açık tutup test ve Git için ikinci terminal kullanın.

Adım 2 - Etkinlik şeklini okuyun

events.js içindeki her özelliğin türünü ve amacını yazın. Kod çalışmadan önce 80 - 62 tahminini yapın; Console'da başlangıç tablosunu doğrulayın.

Adım 3 - Her seferinde bir fonksiyon yazın

Önce seatsRemaining(event), sonra full, almost full ve open sınırlarını içeren eventStatus(event) fonksiyonunu uygulayın. Kanıt toplamadan tüm fonksiyonları birden yazmayın; küçük başarısızdan başarılıya döngü hatayı yerelleştirir.

Adım 4 - Koleksiyonu dönüştürün

filterEvents(allEvents, category) ve summarizeEvents(allEvents) fonksiyonlarını uygulayın. Girdi dizisini değiştirmeyin; geçici mekanik yerine alan anlamını belirten adlar kullanın.

Adım 5 - Sınırı doğrulayın

Eksik veya sayı olmayan capacity ve registered değerlerini yararlı TypeError ile reddedin. Bir bilinçli başarısızlık gözlemleyin, stack'i okuyun, girdiyi onarın ve testleri yeniden çalıştırın.

Adım 6 - Çıktıyı kodla karşılaştırın

Tarayıcıyı yenileyin. Üç etkinlik başlığını, durum sözcüklerini ve kalan koltuk sayılarını kontrol edin. Görsel eşleşme tek başına yeterli değildir; otomatik testler de geçmelidir. Şekil 4.10 test edilmiş modülün üç nesneyi görünür uygunluk özetlerine dönüştürdüğünü gösterir.

Adım 7 - Kanıtı commit edin

git status ve git diff çalıştırın; tamamlanan Hafta 4 dosyalarını davranışı açıklayan bir mesajla commit edin: Complete event availability logic.

Tablo 4.6 - Hafta 4 laboratuvarı tamamlanma kaydı

Kontrol	Beklenen kanıt
Depo kökü	npm komutlarından önce package.json görünür.
Otomatik testler	Normal, sınır ve geçersiz girdiler geçer.
Tarayıcı	Üç doğru özet yükleme mesajının yerini alır.
Console	Yakalanmamış hata veya başarısız modül isteği yoktur.
Veri bütünlüğü	Filtreleme ve özetleme kaynak diziyi değiştirmez.
Git	Tek odaklı commit davranışı kaydeder.

Alıştırmalar

Temel

Alıştırma 1 - Değerler ve türler: typeof 42, typeof "42", typeof null, Array.isArray([]) ve typeof (() => 1) ifadelerinin türünü ve değerini yazın. Alıştırma 2 - Bağlama mı nesne değişikliği mi?: const event = { capacity: 80 }; ardından event.capacity = 90 ve event = { capacity: 100 } işlemlerinden hangisinin neden başarısız olduğunu açıklayın. Alıştırma 3 - Strict karşılaştırma: 0 === false, 0 == false, "" || "Untitled" ve "" ?? "Untitled" sonuçlarını dönüşüm veya kısa devre kuralıyla açıklayın. Alıştırma 4 - Dalı izleyin: remaining değerleri 0, 8 ve 18 için eventStatus dalını izleyip durumu verin.

Uygulama

Alıştırma 5 - Total function yazın: occupancyPercent(event), pozitif olmayan kapasiteyi reddedip yuvarlanmış yüzde döndürsün; hata seçiminizi açıklayın. Alıştırma 6 - Dizi hattı: laboratuvar verisinden 15'ten fazla kalan koltuğu olan featured etkinlik başlıklarını üretin; kaynak diziyi değiştirmeyin. Alıştırma 7 - Scope teşhisi: fonksiyon içindeki const remaining neden dönüşten sonra okunamaz? Döndürerek veri akışını onarın. Alıştırma 8 - Kimlik teşhisi: aynı etkinlik nesnesine bakan iki bağlamanın değişikliği neden birbirinde göründüğünü açıklayın ve sığ kopya güncellemesi gösterin.

Zorluk

Alıştırma 9 - Sınır tablosu: eventStatus için açık, tam 10, bir, sıfır, fazla kayıt ve geçersiz sayısal veri durumlarını tasarlayın. Beklenen sonucu belirtin. Alıştırma 10 - Başarısız hesabı ayıklayın: form alanlarından gelen kapasite ve kayıtları topladıktan sonra Console "8062" gösteriyor. Nedenini teşhis edip örtük dönüşüme güvenmeden onarın.

Alıştırma 11 - Döngü mü dönüşüm mü?: Her başlığı yazdırma, technology etkinliklerini seçme, özet nesneleri oluşturma ve ilk full etkinliği bulma görevleri için loop, filter, map ve find seçin. Alıştırma 12 - Test kalitesi: yalnızca tek sıradan geçerli etkinlikle seatsRemaining çağıran testi eleştirin; daha güçlü kısa küme önerin. Alıştırma 13 - Kodu ayıklamadan ortamı kanıtlayın: terminal C:\Users\Student> gösteriyor, dir package.json dosyayı bulamıyor ve npm run serve:week04 ENOENT veriyor. Kanıtın neyi gösterdiğini, neyi göstermediğini ve sıradaki üç eylemi yazın. Alıştırma 14 - Kaynak, komut ve çıktıyı ayırın: console.log(2 + 3);, node labs/week-04/hello.js ve 5'i sınıflandırın; çıktıyı shell'e tekrar yazmanın neden kategori hatası olduğunu açıklayın. Alıştırma 15 - Tarayıcı script yüklemesini izleyin: HTML 200, hello-brower.js 404 ve Console boş. Kusuru, kanıtı ve onarımı belirtin.

Çözümlü cevaplar

Cevap 1: Sonuçlar sırasıyla "number", "string", "object", true ve "function"dır. null sonucu tarihsel typeof davranışıdır; Array.isArray bilinçli dizi testidir. Cevap 2: event.capacity değişir çünkü nesne değişebilirdir. Yeni nesne atamak const bağlamasını yeniden bağlamaya çalışır ve başarısız olur. const nesneyi özyinelemeli dondurmaz. Cevap 3: 0 === false türler farklı olduğu için false; 0 == false gevşek dönüşüm sonrası true; boş string falsy olduğu için || Untitled; ?? ise yalnız null/undefined'da sağ tarafı kullanır, dolayısıyla boş stringi döndürür. Cevap 4: 0 full, 8 almost full, 18 open dalına gider.

Cevap 5: capacity sonlu ve pozitif değilse RangeError, registered sonlu değilse TypeError atın; ardından Math.round((registered / capacity) * 100) döndürün. Hata türü sözleşmenin hangi kısmının ihlal edildiğini belirtir.

```
function occupancyPercent(event) {
  if (!Number.isFinite(event.capacity) || event.capacity <= 0) {
    throw new RangeError("Capacity must be a positive finite number.");
  }
  if (!Number.isFinite(event.registered)) {
    throw new TypeError("Registered must be a finite number.");
  }
  return Math.round((event.registered / event.capacity) * 100);
}
```

Cevap 6: filter ile featured ve seatsRemaining(event) > 15 koşulunu seçip map ile title döndürün; iki yöntem yeni diziler verir ve kaynak nesneleri okumakla yetinir. Cevap 7: Yerel bağlama fonksiyonun lexical scope'una aittir. Değeri return edin ve çağrı sonucunu dışarıda bağlayın. Cevap 8: Atama nesne referansını kopyalar; `{ ...original, registered: 63 }` yeni dış nesne oluşturur. İç içe nesneler yine paylaşılabilir. Cevap 9: 18 open, 10 ve 1 almost full, 0 full; fazla kayıt clamp sonrası full; sayısal olmayan veri TypeError üretmelidir. Cevap 10: Form değerleri string olduğu için + birleştirme yapar. Number ile dönüştürüp Number.isFinite ile doğrulayın; sonra toplayın.

Cevap 11: Her başlığı yazdırmak yan etki için for...of; technology seçmek filter; her etkinlik için özet oluşturmak map; ilk full etkinliği bulmak find'dır. Cevap 12: Tek sıradan test sıfır, fazla kayıt, sınırlar, eksik alanlar ve sonlu olmayan sayıları kaçırır. Normal, full, clamp, tüm durum sınırları ve hata türlerini test edin. Cevap 13: npm C:\Users\Student içinde manifest bulamamıştır; Node veya JavaScript'in bozuk olduğunu kanıtlamaz. Yetkili klasörü açın/cd edin, dir package.json ile doğrulayın, sonra aynı kökten npm run serve:week04 çalıştırın. Cevap 14: İlk öğe JavaScript kaynak kodu, ikincisi shell komutu, 5 çalışma zamanı çıktısıdır. 5'i shell'e yazmak kanıtı komut sanmaktır. Cevap 15: browser kelimesindeki s eksik. HTML 200 sunucu ve belgeyi, script 404 ise src URL'sinin dosyayla eşleşmediğini kanıtlar. src'yi hello-browser.js yapın, kaydedin, 200 yanıtını doğrulayın.

Hata ayıklama kliniği

ReferenceError remaining is not defined için ilk uygulama frame'ini, yazımı ve lexical scope'u inceleyin; parametre veya return ile veri akışını düzeltin. undefined özelliği okunuyorsa soldaki ifadeyi izleyin; array lookup veya return eksik olabilir. Rakamlar birleşiyorsa typeof ile girdileri inceleyip sınırda dönüştürün. Bir test diğerinin verisini değiştiriyorsa paylaşılan mutable nesne/dizi arayın ve her testte yeni fixture oluşturun.

Geri çağırma quiz'i

1. ECMAScript ile browser API arasındaki fark nedir? 2. İfade ne üretir? 3. Hangi durumda let, const yerine kullanılmalıdır? 4. const nesnesinin özelliği neden değişebilir? 5. typeof null neden yeterli null testi değildir? 6. === ile == arasındaki fark nedir? 7. ?? ne zaman ||'dan güvenlidir? 8. Döngünün sonlanması için ne değişmelidir? 9. Fonksiyon return olmadan biterse ne olur? 10. Parametre bağlaması nerede geçerlidir? 11. map neden derin değişmezliği garanti etmez? 12. Nokta ve köşeli ayraç erişimi farkı nedir? 13. İki bağlama aynı mutation'ı neden görür? 14. Fonksiyon ne zaman sonuç uydurmak yerine hata atmalıdır? 15. Normal, sınır ve geçersiz testler ne sağlar? 16. Node.js'in geçerli terminalde kullanılabildiğini hangi kanıt gösterir? 17. npm komutundan önce package.json neden görünmelidir? 18. Dosyayı kaydetmek ve çalıştırmak farkı nedir? 19. JavaScript isteği başarısızken HTML neden görünebilir? 20. hello-browser.js durumunu hangi panel kanıtlar?

Quiz yanıtları

1. ECMAScript dili tanımlar; browser API host tarafından sağlanır. 2. İfade bir değere değerlendirilir; statement yürütmeyi yönetir. 3. Algoritma bağlamayı bilinçli olarak yeniden atıyorsa let kullanılır. 4. const yeniden bağlamayı engeller, referans verilen nesnenin mutation'ını değil. 5. typeof null tarihsel olarak object döndürür; value === null kullanın. 6. === örtük dönüşüm yapmadan karşılaştırır; == dönüştürebilir. 7. 0, false veya boş string geçerli kalacaksa ?? daha güvenlidir. 8. Koşulla ilgili durum sonlanmaya ilerlemelidir. 9. Çağrı undefined döndürür. 10. Parametre çağrının fonksiyon scope'unda yereldir.

11. map yeni dizi oluşturur fakat iç içe nesne referanslarını koruyabilir. 12. Nokta kaynakta bilinen adı, köşeli ayraç ifade ile hesaplanan anahtarı kullanır. 13. İki bağlama aynı nesne kimliğini içerir. 14. Girdi sözleşmeyi ihlal ediyor ve dürüst bir alan sonucu yoksa hata atın. 15. Normal davranışı, karar sınırlarını ve geçersiz sözleşmelerin reddini kanıtlar. 16. node --version beklenen sürümü o terminalde yazdırmalıdır; yükleyici dosyası runtime kanıtı değildir. 17. npm script'ler ve proje metadatası için manifesti okur; yokluğu çoğunlukla yanlış klasördür. 18. Kaydetmek editör içeriğini diske yazar; çalıştırmak runtime'ın kaydedilmiş kaynağı okumasıdır. 19. HTML ve JavaScript ayrı isteklerdir; ilki başarılı, ikincisi 404 olabilir. 20. Network URL ve HTTP status'u, Console yürütme mesaj ve hatalarını gösterir.

Oturum özeti

JavaScript dersin ana programlama dilidir; host'lar ek API'ler sağlar. Değerlerin türleri, bağlamaların lexical scope içindeki adları vardır. Varsayılan const, bilinçli yeniden atama için let kullanın. Strict equality ve açık dönüşüm sınırları görünür kılar. Dallar yol seçer, döngüler ilerlemeyle tekrarlar, fonksiyonlar sözleşme kurar. Diziler sıralı koleksiyon, nesneler adlandırılmış özelliklerdir. Nesne ataması kimliği paylaşır, spread yalnızca sığ kopyadır. Hatalar ve başarısız assertion'lar yapılandırılmış kanıttır. Küçük otomatik testler beklentileri tekrarlanabilir yapar fakat yalnızca test edilen davranışı kanıtlar.

Sözlük - Tablo 4.7

Tablo 4.7 bölümün çalışma sözcüklerini toplar: argument çağrıda verilen değer; array indeksli sıralı nesne; binding ad ile değer ilişkisi; boolean true/false ilkel türü; branch kontrol akışının seçtiği yol; callback başka işleme verilen fonksiyon; coercion dil kurallarıyla dönüşüm; ECMAScript JavaScript'in uyguladığı standart; expression değer üreten kaynak; function parametre ve sonuç arkasında davranış paketleyen çağrılabilir nesne; mutation mevcut nesne durumundaki değişiklik; identifier yasal kaynak adı; literal doğrudan yazılan değer; object kimlik ve anahtarlı özellikleri olan değer; parameter gelen argüman için yerel bağlama; primitive değişmez nesne olmayan değer; scope bağlamanın erişilebilir olduğu bölge; statement yürütme adımı; test assertion gözlenen/beklenen karşılaştırması; runtime talimatları çalıştıran yazılım; source code dosyada saklanan insan yazımı; terminal shell komutlarını çalıştıran metin arayüzüdür.

Birincil başvurular ve ileri okuma

1. Ecma International, ECMAScript 2026 Language Specification - dil semantiği için normatif kaynak. 2. MDN, JavaScript Guide - erişilebilir açıklamalar ve referanslar. 3. MDN, What is JavaScript? - HTML, CSS, JavaScript ve browser API ilişkisi. 4. Node.js, Official download page - eski bağlantı yerine güncel desteklenen yama. 5. Visual Studio Code, Getting started - kurulum, workspace ve editör arayüzü. 6. Node.js, Test runner documentation - test komutu ve yaşam döngüsü. 7. Node.js, Strict assertion documentation - eşitlik hataları ve diff'ler. 8. Marijn Haverbeke, Eloquent JavaScript, 4. baskı - ek dil pratiği. 9. Axel Rauschmayer, Exploring JavaScript - ilk okumadan sonra kesin dil eşlikçisi.

Neler daha sık değişir?

Değer, kontrol akışı, fonksiyon, dizi ve nesne semantiği kararlıdır. Yeni ECMAScript sürümleri, Node.js indirme sürümleri, tanı araçları ve test runner özellikleri gelişir; tarayıcı ve editör arayüzleri değişebilir. Ders motoruyla uyumlu en yeni Node.js 24 LTS yamasını kullanın. Teknik inceleme tarihi: 21 Ekim 2026.

Öğretim elemanı rehberi - önerilen sekiz saat

1. Araç rolleri, kurulum ve klasör/terminal doğrulaması - 45 dakika. 2. Kanıtla ilk Node.js ve tarayıcı programları - 45 dakika. 3. Runtime sınırı, değerler, bağlamalar ve ifadeler - 55 dakika. 4. İşleçler, eşitlik, dönüşüm, dallar ve iz tabloları - 60 dakika. 5. Döngüler, fonksiyonlar, return ve scope - 60 dakika. 6. Diziler, nesneler, kimlik ve dönüşümler - 60 dakika. 7. Hatalar, testler ve hata ayıklama gösterimi - 35 dakika. 8. Yönlendirmeli laboratuvar ve kanıt incelemesi - 120 dakika.

Yanılgıları ortaya çıkaran gösterimler

• Browser ve Node.js'te typeof document çalıştırarak dil ile host'u ayırın. • const'u yeniden atayın, sonra const nesnesinin özelliğini değiştirin. • 0, "0", false, boş string ve boş diziyi strict karşılaştırma ve koşullarda karşılaştırın. • eventStatus() için 0, 10 ve 11 koltukta adım adım ilerleyin. • Spread ile diziyi kopyalayın, iç nesneyi değiştirin ve sığ paylaşımı gözleyin. • Bir assertion'ı bozup actual, expected, operator ve stack kanıtını okuyun.

Kapsayıcı değerlendirme

İz tablosu, sözlü açıklama veya debugger adım kaydı aynı akıl yürütmeyi gösteriyorsa biçimlendirici kanıt olarak kabul edin. Kodu görsel yerine metin olarak sağlayın, sözdizimi yazımı için ek süre tanıyın ve sözleşme akıl yürütmesini yazma hızından ayrı değerlendirin. Tam teslim yine aynı davranış testlerini geçmeli ve bir hata onarımını açıklamalıdır.

Öğrenci öz değerlendirmesi

Bir değerin türünü ve bir bağlamanın kapsamını kanıtla açıklayabiliyor musunuz? Bir ENOENT, 404 veya TypeError mesajını dosyayı rastgele değiştirerek değil sınırı inceleyerek teşhis edebiliyor musunuz? Fonksiyon sözleşmesini, test girdilerini ve Git commit kanıtını bir başkasının yeniden üretebileceği biçimde yazabiliyor musunuz?

Oturum 4 kapanış ölçütü

Oturumun sonunda öğrenci ilk JavaScript dosyasını doğru klasörden çalıştırabilmeli, Node.js ile tarayıcı host'unu ayırabilmeli, değerleri ve bağlamaları tür/kapsam kanıtıyla açıklayabilmeli, branch ve loop yürütmesini izleyebilmeli, açık sözleşmeli fonksiyonlar yazabilmeli, dizileri ve nesneleri mutation riskini gözeterek dönüştürebilmeli, stack trace'i okuyabilmeli ve normal, sınır ve geçersiz girdileri kapsayan testler çalıştırabilmelidir.

Bu ölçüt yalnızca programın bir kez çalışmasına değil, öğrencinin sonucun neden doğru olduğunu ve başarısız olduğunda hangi kanıtla onardığını açıklamasına dayanır.

Ek A - Test edilmiş laboratuvar referans çözümleri

Bu dosyalar, depo kontrollerinde kullanılan aynı test edilmiş kaynaklardan PDF'ye üretilmiştir. Laboratuvarı tamamladıktan sonra kendi commit edilmiş çalışmanızla karşılaştırın; çözümü önceden kopyalamayın.

Hafta 01 - index.html

```
<!doctype html>
<html lang="en">
  <head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1"><title>Campus Hub - Event
  <body><header><h1>Campus Hub</h1><nav aria-label="Primary navigation"><ul><li><a href="#events">Events</a></li><li><a hre
    <main><section id="events"><h2>Upcoming events</h2><article><h3>Robotics laboratory open evening</h3><p><time datetime=
    <footer><p>Campus Hub is a fictional student learning project.</p></footer></body>
</html>
```

Hafta 02 - index.html

Bu başlangıç dosyası erişilebilir bir etkinlik gönderim formudur. Kaynak kodundaki İngilizce görünen etiketler ve URL'ler çalıştırılabilir örneğin parçasıdır; öğrenciler bunları çeviri metniyle değiştirebilir, ancak name, id, for, action ve method ilişkilerini bozmamalıdır.

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1"><meta name=
<body><a href="#main-content">Skip to event form</a><header><p><a href="/">Campus Hub</a></p><nav aria-label="Primary navig
<main id="main-content"><h1>Submit a campus event</h1><p>Required fields are marked in their labels.</p><form action="/even
```

Formun tam kaynak dosyasında start time, radio group, açıklama textarea'sı, aria-describedby ve footer da bulunur. Her kontrolün etiketi, name değeri ve doğrulama sınırı birlikte test edilmelidir.

Hafta 03 - index.html

Responsive Campus Hub başlangıcı, skip-link, semantik başlıklar, filtre formu, event grid ve üç event-card içerir. Uzun başlık özellikle taşma ve responsive davranış testleri için bırakılmıştır.

```
<div class="page-shell"><header class="site-header"><a class="brand" href="/">Campus Hub</a><nav class="site-navigation" ar
```

Tam dosyada communities bölümü, iki ek kart ve footer bulunur. Öğrenciler kaynak sırasını, heading hiyerarşisini ve filtre kontrolünün label ilişkisini korumalıdır.

Hafta 03 - styles.css (reset, temel ve layout katmanları)

```
@layer reset, base, layout, components;

@layer reset {
  *, *::before, *::after { box-sizing: border-box; }
  img, svg, video { max-inline-size: 100%; block-size: auto; }
  button, input, select, textarea { font: inherit; }
}

@layer base {
  :root { --color-ink: #172033; --color-accent: #146c94; --color-surface: #f7fafc; --color-border: #cad4df; --color-focus:
  body { margin: 0; color: var(--color-ink); background: var(--color-surface); font: 1rem/1.6 system-ui, sans-serif; }
  h1 { font-size: clamp(2rem, 6vw, 4rem); line-height: 1.1; max-inline-size: 16ch; }
  :focus-visible { outline: 3px solid var(--color-focus); outline-offset: 3px; }
}

@layer layout {
  .page-shell { width: min(100% - 2rem, 72rem); margin-inline: auto; }
  .site-header { display: flex; flex-wrap: wrap; align-items: center; justify-content: space-between; gap: var(--space-4);
  .event-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(100%, 17rem), 1fr)); gap: var(--space-4);
}
```

Katmanlar seçici özgüllüğünden ayrı bir öncelik düzeni kurar. Mantıksal özellikler, clamp ve auto-fit içeriğin ve kullanıcı ayarlarının değişmesine izin verir.

Hafta 03 - styles.css (components, container query ve medya tercihleri)

```
@layer components {
  .skip-link { position: absolute; inset-inline-start: 1rem; inset-block-start: -5rem; padding: .75rem 1rem; background: wh
  .skip-link:focus { inset-block-start: 1rem; }
  button, select { min-block-size: 2.75rem; padding-block: .6rem; padding-inline: .9rem; border: 1px solid var(--color-acce
  .event-card { min-inline-size: 0; padding: var(--space-6); border: 1px solid var(--color-border); border-radius: .65rem;
  .event-card__content { display: grid; gap: .5rem; }
  .event-card h3 { overflow-wrap: anywhere; }
}

@container event-card (min-width: 20rem) {
  .event-card__content { grid-template-columns: 1fr auto; align-items: end; }
  .event-card__title, .event-card__content p { grid-column: 1 / -1; }
  .event-card__action { justify-self: end; }
}
```

Kalan kaynakta 48rem medya sorgusu, reduced-motion ve forced-colors kuralları bulunur. Bu kurallar görsel hareketi azaltır, yüksek kontrast ortamında sınırları korur ve odak göstergesini silmez.

Hafta 04 - events.js

```
export const events = [
  { title: "Robotics night", category: "technology", capacity: 80, registered: 62, featured: true },
  { title: "Campus orchestra", category: "arts", capacity: 120, registered: 120, featured: false },
  { title: "Sustainable gardens", category: "community", capacity: 40, registered: 18, featured: true },
];

export function seatsRemaining(event) {
  if (!event || !Number.isFinite(event.capacity) || !Number.isFinite(event.registered)) {
    throw new TypeError("An event must contain numeric capacity and registered values.");
  }
  return Math.max(0, event.capacity - event.registered);
}

export function eventStatus(event) {
  const remaining = seatsRemaining(event);
  if (remaining === 0) return "full";
  if (remaining <= 10) return "almost full";
  return "open";
}
```

Tam modül filterEvents, summarizeEvents, console.table ve document bulunuyorsa #program-output alanını güncelleyen kodu da içerir. Modülün amacı aynı veriyi hem testlerde hem tarayıcıda kullanmaktır.

Hafta 04 - events.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import { eventStatus, filterEvents, seatsRemaining, summarizeEvents } from "./events.js";

globalThis.document = { querySelector: () => null };
const sample = { title: "Robotics night", category: "technology", capacity: 80, registered: 62 };

test("seatsRemaining subtracts registrations without returning a negative value", () => {
  assert.equal(seatsRemaining(sample), 18);
  assert.equal(seatsRemaining({ ...sample, registered: 90 }), 0);
});

test("eventStatus applies the documented boundaries", () => {
  assert.equal(eventStatus(sample), "open");
  assert.equal(eventStatus({ ...sample, registered: 72 }), "almost full");
  assert.equal(eventStatus({ ...sample, registered: 80 }), "full");
});

test("invalid event data produces a useful TypeError", () => {
  assert.throws(() => seatsRemaining({ capacity: "80", registered: 10 }), TypeError);
});
```

Tam kaynakta filterEvents'in yeni dizi döndürmesi ve summarizeEvents'in kamuya açık özet şeklini koruması da assert.deepEqual ile test edilir. Testler davranış sözleşmesini belgeleyen çalıştırılabilir örneklerdir.

Hafta 04 - hello-browser.html ve hello-browser.js

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1"><title>Camp
```

```
const courseName = "Web Programming";
const sum = 2 + 3;

console.log(`Hello from ${courseName}!`);
console.log(`Two plus three is ${sum}.`);
```

HTML dosyası script kaynağını ister; JavaScript dosyası Console'a iki mesaj yazdırır. Network'te HTML ve script isteklerini, Console'da yürütme çıktısını ayrı kanıtlar olarak inceleyin.

Hafta 04 - hello.js, index.html ve styles.css

```
const courseName = "Web Programming";
const sum = 2 + 3;

console.log(`Hello from ${courseName}!`);
console.log(`Two plus three is ${sum}.`);
```

Hafta 04 index.html, styles.css ile bir module script yükler ve #program-output içinde Loading event summary... başlangıç metnini gösterir. events.js başarılı olduğunda bu alan üç etkinliğin başlık, durum ve kalan koltuk özetleriyle değiştirilir.

```
:root { color-scheme: light; font-family: system-ui, sans-serif; }
* { box-sizing: border-box; }
body { margin: 0; min-block-size: 100vh; display: grid; place-items: center; background: #eaf5f5; color: #12355b; }
main { inline-size: min(44rem, calc(100% - 2rem)); padding: 2rem; border: 1px solid #a9c7d8; border-radius: 1rem; backgroun
pre { padding: 1rem; overflow: auto; white-space: pre-wrap; }
```

Bu örnekte `pre` çıktının satır sonlarını korur; overflow:auto uzun çıktının taşmasını kontrol eder; min() sayfanın dar görünüm alanlarında kenar boşluklarını korur.

Ek B - Hızlı komutlar

Öğrenci coursework repository'sinde sağlanan kurulumu doğrulayın:

```
npm test
```

Hafta 1'i http://localhost:8000/ adresinde sunun:

```
npm run serve:week01
```

Ders kitabı kaynak repository'sinde bu PDF baskısını doğrulayın, oluşturun ve yeniden doğrulayın:

```
npm test
npm run pdf
npm run pdf:verify
```

Komutları her zaman doğru repository kökünden çalıştırın. `package.json` görünür değilse npm komutu çalıştırmayın; önce terminalin geçerli klasörünü düzeltin.